A Practical Guide to Control Theory

For Engineering Students in Robot Competitions

Skywalker Liu
ltx2499323877@outlook.com
github.com/liuskywalkerjskd

October 2025

Contents

1 Introduction **7**

Two Numbers That Run the World: e and π **8**

2 Digital Signal Processing **11**
2.1 Understanding the Frequency Domain . . . 11
2.2 Laplace Transform and Fourier Transform . . . 11
2.2.1 Laplace Transform . . . 11
2.2.2 A First Look at Poles and Zeros . . . 12
2.2.3 Fourier Transform . . . 12
2.2.4 Frequency Spectrum . . . 13
2.3 Frequency-Based Filter Design . . . 14
2.3.1 Low-Pass Filter . . . 14
2.3.2 Band-Pass Filter . . . 16
2.3.3 High-Pass Filter . . . 16
2.3.4 Notch Filter (Band-Reject Filter) . . . 16
2.3.5 Filter Type Selection and Parameter Tuning . . . 17
2.4 Z-Transform . . . 17

3 Sensor Calibration and Selection **19**
3.1 The Sensor Error Model . . . 19
3.2 IMU Calibration . . . 19
3.2.1 Accelerometer: 6-Position Static Calibration . . . 19
3.2.2 Gyroscope: Static Bias and Noise . . . 20
3.2.3 Gyroscope Scale Factor . . . 21
3.3 Encoder: Resolution vs Noise Tradeoff . . . 21
3.3.1 Velocity Estimation from Position . . . 21
3.3.2 Encoder Types and Selection . . . 22
3.4 Magnetometer Calibration . . . 22
3.5 Current Sensor Calibration . . . 22
3.6 Sensor Fusion Architecture . . . 23
3.7 Choosing the Right Sensor . . . 23
3.8 Practical: Verifying Calibration Quality . . . 24
3.9 Calibration Checklist . . . 25

4 System Description **26**
4.1 Mathematical Models of Systems . . . 26
4.1.1 Differential Equations . . . 26
4.1.2 Transfer Functions . . . 26
4.1.3 State-Space Representation . . . 27
4.2 System Analysis via Transfer Functions . . . 28
4.3 System Analysis via State-Space . . . 31
4.4 Motor Modeling: The Most Common Plant . . . 31

5 Classical Control Theory **35**
5.1 The Concept of Feedback Control . . . 35
5.2 System Performance Metrics . . . 36
5.2.1 Stability Margins . . . 36
5.3 Controller Design via Transfer Functions . . . 37
5.4 PID Controller . . . 38
5.4.1 PID Controller – Continuous Form . . . 38
5.4.2 PID Controller – Discrete Form . . . 40
5.4.3 Improved PID Controllers . . . 40
5.5 Advanced PID Improvements . . . 41
5.5.1 Integral Separation . . . 41
5.5.2 Derivative Placement: PID vs PI-D vs I-PD . . . 42
5.5.3 Filtered PID: Frequency Domain Proof . . . 43

5.5.4 Two-Degree-of-Freedom PID (2-DOF) . . . 45
5.5.5 Fuzzy PID . . . 46
5.6 PID Tuning Methods . . . 47
5.6.1 Empirical Tuning . . . 47
5.6.2 Tuning with MATLAB Simulink . . . 47
5.7 System Identification: Figuring Out What Your Plant Actually Does . . . 48
5.7.1 Step Response Method (Time Domain) . . . 48
5.7.2 Frequency Response Method (Bode Identification) . . . 48
5.7.3 Least-Squares Identification (Automated) . . . 49
5.7.4 Practical Sysid Workflow for Competition Robots . . . 49
5.8 Feedforward Control . . . 50
5.9 PID Architectures: Single, Cascaded, and Parallel-Cascaded . . . 51
5.9.1 Architecture 1: Single-Loop PID . . . 51
5.9.2 Architecture 2: Cascaded (Series) PID . . . 52
5.9.3 Architecture 3: Parallel-Cascaded PID (Multi-Axis) . . . 52
5.9.4 Comparison Summary . . . 53
5.9.5 Output Conflict in Parallel PID . . . 54
5.9.6 Food for Thought: Is Inner-Loop K_p the Same as Outer-Loop K_d? . . . 54

6 Multi-Loop Architecture Design **56**
6.1 The Full Control Stack . . . 56
6.2 Cascade Control Architecture . . . 56
6.2.1 Current → Velocity → Position . . . 56
6.2.2 The Bandwidth Separation Principle . . . 57
6.2.3 Tuning Order: Inside Out . . . 58
6.3 Task-Level vs Joint-Level Control . . . 58
6.4 Supervisor Patterns . . . 59
6.4.1 State Machines for Autonomous Operation . . . 59
6.4.2 Mode Switching and Bumpless Transfer . . . 60
6.5 Timing Architecture . . . 60

7 Discretization and Implementation **63**
7.1 From Continuous to Discrete . . . 63
7.1.1 Zero-Order Hold (ZOH) . . . 63
7.1.2 Tustin (Bilinear) Transform . . . 63
7.1.3 Matched Pole-Zero Method . . . 64
7.2 Digital Filter Structures: FIR vs. IIR . . . 64
7.2.1 FIR (Finite Impulse Response) . . . 64
7.2.2 IIR (Infinite Impulse Response) . . . 65
7.2.3 FIR vs. IIR: Head-to-Head Comparison . . . 66
7.2.4 Practical Guidelines and Biquad Implementation . . . 66
7.3 Fixed-Point vs Floating-Point Arithmetic . . . 67
7.4 Sampling Rate Selection . . . 68
7.5 Implementation on Embedded Systems . . . 68
7.5.1 Timer and Interrupt Configuration . . . 68
7.5.2 Computation Latency and Jitter . . . 69
7.5.3 Anti-Windup in Practice . . . 69

8 Simulation Workflow **72**
8.1 Why Simulate Before Deploying . . . 72
8.2 Model-in-the-Loop (MIL) . . . 72
8.2.1 Building a DC Motor + PID Simulation from Scratch . . . 72
8.2.2 Comparing Simulation with Theory . . . 73
8.3 Software-in-the-Loop (SIL) . . . 74
8.4 Hardware-in-the-Loop (HIL) . . . 74
8.5 Making Simulations Realistic . . . 75
8.5.1 Sensor Noise . . . 75
8.5.2 Disturbances . . . 75
8.5.3 Friction . . . 75

8.5.4 Communication Delay . . . 75
8.5.5 Actuator Saturation . . . 75
8.6 Quick Python Simulation Template . . . 76

9 Practical Debugging and Tuning Cookbook **78**
9.1 The Debugging Mindset . . . 78
9.2 The Diagnostic Flowchart . . . 78
9.2.1 Symptom: Oscillation . . . 78
9.2.2 Symptom: Overshoot . . . 79
9.2.3 Symptom: Steady-State Error / Drift . . . 79
9.2.4 Symptom: No Response . . . 79
9.3 Is It Mechanical, Electrical, Software, or Control? . . . 80
9.3.1 The Isolation Test . . . 80
9.3.2 Common Electrical Problems . . . 80
9.3.3 Common Software Problems . . . 80
9.4 Oscilloscope and Logic Analyzer Techniques . . . 81
9.5 PID Tuning on Real Hardware: A Complete Worked Example . . . 82
9.5.1 System: DJI 3508 Motor Velocity Control . . . 82
9.5.2 Step-by-Step Tuning Procedure . . . 82
9.5.3 When Ziegler-Nichols Fails . . . 83
9.6 Common Failure Modes and Their Signatures . . . 83
9.6.1 Integral Windup in Practice . . . 83
9.6.2 Actuator Saturation . . . 83
9.6.3 Sensor Aliasing . . . 84
9.6.4 Quantization Limit Cycling . . . 84
9.6.5 Communication Latency . . . 84
9.7 Pre-Competition Checklist . . . 84
9.8 Competition Day Emergency Fixes . . . 85
9.9 Building a Debugging Toolkit . . . 86

10 Modern Control Theory **88**
10.1 Relationship Between State-Space and Transfer Functions . . . 88
10.2 System Performance Metrics . . . 88
10.2.1 Controllability and Observability . . . 88
10.2.2 Closed-Loop System Stability . . . 90
10.3 Controller Design via State-Space . . . 91
10.3.1 Bellman Equations and Dynamic Programming: The Engine Behind LQR and MPC 91
10.3.2 LQR (Linear Quadratic Regulator) . . . 93
10.3.3 A Brief Primer on Optimization and Convexity . . . 94
10.3.4 MPC (Model Predictive Control) . . . 94
10.3.5 TinyMPC: Riccati-Accelerated MPC for Embedded Systems . . . 96
10.4 Observer Design via State-Space . . . 97
10.4.1 Luenberger Observer . . . 97
10.4.2 The Deep Duality: Controller and Observer as Mirror Images . . . 98
10.4.3 Kalman Filter . . . 99
10.4.4 Extended Kalman Filter (EKF) . . . 100
10.4.5 Quaternion-Based EKF: Avoiding Gimbal Lock . . . 102
10.4.6 Adaptive EKF: Auto-Tuning Q and R Online . . . 103
10.4.7 Luenberger Observer vs. Kalman Filter: A Comprehensive Comparison . . . 105
10.4.8 LQG (Linear Quadratic Gaussian) . . . 106
10.5 Trajectory Generation: Telling Your Motor Where to Go . . . 111
10.5.1 Trapezoidal Velocity Profile . . . 111
10.5.2 S-Curve Profile . . . 111
10.5.3 Bézier Curves and Spline Interpolation . . . 112

11 Nonlinear Control Basics **114**
11.1 Why Linear Methods Fail . . . 114
11.2 Feedback Linearization . . . 114
11.2.1 The Idea: Cancel the Nonlinearity . . . 114
11.2.2 Example: Inverted Pendulum Swing-Up . . . 115
11.2.3 Limitations . . . 116
11.3 Sliding Mode Control . . . 116
11.3.1 The Idea: Robustness Through Discontinuous Switching . . . 116
11.3.2 Example: Velocity Control with Unknown Friction . . . 117
11.3.3 The Chattering Problem . . . 117
11.4 Gain Scheduling . . . 118
11.5 Lyapunov-Based Design . . . 119
11.5.1 Energy Shaping for Swing-Up . . . 119
11.6 When to Use What: A Decision Guide . . . 120

12 Attitude and Rotation Representation **122**
12.1 Coordinate Frames . . . 122
12.1.1 The Right-Hand Rule . . . 122
12.1.2 Right-Hand vs Left-Hand Frames . . . 122
12.1.3 Common Frames in Robotics . . . 123
12.2 Rotation Matrices . . . 123
12.2.1 Definition and Properties . . . 123
12.2.2 What Rotation Matrices Actually Do . . . 124
12.2.3 Composing Rotations: Left vs Right Multiplication . . . 125
12.3 Euler Angles . . . 125
12.3.1 Definition and Conventions . . . 126
12.3.2 Gimbal Lock . . . 126
12.4 Quaternions . . . 128
12.4.1 Axis-Angle Rotation: The Gateway to Quaternions . . . 128
12.4.2 From Axis-Angle to Quaternions . . . 129
12.4.3 Quaternion Operations . . . 129
12.4.4 Why Quaternions Win . . . 130
12.5 Practical Summary . . . 130

13 Outlook: Data-Driven and Learning-Based Control **131**
13.1 Data-Driven Control . . . 131
13.1.1 Motivation: From “Model First, Control Second” to “Control Directly from Data” 131
13.1.2 Willems’ Fundamental Lemma . . . 131
13.1.3 Data-Driven LQR . . . 131
13.1.4 Data-Enabled Predictive Control (DeePC) . . . 131
13.1.5 Koopman Operator Theory . . . 131
13.2 Adaptive Control . . . 132
13.2.1 Model Reference Adaptive Control (MRAC) . . . 132
13.2.2 L1 Adaptive Control . . . 132
13.2.3 When to Use What . . . 132
13.3 Reinforcement Learning and Control . . . 132
13.3.1 Core Concepts . . . 133
13.3.2 Two Paradigms . . . 133
13.3.3 Sim-to-Real Transfer . . . 133
13.3.4 Success Stories and Current Status . . . 133
13.4 Trajectory Optimization and Optimal Control . . . 133
13.4.1 Two Solution Strategies . . . 133
13.4.2 iLQR and DDP . . . 134
13.5 Neural Networks Meet Control . . . 134
13.5.1 Neural Networks as Controller Components . . . 134
13.5.2 Neural ODEs and Physics-Informed Neural Networks . . . 134
13.5.3 Diffusion Policies for Robot Learning . . . 134
13.5.4 Foundation Models for Robotics (VLAs) . . . 135

13.5.5 Safety Constraints and Verifiability . . . 135
13.6 Event-Triggered Control . . . 135
13.7 Method Comparison at a Glance . . . 136
13.8 Advice for the Reader . . . 136
13.9 Further Reading . . . 136

14 Appendix: Plug-and-Play C++ Modules 138
14.1 Low-Pass Filter . . . 138
14.2 PID Controller . . . 139
14.3 Setpoint Ramp (LPF Soft-Start) . . . 140
14.4 Complementary Filter (IMU Tilt) . . . 141
14.5 Lightweight Matrix Library . . . 141
14.6 Kalman Filter . . . 143
14.7 LQR Gain (Offline Computation Helper) . . . 145
14.8 Cascaded PID . . . 146
14.9 Advanced PID (Integral Separation + PI-D + 2-DOF) . . . 147
14.10 EKF for 6-DOF IMU (Pose Estimation) . . . 148
14.11 TinyMPC Solver . . . 151
14.12 Cubic Bézier Trajectory . . . 153
14.13 Module Dependency Map . . . 155

1 Introduction

This document is **not** a textbook. Textbooks are great—rigorous, comprehensive, and excellent at putting you to sleep at 2 AM. This is a *lecture note*, written by someone who learned control the hard way: by breaking robots first, then reading the theory to figure out why.

The goal is simple: give you enough theoretical foundation to **design, tune, and debug controllers** on real hardware. We will cover the math, but for every concept we will always ask: "What is this actually for when the robot hits the field?"

Who Is This For?

Undergraduate students in their early and middle years. Prerequisites: calculus and basic linear algebra (taken or currently taking). If you know what a derivative is and can multiply matrices without breaking down, you are ready.

What Is Special About This Note's Structure?

Almost no control textbook on the market has a similar content arrangement.

We start with **Digital Signal Processing**—because before you can control anything, you need clean data from your sensors. Then we learn how to **describe systems** mathematically—a fancy way of saying "write down what your motor actually does." Next comes **Classical Control Theory**, PID's home turf—the bread and butter of competition robotics. After that, **Discretization and Implementation** bridges the gap between elegant math and the harsh reality of a microcontroller running at a fixed clock rate. Finally, **Modern Control Theory** gives you the tools that turn a "working" robot into a "good" robot: LQR, MPC, and Kalman filters.

Let us begin.

Two Numbers That Run the World: e and π

Before we dive into filters, transfer functions, and all the machinery of control, let us spend a moment with the two irrational numbers that quietly power nearly everything in this book: π and e.

You have met them in calculus. You may have memorized their digits. But here is the one-sentence version of *why they matter in engineering*:

> **π is the language of things that repeat.**
> **e is the language of things that grow or decay.**

That is it. Almost every equation you will see in this book is built on one of these two ideas—or both at the same time.

π: The Heartbeat of Periodicity

Spin a wheel. Watch a pendulum swing. Listen to a tuning fork hum. All of these are *periodic*—they trace the same path over and over again. π is the number that connects the geometry of a circle to the algebra of repetition.

Consider a point moving around a unit circle at constant speed. Its horizontal position at time t is $\cos(t)$ and its vertical position is $\sin(t)$. One full trip around the circle takes exactly 2π units of time. That is not a coincidence—it is the *definition* of where π comes from. A circle's circumference is $2\pi r$; dividing by the radius gives 2π, the "natural period" of rotation.

In control engineering, periodic phenomena are everywhere:

- The vibration of a mechanical structure,
- The AC voltage from a power supply,
- The oscillation of an under-damped step response (when a system overshoots its target and rings back and forth before settling—we will study this in detail later),
- The sampling clock of your microcontroller.

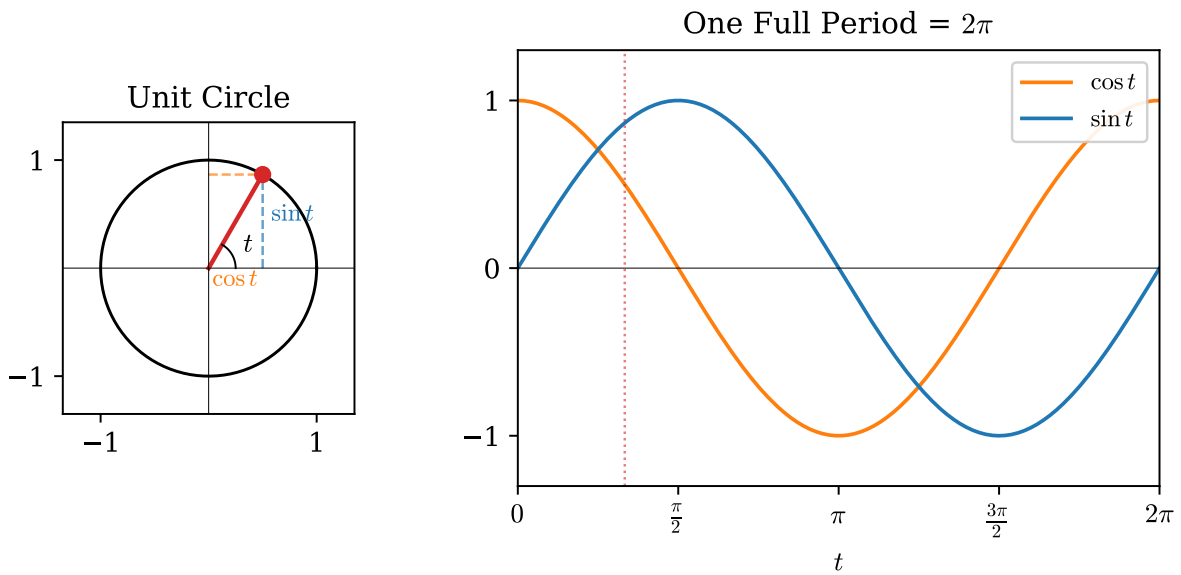

Figure 1: A point on the unit circle traces out $\cos t$ and $\sin t$. One full revolution takes 2π.

Whenever you write $\sin(\omega t)$ or $\cos(\omega t)$, π is hiding inside $\omega = 2\pi f$. The Fourier transform—our main tool for understanding signals—decomposes *any* signal into a sum of sinusoids, each carrying a factor of π. So π is not just about circles; it is the universal currency of "how often does this repeat?"

e: The Meter of Growth and Decay

Now imagine something very different: a capacitor discharging through a resistor. The voltage does not oscillate—it simply fades. You measure it every second and notice that it always loses the *same fraction*

of whatever is left. Half-life in radioactive decay works the same way: what matters is not how much you lose in absolute terms, but what *proportion* remains.

This "constant proportional change" is exactly what e captures. Formally, e is the unique number for which the function $f(x) = e^x$ is its own derivative:

$$\frac{d}{dx} e^x = e^x.$$

In plain language: the rate of change of e^x at any point equals its current value. That makes e^x the natural solution to the equation "the speed of change is proportional to the current state"—which is precisely the differential equation $\dot{x} = ax$.

When $a > 0$, $x(t) = e^{at}$ grows exponentially—like compound interest, or an unstable system running away. When $a < 0$, $x(t) = e^{at}$ decays exponentially—like that discharging capacitor, or a stable system returning to equilibrium.

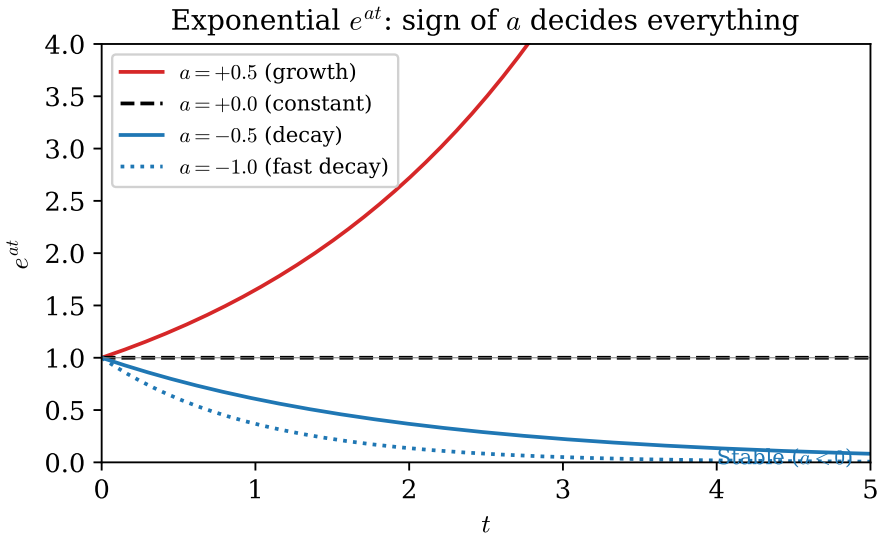

Figure 2: Exponential curves e^{at}. Positive a: unstable growth. Negative a: stable decay.

In control theory, the sign of a (or more precisely, the real part of a system's poles) decides everything about stability. A pole in the left half-plane means e^{at} with $a < 0$: the response dies out. A pole in the right half-plane means $a > 0$: the response blows up. This single idea—does the exponential grow or shrink?—is the foundation of stability analysis.

When They Join Forces: $e^{j\theta}$

The real magic happens when e and π meet. Euler's formula ties them together:

$$e^{j\theta} = \cos\theta + j\sin\theta,$$

where $j = \sqrt{-1}$. This single equation is arguably the most important identity in all of engineering mathematics. It says that the *complex exponential* is simultaneously an exponential and a sinusoid—growth/decay and oscillation, unified in one expression.

When a system has complex poles $\sigma \pm j\omega$, the time response is

$$e^{\sigma t}\big(\cos\omega t + j\sin\omega t\big).$$

The factor $e^{\sigma t}$ (the e part) controls the **envelope**—whether the oscillation is growing, shrinking, or staying constant. The factor $\cos\omega t$ (the π part, since $\omega = 2\pi f$) controls the **frequency**—how fast the system oscillates.

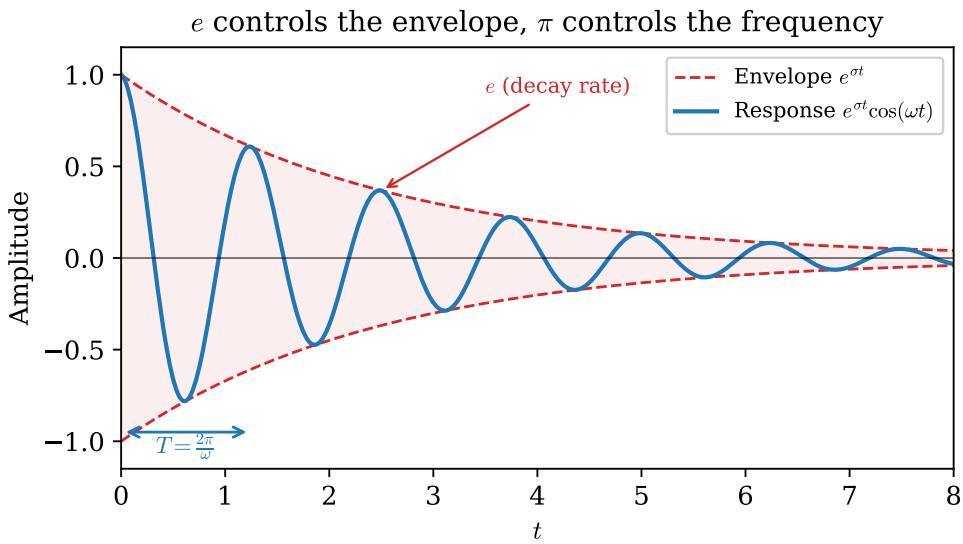

Figure 3: A damped oscillation $e^{\sigma t}\cos(\omega t)$: the envelope decays ($e$) while the signal oscillates (π).

In other words: e **tells you how fast,** π **tells you how often**. Every second-order system you will study in this book—from a mass-spring-damper to a PID-controlled motor—is just these two numbers doing their respective jobs.

With π and e in your toolkit, you are ready for everything that follows.

2 Digital Signal Processing

You might be wondering: "I signed up for control theory, why are we starting with signal processing?" Great question. Here is the answer: **garbage in, garbage out.**

Your controller is only as good as the data it receives. If your IMU (Inertial Measurement Unit—the small chip that combines accelerometers and gyroscopes to measure orientation and motion) reading is bouncing around like a caffeinated squirrel, no amount of PID tuning will save you. So before we learn to control systems, we learn to clean up their signals.

2.1 Understanding the Frequency Domain

In everyday life, we think about signals in the **time domain**—voltage changes over time, position changes over time. This is natural and intuitive. But it turns out that many problems become *much* easier if we look at signals from a different angle: the **frequency domain**.

The core idea is beautifully simple: *any signal can be decomposed into a sum of sinusoids of different frequencies.* Your noisy gyroscope reading? It is actually a clean low-frequency rotation signal plus a bunch of high-frequency noise signals all added together. If we can identify which frequencies are "signal" and which are "noise," we can surgically remove the noise.

Think of it like an audio equalizer on a music player. The bass, mids, and treble are all mixed together in the time-domain waveform. But the equalizer lets you boost or cut specific frequency ranges independently. That is exactly what we do with filters in control systems.

Key intuition: Slow, meaningful changes in your system (like a chassis turning) happen at low frequencies. Fast, meaningless changes (like electrical noise or vibration) happen at high frequencies. The frequency domain lets us tell them apart.

2.2 Laplace Transform and Fourier Transform

These two transforms are your tickets from the time domain to the frequency domain. They are closely related, but they serve slightly different purposes.

2.2.1 Laplace Transform

The Laplace transform takes a time-domain function $f(t)$ and converts it into a function of the complex variable s:

$$F(s) = \mathcal{L}\{f(t)\} = \int_0^\infty f(t)e^{-st}\,dt \tag{1}$$

where $s = \sigma + j\omega$ is a complex number. The real part σ captures exponential growth or decay, and the imaginary part $j\omega$ captures oscillation.

Why do we care? Because the Laplace transform turns differential equations into algebraic equations. Instead of solving

$$m\ddot{x} + b\dot{x} + kx = F(t),$$

(which requires calculus), we can solve

$$(ms^2 + bs + k)X(s) = F(s),$$

(which is just algebra). Derivatives become multiplication by s. Integration becomes division by s. Life becomes easier.

The most important Laplace transforms you will use daily:

Time domain	**s-domain**	**When you see it**
1 (step input)	$\frac{1}{s}$	Commanding a new setpoint
e^{-at}	$\frac{1}{s+a}$	First-order system response
$\sin(\omega t)$	$\frac{\omega}{s^2+\omega^2}$	Oscillation, vibration
$\dot{f}(t)$	$sF(s) - f(0)$	Derivative in the controller
$\int f(t)\,dt$	$\frac{F(s)}{s}$	Integral in the controller

Pro tip: You do not need to memorize the integral formula. You need to understand that s means "derivative" and $1/s$ means "integral." That single insight will carry you through 80% of control theory.

What is e^{-st} actually doing? The kernel $e^{-st} = e^{-\sigma t}e^{-j\omega t}$ is a "probe signal" that asks two questions simultaneously: (1) $e^{-\sigma t}$: "Does the signal grow or decay at rate σ?" (2) $e^{-j\omega t}$: "How much of frequency ω is present?" By integrating $f(t) \cdot e^{-st}$ over all time, we compute the "correlation" between the signal and every possible exponential-sinusoid. The result $F(s)$ is a map of the entire complex plane, telling us exactly how much of each exponential-oscillatory mode exists in the signal.

Common Laplace transform pairs — worked out:

The **unit step** $u(t) = 1$ for $t \geq 0$:

$$\mathcal{L}\{1\} = \int_0^\infty e^{-st}\,dt = \left[-\frac{e^{-st}}{s}\right]_0^\infty = \frac{1}{s}$$

The **exponential decay** e^{-at}:

$$\mathcal{L}\{e^{-at}\} = \int_0^\infty e^{-at}e^{-st}\,dt = \int_0^\infty e^{-(s+a)t}\,dt = \frac{1}{s+a}$$

Notice that $F(s) = \frac{1}{s+a}$ "blows up" when $s = -a$. That special value of s is called a **pole**—we will define it properly in a moment.

2.2.2 A First Look at Poles and Zeros

We just saw that $\mathcal{L}\{e^{-at}\} = \frac{1}{s+a}$. The denominator becomes zero at $s = -a$, which means $F(s)$ shoots to infinity there. That point is called a **pole**.

In general, for any rational function in s—and most **transfer functions** (the ratio of a system's output to its input in the s-domain, which we will derive properly in the next chapter) are rational—we write:

$$G(s) = \frac{N(s)}{D(s)} = \frac{(s-z_1)(s-z_2)\cdots}{(s-p_1)(s-p_2)\cdots}$$

- **Poles** (p_i): roots of the denominator $D(s)$. These are the values of s where $G(s) \to \infty$.
- **Zeros** (z_i): roots of the numerator $N(s)$. These are the values of s where $G(s) = 0$.

Why should you care right now? Because the pole location tells you the time-domain behavior directly:

- A pole at $s = -a$ (negative real) produces e^{-at}—exponential decay. The more negative a is, the faster the decay.
- A pole at $s = +a$ (positive real) produces e^{+at}—exponential growth. The system is unstable.
- A pair of complex poles $s = -\sigma \pm j\omega$ produces $e^{-\sigma t}\cos(\omega t)$—a decaying oscillation. (This should look familiar from the e and π section!)

In short: **poles tell you what exponential and oscillatory modes are hiding inside a system.** Every time we design a filter in this chapter, we are really choosing where to put poles and zeros. We will revisit poles and zeros more rigorously in Section 4.2, where we connect them to stability, speed, and overall system performance.

The **ramp** $t \cdot u(t)$:

$$\mathcal{L}\{t\} = \frac{1}{s^2}$$

Notice: multiplying by t in time corresponds to differentiating with respect to s and flipping the sign: $\mathcal{L}\{t \cdot f(t)\} = -\frac{d}{ds}F(s)$. Since $\mathcal{L}\{1\} = 1/s$, we get $\mathcal{L}\{t\} = -\frac{d}{ds}\frac{1}{s} = \frac{1}{s^2}$.

2.2.3 Fourier Transform

The Fourier transform is the Laplace transform's well-behaved cousin. It is what you get when you set $s = j\omega$ (i.e., $\sigma = 0$):

$$F(j\omega) = \int_{-\infty}^{\infty} f(t)e^{-j\omega t}\,dt \tag{2}$$

While the Laplace transform works with the full complex plane (and can handle unstable, growing signals), the Fourier transform lives strictly on the imaginary axis. It tells you: "How much of each frequency is present in this signal?"

When to use which: Use the **Laplace transform** for analyzing and designing control systems (transfer functions, stability). Use the **Fourier transform** for analyzing signals (noise, vibration, frequency content).

In practice, for digital systems we often use the **Discrete Fourier Transform (DFT)**, computed efficiently via the **Fast Fourier Transform (FFT)** algorithm. If you ever want to see what frequencies are hiding in your sensor data, just throw it into an FFT. MATLAB's `fft()` or Python's `numpy.fft.fft()` will do the job in one line.

2.2.4 Frequency Spectrum

When you apply the Fourier transform to a signal, you get its **frequency spectrum**—a plot showing the amplitude (and phase) of each frequency component.

Reading a frequency spectrum is like reading an X-ray of your signal:

- A tall peak at 50 Hz? That is probably power line interference.
- A broad hump below 10 Hz? That is likely your actual signal of interest.
- Random fuzz spread across all frequencies? That is white noise.

Practical example: Suppose your gimbal encoder gives a noisy velocity reading. You record 1 second of data, compute the FFT, and see a clean peak at 3 Hz (your gimbal is tracking a target moving at 3 Hz) plus a mess of noise above 50 Hz. Now you know: design a low-pass filter with a cutoff around 20–30 Hz, and your signal will be clean.

The frequency spectrum has two parts:

- **Magnitude spectrum** $|F(j\omega)|$: How strong each frequency component is.
- **Phase spectrum** $\angle F(j\omega)$: The time shift of each frequency component.

For filter design, we mostly care about the magnitude spectrum. Phase becomes important later when we analyze control loop stability (*phase margin*—how much extra phase lag the system can tolerate before it becomes unstable; see the Classical Control chapter).

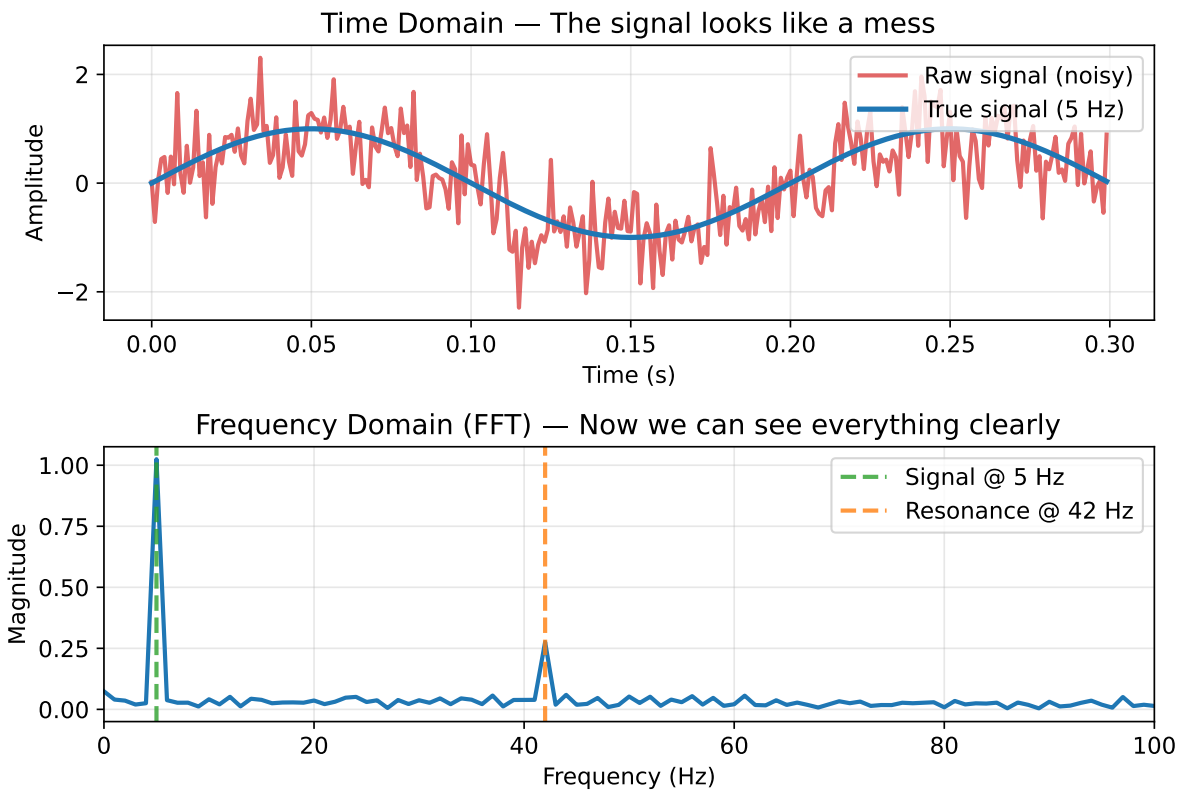

Figure 4: Top: a noisy time-domain signal looks hopeless. Bottom: the FFT instantly reveals a 5 Hz signal and a 42 Hz resonance hiding in the noise.

Case Study: Debugging a Vibrating Chassis with FFT

True story pattern from competition robotics: Your mecanum chassis drives fine at low speed but develops a nasty vibration at high speed. The whole frame buzzes. Everyone suspects a loose screw. You tighten everything. Still vibrates.

The diagnosis: Log the IMU accelerometer data for 5 seconds while driving at the problematic speed. Run an FFT in Python:

```
import numpy as np
freqs = np.fft.rfftfreq(len(data), d=1.0/sample_rate)
spectrum = np.abs(np.fft.rfft(data))
plt.plot(freqs, spectrum)
```

You see a massive spike at exactly 42 Hz. Interesting. You calculate: at high speed, the mecanum wheel rollers contact the ground at... about 42 Hz. The roller contact frequency matches a structural resonance of the chassis frame.

The fix options:

- Add a **notch filter** (band-reject) at 42 Hz in the control loop to prevent the controller from exciting the resonance.
- Add physical damping (rubber mounts) to shift the resonance frequency.
- Stiffen the frame to move the resonance above the operating range.

The lesson: Without the FFT, you would have spent hours tightening screws. With the FFT, the root cause was obvious in 10 seconds. **The frequency domain is a debugging superpower.** Whenever something oscillates and you do not know why, record the data and look at the spectrum.

2.3 Frequency-Based Filter Design

Now that we can see signals in the frequency domain, we can design filters to keep the frequencies we want and reject the ones we do not.

A filter is simply a system that has different gains at different frequencies. We describe this using the filter's **frequency response**—a plot of gain vs. frequency (called a **Bode magnitude plot**).

2.3.1 Low-Pass Filter

The most common filter in robotics. It passes low frequencies and attenuates high frequencies.

First-order low-pass filter:

$$H(s) = \frac{\omega_c}{s + \omega_c} \tag{3}$$

where $\omega_c = 2\pi f_c$ is the cutoff frequency in rad/s. At frequencies below ω_c, the gain is approximately 1 (signal passes through). Above ω_c, the gain rolls off at -20 dB/decade (signal gets attenuated).

In the time domain, this is equivalent to an exponential moving average:

$$y[n] = \alpha \cdot x[n] + (1 - \alpha) \cdot y[n - 1] \tag{4}$$

This follows from a backward Euler discretization of the continuous filter. Starting from $H(s) = \frac{\omega_c}{s+\omega_c}$, the corresponding differential equation is $\dot{y} + \omega_c y = \omega_c x$. Replacing $\dot{y}$ with the backward difference $\frac{y[n]-y[n-1]}{T_s}$ gives:

$$\frac{y[n] - y[n-1]}{T_s} + \omega_c\, y[n] = \omega_c\, x[n]$$

Collecting terms in $y[n]$:

$$y[n]\left(\frac{1}{T_s} + \omega_c\right) = \omega_c\, x[n] + \frac{y[n-1]}{T_s}$$

Dividing both sides by $\left(\frac{1}{T_s} + \omega_c\right)$ and defining $\alpha = \frac{\omega_c T_s}{1+\omega_c T_s} = \frac{T_s}{T_s + 1/\omega_c}$:

$$y[n] = \alpha \cdot x[n] + (1 - \alpha) \cdot y[n - 1]$$

where $\alpha = \frac{T_s}{T_s + \frac{1}{2\pi f_c}}$ (since $\omega_c = 2\pi f_c$) and T_s is the sampling period.

When to use: Smoothing noisy sensor readings (gyroscope, accelerometer, motor current). This is the filter you will use 90% of the time.

Gotcha: A low-pass filter introduces **phase lag**—the output is delayed relative to the input. The more aggressively you filter (lower cutoff), the more lag you introduce. In a fast control loop, this lag can destabilize your system. There is always a trade-off between smooth signals and fast response.

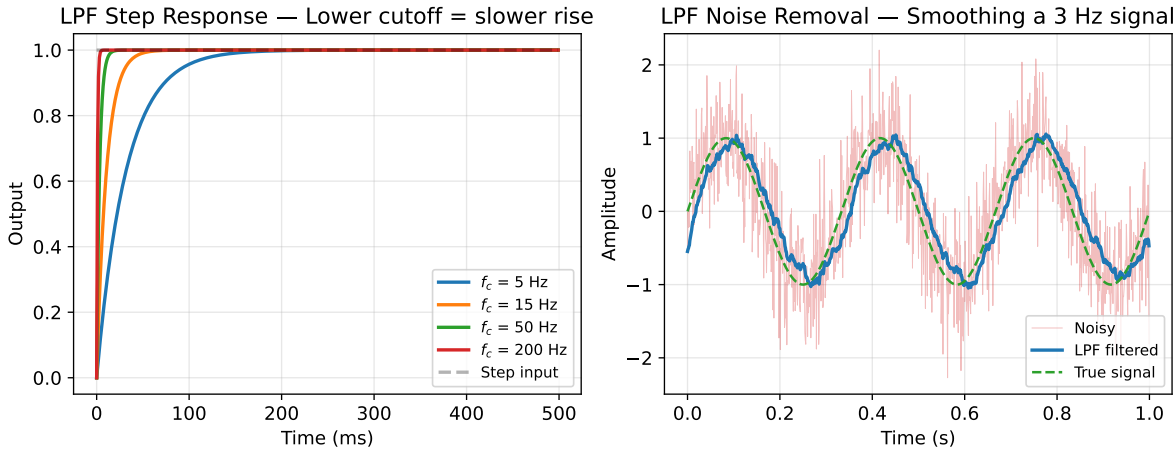

Figure 5: Left: LPF step response — lower cutoff means slower rise but smoother output. Right: filtering a noisy 3 Hz signal — the LPF recovers the true signal while removing high-frequency noise.

Case Study: Why a Low-Pass Filter Gives You "Soft Start" for Free

Suppose you command your motor to jump from 0 to 5000 RPM instantly (a step input). Without any filter, the setpoint changes discontinuously—the motor driver sees a sudden full-voltage command, the gears slam, the chassis lurches, and your teammates spill their coffee.

Now insert a first-order LPF on the setpoint: $r_{\text{filtered}}(s) = \frac{\omega_c}{s+\omega_c} \cdot R(s)$. What happens?

The step input in the s-domain is $R(s) = \frac{5000}{s}$. After the filter:

$$R_{\text{filtered}}(s) = \frac{5000\,\omega_c}{s(s+\omega_c)}$$

Taking the inverse Laplace transform:

$$r_{\text{filtered}}(t) = 5000\left(1 - e^{-\omega_c t}\right)$$

The step has been transformed into a **smooth exponential ramp**. The time constant is $\tau = 1/\omega_c$. After 3τ, the output reaches 95% of the target; after 5τ, it is at 99%.

Why this matters on your robot:

- **Mechanical protection:** Gears and belts have torque limits. A step command can strip teeth or snap belts. The LPF limits the rate of change, keeping forces within safe bounds.
- **Current limiting:** A sudden voltage step causes a huge inrush current ($I = V/R$ when back-EMF is still zero). The LPF ramps up the voltage gradually, keeping current manageable.
- **Tunable aggressiveness:** Want a faster ramp? Increase ω_c (higher cutoff). Want a gentler ramp? Decrease ω_c. One parameter controls the entire behavior.

Comparison with a linear ramp: You could also ramp the setpoint linearly (e.g., increase by 1000 RPM/s). But the LPF is better because it also smooths out *any* discontinuity in the setpoint—not just the initial step, but also sudden changes during operation. It is a "set it and forget it" solution.

The code is trivial:

```
setpoint_filtered += alpha * (setpoint_raw - setpoint_filtered);
```

One line of code. No lookup tables, no state machines. That is the beauty of the LPF as a soft-start mechanism.

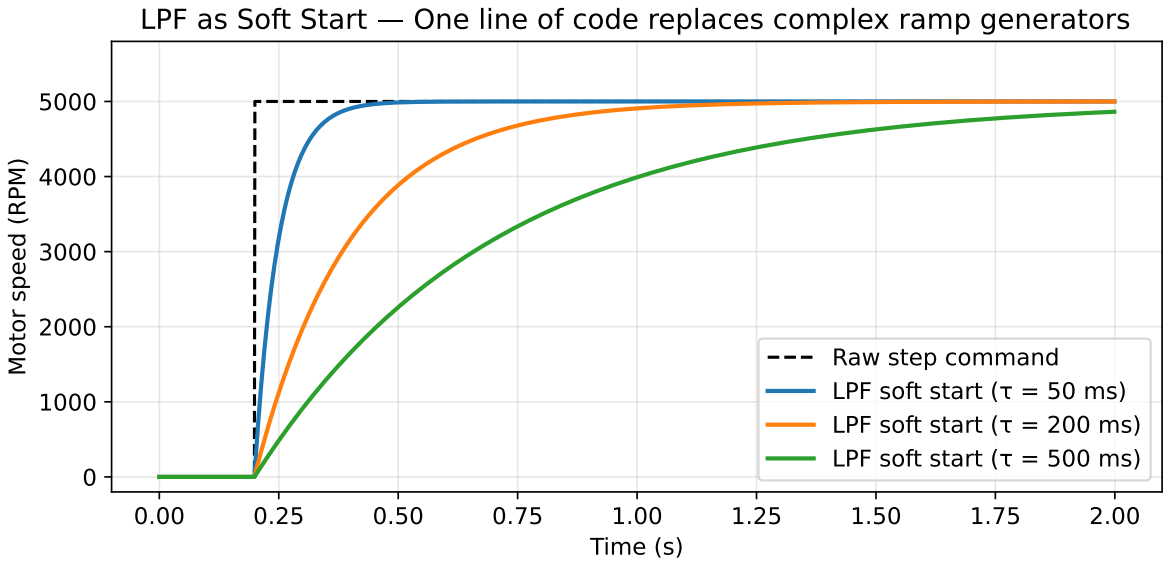

Figure 6: LPF soft start: a single parameter τ controls how aggressively the motor ramps up. The step command becomes a smooth exponential curve.

2.3.2 Band-Pass Filter

Passes a specific range of frequencies and attenuates everything else. Think of it as a low-pass and a high-pass filter combined.

$$H(s) = \frac{\frac{\omega_0}{Q} s}{s^2 + \frac{\omega_0}{Q} s + \omega_0^2} \tag{5}$$

where ω_0 is the center frequency and Q is the quality factor (higher Q = narrower band).

When to use: Extracting a specific frequency component from a signal. For example, if you know your target oscillates at a specific frequency and you want to track it while rejecting everything else.

In robotics, band-pass filters are less common than low-pass filters, but they are useful for vibration analysis and resonance detection.

2.3.3 High-Pass Filter

Passes high frequencies, attenuates low frequencies. It is the opposite of a low-pass filter.

First-order high-pass filter:

$$H(s) = \frac{s}{s + \omega_c} \tag{6}$$

When to use: Removing DC offset or slow drift from a signal. For example, if your accelerometer has a slowly drifting bias, a high-pass filter can remove it while keeping the fast acceleration data intact.

Practical note: A complementary filter (commonly used for IMU sensor fusion) is just a low-pass filter on one sensor and a high-pass filter on another, with their cutoff frequencies matched so the gains add up to 1. The accelerometer gets low-passed (trust it for slow/static orientation) and the gyroscope gets high-passed (trust it for fast rotations). Simple, elegant, and surprisingly effective.

2.3.4 Notch Filter (Band-Reject Filter)

A notch filter is the opposite of a band-pass filter: it *removes* a narrow band of frequencies while leaving everything else untouched. It is your surgical tool for killing a single problematic frequency—a structural resonance, a 50/60 Hz power-line hum, or a motor cogging frequency.

Second-order notch filter:

$$H(s) = \frac{s^2 + \omega_0^2}{s^2 + \frac{\omega_0}{Q}\, s + \omega_0^2} \tag{7}$$

where ω_0 is the frequency to reject and Q is the quality factor (higher Q = narrower notch). At $s = j\omega_0$, the numerator evaluates to zero, so the gain drops to zero at exactly ω_0. The denominator keeps the filter stable and controls the width of the rejection band.

Pole-zero interpretation: The numerator places a pair of *zeros* on the imaginary axis at $s = \pm j\omega_0$—these create the "notch" by driving the gain to zero. The denominator places a pair of *poles* slightly to the left of those zeros—these keep the filter stable and determine how quickly the gain recovers on either side. This is a textbook example of poles and zeros doing exactly what we described in Section 2.2: zeros kill frequencies, poles shape the surrounding response.

When to use:

- A structural resonance is causing oscillation in your control loop (see the mecanum wheel case study earlier in this chapter).
- Power-line interference (50 or 60 Hz) is corrupting a sensor signal.
- A motor's cogging torque at a known frequency is visible in the velocity measurement.

Practical tip: Getting ω_0 wrong by even a few Hz can make the notch miss the target entirely. Always identify the exact resonance frequency from an FFT of your sensor data *before* designing the filter. If the resonance frequency shifts with operating conditions (e.g., with speed or load), consider an **adaptive notch filter** that tracks ω_0 in real time.

Discrete-time implementation: Like all filters in this chapter, the notch filter is implemented as a biquad (second-order section). The bilinear transform with pre-warping at ω_0 converts $H(s)$ to $H(z)$ without shifting the notch frequency:

$$s = \frac{\omega_0}{\tan(\omega_0 T_s/2)} \cdot \frac{z-1}{z+1}.$$

2.3.5 Filter Type Selection and Parameter Tuning

Choosing the right filter comes down to answering three questions:

1. **What frequencies do I want to keep?** This determines the filter type (low-pass, high-pass, band-pass, or notch).
2. **Where is the boundary?** This determines the cutoff frequency f_c (or the rejection frequency f_0 for a notch).
3. **How sharp should the transition be?** This determines the filter order (or Q for a notch).

Rules of thumb:

- Start with a first-order low-pass filter. It is simple, stable, and good enough for most applications.
- Set the cutoff frequency to 5–10× the bandwidth of your signal of interest. Too low and you lose signal; too high and you keep noise.
- If a first-order filter is not enough, try second-order (Butterworth for flat passband, Chebyshev for sharper rolloff).
- **Never** design a filter without looking at the frequency spectrum of your actual data first. Guessing cutoff frequencies is a recipe for frustration.
- When in doubt, plot the filtered output alongside the raw signal and check if the result makes physical sense.

2.4 Z-Transform

Everything we have discussed so far with the Laplace transform lives in the continuous world. But your microcontroller is digital—it works with samples, not continuous signals. The **Z-transform** is the discrete-time equivalent of the Laplace transform.

The Z-transform of a discrete sequence $x[n]$ is:

$$X(z) = \sum_{n=0}^{\infty} x[n]z^{-n} \tag{8}$$

The key correspondence is:

Continuous (s-domain)	**Discrete (z-domain)**
s (derivative)	$\frac{z-1}{T_s}$ (forward difference)
$\frac{1}{s}$ (integral)	$\frac{T_s}{z-1}$ (accumulator)
Stable if $\mathrm{Re}(s) < 0$	Stable if $\lvert z\rvert < 1$
e^{-aT_s} in s-domain	z^{-1} is one-sample delay

Why do we care? Because when you implement a filter or controller on a microcontroller, you are working in the z-domain, whether you realize it or not. That simple line of code:

```
y = alpha * x + (1 - alpha) * y_prev;
```

is actually a z-domain transfer function:

$$H(z) = \frac{\alpha}{1 - (1-\alpha)z^{-1}}$$

Understanding the Z-transform helps you:

- Convert continuous-time filters/controllers to code (covered in Section 5).
- Analyze the stability of your discrete-time system.
- Understand why your controller might behave differently at different sampling rates.

For now, just remember: z^{-1} **means "one time step ago."** That is the single most important thing about the Z-transform.

Common Z-transform pairs:

Discrete signal	**z-domain**	**When you see it**
$\delta[n]$ (unit impulse)	1	Initial kick
$u[n]$ (unit step)	$\frac{z}{z-1}$	Step command
$a^n u[n]$ (geometric decay)	$\frac{z}{z-a}$	Discrete first-order response
$x[n-1]$ (one-step delay)	$z^{-1}X(z)$	Previous sample in code
$x[n] - x[n-1]$ (first difference)	$(1-z^{-1})X(z)$	Discrete derivative
$\sum_{k=0}^{n} x[k]$ (running sum)	$\frac{1}{1-z^{-1}}X(z)$	Discrete integral

The connection between s and z: The exact mapping is $z = e^{sT_s}$. The connection is simple: a continuous-time exponential $x(t) = e^{st}$, when sampled at intervals T_s, becomes the sequence $x[n] = e^{snT_s} = (e^{sT_s})^n$. In the Z-transform world, z^n is the discrete exponential. Matching terms gives $z = e^{sT_s}$.

A continuous pole at $s = -a$ maps to a discrete pole at $z = e^{-aT_s}$. This means:

- A stable continuous pole ($\mathrm{Re}(s) < 0$) maps to $\lvert z\rvert < 1$ (inside the unit circle). Stability is preserved.
- A faster continuous pole (more negative s) maps to a smaller $\lvert z\rvert$ (closer to the origin). Very fast poles map near $z = 0$.
- An oscillatory continuous pole ($s = \sigma \pm j\omega$) maps to $z = e^{\sigma T_s}e^{\pm j\omega T_s}$, which spirals inside (stable) or outside (unstable) the unit circle.

Reading a z-domain transfer function: Any z-domain transfer function can be directly translated to code by replacing z^{-1} with "previous value." For example:

$$H(z) = \frac{b_0 + b_1 z^{-1}}{1 + a_1 z^{-1}}$$

becomes:

```
y = b0 * x + b1 * x_prev - a1 * y_prev;
x_prev = x;  y_prev = y;
```

This is called a **difference equation**, and it is the final form of any digital filter or controller.

3 Sensor Calibration and Selection

Your filters from the previous chapter can remove noise, but they cannot fix a sensor that is fundamentally lying to you. A gyroscope with a 2/s bias will drift 120 per minute no matter how good your low-pass filter is. This chapter teaches you how to make your sensors honest before feeding their data into the control pipeline.

Calibration is not glamorous work. It is the kind of thing you do on a Saturday afternoon with a flat table and a protractor. But it is the single highest-return activity in robotics: ten minutes of calibration can eliminate errors that no amount of software can fix.

3.1 The Sensor Error Model

Every sensor you will ever use obeys roughly the same error model. The measurement y_m is related to the true value y by:

$$y_m = S\,y + b + n \tag{9}$$

where S is the **scale factor** (ideally 1, but in practice 0.98–1.02 for a typical MEMS sensor), b is the **bias** (a constant offset that persists even when the true value is zero), and n is **noise** (zero-mean random fluctuation).

Let us build intuition for each term. The **bias** is the most insidious: if your gyroscope reads 0.5/s when the robot is perfectly still, that half-degree-per-second adds up relentlessly. After one minute you have 30 of phantom rotation. After five minutes, 150. No filter can remove a DC offset from a rate sensor—you must measure it and subtract it.

The **scale factor** error is subtler. If $S = 1.03$ instead of 1.0, then a 90 turn reads as 92.7. You will not notice this during a quick test, but it accumulates over a full match.

The **noise** n is what you already know how to handle: low-pass filters, complementary filters, Kalman filters. It is the easiest error to deal with.

For multi-axis sensors (accelerometers, gyroscopes, magnetometers), the model generalizes to:

$$\mathbf{y}_m = \mathbf{S}\,\mathbf{y} + \mathbf{b} + \mathbf{n} \tag{10}$$

where $\mathbf{S}$ is now a 3×3 matrix. The off-diagonal elements of $\mathbf{S}$ capture **cross-axis coupling**—the x-axis accelerometer picking up a bit of y-axis acceleration due to imperfect mechanical alignment inside the chip. For cheap MEMS sensors, cross-axis coupling is typically 1–3% of the main axis, which is small but measurable.

Error source	Typical magnitude	How to detect	How to fix
Bias (accel.)	±50 mg	Static test: read at rest	Subtract mean
Bias (gyro.)	±1/s	Static test: read at rest	Subtract mean
Scale factor	1–3%	Compare to known reference	Solve calibration eqn.
Cross-axis	1–3%	6-position calibration	Full 3×3 matrix
Noise (accel.)	$\sim 200\ \mu g/\sqrt{\text{Hz}}$	Allan variance or PSD	Low-pass filter
Noise (gyro.)	$\sim 0.01/\text{s}/\sqrt{\text{Hz}}$	Allan variance or PSD	Low-pass filter
Temp. drift	varies	Test at different temps	Online compensation

Temperature drift deserves a mention. MEMS sensor biases shift with temperature—sometimes significantly. High-end IMUs include internal temperature sensors and apply polynomial compensation. For competition robots operating in a gym at roughly constant temperature, you can usually ignore this, but be aware that a robot calibrated indoors in winter may behave differently at an outdoor summer event.

3.2 IMU Calibration

3.2.1 Accelerometer: 6-Position Static Calibration

The idea is beautifully simple: gravity gives you a known, free, highly accurate reference vector. On Earth's surface, $\|\mathbf{g}\| = 9.81\ \text{m/s}^2$ to four significant figures. If you can orient your accelerometer so that gravity aligns with each axis in turn (positive and negative), you get six data points that fully constrain the 3×3 scale-and-cross-axis matrix $\mathbf{S}$ and the 3×1 bias vector $\mathbf{b}$.

Place your IMU on a flat surface in six orientations: $+x$ up, $-x$ up, $+y$ up, $-y$ up, $+z$ up, $-z$ up. In each orientation, collect a few hundred samples and average them. You do not need a precision jig—a flat table and a box with square edges work fine.

In each orientation i, the measurement model gives:

$$\mathbf{a}_{m,i} = \mathbf{S}\,\mathbf{g}_i + \mathbf{b} \tag{11}$$

where $\mathbf{g}_i$ is the known gravity vector in that orientation. For example, when $+z$ is up, $\mathbf{g}_i = [0, 0, 9.81]^T$. When $-x$ is up, $\mathbf{g}_i = [-9.81, 0, 0]^T$.

Stack all six equations into a single matrix equation. Define $\mathbf{A}_m \in \mathbb{R}^{6\times 3}$ as the matrix of measured accelerations (one row per orientation) and $\mathbf{G} \in \mathbb{R}^{6\times 3}$ as the corresponding known gravity vectors. Then:

$$\mathbf{A}_m = \mathbf{G}\,\mathbf{S}^T + \mathbf{1}\,\mathbf{b}^T \tag{12}$$

This is a standard linear system. Augment $\mathbf{G}$ with a column of ones to absorb the bias, and solve via least squares:

$$\begin{bmatrix}\mathbf{S}^T \\ \mathbf{b}^T\end{bmatrix} = \left(\tilde{\mathbf{G}}^T\tilde{\mathbf{G}}\right)^{-1}\tilde{\mathbf{G}}^T\mathbf{A}_m \tag{13}$$

where $\tilde{\mathbf{G}} = [\mathbf{G}\ \ \mathbf{1}]$ is the augmented matrix. Once you have $\mathbf{S}$ and $\mathbf{b}$, the corrected reading is:

$$\mathbf{a}_{\text{true}} = \mathbf{S}^{-1}(\mathbf{a}_m - \mathbf{b}) \tag{14}$$

Here is a minimal Python script that performs the entire calibration:

```
import numpy as np

# a_meas: (6, 3) array of mean accelerometer readings per orientation
# g_ref:  (6, 3) array of known gravity vectors per orientation
G_aug = np.hstack([g_ref, np.ones((6, 1))])          # (6, 4)
params, _, _, _ = np.linalg.lstsq(G_aug, a_meas, rcond=None)
S = params[:3, :].T                                   # (3, 3)
b = params[3, :]                                       # (3,)
S_inv = np.linalg.inv(S)

def calibrate(a_raw):
    return S_inv @ (a_raw - b)
```

After calibration, verify: rotate the IMU to arbitrary orientations and check that $\|\mathbf{a}_{\text{true}}\| \approx 9.81$ m/s^2. If the magnitude wanders by more than ± 0.05 m/s^2, something went wrong—most likely one of your six orientations was not level.

3.2.2 Gyroscope: Static Bias and Noise

Gyroscope calibration is simpler than accelerometer calibration because you only need one test condition: *be perfectly still.*

Place your IMU on a stable surface and record data for 60 seconds. The average reading on each axis is your bias. Subtract it in your code:

```
# During initialization (robot stationary):
gyro_bias = np.mean(gyro_samples, axis=0)    # average over ~60s
# During operation:
gyro_corrected = gyro_raw - gyro_bias
```

That is the entire calibration for most competition applications. Do this every time you power on, because MEMS gyroscope bias drifts slightly from session to session ("turn-on bias").

For a deeper characterization, the **Allan variance** tells you more about your sensor's noise structure. The Allan variance is computed by averaging the gyroscope output over progressively longer time windows τ and plotting the resulting standard deviation $\sigma(\tau)$ on a log-log plot.

The key features of the Allan variance plot are:

At short averaging times ($\tau < 1$ s), the curve slopes downward with slope $-1/2$. This region is dominated by **angle random walk** (white noise)—the fundamental noise floor of the sensor. The value of σ at $\tau = 1$ s gives you the angle random walk coefficient directly.

At intermediate averaging times, the curve reaches a minimum. This minimum is the **bias instability**—the best stability your sensor can achieve. For a $2 MEMS gyroscope, this is typically around 1–10/hr; for a fiber-optic gyroscope, it might be 0.001/hr.

At long averaging times ($\tau > 100$ s), the curve turns upward with slope $+1/2$. This is **rate random walk**—slow drift processes like temperature changes.

For competition robots, do not overthink this. Measure the static bias, subtract it, and move on. Allan variance analysis is useful when you are selecting between sensors or debugging long-term drift, but your matches are 3–5 minutes long, and a simple bias subtraction handles that timescale just fine.

3.2.3 Gyroscope Scale Factor

If you need to calibrate the gyroscope scale factor (not just bias), you need a known rotation rate. The poor-man's approach: mount the IMU on a turntable (a lazy Susan works), rotate it exactly 360, and check whether the integrated gyroscope output agrees. If the gyro says you rotated 354, your scale factor is $360/354 \approx 1.017$. This is rarely necessary for competition robots—MEMS gyroscope scale factors are typically within 1% of nominal—but it takes five minutes and eliminates one more source of error.

A more precise method uses a rate table that spins at a known, constant angular velocity. You compare the gyroscope reading to the known rate at several speeds, fit a line, and extract the slope (scale factor) and intercept (bias) simultaneously. This is overkill for most competition scenarios, but it is how the sensor manufacturers characterize their parts.

3.3 Encoder: Resolution vs Noise Tradeoff

3.3.1 Velocity Estimation from Position

Encoders give you position (angle), but your control loop almost always needs velocity. The naive approach is a finite difference:

$$v[n] = \frac{\theta[n] - \theta[n-1]}{T_s} \tag{15}$$

where T_s is the sample period. This works well at high speeds but falls apart at low speeds due to **quantization**. An encoder with N counts per revolution has a minimum detectable position change of $\Delta\theta = 2\pi/N$ radians. The minimum nonzero velocity you can measure is therefore:

$$v_{\min} = \frac{2\pi/N}{T_s} \tag{16}$$

Let us plug in numbers. A 1024-count encoder sampled at 1 kHz gives $v_{\min} = 2\pi/(1024 \times 0.001) \approx 6.14$ rad/s ≈ 58.6 RPM. Below this speed, the velocity estimate jumps between 0 and $v_{\min}$ with nothing in between. Your motor appears to be stuttering even though it is spinning smoothly.

There are several solutions, in order of increasing sophistication.

Reduce the sample rate for velocity. If you compute velocity every 10 ms instead of every 1 ms, $v_{\min}$ drops by a factor of 10. The downside is a 10× slower velocity update, which may be too slow for your inner control loop.

M/T method. Instead of counting edges per fixed time interval (M method), measure the time between edges (T method), or combine both (M/T method). At low speeds, where edges arrive slowly, measuring the time between them gives much finer velocity resolution. This is what hardware timer capture/compare peripherals are designed for. On an STM32, you configure a timer channel in "input capture" mode: every time an encoder edge arrives, the hardware latches the timer counter value. The time between two captures, divided into the angle per edge, gives you velocity with resolution limited only by the timer clock (typically 72–168 MHz). The M/T hybrid automatically switches between edge-counting at high speed and time-measurement at low speed, giving you the best of both worlds across the entire speed range.

Low-pass filter the finite difference. A simple first-order LPF on the velocity estimate smooths out the quantization jumps. This introduces a small delay but is often the best tradeoff for simplicity:

```
v_raw = (theta - theta_prev) / Ts;
v_filtered += alpha * (v_raw - v_filtered);
theta_prev = theta;
```

Observer-based estimation. A Luenberger observer or a Kalman filter can estimate velocity from noisy position measurements with better performance than a simple filter. The observer uses a model of the motor dynamics (typically a first-order model: $\dot{\omega} = (K_t i - B\omega)/J$) together with the position measurement to produce a smooth velocity estimate. We will develop this idea properly in Chapter 6 when we discuss state observers, so for now just know that this option exists and is the theoretically optimal approach.

3.3.2 Encoder Types and Selection

Not all encoders are created equal. **Incremental encoders** output two quadrature pulse trains (A and B channels) and optionally an index pulse (Z channel) that fires once per revolution. The quadrature encoding gives you direction as well as position, and most microcontroller timer peripherals can decode quadrature signals in hardware with zero CPU overhead. Incremental encoders lose their position when power is removed, so you need a homing procedure at startup.

Absolute encoders report the exact shaft angle at all times, even after a power cycle. They use either optical or magnetic sensing with a digital output (SPI, SSI, or analog). The AS5048A and AS5600 are popular magnetic absolute encoders in the robotics world. Their resolution is typically 12–14 bits (4096–16384 counts per revolution), which is excellent. The downside is that communication latency (a few microseconds for SPI) can matter in very fast control loops, and some cheap magnetic encoders have nonlinearity of ±1–2 LSB that varies with shaft angle.

For most competition robots, a 1024–4096 count incremental encoder on the motor shaft (before the gearbox) gives the best combination of resolution, speed, and simplicity. If you need absolute position (e.g., for a joint that must know its angle at startup without homing), use a magnetic absolute encoder on the output shaft.

3.4 Magnetometer Calibration

A magnetometer measures the Earth's magnetic field to determine heading. In a perfect world, if you rotate the sensor 360 in the horizontal plane, the x-y readings trace out a perfect circle centered at the origin. In reality, you get an offset ellipse, and this is where calibration comes in.

Hard iron distortion comes from permanent magnets or magnetized ferrous materials near the sensor (steel screws, speaker magnets, battery contacts). These produce a constant magnetic field that shifts the circle away from the origin. The fix is to find the center of the shifted circle and subtract it—that is your hard iron offset vector.

Soft iron distortion comes from magnetically permeable materials (soft steel, iron) that distort the Earth's field non-uniformly, stretching the circle into an ellipse. The fix is to fit an ellipse to the data and compute the transformation matrix that maps it back to a circle.

The practical calibration procedure: slowly rotate your robot 360 (or better, tumble it in all orientations), record the magnetometer data, fit an ellipse to the x-y scatter plot, and extract the offset and scaling. Many open-source tools do this automatically.

A word of caution: magnetometers are **unreliable indoors**. Steel structural beams, reinforced concrete, electrical wiring, and nearby motors all distort the magnetic field in unpredictable, location-dependent ways. If your competition is in a building with a steel frame, your magnetometer heading may vary by 10–30 depending on where the robot is on the field. Use the magnetometer for coarse heading initialization, but do not trust it for continuous navigation indoors. Gyroscope integration (with periodic corrections) is almost always more reliable for short-duration matches.

3.5 Current Sensor Calibration

If you are doing field-oriented control (FOC) of a BLDC motor, your current measurements feed directly into the innermost control loop. Errors here propagate everywhere.

Offset calibration: Before the motor starts, with zero current flowing, read the current sensor for a few hundred samples and average. This is your zero-current offset. Subtract it from all subsequent readings. On many motor driver boards, the current sensor is a shunt resistor plus an op-amp with a mid-supply reference voltage, so the "zero" reading is around $V_{cc}/2$, not actually 0 V.

Bandwidth matters. Your current control loop typically runs at 10–20 kHz. If your current sensor has a bandwidth of only 5 kHz, the sensor itself acts as a low-pass filter with significant phase lag at the frequencies where your controller is trying to act. This phase lag reduces your phase margin and can destabilize the current loop. Check your sensor datasheet: the bandwidth should be at least 5× higher than your current loop frequency. Hall-effect current sensors (like the ACS712) are typically limited to a few hundred kHz, which is fine. Shunt-resistor-based sensing with a fast op-amp can reach MHz bandwidth, which is more than enough.

3.6 Sensor Fusion Architecture

Once your sensors are calibrated, the next question is how to combine them. There are two fundamentally different approaches.

Measurement-level fusion blends the raw (calibrated) sensor data directly. The complementary filter from Chapter 2 is the classic example: low-pass the accelerometer, high-pass the gyroscope, add them together. This is simple, lightweight, and works beautifully when you have exactly two sensors with complementary noise characteristics.

State-level fusion estimates the system's internal state (position, velocity, orientation) from all sensor measurements simultaneously, using a dynamic model of the system. The Kalman filter is the gold standard here. It handles any number of sensors, accounts for their individual noise characteristics, and produces statistically optimal estimates under certain assumptions.

How do you choose? Here is a simple decision rule:

If you have 1–2 sensors with well-understood, complementary noise profiles, and the relationship between sensors and the state of interest is simple, use a complementary filter. It is five lines of code, runs in microseconds, and is easy to tune.

If you have 3 or more sensors, or the relationship between measurements and states is nonlinear (e.g., GPS + IMU + magnetometer for full pose estimation), or you want provably optimal performance, use the Extended Kalman Filter from Chapter 6. The EKF handles all of these cases in a unified framework. The cost is more code, more tuning, and more computation—but modern microcontrollers handle it easily.

Most competition robots land in the complementary filter camp. An accelerometer-gyroscope complementary filter for tilt estimation, perhaps augmented with encoder odometry for position. Save the EKF for when you genuinely need it.

A useful mental model: think of sensor fusion as *spectral division of labor*. The accelerometer is trustworthy at low frequencies (it sees gravity, which is constant) but contaminated by vibration at high frequencies. The gyroscope is trustworthy at high frequencies (it responds instantly to rotation) but drifts at low frequencies (bias accumulation). A complementary filter simply assigns each sensor to the frequency range where it excels. The Kalman filter does the same thing, but it computes the optimal crossover frequency automatically from the noise statistics.

3.7 Choosing the Right Sensor

Before you calibrate, you need to pick the right sensor in the first place. Here is a compact decision guide for the most common competition-robot sensing needs.

Orientation (tilt/heading): A 6-axis IMU (accelerometer + gyroscope) covers most needs. Add a magnetometer only if you need absolute heading and your arena does not have large steel structures nearby. The BMI088, ICM-42688, and MPU-6500 are popular choices in order of increasing quality.

Wheel velocity: Use a quadrature encoder with at least $N = 4096$ counts per revolution if you need smooth velocity down to ~ 5 RPM at 1 kHz sampling. If your motor has a gearbox, count on the motor shaft (before the gearbox) for higher resolution.

Distance/ranging: Time-of-flight (ToF) sensors like the VL53L1X give millimeter resolution up to ~ 4 m. They are small, cheap, and easy to use over I2C. For longer range, a 2D lidar (e.g., RPLIDAR A1) gives 360 scans at 5–10 Hz up to 12 m.

Current sensing for FOC: Use inline shunt resistors with a fast op-amp for best bandwidth. Hall-effect sensors (ACS712, ACS714) are easier to integrate but have lower bandwidth and higher noise. At a minimum, your current sensor bandwidth should exceed 5× the current loop frequency.

The general principle: buy the best sensor you can afford, then calibrate it carefully. A well-calibrated $3 sensor will outperform a poorly calibrated $30 sensor every time.

3.8 Practical: Verifying Calibration Quality

Calibration is not a "set and forget" operation. You need to verify that it actually worked, and you need to log the right data so that you can debug problems later.

Accelerometer sanity check: After calibration, hold the IMU in several arbitrary orientations. Compute $\|\mathbf{a}_{\text{cal}}\|$. It should be 9.81 ± 0.05 m/s^2 in every orientation. If it deviates, your calibration matrix is wrong—probably because one of the six positions was not level during the calibration procedure.

Gyroscope sanity check: With the robot stationary, integrate the calibrated gyroscope rate over 60 seconds. The accumulated angle should stay within ± 1. If it drifts further, your bias estimate is off. Re-run the static calibration, making sure the robot is truly motionless (no fans causing vibration, no one bumping the table).

Magnetometer sanity check: After hard/soft iron calibration, rotate the sensor 360 in the horizontal plane and plot the calibrated x-y data. It should form a circle centered near the origin, not an ellipse or an offset blob.

What to log: Always log both raw and calibrated data with timestamps. When something goes wrong on the field, you need the raw data to determine whether the problem is in the sensor or in the calibration. If you only log calibrated data, you cannot tell the difference. Logging is cheap—a few kilobytes per second. The debugging time it saves is enormous.

Encoder sanity check: Spin the wheel exactly one full revolution by hand (use a mark on the wheel). The encoder should report exactly N counts (or $4N$ in quadrature mode, depending on your counting configuration). If it consistently reads $N - 1$ or $N + 1$, you may have electrical noise causing spurious edges—add a small capacitor across the encoder output lines, or reduce the cable length.

Current sensor sanity check: With the motor disconnected, the current reading should be zero (after offset subtraction). Then command a known, steady current and verify with a multimeter or a clamp meter. If the sensor reads 1.0 A but the multimeter says 0.95 A, your gain calibration is off by 5%.

When to recalibrate: Recalibrate the gyroscope bias every time you power on—it shifts by a few tenths of a degree per second between sessions. Accelerometer and magnetometer calibrations are stable unless you change the mounting or add new ferrous components to the chassis. Current sensor offsets should be measured at every startup, before the motor is energized.

Case Study: My Heading Drifts 30 in 5 Minutes

The problem: a competition robot uses a magnetometer for heading estimation. During testing, the heading slowly drifts 30 over a 5-minute run, despite the robot returning to its starting orientation.

The diagnosis starts with plotting the raw magnetometer x-y data while rotating the robot 360. Instead of a circle centered at the origin, the data forms a circle centered at roughly $(150, -80)$ μT. This is classic hard iron distortion—the steel chassis and the neodymium magnets in the drive motors are producing a constant magnetic offset.

The fix: rotate the robot through a full 360 turn (slowly, on a flat surface), record the magnetometer data, compute the center of the circle (c_x, c_y), and subtract it from all future readings:

```
mag_cal_x = mag_raw_x - cx;
mag_cal_y = mag_raw_y - cy;
heading = atan2(mag_cal_y, mag_cal_x);
```

After this three-line fix, the heading error over 5 minutes drops from 30 to less than 3. The remaining error is from soft iron distortion and indoor magnetic field non-uniformity—acceptable for a competition robot that also has gyroscope integration as a backup.

The lesson: always visualize your magnetometer data as a scatter plot before trusting it. If the circle is not centered at the origin, you have a hard iron problem. If it is an ellipse instead of a circle, you also have a soft iron problem.

Case Study: Velocity Goes Crazy Below 10 RPM

The problem: a wheeled robot uses a 1024-count quadrature encoder for wheel velocity feedback.

The PID velocity controller works well above 60 RPM, but below about 10 RPM the motor audibly stutters and the velocity plot looks like a square wave oscillating between 0 and some minimum value.

The analysis: at 1 kHz sampling, the minimum detectable velocity for a 1024-count encoder is:

$$v_{\min} = \frac{2\pi}{1024 \times 0.001 \text{ s}} \approx 6.14 \text{ rad/s} \approx 58.6 \text{ RPM}$$

Below this speed, the position difference $\theta[n] - \theta[n-1]$ is almost always 0 (no edge counted), with occasional jumps of ± 1 count. The finite-difference velocity oscillates between 0 and $v_{\min}$. The PID controller sees "zero velocity" and cranks up the output; then it sees "$v_{\min}$" and backs off. This creates the audible stuttering.

The fix combines two changes. First, compute velocity at a lower rate—every 10 ms instead of every 1 ms—which reduces $v_{\min}$ to 5.86 RPM. Second, apply a first-order low-pass filter ($\alpha = 0.1$) to smooth the remaining quantization steps:

```
// Every 10 ms:
v_raw = (encoder_count - prev_count) * (2*PI/1024) / 0.01;
v_filt += 0.1 * (v_raw - v_filt);
prev_count = encoder_count;
```

After this change, the motor spins smoothly down to about 3 RPM. For even lower speeds, the team later switched to the M/T method using the STM32 timer's input capture mode, which measures the time between encoder edges and provides fine velocity resolution all the way down to near-zero speed.

The lesson: always compute $v_{\min}$ for your encoder-and-sample-rate combination *before* building the robot. If $v_{\min}$ is above the minimum speed you need, either use a higher-resolution encoder, reduce the velocity sample rate, or implement M/T measurement.

3.9 Calibration Checklist

Here is the calibration sequence we recommend running every time you bring up a new robot or make significant hardware changes. It takes about 30 minutes and saves hours of debugging later.

Step	Sensor	Procedure
1	Current sensor	Zero-current offset (motor disconnected)
2	Gyroscope	60-second static bias (robot stationary on flat surface)
3	Accelerometer	6-position static calibration (if first time or remount)
4	Magnetometer	360 rotation for hard/soft iron (if used)
5	Encoder	One-revolution count check (verify N counts exactly)
6	All sensors	Log raw + calibrated data, run sanity checks

Steps 1 and 2 should be done at every power-on. Steps 3–5 only need to be repeated when hardware changes. Step 6 should become a habit—the five minutes you spend verifying calibration will save you from chasing phantom bugs during the competition.

The bottom line of this entire chapter is simple: *calibrate before you filter, filter before you control.* A sensor that tells the truth, even noisily, is infinitely more useful than one that lies quietly. Get the truth first; the rest of this book will teach you what to do with it.

4 System Description

Before you can control something, you need to understand what it *does*. A system is anything that takes an input and produces an output. Your DC brushed motor takes voltage and produces angular velocity. Your chassis takes wheel speeds and produces a trajectory. Your gimbal takes motor torques and produces angles.

The art of control engineering begins with writing down a mathematical description of these input-output relationships. This is called **modeling**, and it is simultaneously the most important and most underappreciated step in the process.

A word of wisdom: A mediocre controller on a good model will outperform a perfect controller on a bad model, every single time. Spend time understanding your system.

We describe systems as either **SISO** (Single-Input, Single-Output) or **MIMO** (Multiple-Input, Multiple-Output). A single motor is SISO. A holonomic chassis with four wheels is MIMO. We will start with SISO because it is simpler, and generalize later.

4.1 Mathematical Models of Systems

There are three main ways to describe a system mathematically, each with its own strengths.

4.1.1 Differential Equations

The most fundamental description. We use physics (Newton's laws, Kirchhoff's laws, etc.) to write equations relating inputs, outputs, and their derivatives.

Example: DC brushed motor

A DC brushed motor is the most common actuator in robotics competitions. Here is its simplified model:

Electrical equation (Kirchhoff's voltage law):

$$V = L\frac{di}{dt} + Ri + K_e\omega \tag{17}$$

Mechanical equation (Newton's second law for rotation):

$$J\frac{d\omega}{dt} = K_t i - b\omega - \tau_{\text{load}} \tag{18}$$

where:

- V: input voltage, i: armature current
- L: inductance, R: resistance
- K_e: back-EMF constant, K_t: torque constant
- J: moment of inertia, b: viscous friction
- ω: angular velocity, τ_{load}: load torque

These are two coupled first-order differential equations. In practice, the electrical dynamics ($L\frac{di}{dt}$) are much faster than the mechanical dynamics, so we often simplify by setting $L \approx 0$, giving us:

$$J\frac{d\omega}{dt} = \frac{K_t}{R}(V - K_e\omega) - b\omega - \tau_{\text{load}} \tag{19}$$

This is a first-order ODE relating input voltage V to output speed ω. Now we can analyze and control it.

4.1.2 Transfer Functions

Take the Laplace transform of the differential equation (assuming zero initial conditions), and you get a **transfer function**: the ratio of output to input in the s-domain.

Let us sketch the derivation. From the simplified equation ($L \approx 0$), Ohm's law gives $I = (V - K_e\omega)/R$. Substituting into the mechanical equation $J\frac{d\omega}{dt} = K_t I - b\omega$:

$$J\frac{d\omega}{dt} = \frac{K_t}{R}(V - K_e\omega) - b\omega$$

Taking the Laplace transform (zero initial conditions): $(Js + b + K_t K_e/R)\,\Omega(s) = (K_t/R)\,V(s)$, hence:

For our simplified DC brushed motor:

$$G(s) = \frac{\Omega(s)}{V(s)} = \frac{K_t/R}{Js + (b + K_t K_e/R)} \tag{20}$$

This can be rewritten in standard first-order form:

$$G(s) = \frac{K}{\tau s + 1} \tag{21}$$

where K is the DC gain (steady-state speed per volt) and τ is the time constant (how fast the motor reaches steady state).

Why transfer functions are great. They are compact—one fraction describes the entire system. Cascading systems is just multiplication: $G_{\text{total}}(s) = G_1(s) \cdot G_2(s)$. Stability analysis is straightforward: check the roots of the denominator (poles). And the frequency response is obtained by simply substituting $s = j\omega$.

Why transfer functions are limited. They only work for linear, time-invariant (LTI) systems. They assume zero initial conditions. They are naturally SISO—extending to MIMO requires transfer function matrices, which get unwieldy. And they hide the internal states of the system. This last point is why we need the state-space representation.

4.1.3 State-Space Representation

The state-space representation describes a system as a set of first-order differential equations in matrix form:

$$\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u} \quad \text{(state equation)} \tag{22}$$

$$\mathbf{y} = C\mathbf{x} + D\mathbf{u} \quad \text{(output equation)} \tag{23}$$

where:
- $\mathbf{x}$: state vector (the internal variables that describe the system's condition)
- $\mathbf{u}$: input vector
- $\mathbf{y}$: output vector
- A: system matrix, B: input matrix, C: output matrix, D: feedthrough matrix

What each matrix physically means:

- A (system/dynamics matrix): Describes how the states evolve on their own, without any input. If you disconnect the motor ($\mathbf{u} = 0$), the system evolves as $\dot{\mathbf{x}} = A\mathbf{x}$. The eigenvalues of A determine the natural behavior: stable decay, oscillation, or divergence. Entry A_{ij} tells you how state j influences the rate of change of state i.

- B (input matrix): Describes how the inputs affect the states. Entry B_{ij} tells you how input j influences the rate of change of state i. If a row of B is all zeros, that state cannot be directly driven by any input—it can only be influenced indirectly through coupling in A.

- C (output matrix): Describes which states you can measure. Entry C_{ij} tells you how much state j contributes to measurement i. If $C = [1\ 0\ \ldots\ 0]$, you are measuring only the first state.

- D (feedthrough matrix): Describes direct input-to-output coupling that bypasses the dynamics. In most physical systems, $D = 0$ because inputs must go through the dynamics before affecting the output. $D \neq 0$ appears when there is a direct algebraic path from input to output (e.g., a resistive voltage divider).

Example: DC brushed motor in state-space

Choose states $\mathbf{x} = \begin{bmatrix} \omega \\ i \end{bmatrix}$ (angular velocity and current):

$$\begin{bmatrix} \dot{\omega} \\ \dot{i} \end{bmatrix} = \begin{bmatrix} -b/J & K_t/J \\ -K_e/L & -R/L \end{bmatrix} \begin{bmatrix} \omega \\ i \end{bmatrix} + \begin{bmatrix} 0 \\ 1/L \end{bmatrix} V \tag{24}$$

$$y = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} \omega \\ i \end{bmatrix} \tag{25}$$

Why state-space is powerful:

- Handles MIMO systems naturally—just make the matrices bigger.
- Exposes internal states, which we can estimate (observers) and control (state feedback).
- Works seamlessly with modern control techniques (LQR, MPC, Kalman filter).
- Easily extended to nonlinear systems (replace $A\mathbf{x}$ with $f(\mathbf{x})$).

The trade-off: State-space is more powerful but more abstract. Transfer functions give you better physical intuition for SISO systems. Good engineers are fluent in both.

4.2 System Analysis via Transfer Functions

Given a transfer function $G(s)$, we can extract a wealth of information about the system.

Poles and zeros revisited. We introduced poles and zeros informally in the DSP chapter. Now let us be precise. Write $G(s) = \frac{N(s)}{D(s)}$. The roots of $N(s)$ are the **zeros**; the roots of $D(s)$ are the **poles**.

- **Poles determine stability.** If all poles have negative real parts (i.e., they are in the left half of the complex plane), the system is stable. If any pole has a positive real part, the system is unstable and will blow up. Poles on the imaginary axis mean the system is marginally stable (perpetual oscillation).
- **Poles determine speed.** Poles farther to the left mean faster dynamics. The *dominant poles* (closest to the imaginary axis) determine the overall system speed.
- **Complex poles mean oscillation.** A pair of complex conjugate poles $s = -\sigma \pm j\omega_d$ produces oscillatory behavior. The real part σ determines how fast the oscillation decays; the imaginary part ω_d determines the oscillation frequency.
- **Zeros shape the response.** Zeros can cancel poles, reshape the transient response, or cause non-minimum phase behavior (the system initially moves in the wrong direction).

Quick trick: determining system order by counting integrations. Before diving into the math, here is an intuitive shortcut. The order of a system equals the number of times you must integrate the input variable to arrive at the output variable in a physical sense. For a DC brushed motor controlled by voltage:

- Voltage $\rightarrow$ current (one integration through the inductance $L\frac{di}{dt} = V - \ldots$) $\rightarrow$ that is one integrator.
- Current $\rightarrow$ angular velocity (one integration through the inertia $J\frac{d\omega}{dt} = K_t i - \ldots$) $\rightarrow$ that is a second integrator.

Two integrations from voltage to velocity $\Rightarrow$ second-order system. If we simplify $L \approx 0$ (current responds instantly), we remove one integration, leaving a first-order system. If the output is *position* instead of velocity, add one more integration ($\theta = \int \omega \, dt$), making it third-order (or second-order with the $L \approx 0$ simplification). This counting trick works for any physical system: springs, thermal systems, fluid tanks—count the independent energy storage elements (inductors, capacitors, masses, springs), and you have the system order.

Standard second-order system. Many physical systems can be approximated as:

$$G(s) = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2} \tag{26}$$

where ω_n is the natural frequency and ζ is the damping ratio. The damping ratio tells you everything about the transient response:

ζ	**Behavior**
$\zeta > 1$	Overdamped: slow, no oscillation
$\zeta = 1$	Critically damped: fastest without oscillation
$0 < \zeta < 1$	Underdamped: oscillation with decay
$\zeta = 0$	Undamped: perpetual oscillation
$\zeta < 0$	Unstable: growing oscillation

For most robotics applications, you want ζ between 0.5 and 0.8—fast response with acceptable overshoot.

Useful formulas for the underdamped case ($0 < \zeta < 1$):

The step response is:

$$y(t) = 1 - \frac{e^{-\zeta\omega_n t}}{\sqrt{1-\zeta^2}} \sin(\omega_d t + \phi), \quad \omega_d = \omega_n\sqrt{1-\zeta^2}, \quad \phi = \arccos(\zeta) \tag{27}$$

where ω_d is the **damped natural frequency** — the actual frequency at which the system oscillates (always less than ω_n).

To derive this, we take the Laplace transform of the step response: $Y(s) = \frac{\omega_n^2}{s(s^2+2\zeta\omega_n s+\omega_n^2)}$. Partial fraction expansion with complex poles at $s = -\zeta\omega_n \pm j\omega_d$ (where $\omega_d = \omega_n\sqrt{1-\zeta^2}$) and inverse Laplace transform yields the result. The overshoot occurs at the first peak, $t_p = \pi/\omega_d$; substituting into $y(t)$ gives M_p.

From this, we can derive closed-form expressions for the performance metrics:

$$\text{Percent overshoot:} \quad M_p = e^{-\pi\zeta/\sqrt{1-\zeta^2}} \times 100\% \tag{28}$$

$$\text{Peak time:} \quad t_p = \frac{\pi}{\omega_d} = \frac{\pi}{\omega_n\sqrt{1-\zeta^2}} \tag{29}$$

$$\text{Settling time (2\%):} \quad t_s \approx \frac{4}{\zeta\omega_n} \tag{30}$$

$$\text{Rise time (approx):} \quad t_r \approx \frac{1.8}{\omega_n} \tag{31}$$

What these formulas tell you:

- Overshoot depends *only* on ζ, not on ω_n. A faster system (ω_n higher) overshoots by the same percentage—it just does it faster.
- Settling time depends on the product $\zeta\omega_n$ — this is the real part of the poles. Faster settling requires poles farther left in the complex plane.
- There is a direct trade-off: lower ζ gives faster rise time but more overshoot. Higher ζ gives less overshoot but slower response.
- For $\zeta = 0.707$ ($1/\sqrt{2}$): overshoot $\approx 4.3\%$, which is often cited as the "optimal" damping for the best compromise between speed and smoothness.

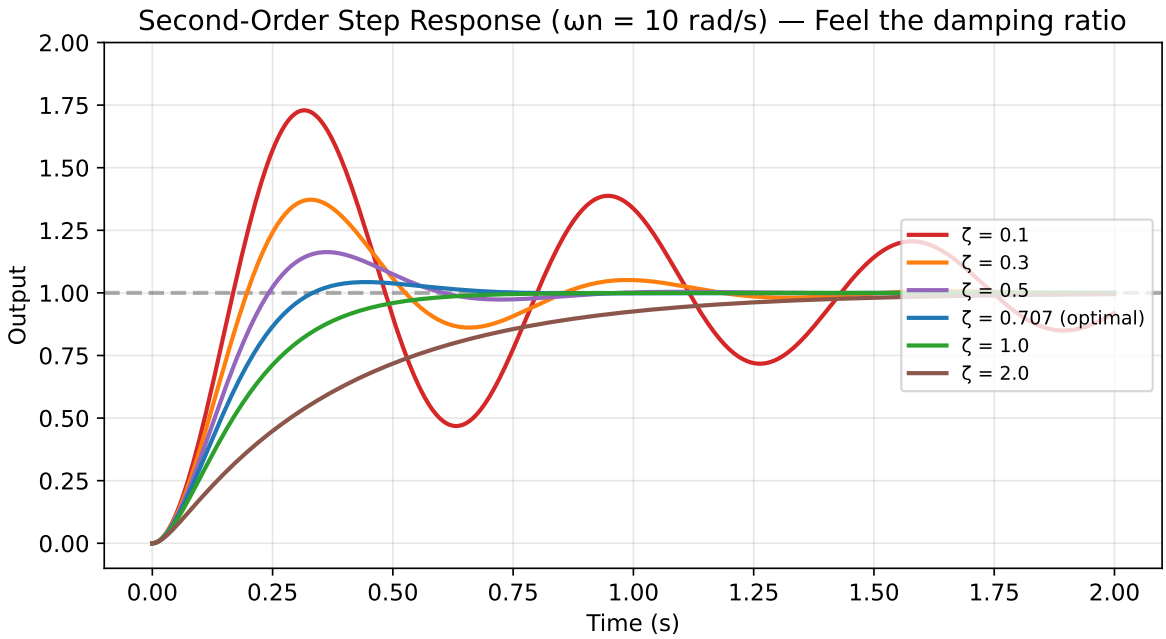

Figure 7: Second-order step response for different damping ratios. Notice how $\zeta = 0.707$ (blue) gives the fastest rise without significant overshoot — this is the "sweet spot" for most control applications.

Case Study: The Spring-Mass-Damper — A System You Can Feel

Before we get more abstract, let us build intuition with the simplest second-order system that you have literally felt with your own body: a car's suspension.

A car hitting a bump is a textbook spring-mass-damper: the car body (mass m) bounces on the suspension spring (stiffness k) while the shock absorber (damping coefficient b) dissipates energy.

$$m\ddot{x} + b\dot{x} + kx = F(t)$$

Transfer function: $G(s) = \frac{1}{ms^2+bs+k} = \frac{1/m}{s^2+\frac{b}{m}s+\frac{k}{m}}$

Natural frequency: $\omega_n = \sqrt{k/m}$. Damping ratio: $\zeta = \frac{b}{2\sqrt{km}}$. These follow from matching coefficients: $ms^2 + bs + k = m(s^2 + 2\zeta\omega_n s + \omega_n^2)$ gives $\omega_n^2 = k/m$ and $2\zeta\omega_n = b/m$, hence $\zeta = b/(2m\omega_n) = b/(2\sqrt{km})$.

Now feel the math:

- **No shock absorber ($\zeta = 0$):** You hit a bump and bounce forever. This is a trampoline, not a car. Undamped.
- **Worn-out shocks ($\zeta = 0.2$):** You hit a bump and bounce 5–6 times before settling. Passengers feel seasick. Underdamped.
- **Good shocks ($\zeta = 0.5$–0.7):** You hit a bump, bounce once or twice gently, and settle quickly. Comfortable. This is the sweet spot.
- **Off-road truck shocks ($\zeta = 1$):** You hit a bump and slowly return to center with no bounce at all. Safe but stiff. Critically damped.
- **Shocks filled with concrete ($\zeta = 3$):** You hit a bump and it takes forever to return to center. You feel every pebble. Overdamped.

The connection to your robot: Your gimbal, your chassis, your robotic arm—they all have natural frequencies and damping. When your gimbal oscillates after a step command, it is underdamped (ζ too low). When it responds sluggishly, it is overdamped (ζ too high). The PID controller's job (Section 4) is essentially to *change the effective ζ and ω_n* of your system by adding virtual springs (P), virtual dampers (D), and memory (I).

Why ω_n matters: A heavy gimbal with a weak motor has a low ω_n—it physically cannot respond fast. No amount of PID tuning can make a system respond faster than its natural frequency

allows. If you need more speed, you need a bigger motor (increase k) or a lighter load (decrease m). This is why hardware design and control design must go hand in hand.

4.3 System Analysis via State-Space

For the state-space model $\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u}$, system behavior is determined by the **eigenvalues of** A.

Here is the beautiful connection: *the eigenvalues of A are exactly the poles of the transfer function.* So everything we said about poles applies here too. The eigenvalue analysis just extends naturally to MIMO systems where transfer functions become awkward.

Matrix exponential. The free response (no input) of the state-space system is:

$$\mathbf{x}(t) = e^{At}\mathbf{x}(0) \tag{32}$$

The matrix exponential e^{At} encodes all the dynamics. If you diagonalize A (when possible), you get:

$$e^{At} = P \begin{bmatrix} e^{\lambda_1 t} & & \\ & \ddots & \\ & & e^{\lambda_n t} \end{bmatrix} P^{-1} \tag{33}$$

Each eigenvalue λ_i contributes an exponential mode. Negative real eigenvalues decay. Positive ones grow. Complex ones oscillate. This is the state-space way of saying "poles determine everything."

4.4 Motor Modeling: The Most Common Plant

Every controller needs a plant model. In robotics competitions, 95% of your plants are electric motors. Whether you are driving wheels, spinning flywheels, or controlling a turret, understanding how to model a motor is the single most useful skill you can have before touching any control algorithm.

DC Brushed Motor Model

A **DC brushed motor** has two coupled subsystems: electrical and mechanical.

Electrical dynamics (KVL around the armature loop):

$$V = L\frac{di}{dt} + Ri + K_e\omega \tag{34}$$

Mechanical dynamics (Newton's second law on the rotor):

$$J\frac{d\omega}{dt} = K_t i - b\omega - \tau_{\text{load}} \tag{35}$$

where V is input voltage, i armature current, ω angular velocity, L inductance, R resistance, K_e back-EMF constant, K_t torque constant, J rotor inertia, b viscous friction, and τ_{load} external load torque.

Taking the Laplace transform (with $\tau_{\text{load}} = 0$) and eliminating $I(s)$ yields the **full second-order transfer function**:

$$\frac{\Omega(s)}{V(s)} = \frac{K_t}{JLs^2 + (JR + bL)s + (bR + K_tK_e)} \tag{36}$$

First-order simplification. For most motors used in robotics competitions, the electrical time constant $\tau_e = L/R$ is on the order of milliseconds—far faster than any mechanical response. Setting $L \approx 0$ collapses the model to a much more tractable first-order system:

$$\frac{\Omega(s)}{V(s)} = \frac{K_m}{\tau_m s + 1} \tag{37}$$

where the **motor gain** and **mechanical time constant** are:

$$K_m = \frac{K_t}{bR + K_tK_e}, \qquad \tau_m = \frac{JR}{bR + K_tK_e} \tag{38}$$

This first-order model is what you will use for PID tuning, feedforward design, and most practical work.

Demystifying the Time Constant τ

The time constant τ appears everywhere in control theory, yet many students struggle to develop an intuition for it. Let us fix that.

What τ really means. For any first-order system $G(s) = \frac{K}{\tau s+1}$, the step response is:

$$y(t) = K\left(1 - e^{-t/\tau}\right)$$

At $t = \tau$, the output has reached $1-e^{-1} \approx 63.2\%$ of its final value. This is where the "63% rule" comes from. But why 63%? Because the first-order system's behavior has a beautiful property: *at every instant, the rate of change is proportional to the remaining distance to the target.* At $t = 0$, the system has full "motivation" and moves the fastest. As it approaches the target, it slows down—like a runner who sprints at the start but slows as the finish line approaches. After one τ, it has covered 63.2% of the gap. After another τ, 63.2% of the *remaining* gap. And so on:

Time	% of final value	Practical meaning
1τ	63.2%	Barely past halfway—still rising noticeably
2τ	86.5%	Getting close, but not there yet
3τ	95.0%	Good enough for most applications
4τ	98.2%	Within 2% error band (settling time!)
5τ	99.3%	Essentially at steady state

A physical analogy: filling a leaky bucket. Imagine filling a bucket that has a hole in the bottom. When the bucket is empty, water flows in fast because there is no pressure pushing it back out. As the bucket fills, the water level creates back-pressure that pushes water out of the hole. The higher the level, the more water leaks out, until eventually inflow equals outflow and the level stops rising. The time it takes to reach 63% of the equilibrium level is τ—and it depends on both the bucket size (capacitance/inertia) and the hole size (resistance/friction).

This is exactly what happens in a motor: voltage drives current into the winding (inflow), but back-EMF and friction resist motion (outflow). The mechanical time constant τ_m tells you how quickly the motor speed "fills up" to its steady-state value.

Why τ connects to everything:

- **Settling time:** $t_s \approx 4\tau$ (for 2% criterion). If your motor has $\tau_m = 50$ ms, settling takes $\sim$200 ms. Too slow? You need a bigger motor or a feedback controller.
- **Bandwidth:** The -3 dB bandwidth of a first-order system is $\omega_{bw} = 1/\tau$. A motor with $\tau_m = 50$ ms has a bandwidth of 20 rad/s $\approx$ 3.2 Hz. It physically cannot track signals faster than this without a controller.
- **Pole location:** The pole of $G(s) = K/(\tau s + 1)$ is at $s = -1/\tau$. Smaller τ means the pole is farther left—faster dynamics. This is the transfer-function way of saying "a faster motor has a smaller time constant."
- **Sampling rate:** Your controller's sampling period should be at most $\tau/10$ for adequate control. For $\tau_m = 50$ ms, sample at ≤ 5 ms (200 Hz minimum).

DC gain K is the other half of the story. The time constant tells you *how fast* the system responds; the DC gain tells you *how far* it goes. For a motor, K_m gives the steady-state speed per volt. If $K_m = 10$ rad/s/V, then 1 V eventually produces 10 rad/s. If your application requires 100 rad/s, you need at least 10 V. No controller can change the DC gain of the plant—it can only change how fast you get there.

Electrical vs. mechanical time constant. A motor actually has *two* time constants: $\tau_e = L/R$ (electrical, typically $\sim$1 ms) and $\tau_m = JR/(bR + K_tK_e)$ (mechanical, typically 20–200 ms). Since $\tau_e \ll \tau_m$, the electrical dynamics "settle" before the mechanical dynamics even get started. This is why we can simplify the second-order model to first-order: by the time you notice the motor moving, the current has already reached its steady-state value. In control theory language, the fast electrical pole is so far left in the s-plane that it has negligible effect on the response—it is not a *dominant pole*.

The dominant pole concept. When a system has multiple time constants, the *slowest* one (largest τ, i.e., the pole closest to the imaginary axis) dominates the response. This is why we can

often approximate high-order systems by simple first- or second-order models—the fast poles contribute only brief transients that die out before we even notice them. This is not sloppy engineering; it is justified approximation, and it is the reason first-order models work so well in practice.

BLDC Motors and FOC

Brushless DC (BLDC) motors are three-phase machines, but modern ESCs running **Field Oriented Control (FOC)** handle the three-phase complexity internally. FOC uses the Park and Clarke transforms to project the three-phase currents onto a rotating d-q reference frame, producing an equivalent DC brushed motor in the q-axis. From your controller's perspective, the ESC exposes a *virtual DC brushed motor*—you command a voltage (or current) and observe angular velocity, and the same transfer function applies directly. You do not need to think about phase commutation at all.

Understanding FOC: Why It Matters and How It Works

A BLDC motor has three stator windings arranged 120° apart. To spin the rotor, these windings must be energized in a specific sequence that keeps the magnetic field always "pulling" the rotor forward. The challenge is: the required currents are sinusoidal and their phase depends on the rotor's angular position. Controlling three time-varying, position-dependent sinusoids simultaneously is messy.

The key idea of FOC: transform the problem until it looks like a DC motor.

Step 1: Clarke transform ($abc \to \alpha\beta$). The three phase currents i_a, i_b, i_c are projected onto a fixed two-axis coordinate system (α, β). This eliminates one redundant variable (since $i_a + i_b + i_c = 0$) without losing any information. The result: two sinusoidal signals instead of three.

Step 2: Park transform ($\alpha\beta \to dq$). The two-axis fixed frame is rotated to align with the rotor's magnetic axis, using the measured rotor angle θ_e. In this **rotating reference frame**:

- The q-axis current (i_q) produces torque—exactly like the armature current in a DC brushed motor.
- The d-axis current (i_d) controls the magnetic field strength. For most applications, i_d is set to zero (no need to weaken or strengthen the field).

After the Park transform, the three time-varying sinusoids have become two *constant* values (i_d and i_q) in steady state. Controlling constants is trivial—just use PI controllers.

Step 3: PI control in dq frame. Two simple PI controllers:

- d-axis PI: keeps $i_d = 0$ (or a reference for field weakening at high speeds).
- q-axis PI: tracks the desired torque current $i_q^* = \tau_{\text{desired}}/K_t$.

Step 4: Inverse transforms. The dq voltage commands are transformed back to three-phase voltages (inverse Park $\to$ inverse Clarke), which are applied to the motor windings via PWM.

Why FOC matters for your robot:

- **Precise torque control.** Since i_q directly determines torque, FOC enables accurate torque (and therefore current) control—essential for force-controlled applications and smooth low-speed operation.
- **Maximum efficiency.** By keeping $i_d = 0$, all current goes to producing torque. No energy is wasted on useless field components. This means less heat and longer battery life.
- **Smooth at all speeds.** Unlike simpler commutation methods (trapezoidal/six-step), FOC produces smooth sinusoidal currents with no torque ripple. Your gimbal does not "cog" at low speeds.
- **Simplifies your work.** The ESC handles all the FOC math internally at 10–40 kHz. Your outer controller sees a clean current (or voltage) interface—a virtual DC brushed motor. You only need to design the velocity and position loops.

What you need to provide: FOC requires accurate rotor position feedback. Typically a magnetic encoder (e.g., AS5048) or Hall sensors. Without position feedback, the Park transform cannot be computed, and sensorless FOC (which estimates position from back-EMF) is used instead—common on drone ESCs but less precise for robotics.

State-Space Form

For observer design, LQR, or MPC, you need the full state-space representation. Using $\mathbf{x} = \begin{bmatrix} i & \omega \end{bmatrix}^T$ and $u = V$:

$$\dot{\mathbf{x}} = A\mathbf{x} + Bu, \qquad A = \begin{bmatrix} -R/L & -K_e/L \\ K_t/J & -b/J \end{bmatrix}, \quad B = \begin{bmatrix} 1/L \\ 0 \end{bmatrix} \tag{39}$$

With $L \approx 0$ the current state i becomes an algebraic variable and the state reduces to $\mathbf{x} = [\omega]$, recovering the first-order model. The full two-state form is what you feed into an LQR or Kalman filter when current sensing is available.

Practical Tip: How to Get Motor Parameters

You rarely have a datasheet with J, b, L. Measure them yourself:

- **Stall test** — apply a known low voltage V_s, hold the shaft still, measure stall current I_s. Then $R = V_s/I_s$.
- **Free-run test** — no load, apply rated voltage V_0, wait for steady state, measure ω_{free}. Then $K_e = (V_0 - I_{\text{free}}R) \,/\, \omega_{\text{free}}$ (SI units: V·s/rad). In SI units, $K_t = K_e$ exactly.
- **Step response** — command a voltage step, log $\omega(t)$. The time to reach 63% of the final value is τ_m.
- **Inertia** — rearrange $\tau_m = JR/(bR + K_tK_e)$ to solve for J, or use a *swing test*: attach a known mass on a known arm, measure angular acceleration α under a known torque, and compute $J = \tau/\alpha$.

Shortcut when using an FOC ESC with a current controller: most modern FOC ESCs close an inner current loop at very high bandwidth. If that loop is active, the current i tracks your command almost instantly. Your effective plant simplifies to *pure mechanical dynamics*:

$$\frac{\Omega(s)}{I_{\text{ref}}(s)} = \frac{K_t}{Js + b}$$

You only need K_t, J, and b—and b is often negligible. This is the plant you will tune your outer velocity loop against.

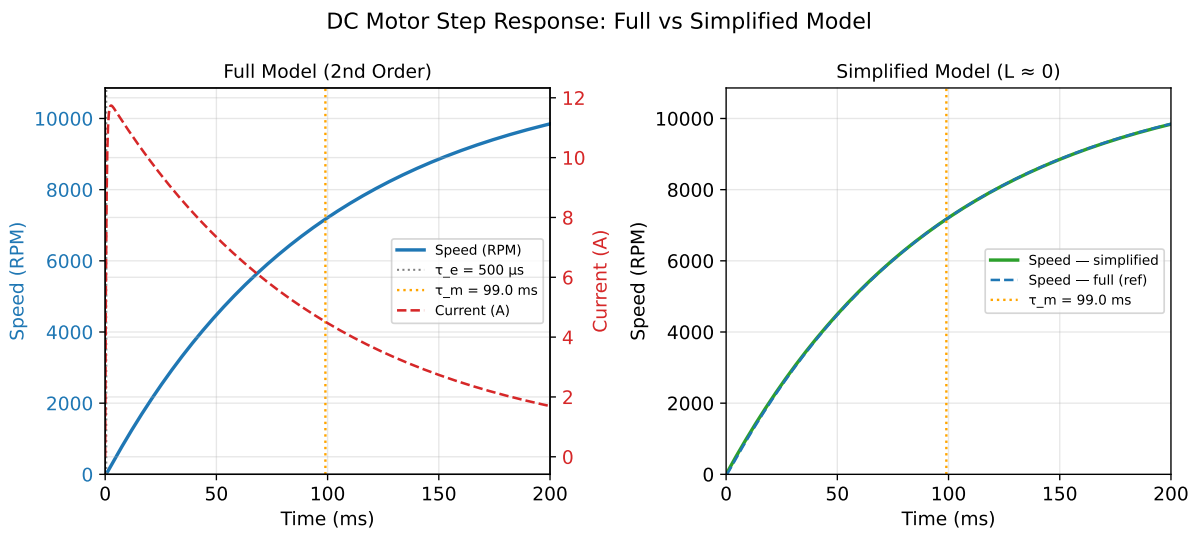

Figure 8: DC brushed motor step response: voltage to angular velocity. Left: full second-order model showing the fast electrical transient ($\tau_e \approx 1$ ms) followed by the slow mechanical rise ($\tau_m \approx 50$ ms). Right: first-order approximation ($L \approx 0$) matches the mechanical response almost perfectly — the electrical pole is too fast to matter.

5 Classical Control Theory

This is the section you have been waiting for. Classical control theory is where we learn to make systems do what we want. And the undisputed champion of classical control—the tool you will use on 95% of your competition robots—is the PID controller.

But before we dive into PID, we need to understand *why* feedback control works in the first place.

5.1 The Concept of Feedback Control

Open-loop control is like cooking without tasting. You follow the recipe exactly, hope for the best, and serve. If the oven temperature is off by 10 degrees, your cake is ruined.

Closed-loop (feedback) control is like cooking while constantly tasting. You taste, adjust the seasoning, taste again, adjust again. Even if the oven is off, you compensate in real time.

Mathematically, a basic feedback loop looks like this:

1. Compute the **error**: $e(t) = r(t) - y(t)$, where r is the desired output (setpoint) and y is the actual output.
2. Feed the error into a **controller** $C(s)$, which produces a control input $u(t)$.
3. The control input drives the **plant** $G(s)$, which produces the output $y(t)$.
4. Measure $y(t)$ and go back to step 1.

The closed-loop transfer function from setpoint R to output Y is derived as follows. From the block diagram: the error is $E(s) = R(s) - Y(s)$, the controller output is $U(s) = C(s)E(s)$, and the plant output is $Y(s) = G(s)U(s)$. Combining: $Y = GCE = GC(R - Y)$, so $Y + GCY = GCR$, giving $Y(1 + GC) = GCR$, hence:

$$\frac{Y(s)}{R(s)} = \frac{C(s)G(s)}{1 + C(s)G(s)} \tag{40}$$

This is the most important equation in classical control. Stare at it. Love it. Memorize it.

Dissecting the closed-loop transfer function:

Let $L(s) = C(s)G(s)$ be the **loop transfer function** (also called open-loop transfer function). Then:

- **Complementary sensitivity** (setpoint tracking): $T(s) = \frac{L(s)}{1+L(s)}$. This is what we wrote above — it tells you how well the output follows the reference. Ideally $T(s) \approx 1$ at low frequencies (good tracking) and $T(s) \approx 0$ at high frequencies (noise rejection).
- **Sensitivity function**: $S(s) = \frac{1}{1+L(s)}$. This tells you how much a disturbance at the output affects the output. Small S = good disturbance rejection. Note that $T(s) + S(s) = 1$ always — you *cannot* have perfect tracking and perfect disturbance rejection at the same frequency. This is a fundamental limitation of feedback.
- **Input sensitivity**: $C(s) \cdot S(s) = \frac{C(s)}{1+L(s)}$. This tells you how much control effort is needed. If this is large at high frequencies, your actuator will work overtime amplifying noise.

The key insight: At frequencies where $|L(j\omega)| \gg 1$ (high loop gain), $T \approx 1$ and $S \approx 0$ — feedback works well. At frequencies where $|L(j\omega)| \ll 1$ (low loop gain), $T \approx 0$ and $S \approx 1$ — feedback does nothing. The **bandwidth** is the frequency where $|L(j\omega)| = 1$ (the crossover), and it marks the boundary between "feedback is helping" and "feedback is not helping."

Why feedback is powerful:

- **Reduces sensitivity to model uncertainty.** Even if $G(s)$ is not perfectly known, feedback compensates.
- **Rejects disturbances.** External forces (friction, wind, collisions) are automatically corrected.
- **Improves dynamic response.** We can make the system faster or more damped by choosing $C(s)$ appropriately.

The catch: Feedback can also *destabilize* a system. If your controller is too aggressive, the correction overshoots, the error reverses, the correction overshoots again, and you get oscillation or complete instability. This is why we need tools to analyze stability.

Case Study: The Shower Temperature Problem

You have experienced feedback control instability in your own bathroom. Here is how:

You step into the shower. The water is cold. You crank the hot knob (large K_p). Nothing happens for 2 seconds (the pipe delay—this is **transport lag**, a real control theory concept). Impatient, you crank it more. Suddenly, scalding water hits you. You jerk the knob to cold. Two seconds later, ice water. You oscillate between scalding and freezing for 30 seconds until you give up.

What went wrong (in control theory terms):

- The **plant** is the water heater + pipes. It has a significant **time delay** ($\sim$2 seconds).
- You (the controller) used a very high K_p—aggressive corrections.
- Time delay adds phase lag. At the frequency where you are adjusting the knob, the total phase lag exceeds 180°. Your phase margin is negative. The system is **unstable**.

The fix: Turn the knob slowly and **wait**. This is equivalent to reducing K_p so that your bandwidth is well below $1/(2 \times \text{delay})$. You are trading speed for stability—exactly the same trade-off you will face with every control loop on your robot.

Lesson: Systems with time delay are fundamentally harder to control. If your robot has communication latency (e.g., CAN bus delay, vision processing delay), you *must* account for it by reducing controller aggressiveness or using a Smith predictor (an advanced technique that compensates for known delays).

5.2 System Performance Metrics

When we design a controller, we need to define what "good" means. The standard metrics for a step response (sudden change in setpoint) are:

- **Rise time** (t_r): Time to go from 10% to 90% of the final value. Faster is generally better.
- **Overshoot** (M_p): How much the output exceeds the setpoint, expressed as a percentage. Less is generally better.
- **Settling time** (t_s): Time until the output stays within ±2% (or 5%) of the final value.
- **Steady-state error** (e_{ss}): The remaining error after the system has settled. Ideally zero.

These metrics often conflict: faster rise time usually means more overshoot. Control design is about finding the right trade-off for your application. A gimbal tracking a target needs low overshoot (no jittering past the target). A chassis accelerating to top speed can tolerate some overshoot if it means getting there faster.

5.2.1 Stability Margins

Even if a system is stable, we want to know *how stable* it is—how close it is to becoming unstable. This is measured by two numbers:

Gain margin (GM): How much you can increase the loop gain before the system goes unstable. Measured in dB. A gain margin of 6 dB means you can double the gain before instability. Typical target: > 6 dB.

Phase margin (PM): How much additional phase lag the system can tolerate before going unstable. Measured in degrees. A phase margin of 45° means the system can tolerate an additional 45° of lag. Typical target: 30°–60°.

You can read both margins from a **Bode plot** of the open-loop transfer function $C(s)G(s)$.

What is a Bode plot? A Bode plot is two graphs stacked vertically, both with frequency on the horizontal axis (log scale). The **top graph (magnitude)** shows the gain $|G(j\omega)|$ in decibels (dB) vs. frequency, where 0 dB means unity gain, +20 dB means gain = 10, and −20 dB means gain = 0.1 (conversion: $\text{dB} = 20\log_{10}(|G|)$). The **bottom graph (phase)** shows the phase angle $\angle G(j\omega)$ in degrees vs. frequency; a phase of −90° means the output lags the input by a quarter cycle.

With magnitude and phase in hand, stability margins can be read directly from these two curves. Find the **phase crossover frequency** ω_{pc} where the phase crosses −180°, and read the magnitude there—the distance from the magnitude curve to 0 dB is your **gain margin**. Then find the **gain**

crossover frequency ω_{gc} where the magnitude crosses 0 dB, and read the phase there—the distance from the phase curve to $-180°$ is your **phase margin**. If both margins are positive, the system is stable.

For hand analysis, a few asymptotic rules let you sketch the Bode plot without a computer. Each pole at $s = -a$ contributes -20 dB/decade rolloff starting at $\omega = a$ and $-45°$/decade phase change centered there. Each zero does the opposite ($+20$ dB/decade and $+45°$/decade). An integrator ($1/s$) contributes -20 dB/decade at all frequencies and a constant $-90°$ phase. The DC gain sets the vertical offset of the magnitude plot.

With these rules, you can sketch approximate Bode plots by hand in under a minute—invaluable during debugging when you need a quick sanity check.

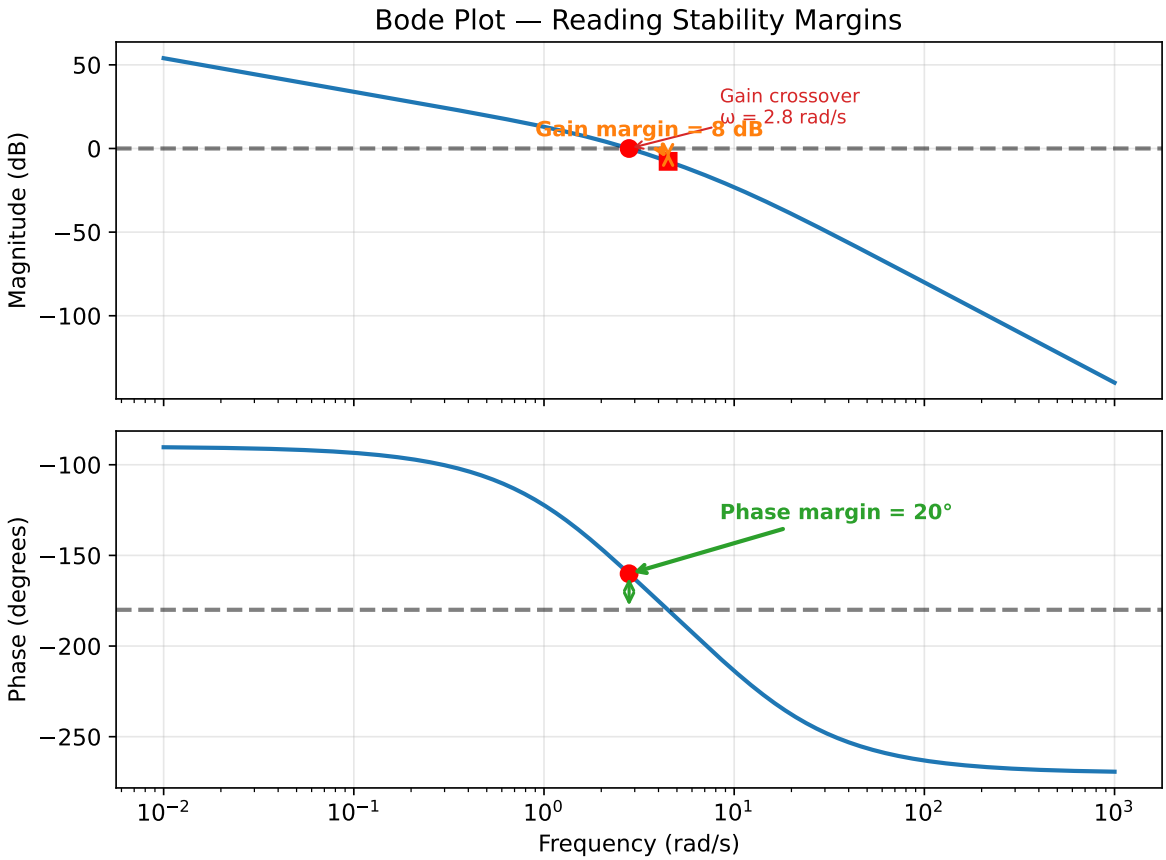

Figure 9: Bode plot of an open-loop system $L(s) = \frac{100}{s(s+2)(s+10)}$. The gain margin (orange) and phase margin (green) are read directly from the plot. Both are positive, so the system is stable.

Rule of thumb: If your system is oscillating, your phase margin is too small. If your system is sluggish, your gain margin is too large (you are being too conservative). A phase margin of around 45° usually gives a good balance.

5.3 Controller Design via Transfer Functions

The game of classical controller design is: given a plant $G(s)$, choose a controller $C(s)$ such that the closed-loop system $\frac{C(s)G(s)}{1+C(s)G(s)}$ meets your performance specifications.

There are many approaches—root locus (a graphical method that plots how the system's poles move in the complex plane as you sweep the controller gain), frequency response shaping (sculpting the Bode plot to meet specs), pole placement (directly choosing where the closed-loop poles should be)—but for competition robotics, the most practical approach is:

1. Start with a PID controller (next section).
2. Tune it using systematic methods.

3. If PID is not enough, add feedforward or switch to modern methods (Section 6).

This might sound unsophisticated, but here is the truth: PID controls an estimated 95% of all industrial control loops in the world. It is simple, robust, and effective. Do not overcomplicate things.

5.4 PID Controller

The PID controller computes the control signal as the sum of three terms based on the error $e(t)$:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau)d\tau + K_d \frac{de(t)}{dt} \tag{41}$$

Each term has a distinct role:

- **P (Proportional):** "The error is big, so push hard. The error is small, so push gently." It is the most intuitive term—a simple spring-like response.
- **I (Integral):** "The error has been nonzero for a while, so there must be a persistent disturbance. Push harder until it is gone." It eliminates steady-state error but can cause overshoot and windup.
- **D (Derivative):** "The error is changing fast, so the system has momentum. Apply the brakes." It adds damping and reduces overshoot but amplifies noise.

An analogy: Imagine you are driving toward a stop sign.

- P says: "The stop sign is 100 meters away, press the gas." Then: "It is 10 meters away, ease off."
- D says: "You are approaching fast—start braking NOW, not when you are close."
- I says: "You stopped 1 meter past the sign. Roll back slowly until you are exactly there."

5.4.1 PID Controller – Continuous Form

The transfer function of a PID controller is:

$$C(s) = K_p + \frac{K_i}{s} + K_d s = K_p \left(1 + \frac{1}{T_i s} + T_d s\right) \tag{42}$$

where $T_i = K_p/K_i$ is the integral time and $T_d = K_d/K_p$ is the derivative time.
Understanding PID in the frequency domain:
The PID controller's Bode plot reveals why it works:

- **At very low frequencies** ($\omega \to 0$): The K_i/s term dominates. The gain goes to infinity ($+\infty$ dB), which is why I eliminates steady-state error — infinite gain at DC means zero error at DC.
- **At mid frequencies**: K_p dominates. The gain is approximately $20\log_{10}(K_p)$ dB. This is the frequency range where your controller does most of its work.
- **At high frequencies** ($\omega \to \infty$): $K_d s$ dominates. The gain increases with frequency at +20 dB/decade, which is why D amplifies noise — it has increasing gain at the very frequencies where noise lives.

The PID has **two break frequencies**: $\omega_1 = 1/T_i$ (where I hands off to P) and $\omega_2 = 1/T_d$ (where P hands off to D). Between them, the controller behaves like pure proportional control. The Bode magnitude plot looks like a "V" shape (or bathtub) on a log scale: high gain at low frequencies (I), flat in the middle (P), rising at high frequencies (D).

Why the alternative form matters: The form $K_p(1 + \frac{1}{T_i s} + T_d s)$ makes the physical meaning clearer: T_i tells you "how long the integral looks back" and T_d tells you "how far ahead the derivative looks." Larger T_i means slower integral action. Larger T_d means more anticipation (but also more noise sensitivity).

Effect of each gain:

Increase	Rise time	Overshoot	Settling time	SS error
K_p	Decrease	Increase	Small change	Decrease
K_i	Decrease	Increase	Increase	Eliminate
K_d	Small change	Decrease	Decrease	No effect

Practical starting point: Start with P-only control. If there is steady-state error, add I. If there is too much overshoot or oscillation, add D. Do not start with all three at once—you will not know which knob is doing what.

Case Study: Tuning a Gimbal Yaw Motor — Watching P, I, D in Action

Let us walk through tuning a RoboMaster gimbal yaw axis step by step. The motor is a GM6020, controlled via CAN bus, and we want it to track a target angle. The setpoint is a 90° step command.

Step 1: P-only ($K_p = 5.0$, $K_i = 0$, $K_d = 0$)

The gimbal swings toward the target quickly but overshoots by about 15°, oscillates 3–4 times, and settles with a **2° steady-state error**. Why the error? Because as the gimbal approaches the target, the error shrinks, and so does the P output. At some point, the P output equals the friction torque, and the gimbal stops *before* reaching the target. P alone cannot eliminate error caused by constant disturbances (like friction or gravity).

Step 2: Add I ($K_p = 5.0$, $K_i = 0.5$, $K_d = 0$)

The steady-state error vanishes! The integral term slowly accumulates the small residual error and pushes the motor until the error is exactly zero. But now the overshoot is worse (∼20°), and there is a slow oscillation that takes 2 seconds to die out. The integral is "remembering" past errors and adding unnecessary force. This is the classic I-term trade-off: it fixes DC error but worsens transient response.

Step 3: Add D ($K_p = 5.0$, $K_i = 0.5$, $K_d = 2.0$)

The overshoot drops dramatically to about 5°, and the oscillation is almost gone. The derivative term senses the rapid approach and applies a braking torque *before* the gimbal reaches the target. It is like an experienced driver who starts braking well before the intersection, not at the last second.

Step 4: Fine-tune ($K_p = 6.0$, $K_i = 0.3$, $K_d = 3.0$)

We increase K_p slightly for a faster rise time, reduce K_i (we only need it for static friction, not aggressive error correction), and increase K_d for more damping. The result: fast, smooth tracking with less than 3° overshoot and zero steady-state error. The gimbal locks onto the target in under 0.3 seconds.

What happens if you crank each parameter too high?

- K_p too high: The gimbal oscillates violently—it overshoots so hard that the correction overshoots in the other direction. This is the onset of instability.
- K_i too high: Slow, sloshy oscillation. The integral "remembers" too aggressively and keeps overcorrecting. The gimbal wobbles like jelly.
- K_d too high: The motor buzzes and vibrates. The derivative amplifies sensor noise, causing rapid back-and-forth torque commands. Your motor sounds like an angry bee.

The lesson: Each parameter has a "sweet spot." Understanding the *physical mechanism* of each term (spring, memory, damper) helps you diagnose problems instantly instead of blindly twiddling knobs.

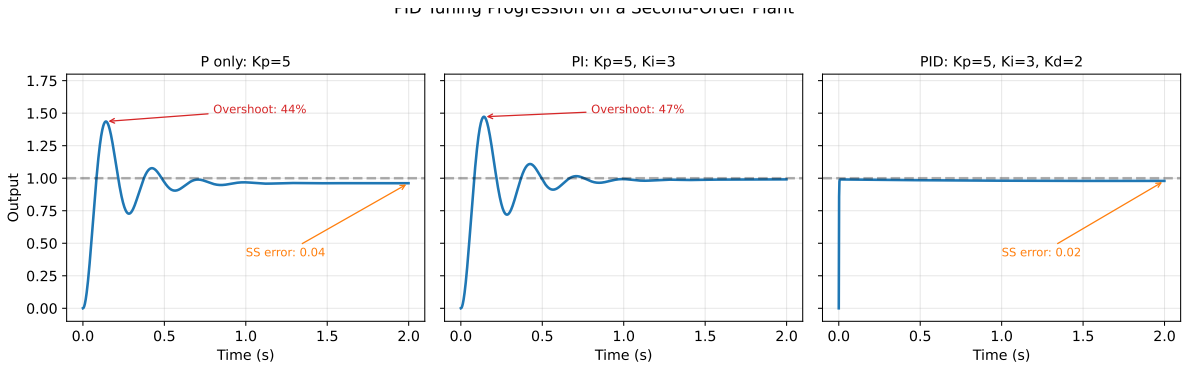

Figure 10: PID tuning progression on a second-order plant. P-only has steady-state error; PI eliminates it but overshoots more; PID damps the overshoot while maintaining zero steady-state error.

5.4.2 PID Controller – Discrete Form

Your microcontroller cannot compute continuous integrals and derivatives. It works with discrete samples at a fixed time step T_s. The discrete PID is:

$$u[n] = K_p\, e[n] + K_i\, T_s \sum_{k=0}^{n} e[k] + K_d \frac{e[n] - e[n-1]}{T_s} \tag{43}$$

Or in a more implementation-friendly **incremental (velocity) form**:

$$\Delta u[n] = K_p(e[n] - e[n-1]) + K_i\, T_s\, e[n] + K_d \frac{e[n] - 2e[n-1] + e[n-2]}{T_s} \tag{44}$$

$$u[n] = u[n-1] + \Delta u[n] \tag{45}$$

The incremental form has a practical advantage: if your controller resets or glitches, it does not produce a sudden large output—it only adjusts by a small increment. This is safer for your hardware.

Pseudocode for a basic discrete PID:

```
error = setpoint - measurement
integral += error * dt
derivative = (error - prev_error) / dt
output = Kp * error + Ki * integral + Kd * derivative
prev_error = error
```

Warning: This naive implementation has three problems that will bite you in practice: integral windup, derivative kick, and noise amplification. Read the next section.

5.4.3 Improved PID Controllers

The textbook PID has several well-known problems. They are not independent—windup, kick, and noise amplification all stem from the same root cause: the three PID terms behave poorly at actuator limits or at high frequencies. Here are the fixes that every competition robot should use.

1. Integral windup and anti-windup

The problem: When the actuator saturates (motor at max voltage), the integral term keeps accumulating error, growing without bound. When the error finally reverses, the huge integral takes a long time to "unwind," causing massive overshoot.

The fix (clamping):

```
integral += error * dt
integral = clamp(integral, -max_integral, max_integral)
```

Better fix (back-calculation): When the output is saturated, reduce the integral term proportionally:

```
output_raw = Kp * error + Ki * integral + Kd * derivative
output = clamp(output_raw, -max_output, max_output)
integral += (error + (output - output_raw) / Ki) * dt
```

2. Derivative kick

Having tamed the integral, the next hazard appears when the setpoint changes abruptly.

The problem: When the setpoint changes suddenly (step command), the error $e = r - y$ jumps instantaneously. The derivative term sees this as an infinite rate of change and produces a huge spike ("kick") in the output.

The fix: Differentiate the *measurement* instead of the error:

```
derivative = -(measurement - prev_measurement) / dt
```

The measurement changes smoothly (your motor cannot teleport), so there is no kick.

3. Derivative noise amplification

Differentiating measurement instead of error solves the kick, but introduces a subtler problem.

The problem: Differentiation amplifies high-frequency noise. If your measurement is noisy, the derivative term will produce wildly fluctuating outputs.

The fix: Low-pass filter the derivative:

$$D(s) = \frac{K_d s}{1 + \frac{K_d}{N K_p} s} \tag{46}$$

In code, this is a first-order low-pass filter on the derivative term, with N (typically 8–20) controlling the filter cutoff.

4. Setpoint weighting

The previous three fixes made the PID robust. This last one makes it flexible.

Sometimes you want the P and D terms to respond differently to setpoint changes vs. disturbances. Setpoint weighting introduces parameters b and c:

$$u = K_p(b \cdot r - y) + K_i \int (r - y)\, dt + K_d \frac{d(c \cdot r - y)}{dt} \tag{47}$$

Setting $b = 1, c = 0$ (derivative on measurement only) is the most common configuration.

5.5 Advanced PID Improvements

The four fixes above are the *minimum* every competition robot needs. But there are deeper structural improvements that can dramatically change performance. Each is analyzed here in the frequency domain and demonstrated with simulation.

5.5.1 Integral Separation

The problem: When the error is large (e.g., right after a step command), the integral term accumulates aggressively. This "pre-charged" integral causes overshoot once the system approaches the setpoint — the integral has "remembered" too much history from the high-error phase.

The fix: Only activate the integral term when the error is smaller than a threshold ε:

$$u_I = \begin{cases} K_i \int e\, dt & \text{if } |e| < \varepsilon \\ 0 & \text{if } |e| \geq \varepsilon \end{cases} \tag{48}$$

Frequency domain interpretation: Integral separation effectively makes the controller **nonlinear** — it behaves like a PD controller (no integral) for large errors and switches to a full PID for small errors. In the frequency domain, this means:

- During large transients: the open-loop gain at DC is finite (PD only), so there is no infinite low-frequency gain. This prevents the integral from "winding up" during the transient.
- Near steady state: the integral activates, providing infinite DC gain and zero steady-state error — exactly what we need.

The controller essentially has **two different Bode plots** depending on the error magnitude: one without the K_i/s pole at low frequencies (PD-like) and one with it (PID). The threshold ε controls when the "switch" happens.

Code:

```
if (fabs(error) < epsilon) {
    integral += Ki * error * dt;
}
// else: integral stays frozen (neither grows nor resets)
```

Choosing ε: Set it to the approximate steady-state error of the PD-only controller. This way, the integral only activates when PD has done most of the work and just needs the final nudge.

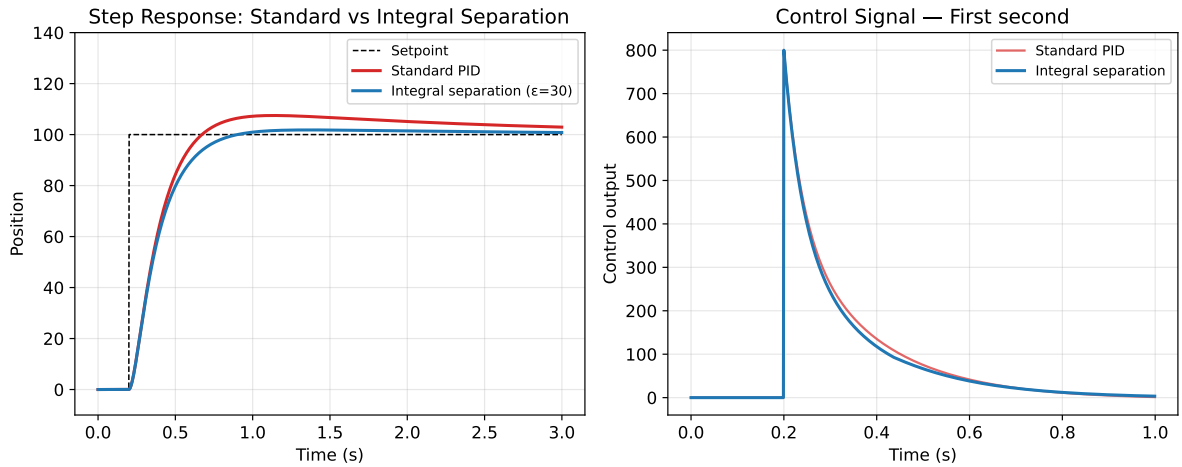

Figure 11: Left: integral separation (blue) reduces overshoot compared to standard PID (red), because the integral does not accumulate during the large initial transient. Right: the control signals during the first second — note how the separated version has a smaller peak.

5.5.2 Derivative Placement: PID vs PI-D vs I-PD

The standard PID applies all three terms to the error $e = r - y$. But there are **three structural variants** that change which signals P and D act on:

Structure	P acts on	I acts on	D acts on
PID	error $(r - y)$	error $(r - y)$	error $(r - y)$
PI-D	error $(r - y)$	error $(r - y)$	measurement $(-y)$
I-PD	measurement $(-y)$	error $(r - y)$	measurement $(-y)$

Frequency domain analysis: All three structures have the *same* closed-loop disturbance rejection transfer function (from disturbance to output). They differ only in the *setpoint-to-output* transfer function:

- **PID:** $\frac{Y}{R} = \frac{(K_d s^2 + K_p s + K_i)G(s)}{s + (K_d s^2 + K_p s + K_i)G(s)}$ — the numerator has a $K_d s^2$ term, which creates a "derivative kick" in the step response (numerator zeros amplify the step's high-frequency content).
- **PI-D:** $\frac{Y}{R} = \frac{(K_p s + K_i)G(s)}{s + (K_d s^2 + K_p s + K_i)G(s)}$ — the $K_d s^2$ is removed from the setpoint path, eliminating the derivative kick. The denominator (stability, disturbance rejection) is unchanged.
- **I-PD:** $\frac{Y}{R} = \frac{K_i \cdot G(s)}{s + (K_d s^2 + K_p s + K_i)G(s)}$ — both $K_d s^2$ and $K_p s$ are removed from the setpoint path. The step response is the smoothest (no overshoot from P or D) but also the slowest.

The key insight: These structures let you independently tune how the system responds to *setpoint changes* (via the numerator) vs. how it rejects *disturbances* (via the denominator). Same denominator = same stability, same disturbance rejection. Different numerator = different setpoint tracking behavior.

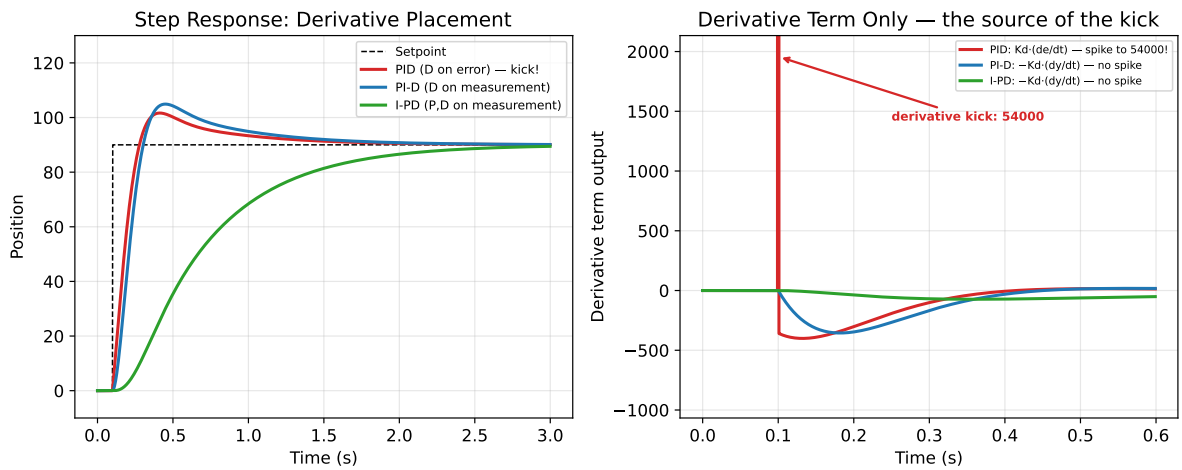

Figure 12: Left: PID (red) shows a derivative kick spike and overshoot. PI-D (blue) eliminates the kick. I-PD (green) is the smoothest but slowest. Right: the control signal reveals the violent kick in standard PID — this spike is what strips gears.

5.5.3 Filtered PID: Frequency Domain Proof

The ideal PID controller $C(s) = K_p + K_i/s + K_d s$ has a fundamental problem: the $K_d s$ term has **unbounded gain at high frequencies**. On a Bode plot, the magnitude of $K_d s$ rises at +20 dB/decade forever. This means:

1. Sensor noise (which lives at high frequencies) gets amplified infinitely.
2. The controller demands infinitely fast actuator response, which is physically impossible.
3. In discrete time, the derivative term aliases high-frequency noise into the control band.

The fix — filtered derivative:

$$C_{\text{filtered}}(s) = K_p + \frac{K_i}{s} + \frac{K_d N s}{s + N} \tag{49}$$

The term $\frac{Ns}{s+N}$ is a first-order high-pass filter with cutoff at $\omega = N$ rad/s. Below N, it behaves like a pure derivative ($\approx s$). Above N, it saturates to a constant gain N. On the Bode plot:

- Below $\omega = N$: the filtered PID is identical to the ideal PID. Same performance.
- Above $\omega = N$: the magnitude **flattens** instead of rising forever. Maximum derivative gain is $K_d \cdot N$. Noise amplification is bounded.

Choosing N: Typical values are $N = 8$–20. The rule: N should be large enough that the derivative still works effectively at the controller bandwidth, but small enough to attenuate noise. Specifically: $N > 5 \times \omega_{\text{bandwidth}}$ (so the filter does not eat into useful derivative action) and $N < \omega_{\text{noise}}/3$ (so noise is attenuated).

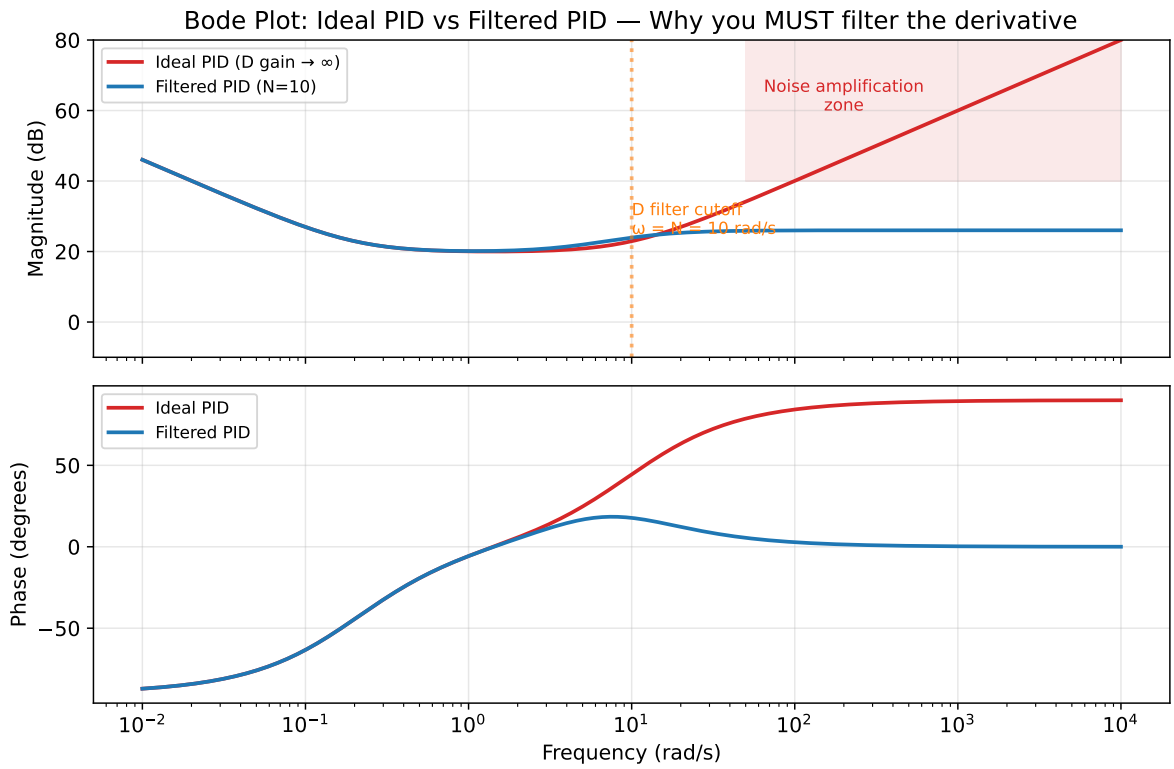

Figure 13: Bode plot: ideal PID (red) has gain rising forever at high frequencies — it will amplify every bit of sensor noise. Filtered PID (blue) flattens above $\omega = N$, bounding the noise amplification. Below the filter cutoff, both are identical.

Output-stage LPF: An alternative approach is to low-pass filter the *entire* PID output before sending it to the actuator:

$$U(s) = \frac{\omega_f}{s + \omega_f} \cdot C(s) \cdot E(s) \tag{50}$$

This is simpler to implement (one LPF after the PID) but it also filters the P and I terms, which can slow down the response. The derivative-only filter is preferred because it targets the problem precisely.

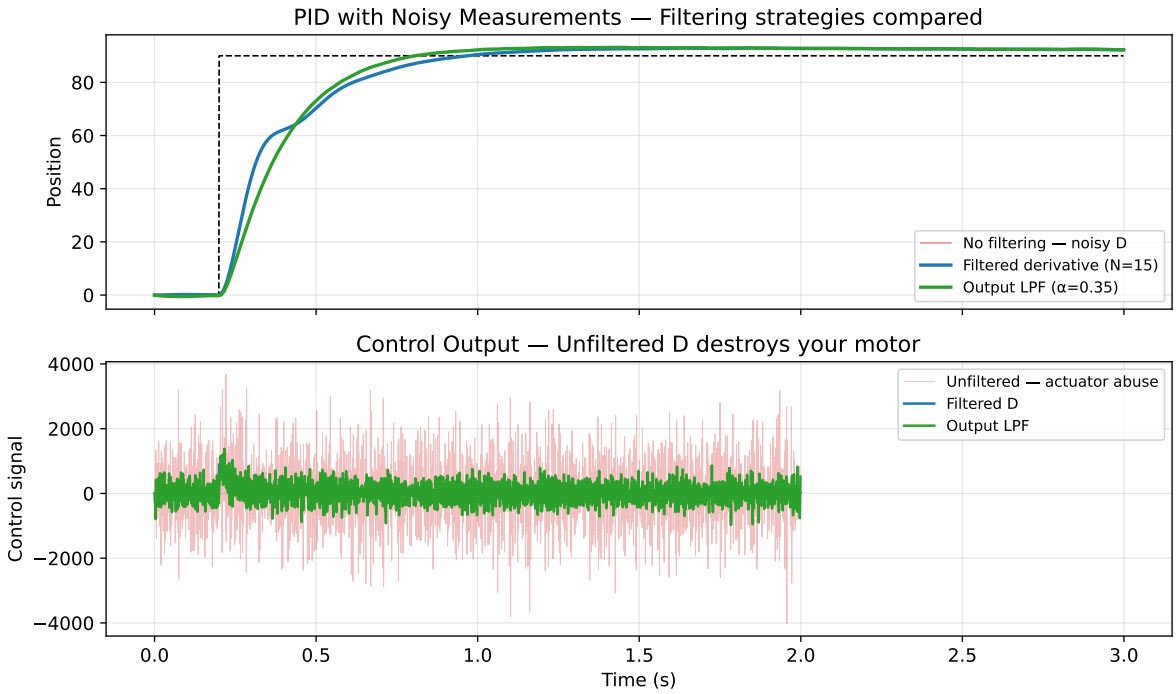

Figure 14: Top: position response with noisy measurements. Unfiltered PID (red) oscillates due to noise-amplified derivative. Filtered derivative (blue) and output LPF (green) both smooth the response. Bottom: the unfiltered control signal is violent — it would destroy a real motor.

5.5.4 Two-Degree-of-Freedom PID (2-DOF)

Standard PID has **one degree of freedom**: you choose K_p, K_i, K_d to meet your specifications. But there is a fundamental conflict: gains that give good *setpoint tracking* (fast, low overshoot for step commands) are different from gains that give good *disturbance rejection* (fast return after a bump).

The 2-DOF PID resolves this by adding setpoint weighting parameters b (on P) and c (on D):

$$u = K_p(b \cdot r - y) + K_i \int (r - y)\, dt + K_d \frac{d(c \cdot r - y)}{dt} \tag{51}$$

Frequency domain analysis: The closed-loop transfer functions become:

$$\frac{Y}{R} = \frac{(bK_p + cK_d s + K_i/s) \cdot G(s)}{1 + C(s)G(s)} \quad \text{(setpoint tracking — tuned by } b,\ c\text{)} \tag{52}$$

$$\frac{Y}{D} = \frac{G(s)}{1 + C(s)G(s)} \quad \text{(disturbance rejection — independent of } b,\ c\text{!)} \tag{53}$$

The magic: b and c *only* affect the setpoint transfer function. The disturbance rejection is controlled entirely by K_p, K_i, K_d. You can now:

1. Tune K_p, K_i, K_d for aggressive disturbance rejection.
2. Separately reduce b (typically 0.5–0.8) to soften the setpoint response and reduce overshoot.
3. Set $c = 0$ to eliminate derivative kick (this is the PI-D structure).

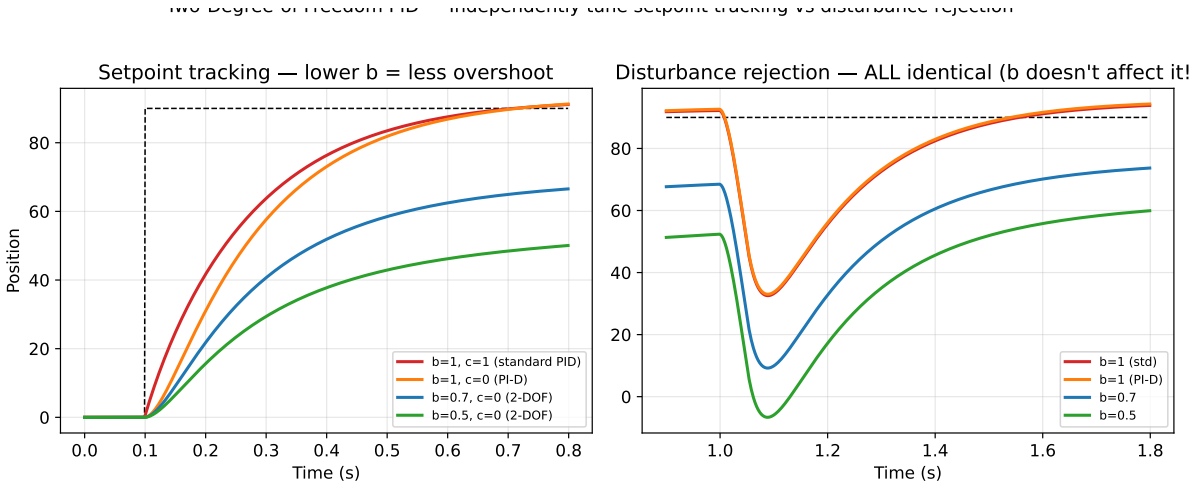

Figure 15: Left: different b values control setpoint overshoot independently. Right: disturbance rejection at $t = 1.0$s is **identical** for all b values — proving that 2-DOF truly decouples the two objectives.

5.5.5 Fuzzy PID

Motivation. A standard PID controller uses fixed gains K_p, K_i, K_d. But the *optimal* gains depend on the operating conditions: when the error is large, an aggressive response is needed; when the error is small, fine control is more appropriate. Fuzzy PID adapts K_p, K_i, K_d in real time based on the current error e and its rate of change $\dot{e}$.

How it works.

1. **Fuzzification.** Define fuzzy sets for the error e and its derivative $\dot{e}$. A common choice uses five linguistic levels: NB (Negative Big), NS (Negative Small), ZO (Zero), PS (Positive Small), PB (Positive Big). Membership functions (typically triangular or trapezoidal) map the crisp values of e and $\dot{e}$ into degrees of membership in each fuzzy set.

2. **Rule table.** A rule table (e.g., 5×5 or 7×7) maps every combination of $(e, \dot{e})$ to adjustments $(\Delta K_p, \Delta K_i, \Delta K_d)$. Rules are written in plain language, for example:

   - "If e is PB and $\dot{e}$ is PB $\Rightarrow$ increase K_p strongly, decrease K_i (to avoid windup), increase K_d."
   - "If e is ZO and $\dot{e}$ is NS $\Rightarrow$ keep K_p moderate, enable K_i for steady-state accuracy, reduce K_d."

3. **Defuzzification.** The fired rules are aggregated (commonly by centre-of-gravity) to produce crisp values of ΔK_p, ΔK_i, ΔK_d.

4. **Gain update.** The actual PID gains are:

$$K_p = K_{p0} + \Delta K_p, \qquad K_i = K_{i0} + \Delta K_i, \qquad K_d = K_{d0} + \Delta K_d$$

   where K_{p0}, K_{i0}, K_{d0} are the baseline gains.

Example rule table for ΔK_p (simplified 3×3).

$e \setminus \dot{e}$	N	Z	P
N	PB	PS	ZO
Z	PS	ZO	NS
P	ZO	NS	NB

Reading: when the error is Negative and getting more negative ($\dot{e} = \text{N}$), increase K_p strongly (PB) to drive the system back; when both are near Zero, no adjustment is needed (ZO).

Practical assessment.

- **Advantage:** Fuzzy PID adapts to changing conditions—load changes, nonlinear operating regions—without requiring an explicit system model.

- **Disadvantage:** The rule table design is largely empirical. In effect, you trade "tuning 3 gains" for "tuning a table of rules," which can be harder to validate rigorously.
- **When to use:** In competition robotics, a well-tuned fixed PID or 2-DOF PID usually suffices. Fuzzy PID is worth considering when the system operates across very different regimes (e.g., a chassis that must handle both empty and fully loaded conditions).
- **Compared to gain scheduling:** Gain scheduling manually switches between pre-tuned PID parameter sets at discrete operating points, which can cause transients at the switching boundaries. Fuzzy PID provides smoother transitions because the membership functions naturally interpolate between rules.

5.6 PID Tuning Methods

You have designed a PID controller. Now comes the eternal question: what values of K_p, K_i, K_d should you use?

5.6.1 Empirical Tuning

The most common method in competitions, because it requires no math and works surprisingly well.

Ziegler-Nichols ultimate gain method:

1. Set $K_i = 0$ and $K_d = 0$.
2. Gradually increase K_p until the system oscillates with constant amplitude. This is the **ultimate gain** K_u, and the oscillation period is T_u.
3. Use the following table to set your PID gains:

Controller	K_p	K_i	K_d
P	$0.5K_u$	—	—
PI	$0.45K_u$	$0.54K_u/T_u$	—
PID	$0.6K_u$	$1.2K_u/T_u$	$0.075K_uT_u$

Warning: Ziegler-Nichols gives aggressive tuning with about 25% overshoot. For many competition applications, you will want to reduce K_p and K_i from these values.

Manual tuning (the practical way):

1. Start with K_p only. Increase until the response is fast but slightly oscillatory.
2. Add K_d to damp the oscillation. Increase until the oscillation is gone.
3. If there is steady-state error, add a small K_i. Increase slowly—too much causes slow oscillation.
4. Fine-tune. Test with different setpoints and disturbances.

Competition tip: Always tune with the *actual load* on the robot, not with the motor running freely. A gimbal with a camera mounted behaves very differently from a bare motor on a test bench.

5.6.2 Tuning with MATLAB Simulink

If you have access to MATLAB, you can model your system and use the **PID Tuner** app or **Simulink Control Design** to find optimal gains automatically.

Basic workflow:

1. Create a Simulink model of your plant (motor + load).
2. Add a PID Controller block.
3. Use the "Tune" button in the PID block to open the PID Tuner.
4. Adjust the response time and robustness sliders.
5. Export the gains to your embedded code.

This is excellent for getting a starting point before fine-tuning on hardware. The key challenge is getting an accurate plant model—if the model is wrong, the tuning will be wrong.

5.7 System Identification: Figuring Out What Your Plant Actually Does

So far, we have assumed that you *know* your plant model — the transfer function $G(s)$ or the state-space matrices A, B, C, D. But where do these come from? You have two options:

1. **First-principles modeling:** Write down the physics equations (Newton's laws, Kirchhoff's laws, etc.) and derive the model analytically. This is what we did for the DC brushed motor in Section 3. It works well when the physics is simple and well-understood.

2. **System identification (sysid):** Apply known inputs to the real system, measure the outputs, and *fit* a mathematical model to the input-output data. This is what you do when the physics is too complex, unknown, or when you want to capture effects that first-principles models miss (nonlinear friction, cable stiffness, gear backlash).

In practice, you almost always use **both**: first-principles to get the model structure ("it is a second-order system with a delay"), then sysid to get the actual parameter values (K, τ, ζ, etc.).

5.7.1 Step Response Method (Time Domain)

The simplest sysid technique. Apply a step input and record the output.

For a first-order system $G(s) = \frac{K}{\tau s+1}$:

1. Apply a step of amplitude A to the motor voltage.

2. Record the velocity response $y(t)$.

3. Read off:
   - K = (final steady-state value) / A = DC gain
   - τ = time to reach 63.2% of the final value = time constant

For a second-order system $G(s) = \frac{K\omega_n^2}{s^2+2\zeta\omega_n s+\omega_n^2}$:

1. Apply a step and record.

2. Read off:
   - K = (final value) / (step amplitude)
   - M_p = percent overshoot $\Rightarrow \zeta = \frac{-\ln(M_p/100)}{\sqrt{\pi^2+\ln^2(M_p/100)}}$. This is obtained by inverting the overshoot formula $M_p = e^{-\pi\zeta/\sqrt{1-\zeta^2}}$: take the logarithm of both sides, $\ln(M_p/100) = -\pi\zeta/\sqrt{1-\zeta^2}$, square both sides, and solve for ζ.
   - t_p = time to first peak $\Rightarrow \omega_n = \frac{\pi}{t_p\sqrt{1-\zeta^2}}$

Practical tips:

- Log data at the highest rate your sensor supports (1 kHz+ for motors).

- Run the test 3–5 times and average to reduce noise.

- Test with different step amplitudes to check linearity. If the response shape changes, the system is nonlinear.

- Record both the rising step (0 → V) and the falling step (V → 0). If they differ, you have asymmetric dynamics (e.g., different friction in forward and reverse).

5.7.2 Frequency Response Method (Bode Identification)

Apply sinusoidal inputs at different frequencies and measure the gain and phase shift at each frequency. This directly constructs the Bode plot of the system.

Procedure:

1. Apply $u(t) = A\sin(\omega t)$ to the system.

2. Wait for transients to die out (several periods).

3. Measure the steady-state output amplitude B and phase lag ϕ.

4. Compute gain: $|G(j\omega)| = B/A$ and phase: $\angle G(j\omega) = \phi$.

5. Repeat for ω from $0.1\times$ to $10\times$ the expected bandwidth.

Why this is powerful: The Bode plot directly tells you the system order (count the rolloff slopes), the resonance frequencies (peaks in the magnitude plot), and the time delays (linear phase decrease). You can then fit a transfer function to match the measured Bode plot.

Faster alternative — chirp signal: Instead of testing one frequency at a time, apply a **chirp** (frequency sweep) that ramps from low to high frequency over a few seconds. Then compute the FFT of both input and output, and divide: $G(j\omega) = \frac{Y(j\omega)}{U(j\omega)}$. One experiment gives you the entire Bode plot.

5.7.3 Least-Squares Identification (Automated)

For digital systems, sysid can be fully automated. Given input-output data $\{u[k], y[k]\}$, fit a discrete transfer function:

$$y[k] = -a_1 y[k-1] - \cdots - a_n y[k-n] + b_0 u[k-1] + \cdots + b_m u[k-m-1] \tag{54}$$

This is a **linear regression** problem. Stack all samples into a matrix equation $\mathbf{y} = \Phi\boldsymbol{\theta}$ and solve by least squares: $\boldsymbol{\theta} = (\Phi^T\Phi)^{-1}\Phi^T\mathbf{y}$.

In Python (3 lines):

```
from scipy.signal import dlsim
from scipy.optimize import curve_fit
# Or use the dedicated library:
import control
sys_id = control.tf(num, den, dt)  # fit to data

# Even simpler with MATLAB:
% data = iddata(y, u, Ts);
% sys = tfest(data, 2);  % fit 2nd-order TF
% bode(sys);  % inspect the result
```

MATLAB's System Identification Toolbox is the gold standard for this. The function `tfest(data, n)` fits an n-th order transfer function to recorded data automatically. Start with $n = 1$, increase until the fit is good (typically $n = 2$ is enough for motors).

5.7.4 Practical Sysid Workflow for Competition Robots

1. **Open-loop step test:** Disable all controllers. Send a known voltage step to the motor. Log encoder/IMU at maximum rate.

2. **Quick estimate:** Eyeball the step response for K and τ (first-order) or K, ζ, ω_n (second-order).

3. **Validate:** Simulate the identified model with the same step input. Overlay the simulation on the real data. If they match, you are done.

4. **Refine if needed:** If the first-order model is not accurate enough, try second-order. If there is a visible delay, add a time delay term $e^{-\tau_d s}$.

5. **Feed into controller design:** Use the identified model for PID tuning (MATLAB PID Tuner), LQR design (A, B matrices), or feedforward computation.

Common gotchas:

- **Friction:** Static friction (stiction) makes the system nonlinear at low speeds. Your linear model will be inaccurate near zero velocity. Consider identifying separate models for "moving" and "starting."

- **Backlash:** Gear backlash creates a dead zone. The system does not respond to small inputs until the gears engage. This shows up as a flat region in the step response.

- **Saturation:** If your step amplitude drives the motor into saturation, the identified model will be wrong. Use a step amplitude within the linear operating range (typically 30–70% of max voltage).
- **Coupling:** On a multi-axis system (gimbal, chassis), moving one axis can excite another. Identify each axis separately with the others locked if possible.

Why sysid matters: A controller designed for a model with 20% parameter error will have 20% worse performance than one designed for the true model. Five minutes of data logging + sysid can save hours of manual PID tuning. This is especially true for LQR and MPC, which depend heavily on accurate A and B matrices.

5.8 Feedforward Control

Feedback is reactive—it waits for an error to occur, then corrects. Feedforward is **proactive**—it predicts what the system needs and provides it immediately.

Example: You want your gimbal to track a target moving at constant velocity. With pure feedback, the controller is always chasing the target, resulting in a tracking error. With feedforward, you calculate the motor voltage needed to maintain that velocity and add it to the feedback output:

$$u(t) = u_{\text{feedforward}}(t) + u_{\text{feedback}}(t) \tag{55}$$

The feedforward term handles the "expected" part of the motion; feedback handles the "unexpected" part (disturbances, model errors).

Gravity compensation is a common feedforward term for robotic arms and gimbals:

$$u_{\text{ff}} = \tau_{\text{gravity}}(\theta) = mgl\sin(\theta) \tag{56}$$

Without this term, your feedback controller wastes effort fighting gravity instead of tracking the reference.

The golden rule of feedforward: Feedforward relies on the model, so it is only as good as the model. Feedback does not rely on the model, so it is always robust. **Use feedforward to improve performance, but always keep feedback as your safety net.**

Case Study: Cascaded PID on a Gimbal — Why Two Loops Beat One

You want your gimbal to point at a specific angle. The naive approach: one PID loop from angle error directly to motor voltage. This works, but it is mediocre. Here is why, and what the pros do instead.

The problem with single-loop control:

The transfer function from voltage to angle is second-order (motor dynamics + integration from velocity to position). A single PID must handle both the fast electrical/current dynamics and the slow mechanical dynamics simultaneously. It is like asking one person to drive the car AND navigate at the same time—doable but stressful.

The cascaded (nested) loop solution:

Use two PID controllers in series:

1. **Inner loop (velocity):** Runs at 1 kHz. Input: desired velocity. Output: motor voltage command. This loop is fast and rejects disturbances (friction, load changes) quickly.
2. **Outer loop (position):** Runs at 200 Hz. Input: desired angle. Output: desired velocity (which becomes the inner loop's setpoint). This loop handles the slower position tracking.

Why cascaded is better:

- Each loop handles one "layer" of dynamics. Simpler to tune—tune inner loop first (ignoring outer loop), then tune outer loop.
- The inner loop acts as a "shield"—it rejects disturbances before the outer loop even sees them. If someone bumps your gimbal, the velocity loop corrects within milliseconds.
- Limits are enforced at each level: the outer loop outputs a velocity command that is naturally bounded, preventing sudden large torques.

Adding feedforward to the mix:

Now add a velocity feedforward from the trajectory planner:

$$\omega_{\text{cmd}} = \underbrace{K_p(\theta_{\text{ref}} - \theta)}_{\text{outer PID}} + \underbrace{\dot{\theta}_{\text{ref}}}_{\text{feedforward}}$$

The feedforward gives the inner loop the velocity it needs to follow the reference trajectory. The outer PID only corrects small tracking errors. Result: smooth, accurate tracking even during fast motion. This is the architecture used by nearly every competitive RoboMaster gimbal.

Rule of thumb for cascaded loops:

- Inner loop should be 5–10× faster than outer loop.
- Tune from inside out: inner first, then outer.
- Each loop should look like a simple first-order system to the loop above it.

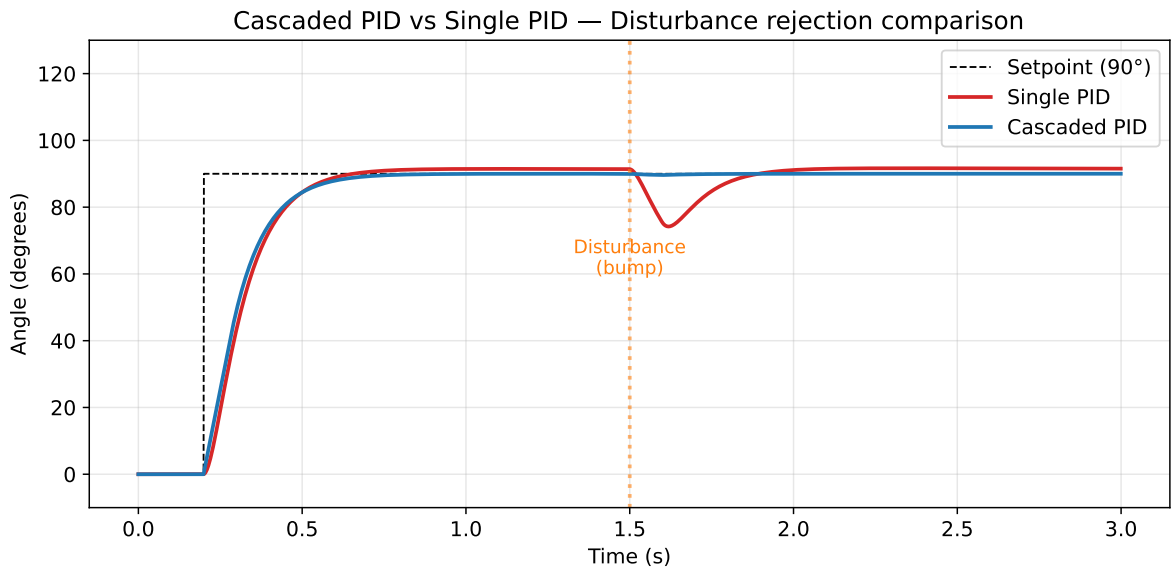

Figure 16: Cascaded PID (blue) vs single PID (red) on a gimbal motor. Both track the 90° setpoint, but when a disturbance hits at $t = 1.5$s, the cascaded architecture recovers much faster because the inner velocity loop absorbs the shock.

5.9 PID Architectures: Single, Cascaded, and Parallel-Cascaded

The case study above demonstrated why cascaded PID outperforms a single-loop PID on a single axis. In practice, competition robots require coordinated multi-axis control (e.g., gimbal yaw + pitch, chassis X + Y + rotation). This section systematically compares three PID architectures and provides ready-to-use code for each.

5.9.1 Architecture 1: Single-Loop PID

The simplest architecture: one PID controller per axis, from position error directly to motor voltage.

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau)\, d\tau + K_d \frac{de(t)}{dt}, \qquad e(t) = r(t) - y(t)$$

Advantages: Simple to implement, only one set of gains to tune per axis, minimal computational cost.

Limitations: The single controller must simultaneously handle fast dynamics (electrical/current) and slow dynamics (mechanical/position). This coupling makes tuning difficult—increasing gains for faster tracking often triggers oscillation. Disturbance rejection is slow because the controller only reacts after the disturbance propagates through the entire plant to the position output.

5.9.2 Architecture 2: Cascaded (Series) PID

Two nested loops per axis: the outer loop generates a velocity (or current) setpoint for the inner loop.

$$\omega_{\text{cmd}} = \text{PID}_{\text{outer}}\big(\theta_{\text{ref}} - \theta\big), \qquad u = \text{PID}_{\text{inner}}\big(\omega_{\text{cmd}} - \omega\big)$$

The main advantage is separation of concerns: each loop handles a single dynamic layer, making tuning straightforward. The inner loop intercepts disturbances before they propagate to the outer loop, dramatically improving disturbance rejection. And because the outer loop's output is a bounded velocity command (not a raw voltage), saturation handling comes for free.

These advantages translate into three concrete design rules. First, the inner loop bandwidth should be 5–10× that of the outer loop, so the outer loop "sees" the inner loop as an obedient, fast actuator. Second, from the outer loop's perspective, the inner loop should behave like a simple first-order system—well-tuned, fast, no overshoot. Third, always tune from inside out: stabilize the inner loop with the outer loop disabled, then close the outer loop.

5.9.3 Architecture 3: Parallel-Cascaded PID (Multi-Axis)

On a multi-axis system (e.g., a gimbal with yaw and pitch), each axis runs its own cascaded PID controller *in parallel*. The axes share sensor data and can optionally include cross-axis coupling compensation.

Why coupling matters: On a gimbal, pitching the camera shifts the center of mass, creating a torque disturbance on the yaw axis. Without compensation, the yaw controller sees this as an unexpected disturbance and reacts slowly. With a cross-coupling feedforward term, the yaw controller anticipates the pitch motion and compensates proactively:

$$u_{\text{yaw}} = \text{CascadedPID}_{\text{yaw}}(\cdot) + k_{\text{cross}} \cdot \dot{\theta}_{\text{pitch}}$$

```
// Parallel-cascaded PID with cross-coupling compensation
struct GimbalController {
    CascadedPID yaw;
    CascadedPID pitch;
    float cross_yaw_from_pitch = 0.0f;  // coupling gain

    struct Output { float yaw_voltage; float pitch_voltage; };

    Output update(float yaw_ref, float yaw_pos, float yaw_vel,
                  float pitch_ref, float pitch_pos, float pitch_vel,
                  float yaw_vel_ff = 0, float pitch_vel_ff = 0) {
        // Cross-coupling compensation: pitch rate disturbs yaw
        float cross_comp = cross_yaw_from_pitch * pitch_vel;

        float u_pitch = pitch.update(pitch_ref, pitch_pos,
                                     pitch_vel, pitch_vel_ff);
        float u_yaw   = yaw.update(yaw_ref, yaw_pos,
                                   yaw_vel, yaw_vel_ff)
                      + cross_comp;
        return {u_yaw, u_pitch};
    }

    void reset() { yaw.reset(); pitch.reset(); }
};
```

5.9.4 Comparison Summary

	Single PID	Cascaded PID	Parallel-Cascaded
Loops per axis	1	2	2
Tuning difficulty	Medium	Low (layered)	Low (layered)
Disturbance rejection	Slow	Fast	Fast
Cross-axis coupling	Not handled	Not handled	Compensated
Code complexity	Minimal	Moderate	Moderate
Typical use case	Simple actuators	Single-axis gimbal	Multi-axis gimbal

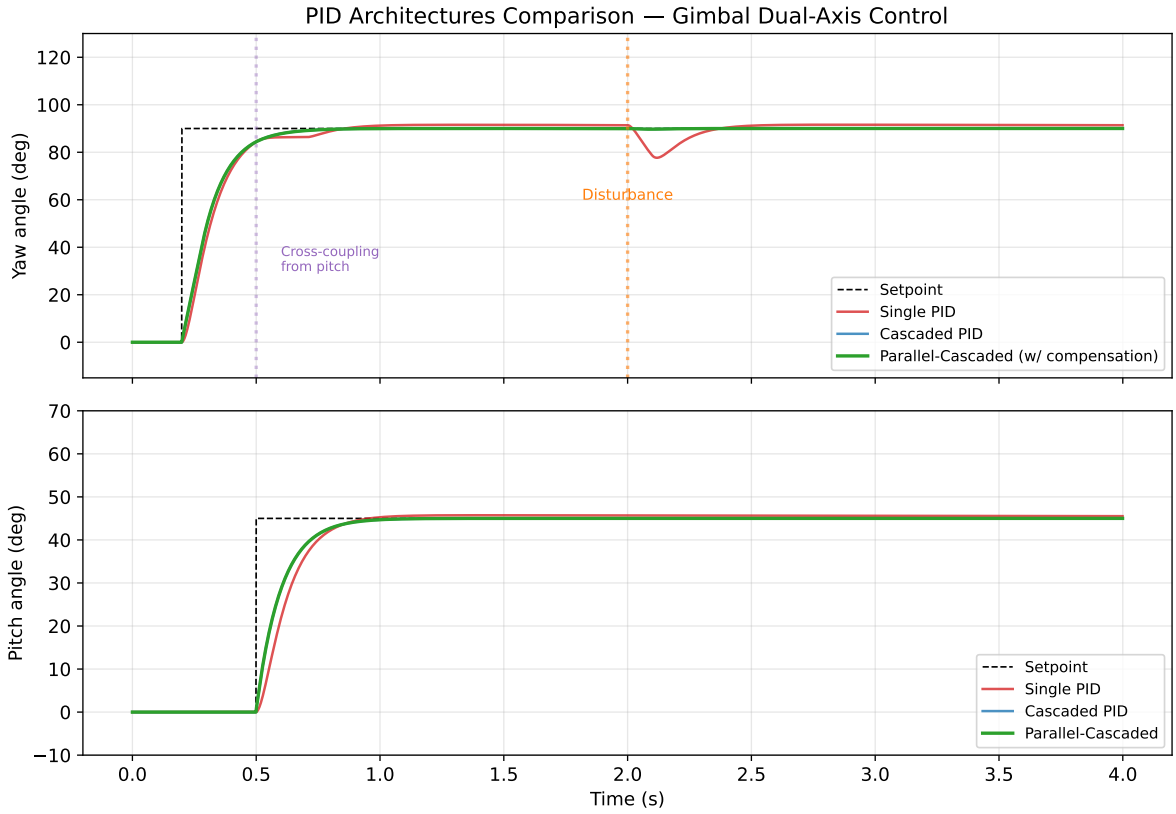

Figure 17: PID architecture comparison on a dual-axis gimbal. Top: yaw axis with disturbance at $t = 2$s and cross-coupling from pitch motion at $t = 0.5$s. Bottom: pitch axis. The parallel-cascaded architecture (green) with cross-coupling compensation provides the cleanest yaw response under both disturbance and coupling.

Practical recommendation: For any system with more than one controlled axis, use the parallel-cascaded architecture. The additional code complexity is minimal (a single struct wrapping two `CascadedPID` objects), while the performance gain from cross-coupling compensation and layered tuning is substantial.

Practical Example: Two-Wheeled Self-Balancing Scooter (Parallel PID)

A two-wheeled self-balancing robot (like a Segway) is a classic *parallel* multi-loop control problem. At least three objectives must be controlled simultaneously: balance (tilt angle), velocity, and yaw (steering). Each objective gets its own controller, and their outputs are *summed* at the motor level.

- **Balance loop (fast):** PD controller on tilt angle ϕ from the IMU. This is the most critical loop—it must run at ≥ 500 Hz and react within milliseconds. The output is a torque command to both wheels (same direction).
- **Velocity loop (medium):** PI controller on wheel velocity (encoder). The reference comes

from the user joystick. The output is added to the balance torque. This loop prevents the robot from drifting while maintaining balance.

- **Yaw loop (slow):** P controller on yaw rate (gyroscope). The output is a *differential* torque—positive to the left wheel, negative to the right (or vice versa)—to turn the robot.

Motor command fusion:

$$u_{\text{left}} = \underbrace{\tau_{\text{balance}}}_{\text{PD on } \phi} + \underbrace{\tau_{\text{velocity}}}_{\text{PI on } v} + \underbrace{\tau_{\text{yaw}}}_{\text{P on } \dot{\psi}}$$

$$u_{\text{right}} = \tau_{\text{balance}} + \tau_{\text{velocity}} - \tau_{\text{yaw}}$$

The sign flip on τ_{yaw} creates differential drive for steering. Each loop is tuned independently: balance first (the robot must stand before it can move), then velocity, then yaw.

5.9.5 Output Conflict in Parallel PID

When multiple PID controllers run in parallel and their outputs are summed into the same actuator, they can *fight each other*. This is the fundamental risk of parallel architectures.

The balancing scooter conflict. Consider the balance loop commanding "tilt forward to stay upright" while the velocity loop simultaneously commands "tilt backward to slow down." Both are correct in their own frame, but their summed output partially cancels, leaving the motor with a weak, confused command. In the worst case, the loops oscillate against each other: the velocity loop creates a tilt disturbance that the balance loop corrects, which changes velocity, which triggers the velocity loop again—an inter-loop limit cycle.

Why this happens: In a cascade, the outer loop *only* talks to the inner loop, and the inner loop *only* talks to the actuator. The hierarchy prevents conflict by construction. In a parallel architecture, all loops talk to the actuator directly, so their objectives must be *naturally compatible*. When they are not, the summed output becomes incoherent.

Mitigation strategies:

1. **Bandwidth separation.** Ensure each loop operates in a distinct frequency band: the fastest loop (balance) dominates at high frequencies, the slowest (yaw) only acts at low frequencies. Conflicting commands are then separated in time scale and do not interfere.
2. **Priority saturation.** Clamp the slower loops' outputs so they cannot override the faster, safety-critical loop. For example, limit $|\tau_{\text{velocity}}|$ so it never exceeds a fraction of $|\tau_{\text{balance}}|$.
3. **Output budget allocation.** Reserve a fixed portion of actuator range for each loop: e.g., 60% for balance, 30% for velocity, 10% for yaw. This guarantees that no loop can starve another.
4. **Convert to cascade.** If conflicts persist, restructure the controller so the velocity loop outputs a *tilt angle reference* for the balance loop, rather than a torque that competes with it. This eliminates the conflict by design—at the cost of increased coupling between loops.

Rule of thumb: parallel PID works well when the controlled variables are *physically orthogonal* (e.g., balance vs. yaw on a scooter—one is sagittal, the other is lateral). It struggles when two loops try to control variables that share the same actuator degree of freedom (e.g., both balance and velocity demand sagittal torque).

5.9.6 Food for Thought: Is Inner-Loop K_p the Same as Outer-Loop K_d?

Consider a position–velocity cascade. The outer loop outputs a velocity reference proportional to position error: $v_{\text{ref}} = K_p^{\text{outer}}(r - y)$. The inner loop then tracks this velocity. Now consider a single PID on position: the derivative term produces $K_d \cdot \dot{e} \approx -K_d \cdot \dot{y}$, which is a feedback proportional to velocity.

Are they equivalent? *Almost, but not quite.* In steady-state linear analysis, the outer K_p acting through an ideal inner velocity loop produces an effect similar to K_d on position error. Specifically, if the inner loop has unity gain and no dynamics (instantaneous velocity tracking), then:

$$u = K_p^{\text{inner}}\big(K_p^{\text{outer}}(r - y) - \dot{y}\big)$$

which expands to terms resembling $K_p \cdot e + K_d \cdot (-\dot{y})$ with $K_d \approx K_p^{\text{inner}}$.

But the cascade is superior in practice because:

1. The inner loop has its *own* integral term that eliminates velocity-level disturbances (friction, load changes) without relying on the outer loop.

2. The inner loop *bandwidth-limits* the outer command, preventing the outer loop from demanding physically impossible accelerations. A raw K_d has no such protection—it amplifies high-frequency noise and can saturate the actuator.
3. The inner loop provides a *measured* velocity signal, not a numerically differentiated one. Differentiation amplifies noise; direct measurement does not.

Bottom line: the derivative term and the inner proportional gain are conceptually related—both inject velocity-dependent damping. But the cascade architecture provides noise rejection, disturbance rejection at the velocity level, and implicit rate limiting that a bare K_d term cannot match.

6 Multi-Loop Architecture Design

In the previous chapter, we designed individual PID controllers. But a real competition robot does not run on a single control loop. A gimbal has a current loop, a velocity loop, and a position loop. A balance robot has a tilt loop and a velocity loop. A chassis has per-wheel velocity loops and an overall trajectory-tracking loop. This chapter teaches you how to stack these loops into a coherent architecture.

The difference between a robot that "sort of works" and one that wins matches often comes down to architecture. Two teams can use the exact same PID algorithm, the exact same motors, the exact same sensors—and one robot is rock-solid while the other oscillates and stutters. The winning team understood how loops fit together. That is what we cover here.

We will start with the big picture—the full control stack from sensor to actuator—then zoom into cascade control and the bandwidth separation principle that makes it work. After that, we discuss task-level versus joint-level control, supervisor state machines, and the timing architecture that keeps everything running on schedule. A detailed gimbal case study ties it all together at the end.

6.1 The Full Control Stack

Every competition robot, whether you realize it or not, implements a layered pipeline:

$$\text{Sensors} \longrightarrow \text{Perception} \longrightarrow \text{Planning} \longrightarrow \text{Control} \longrightarrow \text{Actuation}$$

Sensors produce raw data: encoder counts, IMU accelerations, current-sense ADC values. **Perception** turns raw data into useful state estimates: an EKF fusing IMU and encoders to produce a tilt angle, or a vision pipeline extracting target coordinates. **Planning** decides what the robot should do next: track a trajectory, aim at a target, maintain balance. **Control** computes the commands that make the plan happen: PID outputs, torque commands, PWM duty cycles. **Actuation** applies those commands to the physical world: motor drivers switching H-bridges, solenoids firing.

Consider a competition gimbal. The sensor layer reads a 6-axis IMU at 8 kHz and motor encoders at 10 kHz. The perception layer runs an EKF at 1 kHz to estimate yaw and pitch angles. The planning layer receives target coordinates from a vision co-processor at 120 Hz and computes desired gimbal angles. The control layer runs cascade PID (position → velocity → current) at various rates. The actuation layer is a three-phase motor driver switching at 20 kHz.

The key insight is that *each layer is a client of the layer below it.* The planning layer does not know or care how the motor driver works—it just says "go to 30 degrees." The position PID does not know or care about field-oriented control—it just outputs a velocity command. This separation of concerns is what makes complex robots manageable.

Where does each layer run? In a typical competition robot:

Layer	**Typical Rate**	**Hardware**
Current control	10–20 kHz	Motor driver MCU (e.g., C610/C620 ESC)
Velocity / position control	500 Hz–1 kHz	Main MCU (STM32F4/H7)
Perception (EKF)	500 Hz–1 kHz	Main MCU
Planning / trajectory	50–200 Hz	Main MCU or co-processor
Vision / detection	30–120 Hz	Co-processor (Jetson, RPi, etc.)

Notice the pattern: lower layers run faster, higher layers run slower. This is not a coincidence—it is a direct consequence of the physics, as we will see next.

6.2 Cascade Control Architecture

6.2.1 Current → Velocity → Position

The three-loop cascade is the workhorse of brushless motor control. Let us understand *why* the loops are ordered this way by examining the physics of a DC motor (the same argument applies to FOC-controlled BLDC motors with appropriate coordinate transforms).

The electrical dynamics are:

$$L\frac{di}{dt} = V - Ri - K_e\omega \tag{57}$$

where L is the winding inductance, R is resistance, K_e is the back-EMF constant, and ω is angular velocity. The time constant is $\tau_e = L/R$, typically 0.1–1 ms for small motors. This is the *fastest* dynamics in the system.

The mechanical dynamics are:

$$J\frac{d\omega}{dt} = K_t i - b\omega - \tau_{\text{load}} \tag{58}$$

where J is the rotor inertia, K_t is the torque constant, b is viscous friction, and τ_{load} is the external load. The mechanical time constant $\tau_m = J/b$ is typically 10–100 ms—much slower than the electrical dynamics.

Finally, the kinematic relationship is:

$$\frac{d\theta}{dt} = \omega \tag{59}$$

This is pure integration, which has no natural time constant at all—it is the "slowest" dynamics (an integrator's gain rolls off at -20 dB/decade starting from DC).

The cascade structure matches the physics: the **innermost** loop controls the **fastest** dynamics (current), the **middle** loop controls the next-fastest (velocity), and the **outermost** loop controls the slowest (position). Typical bandwidths:

Loop	**Bandwidth**	**Controller Type**
Current	5–10 kHz	PI (no D needed—plant is first-order)
Velocity	200 Hz–1 kHz	PID or PI
Position	50–200 Hz	PD or PID

The nested block diagram looks like this. The position controller outputs a velocity setpoint. The velocity controller outputs a current setpoint. The current controller outputs a voltage (PWM duty). Each inner loop "wraps around" its own plant dynamics, taming them before the outer loop ever sees them.

$$r_\theta \xrightarrow{C_\theta} r_\omega \xrightarrow{C_\omega} r_i \xrightarrow{C_i} V \xrightarrow{\text{motor}} i,\ \omega,\ \theta \tag{60}$$

6.2.2 The Bandwidth Separation Principle

The cascade architecture works because of a powerful simplification: if the inner loop is *much* faster than the outer loop, the outer loop can pretend the inner loop does not exist.

Let the inner closed-loop transfer function be:

$$T_{\text{inner}}(s) = \frac{C_{\text{inner}}G_{\text{inner}}}{1 + C_{\text{inner}}G_{\text{inner}}} \tag{61}$$

If the inner loop bandwidth ω_{inner} is much larger than the outer loop bandwidth ω_{outer}, then at all frequencies relevant to the outer loop ($\omega \ll \omega_{\text{inner}}$), we have $|T_{\text{inner}}(j\omega)| \approx 1$ and $\angle T_{\text{inner}}(j\omega) \approx 0$. In other words, $T_{\text{inner}}(s) \approx 1$ from the outer loop's perspective. The inner loop appears as a perfect, instantaneous actuator.

The derivation is straightforward. For a well-tuned inner loop with bandwidth ω_{inner}, the closed-loop behaves approximately as a first-order system:

$$T_{\text{inner}}(s) \approx \frac{1}{1 + s/\omega_{\text{inner}}} \tag{62}$$

At a frequency $\omega \ll \omega_{\text{inner}}$, the magnitude is:

$$|T_{\text{inner}}(j\omega)| = \frac{1}{\sqrt{1 + (\omega/\omega_{\text{inner}})^2}} \approx 1 - \frac{1}{2}\left(\frac{\omega}{\omega_{\text{inner}}}\right)^2$$

and the phase is $\angle T_{\text{inner}} \approx -\arctan(\omega/\omega_{\text{inner}}) \approx -\omega/\omega_{\text{inner}}$ radians. For this approximation to hold with less than 5% magnitude error and less than 10° phase lag, we need $\omega_{\text{outer}} < \omega_{\text{inner}}/5$ roughly. This gives us the:

Rule of thumb: The inner loop should be 5–10× faster than the outer loop.

What happens when this separation is violated? Suppose your position loop runs at 200 Hz bandwidth and your velocity loop runs at 300 Hz bandwidth—a ratio of only 1.5×. The position loop now "sees" significant phase lag from the velocity loop. At the position loop's crossover frequency, the velocity loop contributes perhaps 30–40° of additional phase lag. This eats into the position loop's phase margin, causing overshoot or outright oscillation. The two loops are *coupled*—tuning one affects the other, and the nice separation-of-concerns story breaks down.

This is a very common competition bug. The team tunes velocity and position loops separately, each looks fine alone, but together the robot oscillates. The fix is almost always to either speed up the inner loop or slow down the outer loop until the 5× separation is restored.

Worked example. Suppose your velocity loop has a closed-loop bandwidth of 300 Hz, and you set up a position loop with bandwidth 200 Hz. The ratio is only $300/200 = 1.5\times$. At the position loop's crossover frequency (200 Hz), the velocity loop's closed-loop transfer function contributes:

$$\angle T_{\text{vel}}(j \cdot 2\pi \cdot 200) \approx -\arctan\left(\frac{200}{300}\right) = -33.7$$

That is 34° of unexpected phase lag. If your position controller was designed with 45° of phase margin, you now have only $45 - 34 = 11°$—dangerously close to instability. The position loop overshoots violently and may oscillate. If instead you reduce the position loop bandwidth to 60 Hz (a 5× separation), the velocity loop contributes only $-\arctan(60/300) = -11.3°$ of phase lag. Your phase margin drops from 45° to a comfortable 34°, and the system is well-behaved.

6.2.3 Tuning Order: Inside Out

The bandwidth separation principle dictates a strict tuning protocol:

Step 1: Disable all outer loops. Send current commands directly. Tune the current PI controller until the current response is fast and well-damped (overshoot $< 5\%$, settling time < 1 ms). This is usually done by the motor driver manufacturer, so you often skip this step.

Step 2: Close the current loop. Now send velocity commands. The current loop is your "fast actuator"—you do not need to think about it. Tune the velocity PID until the velocity response is satisfactory (overshoot $< 10\%$, settling time < 20 ms). Use derivative-on-measurement to avoid setpoint kick.

Step 3: Close the velocity loop. Now send position commands. The velocity loop is your "medium-speed actuator." Tune the position PD or PID for the desired position response.

The critical rule: *never retune inner loops after closing outer loops*, unless something is fundamentally wrong (like the motor winding resistance changed because you swapped motors). If you find yourself retuning the velocity loop to fix a position-loop oscillation, the real problem is almost certainly a bandwidth separation violation.

A common mistake during competition crunch time: the position loop oscillates, so someone cranks down K_p on the velocity loop to "calm things down." This makes the velocity loop sluggish, which starves the position loop of bandwidth, which makes it even more oscillatory, which prompts someone to reduce position K_p as well, and now the whole system is slow and mushy. The correct procedure is to check the bandwidth ratios, not to randomly detune.

To verify your cascade is healthy, log the setpoints and measurements at each level simultaneously. Plot velocity setpoint vs actual velocity: the actual should track the setpoint with minimal delay and overshoot. If the velocity response is slow or ringy, fix it *before* touching the position loop. Only after the velocity loop looks crisp should you proceed to tune position.

6.3 Task-Level vs Joint-Level Control

There are two philosophies for controlling a multi-actuator robot.

Joint-level control gives each motor its own independent controller. Motor 1 has a PID that tracks its velocity setpoint. Motor 2 has its own PID. Motor 3 has its own PID. They do not talk to each other. This is simple, easy to debug, and works well when the motors are naturally decoupled—which is often the case for wheeled robots where each wheel has its own independent velocity setpoint.

Task-level control works in the *task space* (also called Cartesian space or operational space). The controller computes errors and commands in the space that matters for the task—for example, (v_x, v_y, ω) for a holonomic chassis—and then uses inverse kinematics or a Jacobian to map these commands to individual joint commands.

When do you use which? For a differential-drive chassis, joint-level control is perfectly fine: left wheel velocity and right wheel velocity map trivially to (v, ω). But for a holonomic (omnidirectional) chassis with four mecanum wheels, task-level control is almost essential.

Example: holonomic chassis inverse kinematics. Suppose you want the chassis to move with velocity (v_x, v_y, ω) in the body frame. With four mecanum wheels arranged in the standard X-configuration, each at distance l from the center (measured along the diagonal), the wheel velocities are:

$$\begin{bmatrix}\omega_1\\ \omega_2\\ \omega_3\\ \omega_4\end{bmatrix} = \frac{1}{r}\begin{bmatrix}1 & -1 & -(l_x+l_y)\\ 1 & 1 & (l_x+l_y)\\ 1 & 1 & -(l_x+l_y)\\ 1 & -1 & (l_x+l_y)\end{bmatrix}\begin{bmatrix}v_x\\ v_y\\ \omega\end{bmatrix} \tag{63}$$

where r is the wheel radius, l_x is the half-length (front-to-center), and l_y is the half-width (side-to-center). This matrix is the *inverse kinematics*—it maps task-space commands to joint-space commands. Each wheel then has its own velocity PID tracking the computed ω_i. The trajectory-tracking controller operates entirely in task space, oblivious to individual wheels.

The advantage of task-level control is that the trajectory planner thinks in the coordinates that matter: "move forward at 2 m/s while rotating at 1 rad/s." The inverse kinematics handles the messy details of how four wheels must coordinate to achieve that.

Note that the inverse kinematics matrix above is *not square*—it maps 3 task-space DOFs to 4 wheel velocities. The system is kinematically redundant (more actuators than DOFs). This redundancy is actually beneficial: it means the chassis can achieve any (v_x, v_y, ω) combination, and slight differences in wheel speeds due to manufacturing tolerances are absorbed gracefully. For systems where the number of actuators equals the number of DOFs (e.g., a 3-wheel omnidirectional robot), the matrix is square and invertible in both directions—you can also do *forward kinematics* to estimate the chassis velocity from measured wheel speeds, which is useful for odometry.

For robot arms and multi-DOF mechanisms—turrets with elevation and traverse, or sample-collecting manipulators—the task-level approach becomes essential. The operator or autonomous planner thinks "move the end-effector to (x, y, z)," and the inverse kinematics resolves this into joint angles. Attempting to control each joint independently in such systems leads to curved, unpredictable end-effector paths.

6.4 Supervisor Patterns

6.4.1 State Machines for Autonomous Operation

Control loops compute torques and velocities. But *who decides when the robot should be moving in the first place?* That is the job of the supervisor—a higher-level logic layer that manages the robot's operating mode.

The simplest and most robust supervisor is a **finite state machine** (FSM). A typical competition robot has states like:

$$\texttt{IDLE} \rightarrow \texttt{INITIALIZE} \rightarrow \texttt{RUN} \rightarrow \{\texttt{FAULT}, \texttt{E_STOP}\}$$

In `IDLE`, all actuators are disabled and the robot waits for a start signal (e.g., referee system command). In `INITIALIZE`, the robot performs self-checks: IMU calibration, encoder zeroing, CAN bus health check. Only after all checks pass does the robot transition to `RUN`, where the control loops are active. If any critical fault is detected (motor overtemperature, communication loss, low battery), the FSM transitions to `FAULT`, which disables actuators and logs the error. An emergency stop signal forces an immediate transition to `E_STOP` from any state.

Why does this matter? Without a state machine, a common competition disaster unfolds: the robot powers on, the PID loops start immediately, the IMU has not calibrated yet, so the "tilt angle" reads garbage, and the motors slam to full power. With a proper FSM, the robot cannot enter `RUN` until initialization is complete.

A minimal implementation in C:

```
typedef enum { IDLE, INIT, RUN, FAULT, ESTOP } State;
State state = IDLE;

void supervisor_update(void) {
    switch (state) {
        case IDLE:  if (start_signal()) state = INIT;  break;
        case INIT:  if (all_checks_ok()) state = RUN;  break;
        case RUN:   if (fault_detected()) state = FAULT; break;
        case FAULT: motors_disable(); break;
        case ESTOP: motors_disable(); break;
    }
```

```
    if (estop_pressed()) state = ESTOP;
}
```

Ten lines. Saves your robot from destroying itself at power-on.

6.4.2 Mode Switching and Bumpless Transfer

Competition robots often need to switch between control modes on the fly. A gimbal might switch from manual (operator stick input) to autonomous (vision tracking). A chassis might switch from teleoperation to autonomous navigation. The problem is that at the moment of switching, the new controller's output may differ from the old controller's output, causing a sudden "bump" (discontinuity) in the control signal. The motor jerks. The gimbal whips. The audience gasps.

The fix is called **bumpless transfer**. The idea is simple: when switching to a new controller, initialize its internal state so that its first output matches the last output of the old controller. For a PID controller, this means setting the integrator:

$$I_{\text{new}} = u_{\text{current}} - K_{p,\text{new}} \cdot e_{\text{current}} \tag{64}$$

where u_{current} is the control output at the moment of switching and e_{current} is the current error as seen by the new controller. This ensures that $u_{\text{new}} = K_{p,\text{new}}e + I_{\text{new}} = u_{\text{current}}$ at the first sample after switching. The new controller then smoothly takes over.

If the new controller also has a derivative term, you should also initialize the "previous measurement" variable to the current measurement, so the derivative does not produce a spike.

In competition, the most common mode switch is between operator control and autonomous aiming. During operator control, the gimbal tracks joystick input. When the vision system detects a target and the operator presses the "auto-aim" button, the controller switches to vision-guided position control. Without bumpless transfer, the gimbal jerks to wherever the vision controller's integrator happens to be initialized (usually zero). With bumpless transfer, the transition is seamless—the vision controller picks up exactly where the operator controller left off and smoothly steers toward the target.

6.5 Timing Architecture

All the loop-stacking theory is useless if your code does not execute at the right time. Timing is the skeleton that holds the architecture together.

Loop	Frequency	Priority	Timer / Trigger
Current PI	10 kHz	Highest	ADC end-of-conversion interrupt
Velocity PID	1 kHz	High	TIM6 (1 ms period)
Position PID	500 Hz	Medium	Every 2nd velocity tick
EKF update	1 kHz	Medium	Synchronized with velocity loop
Trajectory planning	100 Hz	Low	Every 10th velocity tick
Supervisor FSM	100 Hz	Low	Same tick as trajectory

Priority scheduling. The current loop is the highest priority because if it misses a deadline, the motor current is uncontrolled for an entire PWM cycle—potentially dangerous. The velocity loop is next. The position loop and planner are lowest priority because they change slowly and can tolerate occasional jitter.

Why jitter matters. Recall the PID derivative term: $D = K_d \cdot (y_{\text{prev}} - y)/\Delta t$. If Δt fluctuates because a higher-priority interrupt delayed your loop, the derivative output fluctuates too—even if the measurement is perfectly smooth. For a loop running at 1 kHz, a 10% jitter (± 0.1 ms) introduces 10% noise on the derivative. This is why deterministic timing is not optional—it is a correctness requirement.

There are two common approaches on microcontrollers:

Bare-metal (nested interrupts): Each loop runs in a timer interrupt. The current loop interrupt has the highest NVIC priority, the velocity loop has a lower priority, and so on. The hardware ensures that a high-priority interrupt can preempt a lower-priority one. This is simple and has minimal overhead, but gets messy when you have many loops with complex interactions.

RTOS (FreeRTOS, RT-Thread): Each loop is a task with an assigned priority. The RTOS scheduler handles preemption. This is cleaner for complex robots with many subsystems.

A minimal FreeRTOS setup:

```
void vel_loop_task(void *arg) {
    TickType_t last = xTaskGetTickCount();
    for (;;) {
        vTaskDelayUntil(&last, pdMS_TO_TICKS(1));  // 1 kHz
        float omega = read_encoder_velocity();
        float cmd = pid_update(&vel_pid, vel_setpoint, omega);
        set_current_setpoint(cmd);
    }
}
// In main(): xTaskCreate(vel_loop_task, "vel", 256, NULL, 3, NULL);
```

The `vTaskDelayUntil` function is critical: it ensures the task runs at a fixed period regardless of how long the computation takes (as long as it finishes within the period). Using `vTaskDelay` instead would accumulate drift—each iteration would be "computation time + 1 ms" instead of exactly 1 ms, and over thousands of iterations the phase shifts noticeably.

One last practical note on timing: be careful with CAN bus communication timing. If your main MCU sends current commands to motor ESCs over CAN at 1 kHz, and the CAN bus is also carrying IMU data, encoder data, and referee system packets, bus contention can cause unpredictable delays. Assign higher CAN priority (lower CAN ID) to time-critical messages like current commands, and consider staggering transmissions so that multiple high-rate messages do not collide on the same millisecond boundary.

Case Study: Complete Gimbal Control Stack—From Sensor to Actuator

Let us trace the full data flow of a competition gimbal pitch axis, from sensor to motor. This is the architecture running on many winning robots, and understanding it end-to-end is the goal of this chapter.

The pipeline:

1. **IMU** (BMI088) samples at 8 kHz. The raw gyroscope reading ω_{raw} passes through a first-order low-pass filter ($f_c = 200$ Hz) to remove high-frequency vibration.
2. **EKF** runs at 1 kHz. It fuses the filtered gyroscope with the accelerometer (corrected for centripetal acceleration) to produce an attitude estimate $\hat{\theta}$ (pitch angle) and a bias-corrected angular velocity $\hat{\omega}$.
3. **Position PID** runs at 500 Hz. It computes $e_\theta = \theta_{\text{cmd}} - \hat{\theta}$ and outputs a velocity command ω_{cmd}. This is typically a PD controller (or PID with a very small K_i), because the position plant already contains an integrator ($\dot{\theta} = \omega$).
4. **Velocity PID** runs at 1 kHz. It computes $e_\omega = \omega_{\text{cmd}} - \hat{\omega}$ and outputs a current command i_{cmd}. This uses derivative-on-measurement to avoid setpoint kick when the position controller changes ω_{cmd} abruptly.
5. **Current PI** runs at 10 kHz inside the motor ESC (C620). It computes $e_i = i_{\text{cmd}} - i_{\text{meas}}$ and outputs a PWM duty cycle. This is a PI controller (no D) because the electrical plant $L\,di/dt = V - Ri - K_e\omega$ is first-order in i when ω is treated as a slow disturbance.
6. **Motor driver** converts the duty cycle to three-phase voltages via space-vector modulation at 20 kHz switching frequency.

Timing summary:

Block	Rate	Bandwidth
Current PI	10 kHz	∼5 kHz
Velocity PID	1 kHz	∼200–500 Hz
Position PID	500 Hz	∼50–100 Hz
EKF	1 kHz	(not a controller, but updates state)

The bandwidth separation is clear: current is 10×–25× faster than velocity, and velocity is 4×–10× faster than position. Each outer loop safely treats its inner loop as approximately unity gain.

Key design decisions:

Why PI for current, not PID? The current plant is first-order (L/R dynamics). A PI controller

gives zero steady-state error for a step input and adds at most 90° of phase—which is enough. Adding a D term would amplify current-sensor noise (current sensors are noisy) with no benefit.

Why derivative-on-measurement in the velocity loop? The velocity setpoint ω_{cmd} changes in discrete jumps every time the position controller runs (every 2 ms). Differentiating the error would create a spike at every jump. Differentiating the measurement avoids this.

Why is the position loop the slowest? Because position is the integral of velocity, and the position controller's job is to generate *smooth* velocity commands. Running it faster than necessary would mean it reacts to every tiny noise blip in the attitude estimate, creating jittery velocity commands.

What happens when you skip the current loop? Some teams send PWM duty directly from the velocity PID, bypassing current control. This "works" in the sense that the motor spins, but you lose *torque linearity*. Without current control, the relationship between your command (duty cycle) and the actual torque ($\tau = K_t i$) depends on back-EMF, supply voltage, temperature, and load. Your velocity PID must compensate for all of these nonlinearities. The gain margin drops, robustness to load changes decreases, and you end up needing to retune whenever the battery voltage drifts. With a proper current loop, the velocity PID sees a nice linear actuator: "I asked for 2 A, I got 2 A." The inner loop absorbs the mess.

Practical tip: When debugging a cascade system, start from the inside out. First verify the current loop with a step response (oscilloscope on the current sensor). Then verify the velocity loop. Then the position loop. If the outer loop misbehaves, the problem is almost never in the outer loop itself—it is usually a poorly tuned inner loop or a bandwidth separation violation.

Numbers from a real system. On a well-tuned competition gimbal pitch axis, typical parameters are: position PD with $K_p = 20$, $K_d = 0.5$ (output in deg/s per deg error); velocity PID with $K_p = 30$, $K_i = 5$, $K_d = 0$ (output in mA per deg/s error); current PI handled by the C620 ESC internally. The position step response (10° step) shows $< 5\%$ overshoot and $\sim$80 ms settling time. The velocity step response (100 deg/s step) shows $< 10\%$ overshoot and $\sim$8 ms settling time. These numbers give a bandwidth ratio of roughly $1/(0.08) : 1/(0.008) \approx 12.5 : 125 = 1 : 10$—right in the sweet spot for bandwidth separation.

The architecture patterns in this chapter—cascade control, bandwidth separation, task-level vs joint-level, state machines, bumpless transfer, and timing discipline—are not exotic theory. They are the standard toolkit of every well-designed competition robot. Master them, and the "integration" phase of your project—where individually working subsystems come together into a single robot—will be smooth instead of catastrophic.

In the next chapter, we will move from continuous-domain design to discrete-time implementation: how to turn these beautiful transfer functions into code that runs on a microcontroller with finite word length and fixed sample rates.

7 Discretization and Implementation

You have designed a beautiful continuous-time controller on paper (or in MATLAB). Now you need to make it run on an STM32 at 1 kHz inside a timer interrupt. This section bridges the gap between theory and embedded reality.

7.1 From Continuous to Discrete

A continuous-time transfer function $C(s)$ must be converted to a discrete-time transfer function $C(z)$ before it can be implemented in code. There are several methods, each with trade-offs.

7.1.1 Zero-Order Hold (ZOH)

The ZOH method assumes the input is held constant between samples (which is exactly what a DAC—Digital-to-Analog Converter, the chip that turns your microcontroller's digital number into a voltage—does). It is the most physically accurate discretization method.

The idea: Between sampling instants, the control signal is constant. We compute exactly what the continuous system would do under this piecewise-constant input.

For a first-order system $G(s) = \frac{a}{s+a}$ with sampling period T_s:

$$G(z) = \frac{1 - e^{-aT_s}}{z - e^{-aT_s}} \tag{65}$$

Derivation sketch. The ZOH discretization computes $G(z) = (1 - z^{-1})\,\mathcal{Z}\left\{\frac{G(s)}{s}\right\}$. The continuous-time step response of $G(s)$ is $y(t) = 1 - e^{-at}$; sampling at $t = nT_s$ gives $y[n] = 1 - e^{-anT_s}$. Taking the $\mathcal{Z}$-transform and multiplying by $(1 - z^{-1})$ yields the result above.

When to use: When you want the discrete system to match the continuous system exactly at the sampling instants. MATLAB's `c2d(sys, Ts, 'zoh')` uses this method.

Downside: The conversion formulas can be complex for higher-order systems. Usually done with software tools, not by hand.

7.1.2 Tustin (Bilinear) Transform

The Tustin method replaces s with:

$$s \leftarrow \frac{2}{T_s} \cdot \frac{z-1}{z+1} \tag{66}$$

This is a simple algebraic substitution. You take your continuous transfer function, replace every s with the expression above, and simplify.

Example: A first-order low-pass filter $H(s) = \frac{\omega_c}{s+\omega_c}$ becomes:

$$H(z) = \frac{\omega_c T_s (z+1)}{(2 + \omega_c T_s)z + (\omega_c T_s - 2)} \tag{67}$$

Why Tustin is popular:

- Simple to compute by hand for low-order systems.
- Preserves stability: a stable continuous system maps to a stable discrete system.
- Good frequency-domain match at low frequencies.

Gotcha: At high frequencies (near the Nyquist frequency), Tustin introduces **frequency warping**—the discrete filter's cutoff frequency is shifted relative to the continuous design. Use **pre-warping** to correct this:

$$\omega_{\text{pre-warped}} = \frac{2}{T_s} \tan\left(\frac{\omega_c T_s}{2}\right) \tag{68}$$

Intuition. The bilinear transform maps the entire continuous frequency axis $\omega \in [0, \infty)$ onto a finite arc of the unit circle $\omega_d \in [0, \pi/T_s)$, compressing high frequencies. The exact relationship is $\omega = \frac{2}{T_s} \tan \frac{\omega_d T_s}{2}$. Pre-warping inverts this mapping: we feed the bilinear transform a "stretched" frequency so that our target frequency ω_c lands at exactly the right place after the compression.

Replace ω_c with $\omega_{\text{pre-warped}}$ in your continuous design, then apply Tustin.

7.1.3 Matched Pole-Zero Method

This method maps the poles and zeros of the continuous system to the z-domain using $z = e^{sT_s}$. It preserves the time-domain response characteristics well.

When to use: For systems where matching the transient response is more important than matching the frequency response. Less common in practice than ZOH or Tustin.

Bottom line: For most competition applications, **Tustin is the go-to method**. It is simple, stable, and accurate enough. Use ZOH when you need exact sample-instant matching. If you are using MATLAB, just use `c2d()` and let it handle the math.

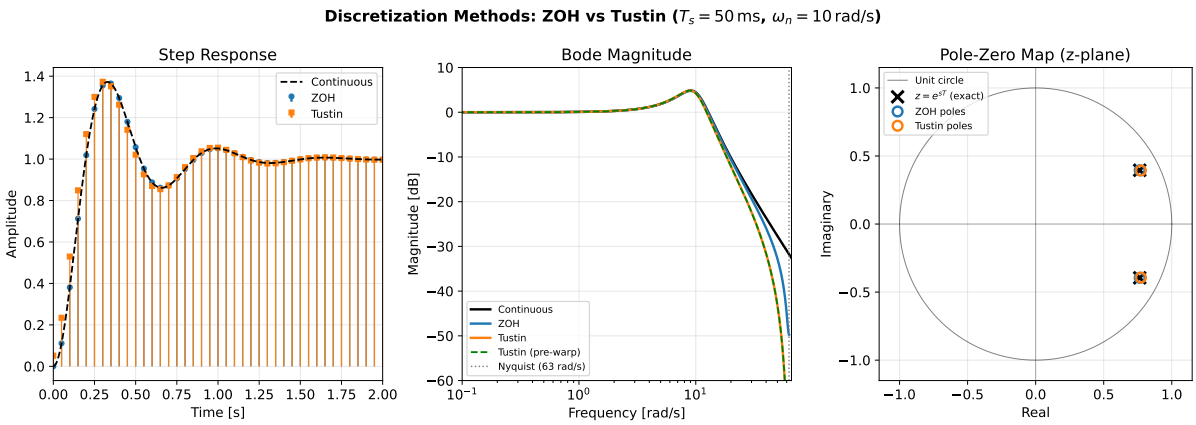

Figure 18: ZOH vs Tustin discretization of an underdamped 2nd-order system ($\omega_n = 10$ rad/s, $\zeta = 0.3$, $T_s = 50$ ms). Left: step response—ZOH matches the continuous system exactly at sample instants. Center: Bode magnitude—Tustin shows frequency warping near the Nyquist frequency; pre-warping (green) corrects this. Right: pole-zero map showing how each method maps continuous poles to the z-plane.

7.2 Digital Filter Structures: FIR vs. IIR

Now that we know how to convert continuous-time designs into discrete-time transfer functions, the natural next question is: what does the resulting digital filter actually look like in code? There are two fundamental structures: **FIR** and **IIR**. Understanding the difference is essential for choosing the right implementation on your microcontroller.

7.2.1 FIR (Finite Impulse Response)

An FIR filter computes the output as a weighted sum of the current and past input samples only—no feedback:

$$y[n] = \sum_{k=0}^{M} b_k \, x[n-k] \tag{69}$$

In the z-domain:

$$H(z) = \sum_{k=0}^{M} b_k \, z^{-k} \tag{70}$$

FIR filters have **only zeros** (no poles except trivially at $z = 0$), which gives them two important properties:

- **Always stable**—no feedback means no risk of runaway oscillation.
- **Linear phase** is achievable by making the coefficients symmetric ($b_k = b_{M-k}$). This means every frequency component is delayed by the same amount, so the signal shape is perfectly preserved.

The price you pay is **filter order**: to get a sharp cutoff you may need $M = 50$–200 taps, which means storing that many past samples and performing that many multiplications per output sample.

Common FIR design methods:

- **Window method**—truncate the ideal impulse response and apply a window function (Hamming, Blackman, Kaiser) to reduce spectral leakage (energy "spilling" from one frequency bin into neighboring bins due to the abrupt truncation of the impulse response).
- **Frequency sampling**—specify the desired magnitude response at DFT sample points and inverse-transform.
- **Parks–McClellan (Remez)**—an optimal algorithm that minimizes the maximum approximation error (equiripple design).

7.2.2 IIR (Infinite Impulse Response)

An IIR filter uses *both* past inputs and past outputs—it has a feedback path:

$$y[n] = \sum_{k=0}^{M} b_k\, x[n-k] \;-\; \sum_{k=1}^{N} a_k\, y[n-k] \tag{71}$$

In the z-domain:

$$H(z) = \frac{\displaystyle\sum_{k=0}^{M} b_k\, z^{-k}}{1 + \displaystyle\sum_{k=1}^{N} a_k\, z^{-k}} = \frac{B(z)}{A(z)} \tag{72}$$

IIR filters have **both poles and zeros**. The poles enable sharp frequency selectivity with very low order (a 4th-order IIR can outperform a 50th-order FIR), but they come with risks:

- **Potentially unstable**—if any pole lies outside the unit circle ($|z| > 1$), the output grows without bound.
- **Nonlinear phase**—different frequencies are delayed by different amounts, which distorts the signal shape.
- **Coefficient sensitivity**—on fixed-point microcontrollers, rounding the coefficients can shift poles enough to cause instability.

Classical IIR design starts from an analog prototype and maps it to discrete time using the methods we just covered (Tustin, ZOH, etc.):

- **Butterworth**—maximally flat passband, monotonic rolloff. The "vanilla" choice; sounds boring but is the right default 80% of the time.
- **Chebyshev Type I**—equiripple in the passband, monotonic in the stopband. Sharper transition than Butterworth for the same order, at the cost of passband ripple.
- **Chebyshev Type II**—monotonic in the passband, equiripple in the stopband. Less common but useful when passband flatness matters more.
- **Elliptic (Cauer)**—equiripple in *both* bands. Gives the sharpest possible transition for a given filter order, but with ripple everywhere.

Note that the bilinear transform and impulse invariance (a method that samples the analog filter's impulse response at the discrete time instants, preserving the time-domain shape but risking aliasing at high frequencies) methods described in the previous subsection are precisely the tools used to convert these analog prototypes into discrete IIR filters.

7.2.3 FIR vs. IIR: Head-to-Head Comparison

Property	FIR	IIR
Stability	Always stable	Must verify (poles inside unit circle)
Phase	Linear phase achievable	Nonlinear phase
Order for sharp cutoff	High (50–200+)	Low (4–10 typical)
Computation per sample	$M + 1$ multiplies	$M + N + 1$, but M, N are small
Latency	Higher (longer filter)	Lower
Design from analog prototype	No	Yes (Butterworth, Chebyshev, Elliptic)
Coefficient sensitivity	Low	High

7.2.4 Practical Guidelines and Biquad Implementation

- Need **linear phase** (audio, symmetric pulse shaping, communications)? Use **FIR**.
- Need a **sharp cutoff with minimal computation** (real-time control loops, resource-constrained MCUs)? Use **IIR**.
- Need **guaranteed stability** without careful analysis? Use **FIR**.
- For higher-order IIR filters, **always** implement as cascaded second-order sections (biquads) to reduce coefficient sensitivity:

$$H(z) = \prod_{i=1}^{L} \frac{b_{0i} + b_{1i}\, z^{-1} + b_{2i}\, z^{-2}}{1 + a_{1i}\, z^{-1} + a_{2i}\, z^{-2}} \tag{73}$$

Each section is a "biquad"—a second-order IIR filter that is simple to implement and numerically well-behaved. Cascading L biquads gives you a $2L$-th order filter while keeping coefficient quantization effects local to each section.

Example: a biquad in C (transposed direct-form II)

```
// One biquad section --- two state variables, numerically stable
float biquad(float x, float *state,
             float b0, float b1, float b2,
             float a1, float a2) {
    float y = b0*x + state[0];
    state[0] = b1*x - a1*y + state[1];
    state[1] = b2*x - a2*y;
    return y;
}
```

Quick Decision Flowchart

1. Is the application phase-sensitive (e.g., symmetric waveform matters)? → **FIR** with symmetric coefficients.
2. Is real-time computation tight (MCU running a fast control loop)? → **IIR** (Butterworth or Chebyshev), implemented as cascaded biquads.
3. Is the filter order moderate and your platform has enough memory? → Either works; default to **FIR** for simplicity and robustness.
4. Do you need the sharpest possible transition band for a given order? → **IIR** with an Elliptic prototype.

Rule of thumb for robotics: Most sensor-smoothing filters on embedded systems are low-order IIR (often just 1st or 2nd order Butterworth). FIR filters shine in audio processing, communications, and any application where linear phase is non-negotiable.

7.3 Fixed-Point vs Floating-Point Arithmetic

Many microcontrollers used in competitions (STM32F1, STM32F4) have hardware floating-point units (FPU). If yours does, **use floating-point**. It is easier, less error-prone, and fast enough for most control loops.

If you are stuck with a microcontroller without an FPU, or if you need maximum speed, you will need **fixed-point arithmetic**. The idea is to represent fractional numbers as scaled integers:

$$\text{fixed_value} = \text{float_value} \times 2^Q \tag{74}$$

where Q is the number of fractional bits. For example, in Q16 format, the number 3.14 is stored as $3.14 \times 65536 = 205,888$.

Pitfalls:

- Overflow: multiplication of two Q16 numbers produces a Q32 result that must be shifted back.
- Range vs. precision trade-off: more fractional bits means more precision but less range.
- Integral accumulation: the integral term in PID can overflow quickly in fixed-point.

Recommendation: Use 32-bit float unless you have a strong reason not to. STM32F4 and above can do single-precision float in one clock cycle.

Why Computation Speed Directly Determines Control Performance

Many students assume that as long as the algorithm is "correct," the control will work. This overlooks a critical fact: *every control algorithm must finish executing within the sampling period* T_s. If the computation takes longer than T_s, the controller misses its deadline, the next cycle is delayed, and the effective control frequency drops. In the worst case, the system becomes unstable.

A concrete example. Suppose your velocity loop runs at 1 kHz ($T_s = 1$ ms) on an STM32F407 at 168 MHz. How much time do different algorithms consume?

Algorithm	Complexity	Approx. time	Fits in 1 ms?
PID (3 multiplies + adds)	$O(1)$	$\sim$1 μs	Yes (1000$\times$ margin)
2nd-order IIR filter	$O(1)$	$\sim$1 μs	Yes
Kalman filter (4×4)	$O(n^3)$, $n = 4$	$\sim$20 μs	Yes (50$\times$ margin)
LQR (gain multiply, 4×1)	$O(n \cdot m)$	$\sim$2 μs	Yes
MPC ($N = 10$, $n = 4$, $m = 2$)	$O(N \cdot n^3)$	$\sim$200–500 μs	Barely
EKF with trig functions	$O(n^3)$ + trig	$\sim$50–100 μs	Yes, but watch out

What happens when computation overruns T_s:

- **Missed update:** The actuator holds its previous command for an extra cycle. The system sees this as an increased time delay. Recall from the shower analogy (Section 4.1): more delay means less phase margin, which means closer to instability.
- **Inconsistent dt:** If the overrun is intermittent (computation is sometimes fast, sometimes slow), the effective dt fluctuates. The integral and derivative terms—which assume a constant dt—accumulate errors. This is *jitter*, and it is often more damaging than a constant delay because it is unpredictable.
- **Cascading failure:** In a cascaded architecture, if the inner loop overruns, the outer loop receives stale velocity data. The outer loop's response degrades, and disturbance rejection—the primary reason for using cascaded control—is compromised.

Practical diagnostics. Toggle a GPIO pin (General-Purpose Input/Output—a digital pin you can set high or low from code) at the start and end of your ISR (Interrupt Service Routine—the function the hardware calls automatically at each timer tick), and observe the pulse width on an oscilloscope. If the pulse width is close to T_s, you are in danger. If it ever exceeds T_s, you have a deadline violation that *must* be fixed before tuning.

What to do when computation is too slow:

1. **Reduce algorithm complexity.** Replace MPC with LQR if the constraints are not active. Replace EKF with a complementary filter if the nonlinearity is mild.

2. **Precompute offline.** LQR gains, lookup tables for trigonometric functions, and MPC gain sequences can all be computed in MATLAB/Python and stored as constants.
3. **Lower the control frequency.** If the system's bandwidth allows it, running at 500 Hz instead of 1 kHz doubles your computation budget. Check whether your system's time constant permits this (recall: $T_s \leq \tau/10$).
4. **Distribute across loops.** Run the expensive outer loop at a lower frequency (e.g., MPC at 100 Hz) while keeping a simple inner loop (PID at 1 kHz). The inner loop maintains control between MPC updates.

The key insight: The best algorithm is not the most sophisticated one—it is the one that reliably finishes within T_s while meeting performance requirements. A perfectly tuned PID running at 1 kHz will always outperform an MPC that overruns its deadline at 200 Hz.

7.4 Sampling Rate Selection

How fast should your control loop run? Too slow and you cannot react to fast disturbances; too fast and you waste CPU time (and the derivative term amplifies quantization noise).

Rule of thumb: The sampling rate should be 10–30× the bandwidth of the closed-loop system.

Application	Bandwidth	Sampling rate
Chassis velocity	∼5 Hz	50–200 Hz
Gimbal position	∼20 Hz	200–1000 Hz
Motor current loop	∼500 Hz	5–20 kHz

Practical constraint: Your sampling rate is bounded by:

- Sensor update rate (no point sampling faster than your encoder/IMU updates).
- Computation time (all control math must finish within one sampling period).
- Communication latency (CAN bus, SPI, I2C can be bottlenecks).

Key insight: For cascaded control loops (e.g., position → velocity → current), the inner loop should run 5–10× faster than the outer loop. Current loop at 10 kHz, velocity loop at 1 kHz, position loop at 200 Hz is a common configuration.

7.5 Implementation on Embedded Systems

7.5.1 Timer and Interrupt Configuration

On most microcontrollers (STM32, etc.), the control loop runs inside a **timer interrupt**. The timer triggers at a fixed period (T_s), and the interrupt service routine (ISR) executes the control algorithm.

Best practices:

- Use a dedicated hardware timer for each control loop.
- Set the timer period to your desired T_s using the prescaler and auto-reload register.
- Keep the ISR **as short as possible**. Read sensor, compute output, set actuator. No printing, no UART, no memory allocation.
- Use `volatile` for variables shared between the ISR and the main loop.
- Set interrupt priorities correctly: current loop > velocity loop > position loop.

Example (STM32 HAL):

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM6) {  // 1 kHz control loop
        float measurement = read_encoder();
        float error = setpoint - measurement;
        float output = pid_compute(&pid, error);
        set_motor_pwm(output);
    }
}
```

7.5.2 Computation Latency and Jitter

Latency is the time between reading the sensor and applying the output. Lower is better. Anything above $T_s/2$ starts degrading performance significantly.

Jitter is the variation in latency from one cycle to the next. If your loop usually takes 10 μs but occasionally takes 500 μs (because another interrupt preempted it), you get inconsistent behavior. Jitter is often more harmful than a constant latency.

How to minimize jitter:

- Give your control timer the highest interrupt priority.
- Avoid disabling interrupts globally for long periods.
- Do not use blocking waits (delay, polling) in any ISR.
- Profile your ISR execution time and make sure it fits comfortably within T_s.

7.5.3 Anti-Windup in Practice

We discussed anti-windup in the PID section, but it deserves special emphasis here because it is the #1 source of "mysterious" PID misbehavior on real hardware.

When does windup happen in practice?

- Motor current limit is reached (you are commanding 20A but the ESC caps at 10A).
- PWM is saturated (duty cycle at 100%, cannot go higher).
- Mechanical hard stop (gimbal hits its limit, wheels are stalled).
- Communication failure (command sent but not received—output is effectively zero).

Implementation checklist:

1. Clamp the integral term to a reasonable range.
2. Clamp the total output to the actuator's actual limits.
3. When the output is saturated, stop accumulating the integral (conditional integration).
4. When switching controllers or modes, reset or smoothly transition the integral term.
5. Test windup recovery explicitly: command a large step, let it saturate, then reverse.

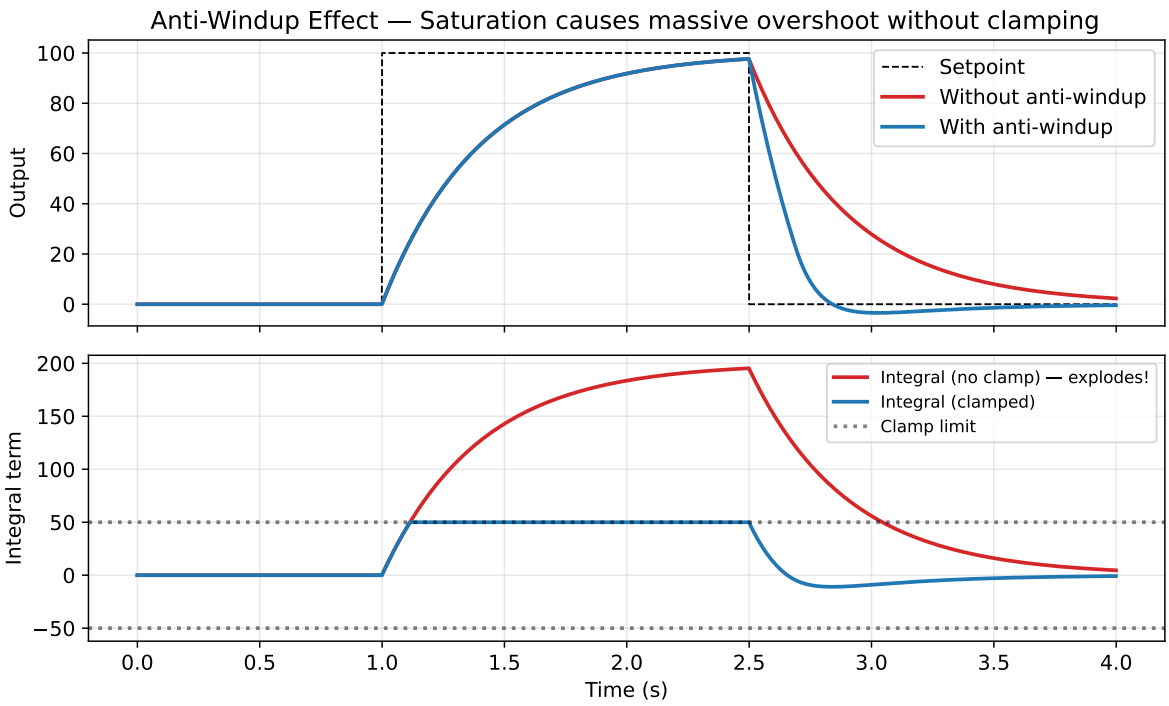

Figure 19: Without anti-windup (red), the integral term explodes during saturation, causing massive overshoot when the setpoint changes. With clamping (blue), the integral stays bounded and the response is clean.

A debugging story: Your robot's wheel motor oscillates wildly after recovering from a stall. You check K_p, K_d—they look fine. You try reducing gains—it helps but makes the system sluggish. The real cause? The integral term accumulated to a huge value during the stall and is now overdriving the motor in the opposite direction. Adding integral clamping fixes it instantly. This happens to everyone. It will happen to you. Now you know what to look for.

Putting It All Together: From Continuous PID to STM32 Code

This example walks through the complete pipeline: start with a continuous-time PID, discretize it, and implement it in an STM32 timer interrupt.

Step 1: Continuous PID. The textbook PID transfer function is:

$$C(s) = K_p + \frac{K_i}{s} + K_d s$$

With $K_p = 6$, $K_i = 0.3$, $K_d = 3$, and $T_s = 1\,\text{ms}$ (1 kHz control loop).

Step 2: Tustin discretization. Apply the bilinear transform $s \leftarrow \frac{2}{T_s}\frac{z-1}{z+1}$ to each term:

- **Integral:** $\frac{K_i}{s}$ $\rightarrow$ `integral += Ki * Ts * 0.5 * (e + e_prev)` (trapezoidal rule)
- **Derivative:** $K_d s$ $\rightarrow$ `deriv = Kd * (e - e_prev) / Ts` (backward difference on error, or better: on measurement)

Step 3: STM32 implementation.

```
// In TIM6 interrupt handler, every 1ms:
void TIM6_DAC_IRQHandler(void) {
    HAL_TIM_IRQHandler(&htim6);

    float y = read_encoder();        // measured output
    float r = get_setpoint();        // reference
    float e = r - y;

    // Tustin integral (trapezoidal)
```

```
    integral += Ki * Ts * 0.5f * (e + e_prev);
    integral = clamp(integral, -IMAX, IMAX);  // anti-windup

    // Derivative on measurement (avoids derivative kick)
    float dy = (y - y_prev) / Ts;

    // PID output
    float u = Kp * e + integral - Kd * dy;
    u = clamp(u, -UMAX, UMAX);

    set_pwm(u);  // actuator output

    e_prev = e;
    y_prev = y;
}
```

Key implementation details:

1. **Derivative on measurement** ($-K_d\dot{y}$), not on error ($K_d\dot{e}$). A step change in r creates an infinite $\dot{e}$ spike that slams the actuator. Using $\dot{y}$ avoids this "derivative kick."
2. **Integral clamping** prevents windup when the actuator saturates.
3. **Output clamping** ensures the PWM stays within hardware limits.
4. **No floating-point in critical path** on MCUs without FPU—use fixed-point Q16 or Q24 arithmetic instead.
5. **All state variables** (`integral`, `e_prev`, `y_prev`) are `static` or file-scope globals, persisting across ISR calls.

8 Simulation Workflow

You have a discretized controller and a mathematical model of your plant. Before burning the code into flash and risking a $200 motor, why not test it on your laptop first? This chapter shows you how to build simulations that catch 90% of bugs before they become hardware damage.

We will start with the case for simulation, then walk through three levels of testing fidelity—Model-in-the-Loop (MIL), Software-in-the-Loop (SIL), and Hardware-in-the-Loop (HIL)—before showing how to inject realism into your models. A copy-paste Python template and a balance-robot case study wrap things up.

8.1 Why Simulate Before Deploying

Hardware debugging is expensive in every dimension that matters during competition season. Broken gears cost money. Burned motor windings cost money *and* a two-week shipping delay. Even when nothing breaks, every hour spent staring at an oscilloscope is an hour not spent tuning your auto-aim or refining your chassis path planner.

Simulation cannot replace hardware testing, but it dramatically reduces the number of hardware iterations you need:

What simulation CAN tell you:

- **Stability.** If the closed-loop poles are in the right half of the z-plane (or equivalently outside the unit circle), your gains are wrong. Simulation catches this instantly; hardware catches it by destroying a gearbox.
- **Approximate performance.** Rise time, overshoot, settling time will be within 20–40% of reality for a decent model.
- **Gain sensitivity.** Sweep a parameter ±50% and see if the controller still works. If simulation shows marginal stability at +30% inertia, your real robot will oscillate the first time someone bolts on a heavier armor plate.

What simulation CANNOT tell you:

- Real friction (stiction, Coulomb, temperature-dependent).
- Backlash in geartrains—unless you explicitly model it.
- Exact sensor noise characteristics (quantization, drift, outliers).
- True communication latency (CAN bus arbitration, DMA jitter).

The simulation-reality gap is unavoidable. Accept it. The goal is not a perfect simulation; the goal is a simulation good enough that your first hardware test produces "it basically works, just needs gain tweaking" instead of "the motor spun backwards and ripped the wiring harness off."

8.2 Model-in-the-Loop (MIL)

MIL is the simplest and most common simulation approach: you write both the plant model and the controller in the same language (Python, MATLAB, or even C), run them in a loop, and look at the results.

8.2.1 Building a DC Motor + PID Simulation from Scratch

We will use the simplified DC motor model from Chapter 3. Neglecting inductance (valid when the electrical time constant $\tau_e = L/R$ is much smaller than the control period T_s), the mechanical dynamics are:

$$J\frac{d\omega}{dt} = \frac{K_t}{R}\big(V - K_e\omega\big) - b\,\omega \tag{75}$$

where J is rotor inertia, K_t the torque constant, K_e the back-EMF constant, R the winding resistance, b the viscous friction coefficient, V the applied voltage, and ω the angular velocity.

Discretize with forward Euler (T_s = sampling period):

$$\omega[n+1] = \omega[n] + T_s \cdot \frac{1}{J}\left[\frac{K_t}{R}\big(V[n] - K_e\,\omega[n]\big) - b\,\omega[n]\right] \tag{76}$$

Pair this with the discrete PID from Chapter 4:

$$u[n] = K_p\,e[n] + K_i\,T_s \sum_{k=0}^{n} e[k] + \frac{K_d}{T_s}\big(e[n] - e[n-1]\big) \tag{77}$$

Here is a complete Python simulation:

```
import numpy as np
import matplotlib.pyplot as plt

# Plant parameters (small brushed DC motor)
J, b, Kt, Ke, R = 0.01, 0.1, 0.01, 0.01, 1.0
Ts, N = 0.001, 2000          # 1 kHz, 2 seconds
omega = np.zeros(N)
# PID gains
Kp, Ki, Kd = 5.0, 10.0, 0.05
e_int, e_prev = 0.0, 0.0
omega_ref = 100.0             # rad/s step reference

for n in range(N - 1):
    e = omega_ref - omega[n]
    e_int += e * Ts
    e_dot = (e - e_prev) / Ts
    V = Kp * e + Ki * e_int + Kd * e_dot
    V = np.clip(V, -24.0, 24.0)          # actuator saturation
    domega = ((Kt / R) * (V - Ke * omega[n]) - b * omega[n]) / J
    omega[n + 1] = omega[n] + Ts * domega
    e_prev = e

t = np.arange(N) * Ts
plt.plot(t, omega); plt.axhline(omega_ref, ls='--', c='r')
plt.xlabel('Time (s)'); plt.ylabel('omega (rad/s)')
plt.title('DC Motor PID Step Response'); plt.grid(); plt.show()
```

Twenty-five lines and you have a working simulation. Run it, and you will see a step response that overshoots slightly and settles in about 0.5 s. If it diverges, your gains are wrong—better to learn that on a laptop than on a $200 motor.

8.2.2 Comparing Simulation with Theory

After collecting the step response, measure the rise time t_r, overshoot M_p, and settling time t_s directly from the simulation data. Then compare with the second-order approximation formulas from Chapter 3:

$$M_p = e^{-\pi\zeta/\sqrt{1-\zeta^2}} \tag{78}$$

$$t_s \approx \frac{4}{\zeta\omega_n} \tag{79}$$

If the numbers match within 10–15%, your simulation is consistent with the linearized model. If they do not match:

1. Your simulation might have a bug (check units, check signs).
2. The closed-loop system is not well-approximated by second order (dominant poles are not a clean conjugate pair).
3. Actuator saturation is clipping the response and invalidating the linear analysis.

The second-order formulas are approximations, not ground truth. When in doubt, trust the simulation—it includes nonlinearities (like saturation) that the formulas ignore.

8.3 Software-in-the-Loop (SIL)

MIL is great for design exploration, but it has a blind spot: your competition robot does not run Python. It runs C on an STM32, with fixed-point arithmetic (or at least 32-bit float), specific integer types, and ISR-based timing.

The idea behind SIL is simple: **compile your actual embedded C controller code and run it against the Python plant model.** This catches an entire class of bugs that MIL misses:

- Fixed-point overflow (your integral term wraps around at 65535).
- Integer division truncation (`int16_t` gains lose precision).
- Variable type mismatches (`uint8_t` holding a value that should be `int32_t`).
- ISR timing assumptions that do not hold.

The implementation is straightforward. Compile your C controller into a shared library and call it from Python using `ctypes`:

```
import ctypes

# Compile: gcc -shared -o pid.so -fPIC pid.c
lib = ctypes.CDLL('./pid.so')
lib.pid_update.argtypes = [ctypes.c_float, ctypes.c_float]
lib.pid_update.restype  = ctypes.c_float
lib.pid_init()

for n in range(N - 1):
    V = lib.pid_update(ctypes.c_float(omega_ref),
                       ctypes.c_float(omega[n]))
    # ... plant update same as before ...
```

Now the PID running in your simulation is byte-for-byte identical to the one that will run on your STM32. If the integral term overflows in C, it overflows in your simulation too—and you catch it before competition day.

When to skip SIL: If your controller is pure Python or MATLAB (e.g., you are doing rapid prototyping and have not written the embedded code yet), MIL is sufficient. SIL only adds value when you have actual C/C++ code to test.

8.4 Hardware-in-the-Loop (HIL)

HIL takes things one step further: your **real MCU** runs your **real code**, but the plant is still simulated on a PC. The PC and MCU communicate via UART, SPI, or CAN—the PC receives the control output from the MCU, simulates the plant dynamics for one time step, and sends back simulated sensor readings.

The setup:

1. MCU boots and starts its control loop normally.
2. Instead of reading a real encoder, it reads a value sent by the PC over UART.
3. It computes the PID output and sends the PWM duty cycle (or voltage command) back to the PC.
4. The PC updates the plant model and sends the new "sensor" value.
5. Repeat at the control rate.

When HIL is worth the effort:

- Safety-critical systems where a controller bug could injure someone.
- Very expensive actuators (industrial servos, hydraulic valves).
- Systems where the real plant is not available yet (waiting for custom-machined parts).

When HIL is overkill: Most competition scenarios. If your motor costs $30 and your gearbox is not going to explode from a gain that is 2x too high, go straight from MIL to real hardware. The time you spend setting up HIL communication protocols would be better spent tuning on the real robot.

Latency warning: HIL at 1 kHz means you have ≤ 1 ms round-trip for UART communication + plant simulation + sensor encoding. At 115200 baud, sending 8 bytes takes about 0.7 ms. Use higher baud rates (921600 or USB FS) or reduce the control rate for HIL testing.

8.5 Making Simulations Realistic

A perfect simulation is useless for controller validation. If your PID only works when the plant model is exact and the sensor is noiseless, it will fail on real hardware. You need to inject the same kinds of imperfections that reality will throw at you.

8.5.1 Sensor Noise

Add white Gaussian noise with variance matched to your sensor's datasheet (or measured Allan variance). For a typical MEMS gyroscope with noise density 0.01 deg/s/$\sqrt{\text{Hz}}$ sampled at 1 kHz, the discrete noise standard deviation is $\sigma = 0.01 \times \sqrt{1000} \approx 0.316$ deg/s.

8.5.2 Disturbances

- **Step disturbance:** apply a sudden load torque at a random time to test disturbance rejection.
- **Sinusoidal disturbance:** inject $d(t) = A\sin(2\pi f t)$ at a known frequency to test frequency-domain rejection.

8.5.3 Friction

The simplest useful friction model combines viscous and Coulomb friction:

$$\tau_{\text{friction}} = b\,\omega + \tau_c \cdot \text{sign}(\omega) \tag{80}$$

where b is the viscous coefficient (already in our motor model) and τ_c is the Coulomb friction torque. This model captures the deadband around zero velocity that makes low-speed tracking difficult.

8.5.4 Communication Delay

Delay the control signal by N_d samples to model CAN bus latency, sensor processing time, or computation delay: $V_{\text{applied}}[n] = V_{\text{computed}}[n - N_d]$.

8.5.5 Actuator Saturation

Clamp the output to $\pm V_{\max}$ (or $\pm\text{PWM}_{\max}$). This is critical—linear analysis assumes unbounded control effort, but your battery is 24 V and your ESC clips at 100% duty.

Here is how to add noise, friction, and saturation to the simulation loop:

```
tau_c = 0.005       # Coulomb friction torque (N*m)
sigma = 0.3         # sensor noise std dev (rad/s)
N_delay = 3         # 3-sample control delay
V_buf = [0.0] * (N_delay + 1)

for n in range(N - 1):
    omega_meas = omega[n] + np.random.randn() * sigma    # noisy sensor
    e = omega_ref - omega_meas
    # ... PID computation ...
    V_buf.append(np.clip(V_cmd, -24, 24))                # saturation
    V = V_buf[-(N_delay + 1)]                             # delayed output
    friction = b * omega[n] + tau_c * np.sign(omega[n])   # friction
    domega = ((Kt / R) * (V - Ke * omega[n]) - friction) / J
    omega[n + 1] = omega[n] + Ts * domega
```

Key insight: A controller that works in a perfect simulation but fails when you add noise and friction is *not* a good controller. If your gains only work in paradise, they will not survive competition day.

8.6 Quick Python Simulation Template

Here is a complete, copy-paste-ready template. Change the plant parameters to match your robot and you have a working simulation in two minutes.

```
import numpy as np, matplotlib.pyplot as plt

def simulate(plant_fn, Kp, Ki, Kd, ref, Ts=0.001, dur=2.0,
             noise_std=0.0, V_max=24.0):
    N = int(dur / Ts)
    y, u = np.zeros(N), np.zeros(N)
    e_int, e_prev, state = 0.0, 0.0, 0.0
    for n in range(N - 1):
        y_meas = y[n] + np.random.randn() * noise_std
        e = ref - y_meas
        e_int += e * Ts
        u_cmd = Kp * e + Ki * e_int + Kd * (e - e_prev) / Ts
        u[n] = np.clip(u_cmd, -V_max, V_max)
        y[n+1], state = plant_fn(y[n], u[n], state, Ts)
        e_prev = e
    t = np.arange(N) * Ts
    fig, (a1, a2) = plt.subplots(2, 1, sharex=True)
    a1.plot(t, y); a1.axhline(ref, ls='--', c='r'); a1.set_ylabel('Output')
    a2.plot(t, u); a2.set_ylabel('Control'); a2.set_xlabel('Time (s)')
    plt.tight_layout(); plt.show()
    return t, y, u
```

Define a plant function for your specific system. For the DC motor:

```
def dc_motor(omega, V, _, Ts,
             J=0.01, b=0.1, Kt=0.01, Ke=0.01, R=1.0):
    domega = ((Kt/R)*(V - Ke*omega) - b*omega) / J
    return omega + Ts * domega, None

t, y, u = simulate(dc_motor, Kp=5, Ki=10, Kd=0.05,
                   ref=100, noise_std=0.3)
```

Copy this, change the plant function parameters to match your robot, and you are running.

Case Study: Tuning LQR in Simulation Before Deploying to a Balance Robot

A two-wheeled balance robot is a classic inverted pendulum on wheels. The state vector is $\mathbf{x} = [\theta,\ \dot{\theta},\ x,\ \dot{x}]^T$—tilt angle and angular rate of the body, plus position and velocity of the wheels. The linearized continuous-time dynamics around the upright equilibrium are:

$$\dot{\mathbf{x}} = A\mathbf{x} + Bu, \qquad A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ \frac{g}{l} & -\frac{b_\theta}{J_\theta} & 0 & 0 \\ 0 & 0 & 0 & 1 \\ -\frac{gm}{M} & 0 & 0 & -\frac{b_x}{M} \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ -\frac{1}{J_\theta l} \\ 0 \\ \frac{1}{M} \end{bmatrix} \tag{81}$$

where g is gravity, l the distance from wheel axle to center of mass, m the pendulum mass, M the total mass, and b_θ, b_x are friction coefficients.

Step 1: Discretize. Using ZOH at $T_s = 5$ ms: $A_d = e^{AT_s}$, $B_d = A^{-1}(A_d - I)B$. In Python, use `scipy.signal.cont2discrete((A, B, C, D), Ts, method='zoh')`.

Step 2: Design LQR. Choose cost matrices Q and R from Chapter 6. Start conservative:

$$Q = \text{diag}(100,\ 1,\ 10,\ 1), \qquad R = 1$$

This penalizes tilt angle 100x more than angular rate, and position 10x more than velocity. Solve the discrete algebraic Riccati equation to get $K = [K_\theta,\ K_{\dot\theta},\ K_x,\ K_{\dot x}]$.

Step 3: Simulate. Run the closed-loop system: $\mathbf{x}[n+1] = A_d\mathbf{x}[n] + B_d(-K\mathbf{x}[n])$ with initial condition $\theta_0 = 5°$.

Now the fun part: **sweep Q and R** to explore the design space.

- Increase Q_{11} (tilt penalty): faster tilt recovery, more aggressive control, larger wheel accelerations.
- Increase Q_{33} (position penalty): robot drifts less but may sacrifice tilt performance.
- Increase R: gentler control, slower response, lower current draw.

After 20 minutes of sweeping in simulation, the team settled on $Q = \text{diag}(500,\ 5,\ 20,\ 1)$, $R = 0.5$, which gave a predicted settling time of 0.8 s with less than 2° overshoot.

Step 4: Deploy. Flash the gains into the STM32, power on the robot, give it a push. Hardware result: settling time 1.1 s, overshoot about 2.5°. The 30% gap is mostly unmodeled friction in the gearbox and slightly higher center of mass than assumed. But—and this is the important part—**the gains worked on the first try**. No motor burned, no gears stripped, no wasted afternoon. One round of minor gain tweaking on hardware brought the settling time down to 0.9 s.

Lesson: Simulation will never be exact. But it gets you into the right neighborhood, and that is worth more than any amount of trial-and-error on hardware.

9 Practical Debugging and Tuning Cookbook

This is the chapter you will actually use on competition day. Every other chapter teaches you how to design controllers; this one teaches you what to do when they do not work. The debugging skills here are not theoretical—they come from hundreds of hours of broken robots, burnt motors, and last-minute fixes.

Your robot just started shaking violently. The match is in twenty minutes. You do not have time to re-derive your transfer function or re-read the Bode plot chapter. You need a systematic procedure that gets you from "something is very wrong" to "it works" as fast as possible. That is what this chapter provides.

9.1 The Debugging Mindset

Before we get to specific symptoms and fixes, internalize three rules. Violating any of these will cost you hours.

Rule 1: The robot is never wrong. If it oscillates, there is a reason. If it drifts, there is a reason. The physics is not lying to you; your model of the physics is wrong. Do not say "the motor is acting weird." Say "the motor is doing X because of Y, and I need to change Z." The robot executes exactly what you told it to—the question is whether what you told it matches what you intended.

Rule 2: Change ONE thing at a time. If you change gains AND hardware simultaneously, you will never know which fixed the problem. If you change K_p and K_d at the same time, and the oscillation goes away, was K_p too high or K_d too low? You have no idea. Disciplined debugging means: change one parameter, test, observe, record. It feels slow, but it is faster than the alternative—randomly flailing through parameter space.

Rule 3: Log everything. "I think the motor was vibrating" is useless. "The velocity oscillated at 42 Hz with ±200 RPM amplitude" is actionable. Good logs let you diagnose problems after the fact, compare before-and-after behavior, and communicate precisely with teammates. If your MCU has UART or USB logging, use it. If you have a logic analyzer, use it. If nothing else, toggle a GPIO pin in your ISR and measure the timing on a scope.

9.2 The Diagnostic Flowchart

When something goes wrong, your first job is to identify the *symptom*. Here are the most common symptoms and their diagnostic procedures.

9.2.1 Symptom: Oscillation

Your robot is shaking, vibrating, or buzzing. This is the single most common competition-day problem.

1. **Is it mechanical?** Disconnect the controller entirely. Command a constant open-loop duty cycle. Spin the motor by hand. Does the mechanism vibrate on its own? If yes, the problem is not your controller—tighten bolts, check bearings, fix backlash.

2. **Is it sensor noise?** Log the raw sensor data with the motor stationary. Is it noisy? If the encoder signal is jittering by several counts, or the IMU reading is bouncing around, that noise is being amplified by your derivative term. Add a low-pass filter (Ch2) to the sensor signal.

3. **Is K_p too high?** Halve K_p. Does the oscillation stop or decrease significantly? If yes, you were operating past the stability margin. Reduce K_p until stable, then carefully increase K_d to recover performance.

4. **Is the sample rate too low?** Toggle a GPIO pin at the start and end of your control ISR. Measure the pulse width on an oscilloscope. Is the actual loop time close to the intended T_s? If the loop is running slower than you think, your effective gains are different from what you designed.

5. **Is there a structural resonance?** FFT the oscillation signal (or at least estimate the frequency by counting cycles on a scope). Is it at a single, sharp frequency unrelated to your PID bandwidth? If yes, you are exciting a mechanical resonance. Add a notch filter at that frequency (Ch2), or reduce K_d which typically excites resonances.

9.2.2 Symptom: Overshoot

The output goes past the setpoint before settling. Moderate overshoot (5–10%) is normal and often acceptable. Excessive overshoot (¿20%) means something needs fixing.

1. **Is K_d too low?** The derivative term is your damping. Increase K_d—does overshoot decrease? If you have no derivative term at all, you are running a PI controller, which inherently overshoots on step inputs.
2. **Is K_i too high?** The integral term accumulates during the transient, and the accumulated value pushes the output past the setpoint. Reduce K_i, or add integral separation (Ch4)—disable the integral term when the error is large, enable it only near the setpoint.
3. **Is the actuator saturating?** Log the raw controller output *before* clamping. If it regularly hits the maximum, the controller is asking for more than the actuator can deliver. While saturated, the system is effectively open-loop, and the integral keeps winding up. Add anti-windup (Ch4).
4. **Is the setpoint step too aggressive?** A step change from 0 to 5000 RPM is asking for infinite acceleration at $t = 0$. No physical system can do that. Add a setpoint filter or soft-start ramp (Ch2) to limit the rate of change of the reference signal.

9.2.3 Symptom: Steady-State Error / Drift

The output settles, but not at the setpoint. There is a persistent offset.

1. **Is there an integral term?** If $K_i = 0$, a purely PD controller will have steady-state error whenever there is a constant disturbance (gravity, friction, load). Add an integral term.
2. **Is the sensor biased?** Check your sensor calibration (Ch2b). An IMU with a 0.5° bias will cause a 0.5° steady-state error no matter how good your controller is.
3. **Is there a mechanical offset?** Gravity, belt tension, spring preload—any constant force that your controller model does not account for will appear as steady-state error. Either add feedforward compensation or let the integral term absorb it (but this is slower).
4. **Is integral windup preventing the integrator from doing its job?** Paradoxically, if your anti-windup is too aggressive, it may clamp the integral before it accumulates enough to eliminate the error. Check that your anti-windup limits (Ch4) are large enough to handle the expected steady-state load.

9.2.4 Symptom: No Response

You command the motor to move. Nothing happens. This is either the easiest or the hardest problem to debug.

1. **Is the motor powered?** Measure the voltage at the motor terminals with a multimeter. This sounds obvious, but blown fuses, loose connectors, and dead batteries account for a shocking number of "my controller doesn't work" complaints.
2. **Is the enable pin active?** Many motor drivers (DRV8301, C610/C620, L298N) have an enable pin that defaults to *disabled*. If you forgot to pull it high in your initialization code, the driver ignores all PWM commands.
3. **Is the PWM polarity correct?** Positive duty cycle should produce positive torque in the direction you expect. Some drivers invert the polarity, or your timer configuration may produce inverted PWM.
4. **Is the control loop actually running?** Toggle a GPIO pin inside your control loop ISR. Check on a scope whether it toggles at the expected rate. If it does not toggle at all, your timer interrupt is not configured correctly.
5. **Is the sign convention correct?** This is the most dangerous software bug. If positive error produces *negative* output, your system is a positive feedback loop. It will not oscillate gently—it will slam to a rail instantly. Check: does a positive setpoint change produce a positive change in your control output?

9.3 Is It Mechanical, Electrical, Software, or Control?

Many debugging sessions go wrong because the student is tuning PID gains when the real problem is a loose setscrew, a sagging power supply, or a sign error in code. Before touching gains, classify the problem.

9.3.1 The Isolation Test

This is the single most valuable debugging technique. It takes two minutes and saves hours.

Step 1: Disconnect the controller. Command a constant open-loop duty cycle (e.g., 30% PWM). Observe the motor.

- If the motor spins smoothly and steadily: the problem is in your controller or software. Go to the software/control checks.
- If the motor vibrates, stalls, or behaves erratically: the problem is mechanical or electrical. Do not touch your code yet.

Step 2: Check mechanical. Disconnect the motor from the mechanism entirely. Spin the output shaft by hand. Does it bind? Is there excessive backlash? Does it feel gritty (bad bearings) or catchy (belt tension, gear mesh)?

Step 3: Check electrical. Measure your power supply voltage with the motor running under load. Does it sag significantly (more than 10%)? If multiple axes move simultaneously and the robot behaves worse, you almost certainly have a power supply problem.

9.3.2 Common Electrical Problems

- **Power supply voltage sag under load.** The battery or power supply cannot deliver enough current. When multiple motors draw simultaneously, the voltage drops, and all controllers lose authority. The symptom is that the robot works fine with one motor, but falls apart when all motors run. Fix: use a higher-capacity battery, add bulk capacitors near the motor drivers, or stagger motor startup.
- **Ground loops.** If your logic board and power board share a ground path, motor current flowing through the shared ground creates a voltage drop that appears as sensor noise. The noise is proportional to motor current—it gets worse when the motor works harder. Fix: use separate ground planes, star grounding, or optically isolated sensor interfaces.
- **EMI from PWM switching.** The fast edges of PWM switching (dV/dt can exceed 1 V/ns) create electromagnetic interference that corrupts nearby sensor signals. Analog sensors (current sense, potentiometers) and slow digital buses (I2C) are especially vulnerable. Fix: keep sensor wires away from motor wires, use shielded cables, add filtering capacitors on sensor inputs, prefer SPI over I2C for noise-sensitive applications.
- **Connector contact resistance.** A corroded or loose connector can add 0.1–1 Ω of resistance in the power path. At 10 A motor current, that is 1–10 V of drop—enough to starve the motor or brown out the MCU. The symptom is intermittent: the robot works fine on the bench but fails under competition loads. Wiggle every power connector while the motor is running and watch for glitches. Fix: clean connectors, use XT60/XT30 connectors with proper crimps, and add a bulk capacitor (100–470 μF) near each motor driver to buffer transient current demands.

9.3.3 Common Software Problems

- **Sign convention error.** If positive voltage produces rotation in the direction that *increases* the error instead of decreasing it, you have positive feedback. The system does not oscillate—it runs away instantly. This is the most common and most dangerous software bug. Always verify: positive error $\rightarrow$ positive output $\rightarrow$ error decreases.
- **Variable overflow.** A 16-bit encoder counter wraps from 65535 to 0. If you compute velocity as `count_now - count_prev` without handling the wraparound, you get a massive spike. Similarly, the integral term can overflow if accumulated in a 16-bit integer. Use 32-bit (or floating-point) arithmetic for all PID computations.

- **Timing errors.** Your control loop runs at the wrong frequency because the timer prescaler is misconfigured. You designed your gains for 1 kHz, but the loop actually runs at 500 Hz. Your effective K_d is twice what you intended (since K_d scales with sample rate), making the system much more aggressive.

- **Data race.** The ISR updates a shared variable (encoder count, sensor reading) while the main loop is reading it. Without the `volatile` keyword (C/C++) or a proper mutex, the compiler may optimize away the read, or you may read a partially-updated value. Always declare shared variables as `volatile`, and disable interrupts briefly when reading multi-byte shared values. Example of a safe read:

```
volatile int32_t encoder_count;  // written in ISR

int32_t read_encoder(void) {
    __disable_irq();
    int32_t val = encoder_count;
    __enable_irq();
    return val;
}
```

- **Uninitialized variables.** If you forget to initialize the integral accumulator to zero, it starts with whatever garbage value was in memory. This can cause a large transient at startup—the motor jerks violently for one cycle, then settles. Always zero all PID state variables in your initialization function.

- **Wrong units or scaling.** Your sensor returns counts but your controller expects radians. Your PWM register takes 0–999 but your PID outputs 0–100. Unit mismatches cause gains to be off by orders of magnitude. Always document the units at each interface boundary with a comment.

9.4 Oscilloscope and Logic Analyzer Techniques

Your eyes and ears are poor measurement instruments. "It seems to vibrate" tells you nothing quantitative. The right tools make invisible problems visible.

PWM duty cycle verification. Put an oscilloscope probe on the motor phase output. Measure the duty cycle. Does it match what your software commands? If your code says 50% duty and the scope shows 30%, your timer configuration is wrong. This takes 30 seconds and eliminates an entire class of bugs.

Control loop timing. Toggle a GPIO pin at the start of your ISR, and clear it at the end. On the scope, the pulse width is your ISR execution time, and the period is your loop period. Both should match your design. If the pulse width is close to the period, your ISR is too slow—it will miss deadlines and the control quality degrades. Here is the code:

```
void TIM_IRQHandler(void) {
    HAL_GPIO_WritePin(DEBUG_GPIO, GPIO_PIN_SET);   // ISR start

    // ... your control code here ...

    HAL_GPIO_WritePin(DEBUG_GPIO, GPIO_PIN_RESET); // ISR end
}
```

Sensor signal integrity. Use a logic analyzer to probe I2C/SPI bus traffic. Verify that the data bytes match what your software reads. If the bus shows correct data but your variable has wrong values, the problem is in your parsing code. If the bus itself shows glitches, the problem is electrical (noise, too-high clock speed, missing pull-ups on I2C).

Logging PID internals. When you are stuck, log *all* internal PID variables, not just the output. A useful logging line prints the timestamp, setpoint, measurement, error, P-term, I-term, D-term, raw output, and clamped output. Here is a minimal UART logging example:

```
// Call once per control cycle (or every N cycles to reduce bandwidth)
void log_pid(uint32_t tick, float sp, float meas,
             float p, float i, float d, float out_raw, float out) {
    printf("%lu,%.1f,%.1f,%.2f,%.2f,%.2f,%.1f,%.1f\r\n",
           tick, sp, meas, p, i, d, out_raw, out);
}
```

Plot these in Python (or even Excel) after the test. The P/I/D breakdown immediately reveals which term is misbehaving. If the I-term is huge while the P-term is small, you have windup. If the D-term is noisy, you need filtering. This one technique will save you more time than any other.

Current measurement. If you have a current-sense resistor or a hall-effect sensor, logging actual motor current is invaluable. The current tells you what the motor is *actually doing*, as opposed to what you are *commanding* it to do. A mismatch between commanded and actual current reveals driver issues, wiring problems, or motor faults.

Pro tip: A $15 logic analyzer (Saleae clone) is the single best debugging tool you can buy for competition robotics. It decodes I2C, SPI, UART, CAN, and PWM. Combined with PulseView (free software), it replaces hundreds of dollars of equipment for 90% of debugging tasks.

9.5 PID Tuning on Real Hardware: A Complete Worked Example

Theory is nice, but you need a concrete procedure you can follow step by step on the bench.

9.5.1 System: DJI 3508 Motor Velocity Control

The DJI 3508 with C620 ESC is ubiquitous in RoboMaster competitions. The relevant parameters:

- Rotor inertia: approximately $J = 0.0005$ kg·m^2
- Torque constant: $K_t \approx 0.3$ N·m/A
- Winding resistance: $R \approx 0.5\ \Omega$
- Current loop: handled internally by the C620 ESC. We treat it as a fast inner loop (bandwidth ~5 kHz) and design only the outer velocity loop.
- Goal: velocity tracking with $< 5\%$ overshoot, < 200 ms settling time.

9.5.2 Step-by-Step Tuning Procedure

Step 1: Find the oscillation gain. Set $K_i = 0$, $K_d = 0$. Send a 500 RPM step command. Slowly increase K_p from 0. At some point, the velocity will exhibit sustained, constant-amplitude oscillation. This is the *critical gain*. Record $K_{p,\text{osc}} \approx 15$. Now set $K_p = 0.5 \times K_{p,\text{osc}} = 7.5$. The system should be stable but sluggish.

Step 2: Add derivative action. Start with $K_d = 0.01$. Increase gradually. The derivative term adds damping, reducing overshoot and improving settling time. Continue increasing until overshoot is below 5%. Typical result: $K_d \approx 0.05$. If increasing K_d causes high-frequency vibration, you are amplifying sensor noise—add a low-pass filter on the derivative path.

Step 3: Add integral action slowly. Start with $K_i = 0.5$. Send the same 500 RPM step. Check: does the steady-state error disappear? Increase K_i until the steady-state error is negligible. Typical result: $K_i \approx 2.0$. If increasing K_i causes overshoot to return, you have gone too far—back off and accept a slight steady-state error, or add integral separation.

Step 4: Disturbance test. With the motor running at 500 RPM, briefly load the shaft by hand (or press a rubber block against the output). The velocity should dip and recover within 200–300 ms. If it recovers slowly, increase K_i slightly. If it oscillates during recovery, decrease K_p or increase K_d.

Step 5: Test across the operating range. Repeat the step response at 100, 500, 2000, and 5000 RPM. The motor's behavior changes with speed (back-EMF, friction). If the controller works at 500 RPM but oscillates at 5000 RPM, you may need gain scheduling or a more conservative set of gains that work across the full range.

At the end of this procedure, you should have a velocity response that rises quickly to the setpoint, overshoots by less than 5%, and settles within 200 ms. The step response should look like a smooth rise with minimal ringing—not a sluggish crawl (under-tuned) or a violent oscillation (over-tuned).

What the response curves look like at each step:

- After Step 1 (K_p only): slow rise with about 30% overshoot and visible ringing (2–3 oscillations before settling). Steady-state error of maybe 5–10%.
- After Step 2 ($K_p + K_d$): faster rise, overshoot reduced to $< 5\%$, settling in $\sim$150 ms. But still a persistent steady-state error.
- After Step 3 ($K_p + K_i + K_d$): same fast rise and low overshoot, but now the error converges to zero within 200 ms. The integral smoothly fills in the last few percent.
- Disturbance test: a brief dip of maybe 50 RPM when the load is applied, full recovery within 200–300 ms.

9.5.3 When Ziegler-Nichols Fails

The Ziegler-Nichols ultimate gain method (which Step 1 above is based on) is a useful starting point, but it fails in several important cases:

- **Systems with large time delay.** If the transport delay is comparable to the system time constant, Z-N gives gains that are far too aggressive. The resulting controller has almost zero phase margin.
- **Systems with nonlinear friction.** If static friction (stiction) is significant, the oscillation test gives inconsistent results at different amplitudes. At low amplitudes the motor sticks, at high amplitudes it moves smoothly. The "critical gain" depends on the test amplitude, which defeats the purpose.
- **Systems where oscillation is dangerous.** For a balance robot or a heavy manipulator arm, inducing sustained oscillation at the critical gain risks mechanical damage or injury.

In all these cases, use the manual step-by-step procedure above. Start with very low gains, increase gradually, and rely on step response measurements rather than the oscillation boundary. It is more tedious but much more forgiving.

9.6 Common Failure Modes and Their Signatures

Learn to recognize these patterns in your data logs. Each has a distinctive "fingerprint" that, once you have seen it, you will immediately identify.

9.6.1 Integral Windup in Practice

Signature: After the actuator saturates, the response is sluggish in one direction, followed by violent overshoot when the error reverses.

What the time series looks like: The output command hits the maximum limit and stays there for an extended period. During this time, the integral term keeps accumulating. When the error finally changes sign, the integral term is so large that it takes a long time to "unwind," keeping the output at maximum long after it should have reversed. The result is a massive overshoot in the opposite direction.

Fix: Implement clamping or back-calculation anti-windup (Ch4). With clamping, stop accumulating the integral when the output is saturated. With back-calculation, feed back the difference between the saturated and unsaturated output to discharge the integrator.

9.6.2 Actuator Saturation

Signature: The actual response matches your simulation at low amplitudes, but diverges at high amplitudes. Performance degrades as the setpoint increases.

Diagnosis: Log the raw controller output before clamping. If it regularly exceeds the actuator limits (e.g., the PID output is ± 20000 but the motor driver accepts ± 16384), your controller is asking for more than the actuator can deliver. During saturation, the system is effectively open-loop.

Fix: Reduce gains to keep the output within the linear range, add output limiting with anti-windup, or simply accept that the system will perform slower at extreme operating points. Sometimes the answer is "your actuator is too small for the task"—no amount of tuning can fix physics.

9.6.3 Sensor Aliasing

Signature: A phantom oscillation appears at an unexpected frequency, unrelated to any PID bandwidth or mechanical resonance.

Cause: The sensor sample rate is too low relative to the signal bandwidth. A high-frequency signal (say, 800 Hz motor vibration) sampled at 1 kHz aliases to 200 Hz (Nyquist, Ch2). The controller sees a 200 Hz disturbance that does not physically exist and tries to fight it.

Fix: Increase the sensor sample rate to at least 2× the highest signal frequency. Or, add an analog anti-aliasing filter (a simple RC low-pass) before the ADC to remove high-frequency content before sampling.

9.6.4 Quantization Limit Cycling

Signature: A small, persistent oscillation around the setpoint that never dies out. The amplitude is typically 1–2 encoder counts.

Cause: The encoder resolution is too coarse relative to the positioning accuracy you are asking for. The controller sees "I am 1 count above the setpoint" and commands a correction. The motor moves 1 count below. Now it is 1 count below, so it corrects back. This alternation is a *limit cycle* caused by quantization.

Fix: Add a small deadband near the setpoint (ignore errors below the quantization threshold), or use a higher-resolution encoder. The deadband should be approximately 1–2 encoder counts wide.

9.6.5 Communication Latency

Signature: Instability that worsens as CAN bus load increases, or as more devices share the communication bus.

Cause: Variable delay in sensor readings means the controller is acting on stale data. If the delay is comparable to the loop period, it adds significant phase lag. The phase margin decreases, and the system approaches instability. Under heavy bus load, messages queue up, delays increase, and the system goes unstable.

Fix: Reduce the controller bandwidth (lower K_p and K_d) so the loop is robust to the delay. Alternatively, add explicit delay compensation using a Smith predictor (Ch4), which models the known delay and compensates for it in the control law.

9.7 Pre-Competition Checklist

Run through this list the night before and the morning of every competition. It takes 15 minutes and prevents 80% of match-day failures.

1. **Power.** Charge all batteries to full. Measure the voltage with a multimeter—do not trust the charger indicator. Bring spares.

2. **Mechanical.** Check every setscrew, every bolt, every belt tension. Vibration loosens fasteners. If it moved during practice, it will fall off during the match. Apply Loctite to critical fasteners.

3. **Wiring.** Tug-test every connector. A connector that "feels" secure but is not fully seated will make intermittent contact, causing the most maddening bugs. Secure loose wires with zip ties or tape so they cannot snag.

4. **Communication.** Power on all modules and verify CAN/UART heartbeat messages appear. Check that no CAN IDs conflict. Verify firmware versions match what you tested with.

5. **Sensors.** Zero and calibrate each sensor. Check IMU bias, encoder zero position, current sense offset. Log 2 seconds of stationary data—does it look clean?

6. **Control loops.** Run each axis through its full range of motion. Verify no oscillation, no stalls, no unexpected behavior at the extremes. Pay special attention to transitions between operating modes (e.g., from stationary to spinning, from free motion to contact).

7. **Emergency stops.** Verify the kill switch works. Verify the software watchdog triggers correctly. Verify that losing communication causes the robot to stop (not run away at full speed).

8. **Log review.** Do one complete test run with full logging. Review the logs. Are all signals within expected ranges? Any anomalies?

The 30-Second Smoke Test

When you are rushed and cannot do the full checklist, do this minimum viable test:

1. Power on. Wait 3 seconds. Listen for abnormal sounds.
2. Command each motor to 10% speed. Does it respond? Correct direction?
3. Command 50% speed. Smooth? No oscillation?
4. Command full speed. No vibration? Power supply holding up?
5. Stop. Verify clean shutdown.

If all five pass, you are probably fine. If any fails, you have found the problem before the match—which is always better than finding it during the match.

9.8 Competition Day Emergency Fixes

10-Minute Emergency Reference

When the match is imminent and something breaks, do not think deeply. Follow this table.

Symptom	30-Second Diagnosis	2-Minute Fix
Violent oscillation	Halve ALL gains	If still oscillating, set $K_d = 0$
Overshoot	Reduce K_i by 50%, increase K_d by 50%	Add setpoint filter / ramp
Drift / steady-state error	Check sensor zero, reset integrators	Add or increase K_i
No response	Check wiring, enable pin, PWM output	Verify sign convention
Erratic behavior	Reduce bandwidth (halve K_p and K_d)	Add output rate limiting
Works on bench, fails on field	Surface friction/conditions changed	Increase K_i, decrease K_d

Golden rule: When in doubt, reduce all gains by 50%. A sluggish robot that works beats a fast robot that crashes.

Case Study: The Balance Robot That Worked Yesterday

Scenario: Your balance robot works perfectly on the smooth lab floor. At the competition venue, it falls over within 3 seconds on the carpet surface.

Root cause: The carpet has much higher friction than the smooth floor. The wheels grip differently, the effective damping changes, and—most importantly—the integral term accumulates differently because the friction creates a larger constant disturbance that the integrator must fight. On smooth floor, the integrator barely activates. On carpet, it saturates.

Diagnosis: You log the integral term on both surfaces. On smooth floor, it stays near zero. On carpet, it ramps to the saturation limit within 2 seconds and stays there. The anti-windup kicks in, but by then the robot has already tilted past the recoverable angle.

Fix (2-minute version): Increase K_d by 50% (the physical damping from carpet partially replaces the need for derivative action, but the higher friction also creates lag that K_d helps with). Reduce K_i by 40% to avoid premature saturation. Raise the anti-windup limit slightly.

Fix (proper version): Add a friction feedforward term that estimates the friction force from the wheel velocity sign and adds a compensating torque. This removes the burden from the integral term entirely.

Lesson: Always test on the actual competition surface before the match. And always have a "high friction" parameter set ready to switch to.

Case Study: The Gimbal That Resonates at 23 Hz

Scenario: Your gimbal vibrates at 23 Hz. You are confused because your PID bandwidth is only 10 Hz. No PID parameter is anywhere near 23 Hz.

Root cause: The gimbal frame has a structural resonance at 23 Hz. The derivative term in your PID has gain that extends well beyond the PID bandwidth (remember, differentiation amplifies high frequencies). At 23 Hz, the derivative term has enough gain to excite the resonance, and the resonance feeds back through the IMU, creating a self-sustaining oscillation.

Diagnosis: FFT the motor current command. A sharp peak appears at 23 Hz. Reduce K_d by 50%—the vibration amplitude drops proportionally. This confirms the derivative term is the excitation source.

Fix (quick): Reduce K_d and accept slightly slower transient response. This is the competition-day fix.

Fix (proper): Add a 23 Hz notch filter (Ch2) in the feedback path. The notch removes the resonance frequency from the signal that the PID sees, so the derivative term cannot excite it. The rest of the spectrum is unaffected, so transient performance is preserved. Design the notch with a Q-factor of 5–10 (narrow enough to not affect nearby frequencies, wide enough to catch the resonance even if it shifts slightly with temperature or load).

Lesson: The PID bandwidth is not the only frequency that matters. The derivative term has gain at *all* frequencies above the bandwidth. Any mechanical resonance within the derivative's effective range is a potential victim. When you design a gimbal, always characterize the structural resonances *before* tuning the controller.

9.9 Building a Debugging Toolkit

Over the course of a competition season, you should accumulate a set of debugging tools and habits. Here is what an experienced team's toolkit looks like:

Hardware tools:

- Multimeter (for voltage, current, and continuity checks)
- Oscilloscope (even a cheap USB scope like the Hantek 6022BE is enough for PWM and timing)
- Logic analyzer (Saleae clone + PulseView for I2C/SPI/CAN/UART decoding)
- Spare batteries, connectors, and wires
- A known-good motor and driver (for isolation testing)

Software tools:

- A serial plotting tool (SerialPlot, PlotJuggler, or a simple Python script with matplotlib)
- A parameter tuning interface—either a GUI or a simple serial command interface that lets you change K_p, K_i, K_d without reflashing firmware
- A logging framework that can dump all PID internal variables at loop rate
- A test script that runs a standard step response and automatically computes overshoot, settling time, and steady-state error

Software parameter tuning example. A minimal serial command parser that lets you change gains at runtime:

```
// Parse "Kp=7.5" or "Ki=2.0" from UART input
void parse_gain_command(const char *cmd) {
    float val;
    if (sscanf(cmd, "Kp=%f", &val) == 1) pid.Kp = val;
    if (sscanf(cmd, "Ki=%f", &val) == 1) pid.Ki = val;
    if (sscanf(cmd, "Kd=%f", &val) == 1) pid.Kd = val;
}
```

This saves you from the reflash-reboot-retest cycle, which can easily burn 2 minutes per iteration. Over 50 tuning iterations, that is over an hour saved.

The debugging log. Keep a simple text file (or a notebook, or a shared document) that records every debugging session:

- Date and time
- Symptom observed
- Hypothesis tested
- What was changed
- Result

This sounds tedious, but it prevents the most common meta-debugging problem: "Wait, did we already try that? What happened?" When three teammates are making changes at 11 PM the night before competition, a log is the difference between systematic progress and circular chaos.

Final thought. Debugging is not a sign that you did something wrong. Every real system needs debugging. The difference between a good engineer and a frustrated student is not the absence of bugs—it is the speed at which bugs are found and fixed. The techniques in this chapter will not eliminate problems, but they will dramatically reduce the time from "something is wrong" to "I know what is wrong and how to fix it." That speed is what wins competitions.

10 Modern Control Theory

Classical control (PID, frequency response, root locus) got us far. It can handle most single-input, single-output systems beautifully. But some problems demand more:

- Your holonomic chassis has four motors that must be coordinated simultaneously (MIMO).
- You want *optimal* performance, not just "good enough" (optimal control).
- You need to estimate quantities you cannot measure directly, like velocity from position data or tilt angle from a noisy IMU (state estimation).
- You want to explicitly handle constraints like maximum motor current or joint angle limits (constrained optimization).

This is where modern control theory shines. It works directly with the state-space representation and uses linear algebra as its primary tool.

10.1 Relationship Between State-Space and Transfer Functions

These are two representations of the same system, and you can convert between them.

State-space to transfer function:

$$G(s) = C(sI - A)^{-1}B + D \tag{82}$$

Taking the Laplace transform of $\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u}$ (with zero initial conditions) gives $s\mathbf{X}(s) = A\mathbf{X}(s) + B\mathbf{U}(s)$, hence $\mathbf{X}(s) = (sI - A)^{-1}B\mathbf{U}(s)$. Substituting into $\mathbf{Y}(s) = C\mathbf{X}(s) + D\mathbf{U}(s)$ yields $G(s) = C(sI - A)^{-1}B + D$.

Transfer function to state-space: Not unique—many different state-space realizations can give the same transfer function. Common forms include *controllable canonical form* (the denominator coefficients appear directly in the last row of A, making it easy to verify controllability) and *observable canonical form* (the denominator coefficients appear in the last column of A, making observability transparent). In practice, MATLAB's `tf2ss` gives you one realization; which canonical form you use matters mainly for hand analysis.

The key difference: Transfer functions describe the input-output behavior. State-space describes the internal dynamics. A system can have identical transfer functions but different internal structures—the state-space representation captures this extra information.

Why it matters: Some internal modes might be unstable but invisible in the transfer function (if they are cancelled by zeros). State-space reveals them. This is why modern control prefers state-space—it sees everything.

10.2 System Performance Metrics

10.2.1 Controllability and Observability

These are two fundamental questions that you should ask before designing any controller or observer.

Controllability: "Can I drive every state to any desired value using the available inputs?"

A system (A, B) is controllable if the **controllability matrix** has full rank:

$$\mathcal{C} = \begin{bmatrix} B & AB & A^2B & \cdots & A^{n-1}B \end{bmatrix}, \quad \text{rank}(\mathcal{C}) = n \tag{83}$$

Derivation: Why the Controllability Matrix Works

Where does $\mathcal{C} = [B \;\; AB \;\; \cdots \;\; A^{n-1}B]$ come from? It is not pulled out of thin air—it follows directly from the state transition equation.

Step 1: Iterative state propagation. For the discrete-time system $\mathbf{x}[k+1] = A\mathbf{x}[k] + B\mathbf{u}[k]$,

start from $\mathbf{x}[0] = \mathbf{0}$ and propagate forward:

$$\mathbf{x}[1] = B\mathbf{u}[0] \tag{84}$$
$$\mathbf{x}[2] = A\mathbf{x}[1] + B\mathbf{u}[1] = AB\mathbf{u}[0] + B\mathbf{u}[1] \tag{85}$$
$$\mathbf{x}[3] = A^2B\mathbf{u}[0] + AB\mathbf{u}[1] + B\mathbf{u}[2] \tag{86}$$
$$\vdots$$
$$\mathbf{x}[n] = A^{n-1}B\mathbf{u}[0] + A^{n-2}B\mathbf{u}[1] + \cdots + B\mathbf{u}[n-1] \tag{87}$$

Step 2: Rewrite as a matrix equation. Stack the inputs into a vector:

$$\mathbf{x}[n] = \begin{bmatrix} A^{n-1}B & A^{n-2}B & \cdots & AB & B \end{bmatrix} \begin{bmatrix} \mathbf{u}[0] \\ \mathbf{u}[1] \\ \vdots \\ \mathbf{u}[n-1] \end{bmatrix} = \mathcal{C} \cdot \mathbf{u}_{\text{seq}} \tag{88}$$

(The columns of $\mathcal{C}$ are just reordered—rank is unchanged.)

Step 3: The rank condition. We want to reach *any* target state $\mathbf{x}[n] \in \mathbb{R}^n$. This requires the equation $\mathcal{C} \cdot \mathbf{u}_{\text{seq}} = \mathbf{x}[n]$ to have a solution for every right-hand side. By linear algebra, this is possible if and only if $\text{rank}(\mathcal{C}) = n$.

Why only n columns? The Cayley–Hamilton theorem guarantees that A^n can be written as a linear combination of $I, A, A^2, \ldots, A^{n-1}$. Therefore A^nB is already in $\text{span}\{B, AB, \ldots, A^{n-1}B\}$, and higher powers add nothing new. You never need more than n columns.

Continuous-time version: For $\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u}$, the reachable set from $\mathbf{x}(0) = \mathbf{0}$ at time t is:

$$\mathbf{x}(t) = \int_0^t e^{A(t-\tau)} B\mathbf{u}(\tau)\, d\tau$$

Expanding the matrix exponential $e^{A(t-\tau)} = \sum_{k=0}^{\infty} \frac{(t-\tau)^k}{k!} A^k$ and again applying Cayley–Hamilton to collapse powers $\geq n$, the reachable set is $\text{span}\{B, AB, \ldots, A^{n-1}B\}$—the same controllability matrix.

Derivation: Why the Observability Matrix Works

The observability matrix arises from the dual question: given outputs $\mathbf{y}[0], \mathbf{y}[1], \ldots$, can we recover the initial state $\mathbf{x}[0]$?

Step 1: Collect output equations. Assume $\mathbf{u} = \mathbf{0}$ (we can always subtract the known input contribution). The free response gives:

$$\mathbf{y}[0] = C\mathbf{x}[0] \tag{89}$$
$$\mathbf{y}[1] = C\mathbf{x}[1] = CA\mathbf{x}[0] \tag{90}$$
$$\mathbf{y}[2] = CA^2\mathbf{x}[0] \tag{91}$$
$$\vdots$$
$$\mathbf{y}[n-1] = CA^{n-1}\mathbf{x}[0] \tag{92}$$

Step 2: Stack into a matrix equation.

$$\begin{bmatrix} \mathbf{y}[0] \\ \mathbf{y}[1] \\ \vdots \\ \mathbf{y}[n-1] \end{bmatrix} = \underbrace{\begin{bmatrix} C \\ CA \\ CA^2 \\ \vdots \\ CA^{n-1} \end{bmatrix}}_{\mathcal{O}} \mathbf{x}[0] \tag{93}$$

Step 3: The rank condition. To uniquely recover $\mathbf{x}[0]$ from the output sequence, the equation $\mathcal{O} \cdot \mathbf{x}[0] = \mathbf{y}_{\text{seq}}$ must have a unique solution. This requires $\text{rank}(\mathcal{O}) = n$ (full column rank).

Why only n rows? Same reason as controllability—Cayley–Hamilton makes CA^n a linear combination of $C, CA, \ldots, CA^{n-1}$, so additional measurements add no new information.

The duality is now obvious: Controllability asks "can $\mathcal{C}$ map the input space *onto* the state space?" Observability asks "can $\mathcal{O}$ map the state space *into* distinguishable outputs?" Transpose one and you get the other—which is exactly the mathematical statement of duality: (A, B) controllable $\Leftrightarrow$ (A^T, B^T) observable (with B^T playing the role of C).

Physical interpretation: If a state is uncontrollable, no input can influence it. For example, if your robot's chassis has a broken motor, you have lost controllability in certain directions.

Worked example: Consider a simplified 2D chassis with states $\mathbf{x} = \begin{bmatrix} v_x \\ v_y \end{bmatrix}$ (velocities in x and y) and a single forward-facing motor:

$$A = \begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix}, \quad B = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

The controllability matrix: $\mathcal{C} = \begin{bmatrix} B & AB \end{bmatrix} = \begin{bmatrix} 1 & -1 \\ 0 & 0 \end{bmatrix}$. Rank $= 1$, but $n = 2$. **Not controllable!** The motor can only drive v_x; no input can change v_y. Physically obvious: a car cannot move sideways. Add a second motor facing sideways ($B = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$) and controllability is restored.

Observability: "Can I determine every state from the available outputs (measurements)?"

A system (A, C) is observable if the **observability matrix** has full rank:

$$\mathcal{O} = \begin{bmatrix} C \\ CA \\ CA^2 \\ \vdots \\ CA^{n-1} \end{bmatrix}, \quad \text{rank}(\mathcal{O}) = n \tag{94}$$

Physical interpretation: If a state is unobservable, you cannot infer it from measurements. For example, if you only measure motor position but not current, the current state might be unobservable (depending on the model).

The controllability-observability duality: A system (A, B, C) is controllable if and only if the dual system (A^T, C^T, B^T) is observable. This elegant symmetry means that techniques for controller design (pole placement) directly translate to observer design.

Bottom line: Always check controllability before designing a controller, and observability before designing an observer. If these conditions are not satisfied, no amount of mathematical sophistication will save you—you need to change the hardware (add sensors, fix actuators).

10.2.2 Closed-Loop System Stability

For the closed-loop state-space system with state feedback $\mathbf{u} = -K\mathbf{x}$:

$$\dot{\mathbf{x}} = (A - BK)\mathbf{x} \tag{95}$$

The system is stable if and only if all eigenvalues of $(A - BK)$ have negative real parts. The gain matrix K is what we design to place these eigenvalues where we want.

Lyapunov stability: A more general tool for stability analysis. A system is stable if there exists a positive definite function $V(\mathbf{x})$ (an "energy-like" function) that always decreases along trajectories ($\dot{V} < 0$). Finding such a V proves stability; failing to find one does not prove instability (it might just mean you did not look hard enough).

For linear systems, the Lyapunov equation is:

$$A^T P + PA = -Q \tag{96}$$

Choose $V(\mathbf{x}) = \mathbf{x}^T P \mathbf{x}$ as a candidate Lyapunov function. Its time derivative along trajectories of $\dot{\mathbf{x}} = A\mathbf{x}$ is $\dot{V} = \dot{\mathbf{x}}^T P \mathbf{x} + \mathbf{x}^T P \dot{\mathbf{x}} = \mathbf{x}^T (A^T P + PA)\mathbf{x}$. For $\dot{V} < 0$ (stability), we need $A^T P + PA = -Q$ for some positive definite Q.

If $Q > 0$ and the solution $P > 0$, the system is stable. MATLAB's `lyap()` solves this directly.

10.3 Controller Design via State-Space

10.3.1 Bellman Equations and Dynamic Programming: The Engine Behind LQR and MPC

Before we dive into LQR and MPC, we need to understand the mathematical engine that powers both: **dynamic programming (DP)** and the **Bellman equation**. If you have done competitive programming (USACO, Codeforces, LeetCode), you already know DP — the control theory version is the same idea wearing a different hat.

The Bellman Principle of Optimality: *An optimal policy has the property that, regardless of the initial state and initial decision, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision.*

In plain English: if you know the optimal strategy from every intermediate state onward, you can figure out the optimal first move by trying all options and picking the one that minimizes (first-step cost) + (optimal cost-to-go from the resulting state).

Warm-Up: Grid-World Shortest Path (DP You Already Know)

Consider a classic DP problem: you are on an $n \times n$ grid. You start at $(0, 0)$ and want to reach $(n-1, n-1)$. Each cell (i, j) has a cost $c(i, j)$. You can move right or down. Find the minimum-cost path.

State: Your position (i, j).

Decision (control): Move right or move down.

Cost-to-go: $V(i, j)$ = minimum total cost to reach the goal from (i, j).

Bellman equation:

$$V(i, j) = c(i, j) + \min\{V(i + 1, j),\ V(i, j + 1)\}$$

Boundary: $V(n-1, n-1) = c(n-1, n-1)$.

Solution: Fill the table *backward* from the goal to the start. At each cell, the optimal decision is whichever neighbor has the smaller cost-to-go. Total complexity: $O(n^2)$.

You have been doing this since your first DP problem. Now watch what happens when we change the vocabulary:

DP (programming)	Optimal control	What it means
State (i, j)	State $\mathbf{x}_k$	Where you are now
Decision (right/down)	Control input $\mathbf{u}_k$	What you do next
Transition $(i, j) \to (i + 1, j)$	$\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k)$	Physics/dynamics
Step cost $c(i, j)$	Stage cost $\ell(\mathbf{x}_k, \mathbf{u}_k)$	Penalty for this step
Cost-to-go $V(i, j)$	Value function $V_k(\mathbf{x})$	Optimal future cost
Bellman recursion	Bellman equation	The DP update rule
Fill table backward	Backward Riccati recursion	How we solve it
Optimal action at (i, j)	Optimal policy $\mathbf{u}_k^* = \pi(\mathbf{x}_k)$	What to do at each state

It is *literally the same algorithm.* The only difference is that in control, the "grid" is continuous (states are real numbers, not integers), and the "decisions" are real-valued control inputs instead of discrete left/right choices.

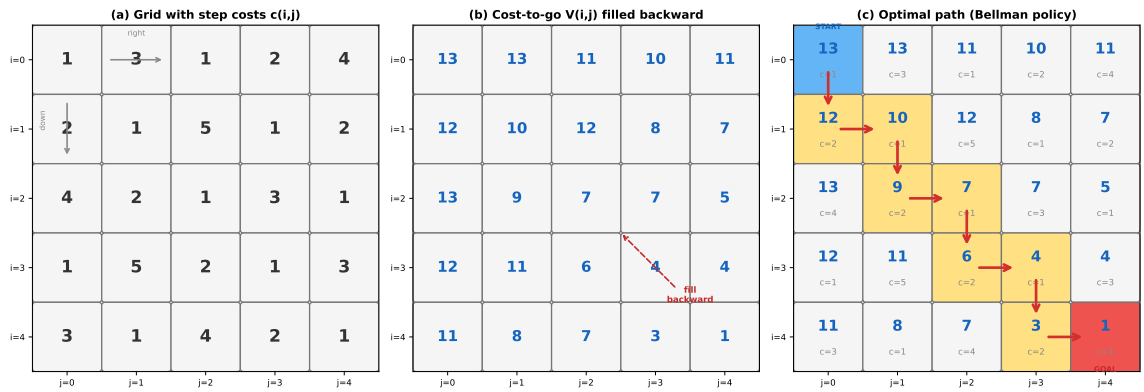

Figure 20: Grid-world DP example (5×5, moves: right or down). (a) Each cell has a step cost $c(i, j)$. (b) The cost-to-go $V(i, j)$ is filled *backward* from the goal — this is the DP table (= Riccati recursion in control). (c) The optimal path (total cost = 13) follows the Bellman policy: at each cell, go toward the neighbor with the smaller V. Replace "cell" with "state" and "right/down" with "control input" and you have LQR.

The Bellman equation for optimal control (discrete time):

Given a system $\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k)$ with stage cost $\ell(\mathbf{x}_k, \mathbf{u}_k)$ and terminal cost $\ell_f(\mathbf{x}_N)$, the optimal cost-to-go (value function) satisfies:

$$V_k(\mathbf{x}) = \min_{\mathbf{u}} \left[\ell(\mathbf{x}, \mathbf{u}) + V_{k+1}\left(f(\mathbf{x}, \mathbf{u})\right)\right] \tag{97}$$

with boundary condition $V_N(\mathbf{x}) = \ell_f(\mathbf{x})$.

The optimal control at step k is:

$$\mathbf{u}_k^*(\mathbf{x}) = \arg\min_{\mathbf{u}} \left[\ell(\mathbf{x}, \mathbf{u}) + V_{k+1}\left(f(\mathbf{x}, \mathbf{u})\right)\right] \tag{98}$$

This is solved backward in time, from $k = N$ to $k = 0$, just like filling the DP table from the goal to the start.

From Grid-World to LQR: The "Continuous Grid" Problem

Now imagine the grid-world becomes continuous. Instead of cells, your "position" is a real-valued state $\mathbf{x} \in \mathbb{R}^n$. Instead of right/down, your "move" is a real-valued control $\mathbf{u} \in \mathbb{R}^m$. The transition is linear: $\mathbf{x}_{k+1} = A\mathbf{x}_k + B\mathbf{u}_k$. The step cost is quadratic: $\ell(\mathbf{x}, \mathbf{u}) = \mathbf{x}^T Q\mathbf{x} + \mathbf{u}^T R\mathbf{u}$.

Plug into the Bellman equation:

$$V_k(\mathbf{x}) = \min_{\mathbf{u}} \left[\mathbf{x}^T Q\mathbf{x} + \mathbf{u}^T R\mathbf{u} + V_{k+1}(A\mathbf{x} + B\mathbf{u})\right]$$

Here comes the magic: because the dynamics are linear and the cost is quadratic, the value function is **also quadratic**: $V_k(\mathbf{x}) = \mathbf{x}^T P_k \mathbf{x}$ for some matrix P_k. Substituting this ansatz and taking the derivative with respect to $\mathbf{u}$:

$$\frac{\partial}{\partial \mathbf{u}} \left[\mathbf{u}^T R\mathbf{u} + (A\mathbf{x} + B\mathbf{u})^T P_{k+1}(A\mathbf{x} + B\mathbf{u})\right] = 0$$

Solving gives:

Expand the Bellman equation: $V_k(\mathbf{x}) = \min_{\mathbf{u}}[\mathbf{x}^T Q\mathbf{x} + \mathbf{u}^T R\mathbf{u} + (A\mathbf{x} + B\mathbf{u})^T P_{k+1}(A\mathbf{x} + B\mathbf{u})]$. Differentiate the bracketed expression with respect to $\mathbf{u}$ and set to zero: $2R\mathbf{u} + 2B^T P_{k+1}(A\mathbf{x} + B\mathbf{u}) = 0$, which gives $\mathbf{u}^* = -(R + B^T P_{k+1} B)^{-1} B^T P_{k+1} A\mathbf{x}$. Substituting back yields the Riccati recursion for P_k.

$$\mathbf{u}_k^* = -\underbrace{(R + B^T P_{k+1} B)^{-1} B^T P_{k+1} A}_{K_k}\, \mathbf{x}_k \tag{99}$$

$$P_k = Q + A^T P_{k+1} A - A^T P_{k+1} B (R + B^T P_{k+1} B)^{-1} B^T P_{k+1} A \tag{100}$$

This backward recursion for P_k is the **discrete Riccati equation** — exactly the "fill the DP table backward" step. When the horizon $N \to \infty$, P_k converges to a constant P, and we get the steady-state **LQR** gain $K = (R + B^T PB)^{-1} B^T PA$.

In the grid-world analogy:

- P_k is the cost-to-go matrix — it tells you the "remaining cost" from any state, just like $V(i, j)$ in the grid.
- K_k is the optimal policy at step k — it tells you what to do at each state, just like "go right if $V(i + 1, j) < V(i, j + 1)$."
- The Riccati recursion is the DP table fill — same $O(N)$ backward pass.

Connection to MPC: MPC is the same Bellman/DP framework, but with two additions:

1. **Constraints** on states and inputs (like adding walls to the grid-world — some cells are forbidden).
2. **Receding horizon**: instead of solving once from $k = 0$ to N, re-solve at every timestep using the latest measurement as the new "start position."

The constraints make the Bellman equation harder to solve (the min is now a constrained optimization), which is why MPC needs a QP solver or the ADMM approach of TinyMPC. But the underlying principle is identical: backward recursion to find the optimal cost-to-go, forward application of the optimal policy.

Connection to reinforcement learning: If you are familiar with RL, the Bellman equation is exactly the same as the **Bellman optimality equation** in RL ($V^*(s) = \max_a[r(s, a) + \gamma V^*(s')]$). LQR is the "closed-form RL" for linear-quadratic problems. When the dynamics are unknown or nonlinear, RL methods (Q-learning, policy gradient) approximate the solution that LQR computes exactly.

10.3.2 LQR (Linear Quadratic Regulator)

With the Bellman/DP framework fresh in mind, LQR is simply the steady-state solution to the linear-quadratic Bellman equation. Instead of manually placing poles, you specify a cost function that balances performance against effort:

$$J = \int_0^\infty \left(\mathbf{x}^T Q\mathbf{x} + \mathbf{u}^T R\mathbf{u}\right) dt \tag{101}$$

The matrix Q penalizes state deviations—larger Q means more aggressive tracking and faster response—while R penalizes control effort—larger R means gentler inputs and energy saving. The balance between Q and R is the fundamental design knob of LQR.

The optimal state feedback gain is:

$$K = R^{-1} B^T P \tag{102}$$

where P is the solution to the **continuous algebraic Riccati equation (CARE)**:

$$A^T P + PA - PBR^{-1} B^T P + Q = 0 \tag{103}$$

Do not solve this by hand. Use `lqr(A, B, Q, R)` in MATLAB or `scipy.linalg.solve_continuous_are()` in Python.

What is the Riccati equation actually computing? The matrix P represents the **optimal cost-to-go**: P_{ij} tells you how much "future cost" is associated with the interaction between states i and j. Think of it as a map of "how expensive is it to be in each state?" States that are hard to control (require lots of effort to bring back to zero) get large entries in P. The optimal gain $K = R^{-1} B^T P$ then says: "Apply control effort proportional to how costly each state is." Expensive states get corrected aggressively; cheap states get gentle treatment.

Tuning LQR. The art is in choosing Q and R. Start with $Q = I$ (identity matrix) and $R = \rho I$ where ρ is a single scalar. A small ρ gives aggressive, fast control that uses lots of actuator effort; a large ρ is conservative and gentle on actuators. To penalize specific states more, increase the corresponding diagonal elements of Q—for example, if you care more about position than velocity, make $Q_{11} > Q_{22}$.

Why LQR is great for competitions. It provides guaranteed stability (for controllable systems) and guaranteed robustness margins—at least 6 dB gain margin and 60° phase margin for SISO systems. It handles MIMO systems effortlessly, and the Q/R tuning is more intuitive than PID's three gains for multi-axis systems.

Example: Balancing a two-wheeled robot. The state is $\mathbf{x} = [\theta, \dot{\theta}, x, \dot{x}]^T$ (tilt angle, tilt rate, position, velocity). PID would need multiple nested loops; LQR handles it with a single K matrix.

Limitation: LQR requires full state feedback ($\mathbf{u} = -K\mathbf{x}$), which means you need to know all states. If you cannot measure all states, you need an observer (see Section 6.4).

10.3.3 A Brief Primer on Optimization and Convexity

MPC solves an optimization problem at every time step. Before diving in, let us establish the minimum optimization vocabulary you need.

The general optimization problem:

$$\min_{\mathbf{x}} \; f(\mathbf{x}) \quad \text{subject to} \quad g_i(\mathbf{x}) \leq 0, \;\; h_j(\mathbf{x}) = 0$$

where f is the **objective function** (cost), g_i are **inequality constraints**, and h_j are **equality constraints**. The optimizer searches for the decision variable $\mathbf{x}$ that minimizes f without violating any constraint.

What makes a problem convex? A problem is convex if:

1. The objective f is a **convex function**: for any two points $\mathbf{a}, \mathbf{b}$ and $0 \leq \lambda \leq 1$,

$$f\big(\lambda\mathbf{a} + (1-\lambda)\mathbf{b}\big) \leq \lambda f(\mathbf{a}) + (1-\lambda) f(\mathbf{b})$$

   Geometrically: the function lies below any chord. Quadratic functions $f(\mathbf{x}) = \mathbf{x}^T Q\mathbf{x}$ with $Q \succeq 0$ (positive semidefinite) are convex.
2. The **feasible set** (defined by the constraints) is convex: any line segment between two feasible points stays feasible. Linear inequalities ($A\mathbf{x} \leq \mathbf{b}$) always define convex sets.

Why convexity matters: A convex problem has a single global minimum—no local minima traps. Any solver that finds a solution has found *the* solution. Non-convex problems can have many local minima, and solvers may get stuck in a bad one.

Quadratic Programs (QP): The most common optimization problem in control. The objective is quadratic, the constraints are linear:

$$\min_{\mathbf{x}} \; \tfrac{1}{2}\mathbf{x}^T H\mathbf{x} + \mathbf{c}^T\mathbf{x} \quad \text{s.t.} \quad A\mathbf{x} \leq \mathbf{b}$$

With $H \succeq 0$, this is convex and can be solved efficiently even on embedded systems (TinyMPC uses this structure). LQR without constraints is an *unconstrained* QP whose solution is the Riccati equation. MPC with box constraints on inputs and states is a *constrained* QP.

Connection to control: Every optimal control problem we encounter in this chapter—LQR, MPC, Kalman filter (dual of LQR)—is a convex QP. This is not a coincidence: the linear dynamics and quadratic cost were chosen *precisely because* they make the problem convex and therefore tractable in real time.

10.3.4 MPC (Model Predictive Control)

MPC is the most powerful and flexible controller in the modern control toolbox. It solves an optimization problem at every time step:

"Given where I am now, what sequence of inputs over the next N steps minimizes my cost function while respecting all constraints?"

At each time step:

1. Measure (or estimate) the current state $\mathbf{x}[k]$.
2. Solve the optimization problem:

$$\min_{\mathbf{u}[k],\ldots,\mathbf{u}[k+N-1]} \sum_{i=0}^{N-1} \big(\mathbf{x}[k+i]^T Q\mathbf{x}[k+i] + \mathbf{u}[k+i]^T R\mathbf{u}[k+i]\big) + \mathbf{x}[k+N]^T P_f \mathbf{x}[k+N] \tag{104}$$

   subject to:

$$\mathbf{x}[k+i+1] = A_d\mathbf{x}[k+i] + B_d\mathbf{u}[k+i] \tag{105}$$

$$\mathbf{u}_{\min} \leq \mathbf{u}[k+i] \leq \mathbf{u}_{\max} \tag{106}$$

$$\mathbf{x}_{\min} \leq \mathbf{x}[k+i] \leq \mathbf{x}_{\max} \tag{107}$$

3. Apply only the first input $\mathbf{u}[k]$.
4. Repeat at the next time step (receding horizon).

Understanding the cost function terms:

- $\mathbf{x}^T Q\mathbf{x}$: **Stage cost on states.** Penalizes deviation from the desired state at each step. Larger Q_{ii} means "I care more about keeping state i close to zero (or the reference)."
- $\mathbf{u}^T R\mathbf{u}$: **Stage cost on inputs.** Penalizes control effort. Larger R means "go easy on the actuators." This prevents the optimizer from finding mathematically optimal but physically violent solutions.
- $\mathbf{x}[k+N]^T P_f \mathbf{x}[k+N]$: **Terminal cost.** Penalizes the state at the end of the prediction horizon. Without this, MPC can "cheat" by pushing all the error to the end of the horizon (like a student who does nothing until the last day). Setting P_f to the solution of the LQR Riccati equation is a common choice that guarantees stability.
- N (**prediction horizon**): How many steps ahead MPC looks. Longer horizon = better anticipation but heavier computation. Too short = myopic behavior (MPC cannot "see" upcoming turns or obstacles). A good starting point: $N = 10$–30 steps.

The receding horizon concept: MPC computes an *entire* optimal trajectory but only executes the first step, then replans. This sounds wasteful, but it is what makes MPC robust: at each step, it uses the latest measurement to correct for model errors and disturbances. It is like a chess player who plans 10 moves ahead but re-evaluates after each opponent's move.

Why MPC is powerful:

- **Handles constraints explicitly.** Motor current limits, joint angle limits, velocity bounds—they are all part of the optimization.
- **Handles MIMO systems naturally.** Multiple inputs and outputs are just more variables in the optimization.
- **Preview capability.** If you know the future reference trajectory (e.g., a planned path), MPC can look ahead and act proactively.
- **Intuitive tuning.** Like LQR, you tune Q and R. The prediction horizon N is the main additional parameter.

The catch: MPC requires solving an optimization problem at every time step. For linear systems with quadratic costs (linear MPC), this is a quadratic program (QP) that can be solved efficiently. For nonlinear systems (nonlinear MPC), it is much harder.

Practical considerations for competition robots: For simple systems (chassis velocity control), PID or LQR is usually sufficient—MPC is overkill. But for trajectory tracking with constraints (autonomous navigation, robotic arms), MPC is worth the additional implementation effort. Libraries like OSQP, qpOASES, and acados can solve the QP fast enough for real-time embedded use. When even that is too expensive, *explicit MPC* precomputes the solution offline as a lookup table indexed by the current state.

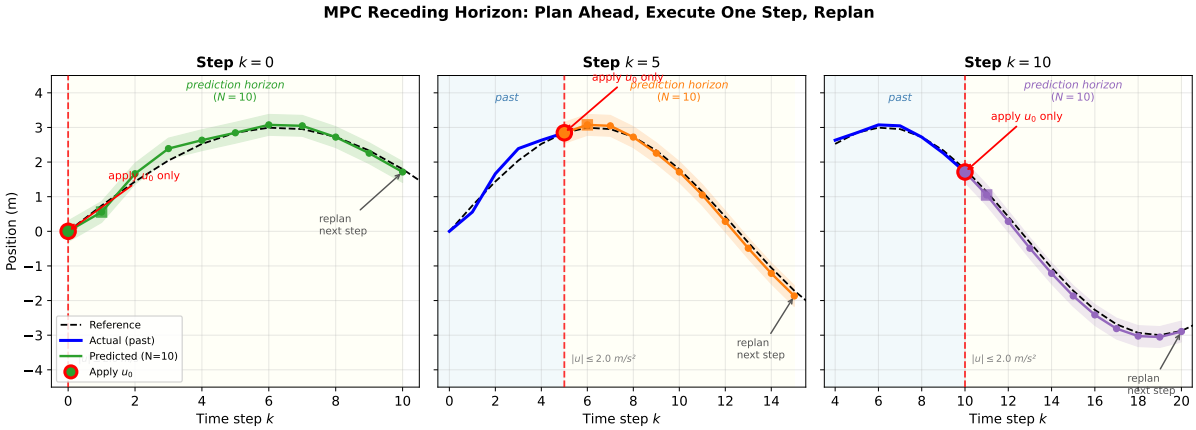

Figure 21: MPC receding horizon illustrated on a position-tracking problem. At each timestep, MPC plans an optimal trajectory over the next $N = 10$ steps (green), but only applies the **first** control action (circled). Then it re-measures, replans, and repeats. The prediction window slides forward with each step—this "plan ahead, execute one, replan" cycle is what makes MPC robust to model errors and disturbances.

10.3.5 TinyMPC: Riccati-Accelerated MPC for Embedded Systems

Standard MPC solves a full QP (quadratic program) at every timestep. For a system with n states, m inputs, and horizon N, this means solving a problem with $N(n+m)$ decision variables — feasible on a laptop but brutal on a microcontroller. **TinyMPC** (Nguyen et al., 2024) is a breakthrough that makes MPC run on resource-constrained embedded systems (Teensy, STM32, even Arduino) by exploiting the Riccati equation structure.

The key insight: For an unconstrained linear MPC, the solution is just a time-varying LQR — you can compute it via a backward Riccati recursion. TinyMPC precomputes this Riccati recursion **offline**, then handles constraints **online** with a lightweight ADMM (Alternating Direction Method of Multipliers) loop.

Algorithm structure:

Step 1 — Offline precomputation (once, or when model changes):

Run the backward Riccati recursion from the terminal cost $P_N = Q_f$ to compute the optimal feedback gains at each step:

$$S_k = R + B_d^T P_{k+1} B_d \tag{108}$$

$$K_k = S_k^{-1} B_d^T P_{k+1} A_d \tag{109}$$

$$P_k = Q + A_d^T P_{k+1} A_d - A_d^T P_{k+1} B_d K_k \tag{110}$$

This gives you N feedback gain matrices $K_0, K_1, \ldots, K_{N-1}$ and $N+1$ cost-to-go matrices $P_0, P_1, \ldots, P_N$. These are stored in memory and *never recomputed* (unless the model or cost matrices change).

Step 2 — Online solve (every control cycle):

Given the current state $\mathbf{x}_0$:

1. **Forward rollout:** Compute the unconstrained optimal trajectory using the precomputed gains: $\mathbf{u}_k = -K_k(\mathbf{x}_k - \mathbf{x}_k^{\text{ref}})$, then $\mathbf{x}_{k+1} = A_d\mathbf{x}_k + B_d\mathbf{u}_k$.

2. **ADMM projection:** If any $\mathbf{u}_k$ or $\mathbf{x}_k$ violates constraints, run a few ADMM iterations (typically 5–20) to find the nearest feasible solution. Each ADMM iteration is just:
   - A backward Riccati-like pass (using the *precomputed* K_k, so it is cheap).
   - A projection step: $z_k = \text{clamp}(u_k + \lambda_k, u_{\min}, u_{\max})$ — element-wise clamping.
   - A dual update: $\lambda_k \leftarrow \lambda_k + u_k - z_k$.

3. **Apply first input:** Send $\mathbf{u}_0$ to the actuator. Discard the rest (receding horizon).

Why this is so fast:

- The Riccati recursion (the expensive part) is done **offline**. Online work is just matrix multiplies and clamping.

- No QP solver library needed. No matrix factorization online. Just multiply-add and clamp.
- Each ADMM iteration is $O(N \cdot n^2)$ — linear in horizon, quadratic in state dimension. For a 4-state system with $N = 10$: about 1600 multiply-adds per iteration. At 10 iterations, that is 16,000 multiply-adds — an STM32F4 at 168 MHz finishes this in $< 100\,\mu$s.
- Memory usage: $N \times (n \times n + m \times n)$ floats for the precomputed gains. For 4 states, 2 inputs, $N = 10$: about 240 floats = 960 bytes.

Comparison with standard QP-based MPC:

	Standard MPC (OSQP)	**TinyMPC**
Online computation	Full QP solve	Riccati rollout + ADMM
Typical solve time (STM32)	1–10 ms	50–200 μs
Memory	KBs (sparse matrices)	$<$ 1 KB for small systems
Dependencies	QP solver library	None (just matrix ops)
Constraint handling	Exact (to tolerance)	Approximate (ADMM)
Warm starting	Possible	Natural (previous solution)

When to use TinyMPC:

- You need MPC on a microcontroller with limited RAM/CPU.
- Your model is linear (or can be linearized at each step for nonlinear systems).
- Hard constraints on inputs are important (motor current limits, joint angle limits).
- Horizon is short to moderate ($N \leq 30$).

When NOT to use TinyMPC:

- Complex nonlinear models (use acados or CasADi on a more powerful computer).
- Very long horizons ($N > 50$) with many state constraints.
- If your problem is unconstrained — just use LQR, it is even simpler.

Reference: The full TinyMPC paper and open-source implementation are at `tinympc.org`. The C implementation fits in under 1000 lines and runs on ARM Cortex-M microcontrollers. Our C++ module in the Appendix follows the same Riccati + ADMM structure.

10.4 Observer Design via State-Space

LQR (and state feedback in general) requires knowing all states. But in practice, you can only measure some of them. An **observer** (or **estimator**) reconstructs the full state vector from the available measurements.

10.4.1 Luenberger Observer

The Luenberger observer is a copy of the system model, with a correction term that uses the measurement error to drive the estimate toward the true state.

$$\dot{\hat{\mathbf{x}}} = A\hat{\mathbf{x}} + B\mathbf{u} + L(\mathbf{y} - C\hat{\mathbf{x}}) \tag{111}$$

where:
- $\hat{\mathbf{x}}$ is the estimated state.
- $\mathbf{y} = C\mathbf{x}$ is the actual measurement.
- L is the observer gain matrix (we design this).

The estimation error $\mathbf{e} = \mathbf{x} - \hat{\mathbf{x}}$ evolves as:

$$\dot{\mathbf{e}} = (A - LC)\mathbf{e} \tag{112}$$

Subtract the observer equation $\dot{\hat{\mathbf{x}}} = A\hat{\mathbf{x}} + B\mathbf{u} + L(\mathbf{y} - C\hat{\mathbf{x}})$ from the true system $\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u}$. The $B\mathbf{u}$ terms cancel, and substituting $\mathbf{y} = C\mathbf{x}$ gives $\dot{\mathbf{e}} = A\mathbf{e} - LC\mathbf{e} = (A - LC)\mathbf{e}$.

If we choose L such that all eigenvalues of $(A - LC)$ have negative real parts, the estimation error decays to zero. The observer "catches up" to the true state.

Design rule of thumb: Place the observer poles 2–5× faster than the controller poles. The observer should converge faster than the controller acts, so the controller always has a good state estimate.

Separation principle: You can design the controller gain K and observer gain L independently. The combined controller-observer system is stable if each part is stable individually. This is a beautiful theoretical result that simplifies design enormously.

When to use Luenberger: When your model is accurate and the noise characteristics are not well known. It is simple, deterministic, and easy to debug.

10.4.2 The Deep Duality: Controller and Observer as Mirror Images

This is one of the most elegant results in all of control theory, and understanding it deeply will make you a better engineer. Let us lay out the parallel structure explicitly.

The controller problem: Given the system $\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u}$, find a gain K such that $\mathbf{u} = -K\mathbf{x}$ makes the closed-loop system $\dot{\mathbf{x}} = (A - BK)\mathbf{x}$ stable and well-behaved.

The observer problem: Given the system $\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u}$, $\mathbf{y} = C\mathbf{x}$, find a gain L such that the estimation error $\dot{\mathbf{e}} = (A - LC)\mathbf{e}$ decays to zero.

Now stare at the two closed-loop matrices side by side:

	Controller	**Observer**
Closed-loop matrix	$A - BK$	$A - LC$
Gain to design	K	L
Required condition	Controllability (A, B)	Observability (A, C)
Design goal	Place eigenvalues of $A - BK$	Place eigenvalues of $A - LC$
Information flow	Input $\to$ states	Measurements $\to$ state estimates
Riccati equation	$A^T P + PA - PBR^{-1}B^T P + Q = 0$	$AP_o + P_o A^T - P_o C^T R_n^{-1} C P_o + Q_n = 0$

The mathematical duality: If you transpose the observer Riccati equation, replacing $A \to A^T$, $C^T \to B$, $Q_n \to Q$, $R_n \to R$, you get exactly the controller Riccati equation. They are literally the same equation with transposed matrices. This means:

- Any algorithm that designs a controller can design an observer (just transpose the inputs).
- If you can place poles for a controller using `place(A, B, poles)`, you can place observer poles using `place(A', C', poles)'` — note the transposes.
- The LQR controller (K from the control Riccati) has a dual: the **Kalman filter** (L from the observer Riccati). LQR is the optimal controller; Kalman is the optimal observer. They are dual problems.

A physical analogy: Think of the controller as a *speaker* and the observer as a *microphone*. The speaker (controller) pushes energy into the system through the inputs B to shape the state trajectory. The microphone (observer) listens to the system through the outputs C to reconstruct what is happening inside. Controllability asks: "Can the speaker reach every corner of the room?" Observability asks: "Can the microphone hear every corner of the room?" They are symmetric questions about the same room (system).

Why this matters in practice: If you have already tuned a state feedback controller and it works well, you have a strong intuition for how pole placement affects performance. That *exact same intuition* applies when designing an observer. Faster poles = faster convergence but more noise sensitivity. Slower poles = smoother estimates but slower tracking. You are turning the same knob, just on the other side of the mirror.

10.4.3 Kalman Filter

The Kalman filter is the optimal observer for linear systems with Gaussian noise. It is the Luenberger observer's probabilistic cousin—instead of placing observer poles manually, it computes the optimal gain L based on the noise statistics.

System model with noise:

$$\mathbf{x}[k+1] = A_d\mathbf{x}[k] + B_d\mathbf{u}[k] + \mathbf{w}[k], \quad \mathbf{w} \sim \mathcal{N}(0, Q_n) \tag{113}$$

$$\mathbf{y}[k] = C\mathbf{x}[k] + \mathbf{v}[k], \quad \mathbf{v} \sim \mathcal{N}(0, R_n) \tag{114}$$

where $\mathbf{w}$ is process noise (model uncertainty) and $\mathbf{v}$ is measurement noise. Q_n and R_n are their covariance matrices.

The Kalman filter algorithm (discrete):

Predict:

$$\hat{\mathbf{x}}[k|k-1] = A_d\hat{\mathbf{x}}[k-1|k-1] + B_d\mathbf{u}[k-1] \tag{115}$$

$$P[k|k-1] = A_dP[k-1|k-1]A_d^T + Q_n \tag{116}$$

Update:

$$K_k = P[k|k-1]C^T(CP[k|k-1]C^T + R_n)^{-1} \tag{117}$$

$$\hat{\mathbf{x}}[k|k] = \hat{\mathbf{x}}[k|k-1] + K_k(\mathbf{y}[k] - C\hat{\mathbf{x}}[k|k-1]) \tag{118}$$

$$P[k|k] = (I - K_kC)P[k|k-1] \tag{119}$$

Intuition: The Kalman gain K_k automatically balances trust between the model prediction and the measurement:

- If process noise is large (Q_n big) relative to measurement noise (R_n small), the filter trusts the measurement more.
- If measurement noise is large relative to process noise, the filter trusts the model more.

Tuning the Kalman filter comes down to choosing Q_n and R_n:

- R_n can often be measured directly (record sensor data when the system is stationary, compute the variance).
- Q_n is harder—it represents model uncertainty. Start with small values and increase if the filter is too sluggish (not tracking fast enough).
- A common cheat: treat Q_n and R_n as tuning knobs. Increase Q_n/R_n ratio for faster tracking (more noise). Decrease for smoother estimates (more lag). Sounds familiar? It is the same trade-off as low-pass filter cutoff frequency, just expressed probabilistically.

Why the Kalman filter is everywhere in robotics:

- **Sensor fusion:** Combine IMU, encoder, and vision data into one coherent state estimate.
- **Velocity estimation:** Get smooth velocity from noisy position measurements without a pure differentiator.
- **Optimal in a well-defined sense:** It minimizes the mean-squared estimation error for linear Gaussian systems.

A practical note: The Kalman filter needs a matrix inverse at each step (the K_k computation). For small state dimensions ($n \leq 6$), this is trivial. For larger systems, use efficient implementations (e.g., Cholesky decomposition). On an STM32F4, a 6-state EKF runs comfortably at 1 kHz.

10.4.4 Extended Kalman Filter (EKF)

The standard Kalman filter assumes linear dynamics ($\mathbf{x}_{k+1} = A\mathbf{x}_k + B\mathbf{u}_k$) and linear measurements ($\mathbf{y}_k = C\mathbf{x}_k$). But real robotics systems are **nonlinear**: the relationship between Euler angles and angular rates involves trigonometric functions, the accelerometer measurement model depends on the sine/cosine of the tilt angles, and so on.

The **Extended Kalman Filter** handles nonlinearity by *linearizing* the system at each timestep around the current estimate. The idea is simple: at each step, pretend the system is locally linear, apply the standard Kalman filter, then re-linearize at the new estimate.

Nonlinear system model:

$$\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k) + \mathbf{w}_k \quad \text{(nonlinear dynamics)} \tag{120}$$

$$\mathbf{y}_k = h(\mathbf{x}_k) + \mathbf{v}_k \quad \text{(nonlinear measurement)} \tag{121}$$

The Jacobian matrix — what it is and why we need it. The Kalman filter's covariance propagation ($P_{k|k-1} = AP_{k-1|k-1}A^T + Q$) requires a *matrix* A. But for a nonlinear system, there is no single matrix A—the "slope" of f changes everywhere. The solution is to use the **Jacobian matrix**, which is simply the multivariable generalization of the derivative.

Recall from calculus: for a scalar function $y = f(x)$, the derivative $f'(x_0)$ tells you the best linear approximation near x_0: $f(x) \approx f(x_0) + f'(x_0)(x - x_0)$. The Jacobian does the same thing for a *vector-valued* function of *multiple variables*. If $\mathbf{f} : \mathbb{R}^n \to \mathbb{R}^m$, the Jacobian is the $m \times n$ matrix of all partial derivatives:

$$F = \frac{\partial \mathbf{f}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}.$$

Each row tells you how one output component changes with respect to all inputs. Evaluated at a specific point $\mathbf{x}_0$, the Jacobian gives the best local linear approximation: $\mathbf{f}(\mathbf{x}) \approx \mathbf{f}(\mathbf{x}_0) + F(\mathbf{x} - \mathbf{x}_0)$—exactly the "pretend the system is locally linear" step the EKF needs.

EKF algorithm:

Predict:

$$\hat{\mathbf{x}}_{k|k-1} = f(\hat{\mathbf{x}}_{k-1|k-1}, \mathbf{u}_{k-1}) \tag{122}$$

$$F_k = \left.\frac{\partial f}{\partial \mathbf{x}}\right|_{\hat{\mathbf{x}}_{k-1|k-1}} \quad \text{(Jacobian of dynamics)} \tag{123}$$

$$P_{k|k-1} = F_k P_{k-1|k-1} F_k^T + Q \tag{124}$$

Update:

$$H_k = \left.\frac{\partial h}{\partial \mathbf{x}}\right|_{\hat{\mathbf{x}}_{k|k-1}} \quad \text{(Jacobian of measurement)} \tag{125}$$

$$K_k = P_{k|k-1} H_k^T (H_k P_{k|k-1} H_k^T + R)^{-1} \tag{126}$$

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + K_k \left(\mathbf{y}_k - h(\hat{\mathbf{x}}_{k|k-1})\right) \tag{127}$$

$$P_{k|k} = (I - K_k H_k) P_{k|k-1} \tag{128}$$

The only difference from the standard Kalman filter:

1. The prediction uses the *nonlinear* function $f(\cdot)$ instead of the matrix A.
2. The measurement innovation uses the *nonlinear* function $h(\cdot)$ instead of the matrix C.
3. The covariance propagation uses the *Jacobians* F_k and H_k (evaluated at the current estimate) instead of the constant matrices A and C.

Everything else — the Kalman gain formula, the covariance update, the predict-update cycle — is *identical*.

Case Study: 6-DOF IMU Pose Estimation with EKF

This is the single most common EKF application in competition robotics: estimating the 3D orientation (roll, pitch, yaw) of your robot from a 6-axis IMU (3-axis accelerometer + 3-axis gyroscope).

State vector: $\mathbf{x} = [\phi, \theta, \psi, b_x, b_y, b_z]^T$

- ϕ: roll, θ: pitch, ψ: yaw (Euler angles in radians)
- b_x, b_y, b_z: gyroscope biases (rad/s) — the EKF estimates these automatically

Dynamics model $f(\mathbf{x}, \mathbf{u})$: The angular rates from the gyroscope (after bias subtraction) drive the Euler angles via the kinematic equations:

$$\omega_x = \text{gyro}_x - b_x, \quad \omega_y = \text{gyro}_y - b_y, \quad \omega_z = \text{gyro}_z - b_z \tag{129}$$

$$\dot{\phi} = \omega_x + \sin\phi \tan\theta \cdot \omega_y + \cos\phi \tan\theta \cdot \omega_z \tag{130}$$

$$\dot{\theta} = \cos\phi \cdot \omega_y - \sin\phi \cdot \omega_z \tag{131}$$

$$\dot{\psi} = \frac{\sin\phi}{\cos\theta} \cdot \omega_y + \frac{\cos\phi}{\cos\theta} \cdot \omega_z \tag{132}$$

$$\dot{b}_x = 0, \quad \dot{b}_y = 0, \quad \dot{b}_z = 0 \quad \text{(biases modeled as random walk)} \tag{133}$$

The Jacobian $F = \frac{\partial f}{\partial \mathbf{x}}$ is a 6×6 matrix with trigonometric entries. It must be recomputed at every timestep (this is the "extended" part).

Measurement model $h(\mathbf{x})$: The accelerometer measures gravity direction, which gives us roll and pitch (but NOT yaw):

$$h(\mathbf{x}) = \begin{bmatrix} \phi \\ \theta \end{bmatrix} = \begin{bmatrix} \arctan 2(a_y, a_z) \\ \arctan 2(-a_x, \sqrt{a_y^2 + a_z^2}) \end{bmatrix} \tag{134}$$

The Jacobian is simple: $H = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix}$.

Key design decisions:

- **Yaw is unobservable** from a 6-axis IMU alone (gravity does not tell you which direction is north). Yaw relies entirely on the gyroscope. To correct yaw drift, add a magnetometer (9-DOF IMU) or external heading reference.
- **Q matrix tuning:** The diagonal entries for angles (Q_{00}, Q_{11}, Q_{22}) represent how much the gyro integration can drift per step. Larger = trust gyro less. The bias entries (Q_{33}, Q_{44}, Q_{55}) represent bias random walk rate — typically very small (10^{-4} to 10^{-6}).
- **R matrix tuning:** Represents accelerometer noise. During vibration or linear acceleration, the accelerometer readings are corrupted. Increase R during high-acceleration maneuvers (adaptive EKF) to avoid trusting bad accelerometer data.
- **Gimbal lock:** When pitch = $\pm 90°$, the $\tan\theta$ and $1/\cos\theta$ terms in the kinematic equations blow up. For robots that can tilt past $\pm 60°$, use **quaternion representation** instead of Euler angles (same EKF structure, different state parameterization).

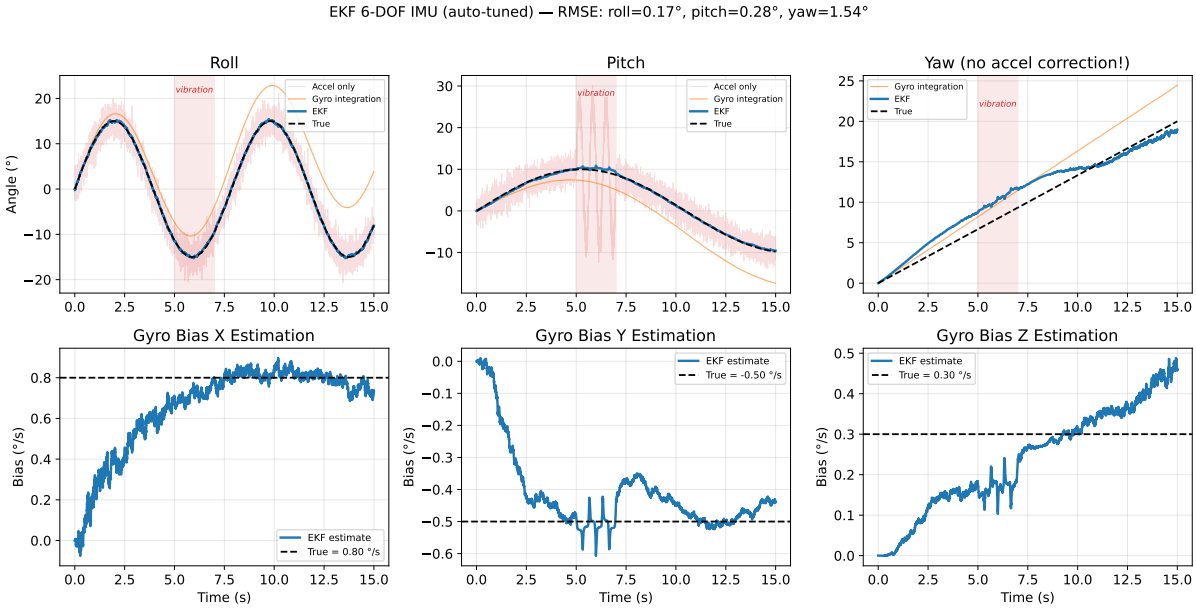

Figure 22: EKF 6-DOF IMU pose estimation with auto-tuned continuous adaptive R. Uses smooth exponential scaling $R = R_{\text{base}} \cdot (1 + \alpha\, e^{\beta \cdot \Delta a})$ with separate R_{base} for roll and pitch, plus pitch-specific x-axis accel deviation detection. Parameters found by offline grid search minimizing weighted RMSE. Top row: roll (RMSE 0.17°), pitch (0.28°), yaw (1.54°). During vibration ($t = 5$–7s, shaded), the adaptive R scales up exponentially, rejecting corrupted accelerometer data. Yaw drift is bounded by bias estimation alone (no heading reference). Bottom row: gyroscope biases converge smoothly to true values.

Beyond EKF: The EKF's linearization can be inaccurate when the state uncertainty is large or the nonlinearity is strong. Alternatives:

- **Unscented Kalman Filter (UKF):** Instead of linearizing, propagates "sigma points" (a small set of carefully chosen sample states) through the nonlinear function. More accurate than EKF for highly nonlinear systems, slightly more expensive ($2n+1$ function evaluations per step where n is the state dimension).
- **Error-State Kalman Filter (ESKF):** Estimates a *small error* around a nominal trajectory instead of the full state. Popular for IMU-based systems because the error state stays small (linearization is accurate) and quaternion singularities are avoided. This is what most production-grade IMU fusion libraries use internally.
- **Complementary filter:** For simple roll/pitch estimation where computational resources are extremely limited, the complementary filter (Section 2) remains a viable one-line alternative.

10.4.5 Quaternion-Based EKF: Avoiding Gimbal Lock

The Euler-angle EKF above works well when pitch stays within ±60°. But when pitch approaches ±90°, the kinematic equations blow up: $\tan\theta \to \infty$ and $1/\cos\theta \to \infty$. This is **gimbal lock**—a mathematical singularity, not a physical one. The robot is fine; your math is broken.

The fix is to represent orientation with a **unit quaternion** $\mathbf{q} = \begin{bmatrix} q_w & q_x & q_y & q_z \end{bmatrix}^T$ instead of Euler angles. Quaternions have no singularities, are compact (4 numbers vs 9 for a rotation matrix), and compose rotations efficiently.

State vector: $\mathbf{x} = \begin{bmatrix} \mathbf{q}^T & \boldsymbol{\beta}^T \end{bmatrix}^T \in \mathbb{R}^7$ where $\boldsymbol{\beta} = \begin{bmatrix} \beta_x & \beta_y & \beta_z \end{bmatrix}^T$ are the gyroscope biases.

Prediction step: Given gyroscope measurement $\boldsymbol{\omega}_m$, the bias-corrected angular velocity is $\boldsymbol{\omega} = \boldsymbol{\omega}_m - \boldsymbol{\beta}$. The quaternion kinematics are:

$$\dot{\mathbf{q}} = \frac{1}{2}\mathbf{q} \otimes \begin{bmatrix} 0 \\ \boldsymbol{\omega} \end{bmatrix} = \frac{1}{2}\Omega(\boldsymbol{\omega})\,\mathbf{q} \tag{135}$$

where $\otimes$ is quaternion multiplication and $\Omega(\boldsymbol{\omega})$ is the 4×4 skew matrix:

$$\Omega(\boldsymbol{\omega}) = \begin{bmatrix} 0 & -\omega_x & -\omega_y & -\omega_z \\ \omega_x & 0 & \omega_z & -\omega_y \\ \omega_y & -\omega_z & 0 & \omega_x \\ \omega_z & \omega_y & -\omega_x & 0 \end{bmatrix} \tag{136}$$

Discrete-time prediction (first-order):

$$\mathbf{q}_{k+1} = \left(I + \frac{\Delta t}{2} \Omega(\boldsymbol{\omega}_k) \right) \mathbf{q}_k, \qquad \boldsymbol{\beta}_{k+1} = \boldsymbol{\beta}_k \tag{137}$$

After prediction, **normalize q** to unit length: $\mathbf{q} \leftarrow \mathbf{q}/\|\mathbf{q}\|$.

Measurement model: The accelerometer measures gravity in the body frame. Given quaternion $\mathbf{q}$, the predicted gravity direction in body frame is:

$$\mathbf{g}_{\text{pred}} = R(\mathbf{q})^T \begin{bmatrix} 0 \\ 0 \\ g \end{bmatrix} = g \begin{bmatrix} 2(q_x q_z - q_w q_y) \\ 2(q_y q_z + q_w q_x) \\ q_w^2 - q_x^2 - q_y^2 + q_z^2 \end{bmatrix} \tag{138}$$

The measurement Jacobian $H = \partial \mathbf{g}_{\text{pred}} / \partial \mathbf{x}$ is a 3×7 matrix (3 accel components, 4 quaternion + 3 bias states). The bias columns are zero since accelerometer readings do not depend on gyro biases directly.

Update step: Standard EKF update applies. The innovation is $\boldsymbol{\nu} = \mathbf{a}_{\text{meas}} - \mathbf{g}_{\text{pred}}$, and after the update, normalize $\mathbf{q}$ again.

Euler Angles vs Quaternions: When to Use Which

- **Euler angles** are fine if your robot operates within $\pm 60°$ pitch (ground vehicles, most turrets, most drones in normal flight). The math is more intuitive and easier to debug.
- **Quaternions** are necessary for robots that can flip, tumble, or point straight up/down (acrobatic drones, underwater ROVs, robot arms near wrist singularities).
- **Tip:** You can use quaternions internally for all computation and convert to Euler angles only for display and human-readable logging: $\phi = \text{atan2}(2(q_w q_x + q_y q_z),\ 1 - 2(q_x^2 + q_y^2))$, etc.
- **The Error-State approach** (ESKF) avoids working with the 4-element quaternion directly in the EKF state. Instead, it estimates a 3-element rotation error $\delta\boldsymbol{\theta}$ and applies it to the quaternion after each update. This keeps the covariance matrix P at 6×6 instead of 7×7 and avoids the unit-norm constraint issue. Most production IMU libraries (e.g., PX4's EKF2, VINS-Mono) use ESKF internally.

10.4.6 Adaptive EKF: Auto-Tuning Q and R Online

The EKF's performance depends critically on the noise covariances Q (process noise) and R (measurement noise). In practice, these are often tuned by hand — a tedious trial-and-error process. Worse, the true noise characteristics *change during operation*: an IMU on a vibrating chassis has very different noise from the same IMU sitting still. Fixed Q and R cannot handle this.

Adaptive EKF methods estimate or adjust Q and R *online*, automatically. There are several approaches, ranging from simple to sophisticated.

Method 1: Innovation-Based Adaptive Estimation (IAE)

This is the simplest and most practical method. The idea: the **innovation sequence** $\boldsymbol{\nu}_k = \mathbf{y}_k - h(\hat{\mathbf{x}}_{k|k-1})$ (the difference between what you measured and what you predicted) contains information about whether your Q and R are correct.

The key insight: If the filter is tuned correctly, the innovation should be **zero-mean white noise** with covariance $S_k = H_k P_{k|k-1} H_k^T + R$. If the innovations are consistently larger than expected, either Q is too small (you are trusting the model too much) or R is too small (you are trusting the sensors too much).

Algorithm: Estimate the actual innovation covariance from a sliding window of N_w recent innovations:

$$\hat{S}_k = \frac{1}{N_w} \sum_{j=k-N_w+1}^{k} \boldsymbol{\nu}_j \boldsymbol{\nu}_j^T \tag{139}$$

Then update R to match:

$$\hat{R}_k = \hat{S}_k - H_k P_{k|k-1} H_k^T \tag{140}$$

If $\hat{R}_k$ has negative eigenvalues (which can happen with small N_w), clamp them to a minimum value. Typical window: $N_w = 20$–100 samples.

Code:

```
// After computing innovation nu = z - h(x_pred):
// Accumulate outer product in sliding window
innov_buffer[buf_idx] = nu;
buf_idx = (buf_idx + 1) % N_WINDOW;

if (samples_seen >= N_WINDOW) {
    // Estimate innovation covariance
    Mat<NZ,NZ> S_hat = Mat<NZ,NZ>::zeros();
    for (int j = 0; j < N_WINDOW; j++) {
        S_hat = S_hat + innov_buffer[j] * innov_buffer[j].T();
    }
    S_hat = S_hat * (1.0f / N_WINDOW);

    // Adapt R
    R = S_hat - H * P_pred * H.T();
    // Clamp diagonal to minimum (prevent negative/zero variance)
    for (int i = 0; i < NZ; i++)
        if (R(i,i) < R_min) R(i,i) = R_min;
}
```

Method 2: Residual-Based Q Adaptation

While IAE adapts R, the process noise Q can be adapted using the **state residuals** (difference between the predicted and updated state):

$$\hat{Q}_k = \frac{1}{N_w} \sum_j (K_j \boldsymbol{\nu}_j)(K_j \boldsymbol{\nu}_j)^T + P_{k|k} - F_k P_{k-1|k-1} F_k^T \tag{141}$$

This is more complex and less numerically stable than IAE. In practice, many systems only adapt R and leave Q fixed, because R varies more (sensor noise changes with environment) while Q is dominated by model uncertainty (which is more constant).

Method 3: Fading Memory (Exponential Forgetting)

The simplest "adaptive" trick: multiply the predicted covariance by a **fading factor** $\alpha > 1$:

$$P_{k|k-1} = \alpha \cdot (F_k P_{k-1|k-1} F_k^T + Q) \tag{142}$$

This artificially inflates the uncertainty at each step, making the filter "forget" old data and trust new measurements more. It is equivalent to making Q larger without explicitly changing it.

- $\alpha = 1.0$: standard EKF (no forgetting).
- $\alpha = 1.01$–1.05: mild adaptation — the filter reacts faster to sudden changes.
- $\alpha > 1.1$: very aggressive — the filter almost ignores the model and follows measurements. Only use during known disturbances.

When to use: When you expect occasional sudden model changes (e.g., a robot arm picks up an unknown load — the inertia changes instantly). The fading memory prevents the filter from "locking on" to outdated dynamics.

Code (one line added to standard EKF):

```
P_pred = F * P * F.T() + Q;
P_pred = P_pred * alpha;   // <-- fading memory, alpha = 1.01
```

Method 4: Multi-Mode Adaptive EKF (for IMU)

For competition robots, the most practical approach is a simple **mode-switching** strategy rather than continuous adaptation:

```
// Compute acceleration magnitude
float accel_mag = sqrt(ax*ax + ay*ay + az*az);
float accel_err = fabs(accel_mag - 9.81f);

if (accel_err < 0.5f) {
    // Stationary or slow motion: trust accelerometer
    ekf.R(0,0) = 0.05f;  ekf.R(1,1) = 0.05f;
} else if (accel_err < 2.0f) {
    // Moderate motion: reduce trust
    ekf.R(0,0) = 0.5f;   ekf.R(1,1) = 0.5f;
} else {
    // High acceleration / vibration: mostly ignore accelerometer
    ekf.R(0,0) = 10.0f;  ekf.R(1,1) = 10.0f;
}
```

The logic: When $|\mathbf{a}| \approx g$ (9.81 m/s^2), the robot is not accelerating and the accelerometer truly measures gravity — trust it (R small). When $|\mathbf{a}|$ deviates from g, there is linear acceleration or vibration contaminating the reading — distrust it (R large).

This is not mathematically elegant, but it is **extremely effective** in practice. Many championship-winning robots use exactly this approach. It is robust, easy to tune (just three thresholds), and computationally free.

Summary: which method to use?

Method	Adapts	Complexity	Best for
Multi-mode switching	R (discrete)	Trivial	IMU on mobile robots
Fading memory	Effective Q	1 line	Sudden model changes
IAE (innovation-based)	R (continuous)	Moderate	Varying sensor noise
Residual-based	Q and R	High	Research / offline

A word of caution: Adaptive methods can *destabilize* the filter if implemented incorrectly. Always clamp adapted Q and R to reasonable ranges, use a minimum window size ($N_w \geq 20$), and validate on recorded data before deploying on hardware.

10.4.7 Luenberger Observer vs. Kalman Filter: A Comprehensive Comparison

Now that you know both observers, let us put them side by side and understand when to use which.

Aspect	Luenberger Observer	Kalman Filter
Design input	Desired pole locations (you choose)	Noise covariances Q_n, R_n (from physics or tuning)
Gain computation	One-time pole placement	Recursive (updates P and K_k every step) or steady-state
Optimality	Not optimal — you pick "reasonable" poles	Optimal (minimizes mean-squared error for linear Gaussian systems)
Noise handling	No explicit noise model — gain is fixed	Explicitly models process and measurement noise
Adaptivity	Fixed gain L — same in all conditions	Gain K_k adapts as uncertainty (P) evolves
Computational cost	Very low (matrix multiply only)	Higher (matrix multiply + inverse each step)
Tuning difficulty	Medium (choose pole locations)	Medium (choose Q_n, R_n)
Confidence output	None	P matrix tells you how confident the estimate is
Sensor fusion	Possible but manual	Natural — just expand C matrix with more sensors

The deep connection: At steady state ($k \to \infty$), the Kalman filter gain K_k converges to a constant matrix K_∞. At that point, the Kalman filter *is* a Luenberger observer with $L = K_\infty$. The difference is

that the Kalman filter *computed* this L optimally from the noise statistics, while the Luenberger approach requires you to choose L by intuition or pole placement.

In fact, the steady-state Kalman gain K_∞ is the solution to the observer Riccati equation — the exact dual of the controller Riccati equation that LQR solves. This completes the beautiful symmetry:

	Control (input) side	**Estimation (output) side**
Manual design	Pole placement for K	Luenberger: pole placement for L
Optimal design	LQR: minimize $\int(\mathbf{x}^T Q\mathbf{x} + \mathbf{u}^T R\mathbf{u})\,dt$	Kalman: minimize $E[\|\mathbf{x} - \hat{\mathbf{x}}\|^2]$
Riccati variable	P (control cost-to-go)	P (estimation uncertainty)
Tuning knobs	Q vs. R (state penalty vs. effort)	Q_n vs. R_n (model trust vs. sensor trust)

10.4.8 LQG (Linear Quadratic Gaussian)

Now that we have both LQR (optimal controller) and the Kalman filter (optimal observer), we can combine them. **LQG (Linear Quadratic Gaussian)** is what happens when you do: LQR handles control, Kalman handles estimation, and the separation principle guarantees the combination is stable.

The LQG architecture:

$$\text{Kalman filter:} \quad \dot{\hat{\mathbf{x}}} = A\hat{\mathbf{x}} + B\mathbf{u} + L(\mathbf{y} - C\hat{\mathbf{x}}) \tag{143}$$

$$\text{LQR controller:} \quad \mathbf{u} = -K\hat{\mathbf{x}} \tag{144}$$

where K comes from the control Riccati (minimizing state cost) and L comes from the estimation Riccati (minimizing estimation error). The controller never sees the true state — it acts on the Kalman estimate.

Why LQG is superior to PID for MIMO/noisy systems:

1. **Optimal noise handling:** PID amplifies noise through the derivative term. LQG's Kalman filter provides the minimum-variance state estimate — the cleanest possible signal for the controller.

2. **MIMO natural:** PID is fundamentally SISO. For a 4-state balance bot, PID needs cascaded loops with hand-tuned interactions. LQG handles all states in one K matrix.

3. **State estimation:** PID cannot estimate unmeasured states. LQG's Kalman filter reconstructs the full state from partial measurements (e.g., estimating velocity from position-only sensors).

4. **Systematic tuning:** PID has 3 knobs per loop. LQG has Q, R (control cost) and Q_n, R_n (noise model) — physically meaningful matrices.

The LQG robustness caveat: Pure LQR has guaranteed stability margins ($\geq$ 6 dB gain, $\geq 60°$ phase). When you add the Kalman filter (making it LQG), these guarantees *vanish.* The LQG controller can have arbitrarily small stability margins. This is the famous **Doyle's counter-example** (1978) — it motivated the development of H_∞ robust control.

Practical implication: LQG works well when your model is accurate and noise is well-characterized. If there is significant model uncertainty (unknown friction, varying load), consider:

- **LQG/LTR (Loop Transfer Recovery):** Modify the Kalman filter tuning to recover LQR's robustness margins at the plant input.

- H_∞ **control:** Design for worst-case model uncertainty instead of average-case noise.

- **Practical approach:** Validate LQG in simulation with $\pm 30\%$ parameter variations. If it stays stable, it is usually fine for competition use.

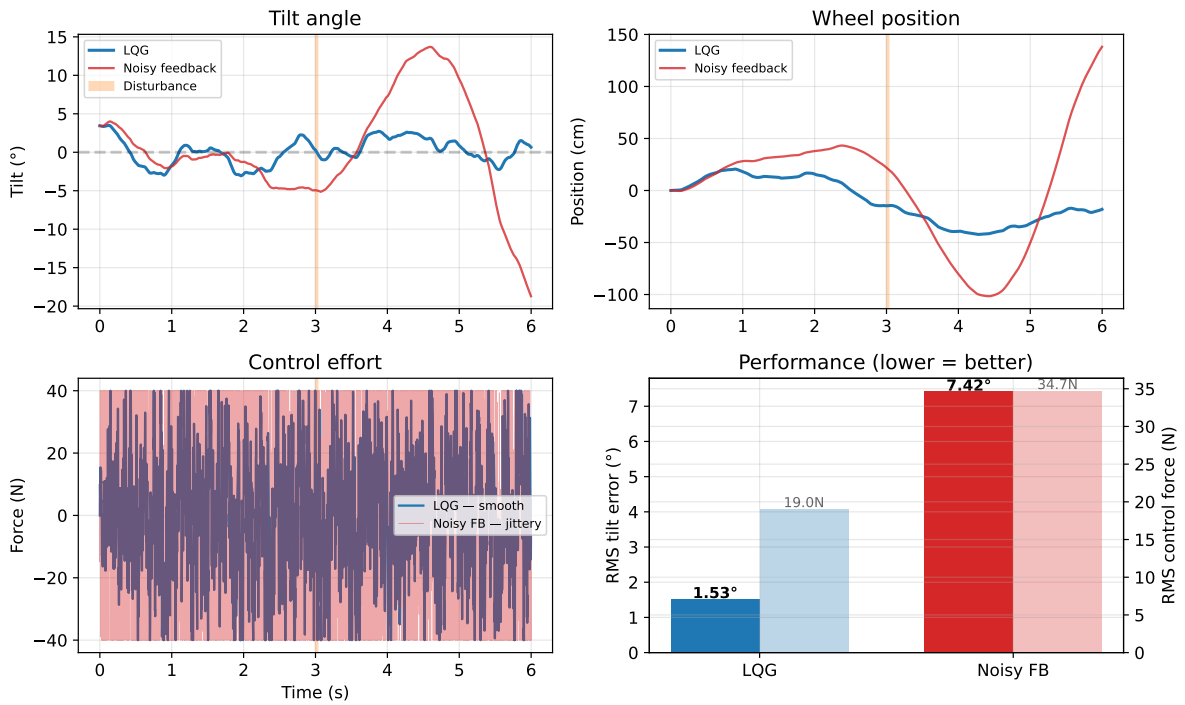

Figure 23: LQG (blue) vs direct noisy feedback (red) on a balance bot with disturbance at $t = 2$s. Both use the same LQR gains, but red uses raw noisy measurements + numerical differentiation while blue uses a Kalman filter. LQG recovers faster (top-left), maintains position (top-right), uses smoother control (bottom-left), and achieves lower RMS error (bottom-right). This demonstrates the value of the Kalman filter: same gains, same structure, dramatically different performance.

Practical decision tree:

1. **Do you need state estimation at all?** If you can measure all states directly (e.g., encoder gives position AND velocity), skip the observer entirely.
2. **Is your model accurate and noise is not a big concern?** Use Luenberger. It is simpler to implement and debug.
3. **Is sensor noise significant or do you need sensor fusion?** Use Kalman. It handles noise optimally and fuses multiple sensors naturally.
4. **Is your system nonlinear?** Use Extended Kalman Filter (EKF). It linearizes at each step, which works well for "mildly" nonlinear systems like IMU orientation estimation.
5. **Is your system highly nonlinear or does EKF diverge?** Use Unscented Kalman Filter (UKF) or consider particle filters (beyond our scope here).

Case Study: IMU Tilt Estimation — Kalman Filter vs. Complementary Filter

You are building a self-balancing robot. You need to know the tilt angle θ accurately and in real time. You have two sensors:

- **Accelerometer:** Measures gravity direction → gives you $\theta_{\text{accel}} = \arctan(a_x/a_z)$. Accurate *on average* but noisy and corrupted by any linear acceleration (when the robot moves, it "sees" phantom tilt).
- **Gyroscope:** Measures angular rate $\dot{\theta}_{\text{gyro}}$. Very clean in the short term, but *drifts* over time because you are integrating: $\theta_{\text{gyro}}(t) = \theta_0 + \int \dot{\theta}_{\text{gyro}}\, dt$. A tiny bias of 0.1°/s becomes 6° of error after one minute.

The insight: The accelerometer is good at low frequencies (static tilt) but bad at high frequen-

cies (vibration, acceleration). The gyroscope is good at high frequencies (fast rotation) but bad at low frequencies (drift). They are *complementary*.

Solution 1: Complementary filter (the quick and dirty way)

$$\theta[n] = \alpha \cdot (\theta[n-1] + \dot{\theta}_{\text{gyro}} \cdot T_s) + (1-\alpha) \cdot \theta_{\text{accel}}$$

This is literally a high-pass filter on the gyroscope and a low-pass filter on the accelerometer, with $\alpha \approx 0.98$. It works surprisingly well for a single line of code.

Solution 2: Kalman filter (the proper way)

State: $\mathbf{x} = \begin{bmatrix} \theta \\ \dot{\theta}_{\text{bias}} \end{bmatrix}$ (angle and gyro bias).

Model: the angle changes by the (bias-corrected) gyro rate:

$$\begin{bmatrix} \theta \\ \dot{\theta}_{\text{bias}} \end{bmatrix}_{k+1} = \begin{bmatrix} 1 & -T_s \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \theta \\ \dot{\theta}_{\text{bias}} \end{bmatrix}_k + \begin{bmatrix} T_s \\ 0 \end{bmatrix} \dot{\theta}_{\text{gyro},k}$$

Measurement: the accelerometer gives us a noisy observation of θ:

$$y_k = \begin{bmatrix} 1 & 0 \end{bmatrix} \mathbf{x}_k + v_k$$

What the Kalman filter does that the complementary filter cannot:

- Automatically **estimates and corrects gyro bias** ($\dot{\theta}_{\text{bias}}$). This eliminates drift without manual calibration.
- Adapts the **trust ratio dynamically** based on noise statistics, rather than using a fixed α.
- Provides a **confidence estimate** (P matrix) for the state, so you know how much to trust the output.
- Generalizes easily to 3D orientation (roll, pitch, yaw) by expanding the state vector.

When is the complementary filter "good enough"? For a simple balance bot or a gimbal where the robot does not accelerate much. The complementary filter is one line of code and requires no matrix math.

When do you need the Kalman filter? When the robot undergoes significant linear acceleration (sentry robot driving fast, drone), when you need gyro bias estimation, or when fusing more than two sensors (add magnetometer for yaw, add wheel encoder for velocity).

Competition tip: Start with the complementary filter. If you see drift or poor performance during aggressive maneuvers, upgrade to a Kalman filter. The complementary filter is also a great "sanity check"—if your Kalman filter gives a wildly different answer, something is wrong with your model.

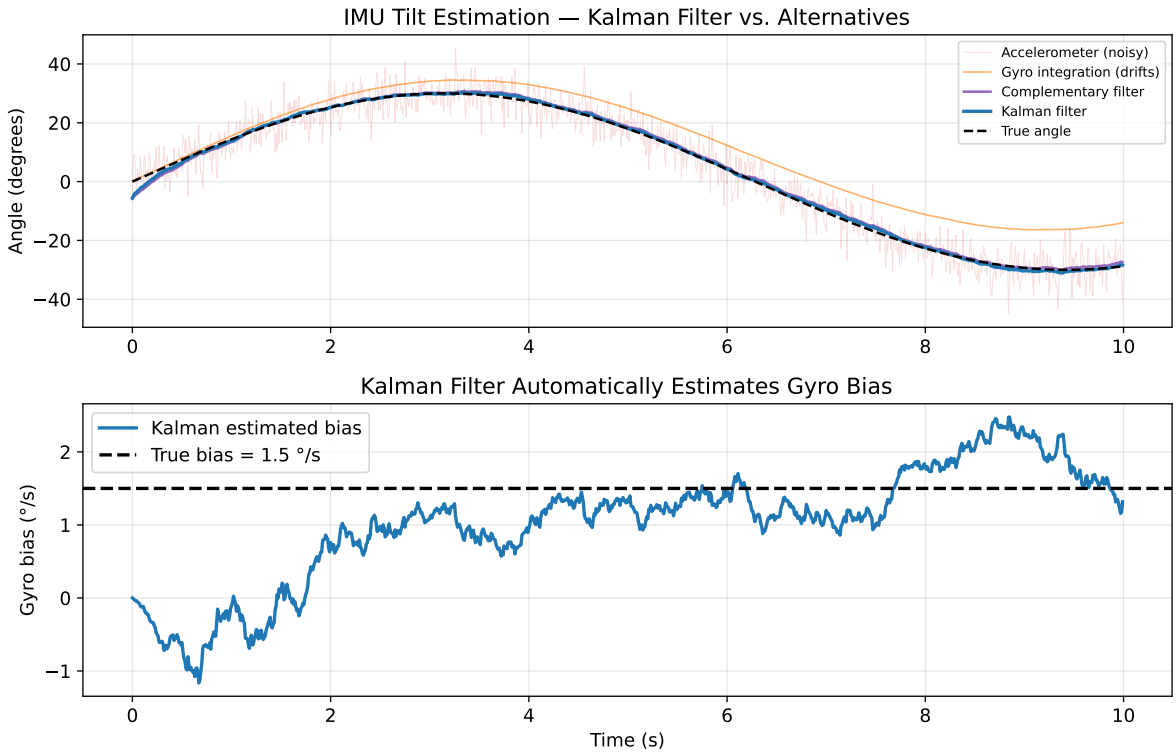

Figure 24: Top: IMU tilt estimation comparison. Pure gyro integration (orange) drifts hopelessly. The accelerometer (red, faint) is noisy. Both the complementary filter (purple) and Kalman filter (blue) track the true angle (black dashed), but Kalman is smoother. Bottom: the Kalman filter automatically estimates and converges to the true gyro bias of 1.5°/s.

Case Study: LQR Balance Bot — From Equations to Riding

Let us design the controller for a two-wheeled self-balancing robot (like a Segway). This is the "hello world" of modern control, and it beautifully showcases why LQR + Kalman is more natural than PID for MIMO systems.

The system: A two-wheeled platform with a tall body that must balance upright while also tracking a desired position.

States: $\mathbf{x} = \begin{bmatrix}\theta & \dot{\theta} & x & \dot{x}\end{bmatrix}^T$ where θ is tilt angle, x is wheel position.

The MIMO challenge: To move forward, the robot must first tilt *forward* (like a human leaning to walk). But tilting is the thing we are trying to prevent! Position control and balance are **coupled and conflicting**. A PID approach would need at least two nested loops with careful cross-coupling management. LQR handles this with a single gain matrix $K = [k_1, k_2, k_3, k_4]$:

$$u = -k_1\theta - k_2\dot{\theta} - k_3(x - x_{\text{ref}}) - k_4\dot{x}$$

Tuning via Q and R:

- Want aggressive balancing? Increase Q_{11} (penalize tilt angle).
- Want tight position tracking? Increase Q_{33} (penalize position error).
- Motors are weak? Increase R (penalize control effort).

A typical starting point: $Q = \text{diag}(100, 1, 10, 1)$, $R = 1$. This says: "I care 100× more about tilt than velocity, and 10× more about position than velocity." Run `K = lqr(A, B, Q, R)` in MATLAB, upload the four gains, and the robot balances.

The complete system:

1. **Kalman filter** estimates $[\theta, \dot{\theta}, x, \dot{x}]$ from IMU + wheel encoders.
2. **LQR controller** computes motor torque from the estimated state.
3. Both run at 500–1000 Hz on an STM32.

This Kalman+LQR architecture is the gold standard for balance bots, drone attitude control, and any system where multiple states are coupled.

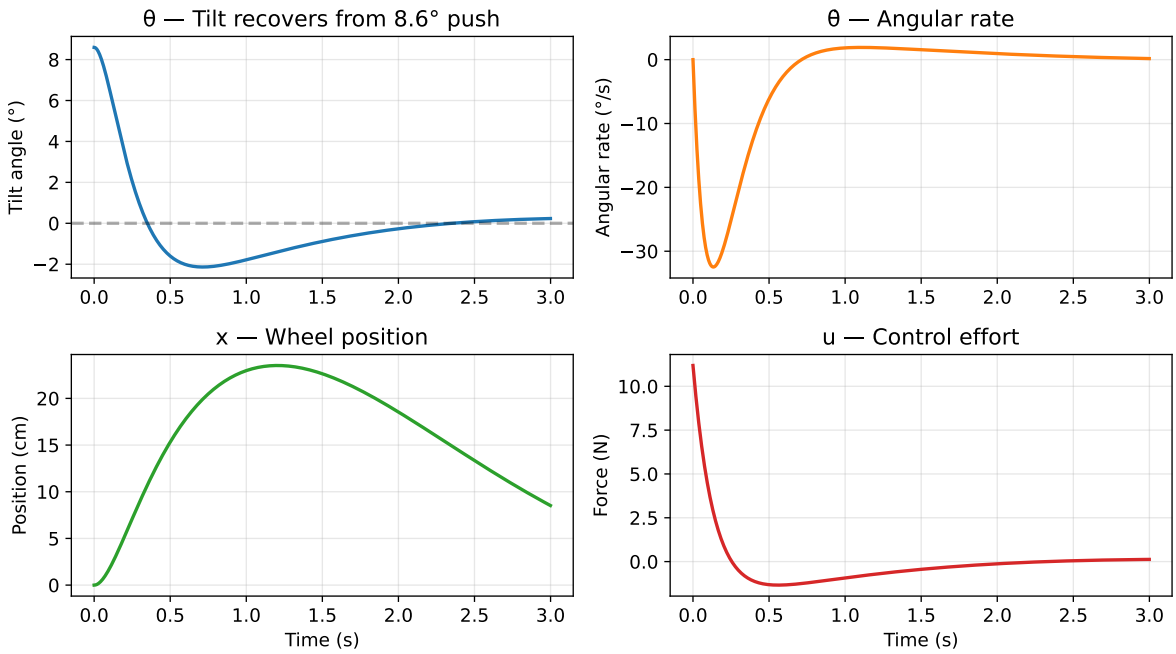

Figure 25: LQR balance bot simulation. Starting from an 8.6° tilt, the controller recovers all four states to zero. Top-left: tilt angle converges in ∼1s. Top-right: angular rate. Bottom-left: wheels move backward to catch the falling body, then return. Bottom-right: control force — note the initial burst followed by smooth decay.

10.5 Trajectory Generation: Telling Your Motor Where to Go

You cannot just command a step from 0° to 90° and expect good things to happen. The motor cannot teleport. You need a smooth **reference trajectory**—a path through position, velocity, and acceleration that respects the physical limits of your actuator. Without one, your PID saturates immediately, integral windup kicks in, and by the time the motor gets anywhere near 90° the controller is fighting itself. Trajectory generation is what separates "my turret kind of works" from "my turret tracks like a top team's."

10.5.1 Trapezoidal Velocity Profile

The trapezoidal profile is the workhorse of motion control. Given a move of distance d with maximum velocity $v_{\max}$ and maximum acceleration $a_{\max}$, it produces three phases:

1. **Accelerate** at $a_{\max}$ from rest up to $v_{\max}$
2. **Cruise** at $v_{\max}$
3. **Decelerate** at $-a_{\max}$ back to rest

Timing and distances:

$$t_{\text{accel}} = \frac{v_{\max}}{a_{\max}}, \qquad d_{\text{accel}} = \frac{1}{2} a_{\max} t_{\text{accel}}^2, \qquad d_{\text{cruise}} = d - 2\, d_{\text{accel}} \tag{145}$$

If $d_{\text{cruise}} < 0$, there is not enough distance to reach $v_{\max}$. The profile becomes **triangular**: the motor accelerates to a peak velocity $v_{\text{peak}} = \sqrt{a_{\max}\, d}$ and immediately decelerates. No cruise phase.

Reference signals at time t:

Let $t_1 = t_{\text{accel}}$, $t_2 = t_1 + d_{\text{cruise}}/v_{\max}$, $t_3 = t_2 + t_{\text{accel}}$.

$$\theta_{\text{ref}}(t) = \begin{cases} \frac{1}{2} a_{\max} t^2 & 0 \le t < t_1 \quad \text{(accelerate)} \\ d_{\text{accel}} + v_{\max}(t - t_1) & t_1 \le t < t_2 \quad \text{(cruise)} \\ d - \frac{1}{2} a_{\max}(t_3 - t)^2 & t_2 \le t \le t_3 \quad \text{(decelerate)} \end{cases} \tag{146}$$

$$\dot{\theta}_{\text{ref}}(t) = \begin{cases} a_{\max}\, t & 0 \le t < t_1 \\ v_{\max} & t_1 \le t < t_2 \\ a_{\max}(t_3 - t) & t_2 \le t \le t_3 \end{cases} \qquad \ddot{\theta}_{\text{ref}}(t) = \begin{cases} +a_{\max} & 0 \le t < t_1 \\ 0 & t_1 \le t < t_2 \\ -a_{\max} & t_2 \le t \le t_3 \end{cases} \tag{147}$$

These three signals are all you need for a fully feedforward-augmented controller (more on that in the practical box below).

10.5.2 S-Curve Profile

The trapezoidal profile has discontinuous acceleration—it jumps from 0 to $a_{\max}$ instantaneously. That sudden jerk excites mechanical resonances, stresses gears, and causes vibration. The **S-curve** (or jerk-limited) profile adds a seventh constraint: the *rate of change* of acceleration (jerk, $j_{\max}$) is bounded.

This turns the three-phase trapezoidal into a seven-phase profile:

1. Jerk up (acceleration increases from 0 to $a_{\max}$)
2. Constant acceleration at $a_{\max}$
3. Jerk down (acceleration decreases from $a_{\max}$ to 0)
4. Cruise at $v_{\max}$
5. Jerk down (acceleration goes to $-a_{\max}$)
6. Constant deceleration at $-a_{\max}$
7. Jerk up (back to zero acceleration)

The result is a position profile that is twice differentiable everywhere—smooth enough that gimbals and precision mechanisms barely feel the motion start. The math is more involved, but in practice you will rarely implement this from scratch: motion control libraries (WPILib's `TrapezoidProfile`, REV's motion magic, etc.) handle it for you.

Practical: Choosing a Profile

Trapezoidal is right for 90% of competition use cases:

- Drive wheels, flywheels, turret slew
- Simple to implement and debug—three phases, piecewise formulas

- Good enough unless you hear gear chatter or see oscillation on fast moves

S-curve when smoothness is non-negotiable:

- Gimbals and two-axis stabilization platforms
- Precision positioning mechanisms (camera sliders, encoder-based arms)
- Anywhere gear wear or vibration is a concern

The full feedforward chain for a trajectory-following turret:

At every control timestep, your trajectory planner outputs three numbers: θ_{ref}, $\dot{\theta}_{\text{ref}}$, $\ddot{\theta}_{\text{ref}}$. Use all three:

$$u = \underbrace{K_p\big(\theta_{\text{ref}} - \theta\big) + K_i \int (\cdot)\, dt + K_d \dot{e}}_{\text{outer position PID}} + \underbrace{k_v\, \dot{\theta}_{\text{ref}}}_{\text{velocity FF}} + \underbrace{k_a\, \ddot{\theta}_{\text{ref}}}_{\text{accel FF}}$$

The acceleration feedforward comes from Newton's second law for rotation: $\tau = J\alpha$, so $k_a \approx J/k_\tau$ where k_τ is the motor's torque constant. The position PID only needs to cancel small tracking errors; the feedforward does the heavy lifting.

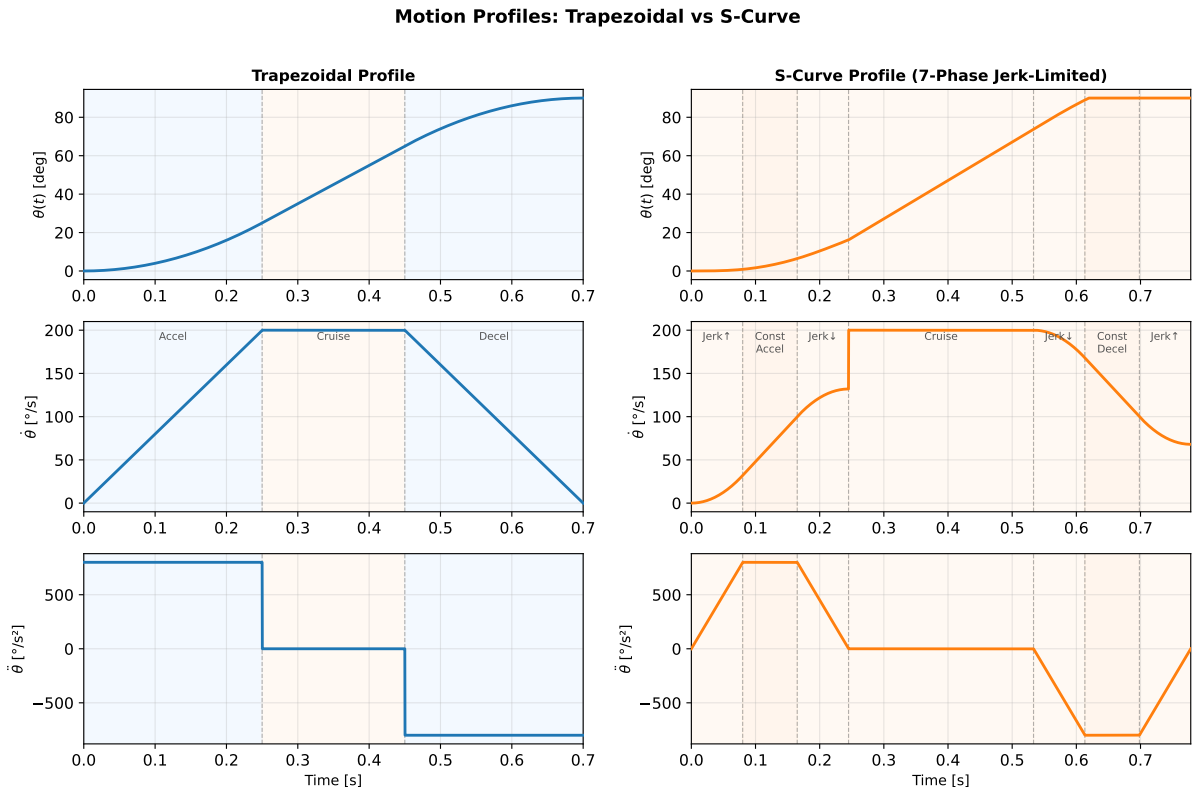

Figure 26: Trapezoidal (left) vs S-curve (right) motion profiles for a 90° move. The trapezoidal profile has discontinuous acceleration (sudden jumps), while the S-curve limits jerk for smoother motion. Both produce position, velocity, and acceleration references for feedforward control.

10.5.3 Bézier Curves and Spline Interpolation

While trapezoidal and S-curve profiles handle single point-to-point moves well, many applications require smooth trajectories through multiple waypoints or along shaped paths in 2D/3D space. **Bézier curves** and **cubic splines** are two widely used tools for this purpose.

Bézier Curves. A Bézier curve of degree n is defined by $n+1$ **control points** $\mathbf{P}_0, \mathbf{P}_1, \ldots, \mathbf{P}_n$. The curve is parameterized by $t \in [0, 1]$:

$$\mathbf{B}(t) = \sum_{i=0}^{n} \binom{n}{i} (1-t)^{n-i}\, t^i\, \mathbf{P}_i$$

The most common variant is the **cubic Bézier** ($n = 3$), which uses four control points. The curve passes through $\mathbf{P}_0$ (at $t = 0$) and $\mathbf{P}_3$ (at $t = 1$), while $\mathbf{P}_1$ and $\mathbf{P}_2$ shape the curve without lying on it. A key property is that the tangent at $\mathbf{P}_0$ points toward $\mathbf{P}_1$, and the tangent at $\mathbf{P}_3$ points from $\mathbf{P}_2$—this gives intuitive control over entry and exit directions. Bézier curves are well suited for **waypoint-based path planning** where smooth curves with controllable entry/exit directions are needed (e.g., swerve drive paths, robotic arm end-effector trajectories).

Cubic Spline Interpolation. To pass through a sequence of waypoints $\{q_0, q_1, \ldots, q_N\}$, one can use **piecewise cubic polynomials** joined at the waypoints. Each segment takes the form:

$$q_i(t) = a_i + b_i\, t + c_i\, t^2 + d_i\, t^3$$

At each junction, **continuity conditions** require that position, velocity, and acceleration match between adjacent segments, yielding C^2 continuity (continuous up to the second derivative). Two common boundary conditions are:

- **Natural spline**: the second derivative is set to zero at both endpoints.
- **Clamped spline**: the velocity (first derivative) is specified at both endpoints.

Compared to Bézier curves, cubic splines are **guaranteed to pass through all waypoints**—Bézier curves only interpolate the first and last control points. Compared to trapezoidal profiles, cubic splines provide **smooth velocity and acceleration everywhere** with no discontinuities.

When to Use What.

- **Trapezoidal / S-curve**: single point-to-point moves with explicit velocity and acceleration limits.
- **Bézier curves**: smooth 2D/3D paths with shape control (e.g., swerve drive paths, end-effector trajectories).
- **Cubic splines**: multi-waypoint trajectories where passing through every point is required (e.g., autonomous navigation sequences).

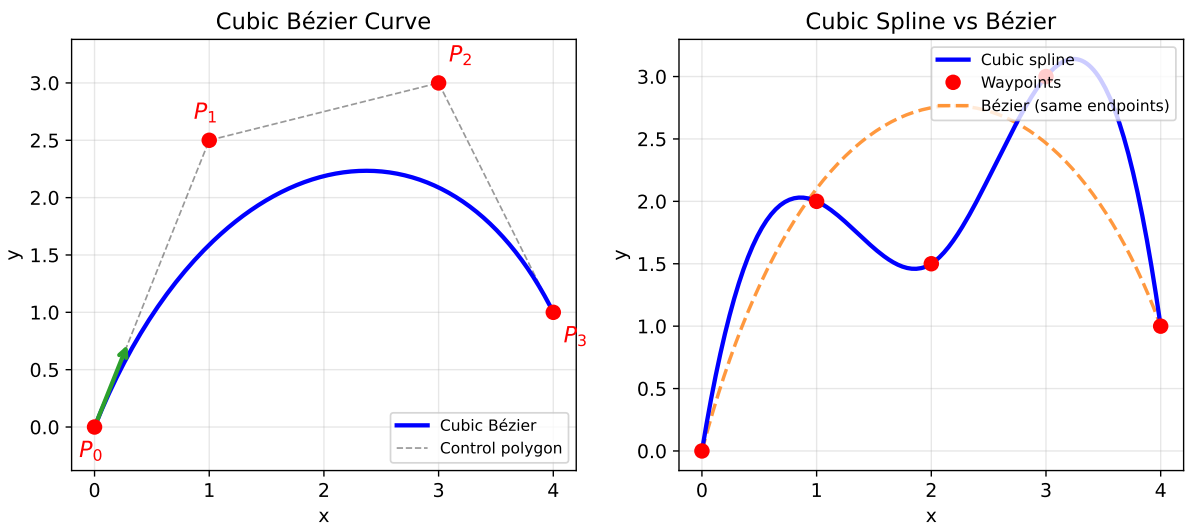

Figure 27: Left: a cubic Bézier curve defined by four control points P_0–P_3. The curve passes through P_0 and P_3; P_1 and P_2 shape the curve. Green and red arrows show the tangent directions at the endpoints. Right: cubic spline interpolation passes through all waypoints (red dots), while a Bézier curve with the same endpoints only interpolates the first and last points.

11 Nonlinear Control Basics

LQR and MPC from the previous chapter are powerful, but they rest on a linear model. What happens when your robot operates far from the equilibrium point? An inverted pendulum near the upright position is approximately linear—but during swing-up, the $\sin(\theta)$ nonlinearity dominates and linear methods fail completely. This chapter gives you the tools to handle nonlinearity directly.

We will build up four techniques—feedback linearization, sliding mode control, gain scheduling, and Lyapunov-based design—and then show how they combine in a real swing-up-to-stabilization controller. The math is heavier than PID tuning, but the payoff is enormous: these methods let you control systems that linear theory simply cannot touch.

You do not need to master all four methods. Understanding the *idea* behind each one—and knowing when to reach for which—is more valuable than memorizing every derivation. That said, we will work through the derivations where they illuminate the design, because "plug in the formula" is not enough when something goes wrong at 2 AM before a competition.

11.1 Why Linear Methods Fail

Every linear controller you have seen so far relies on the same trick: approximate a nonlinear system by its Jacobian at an operating point, then design a controller for the resulting linear model. This is *linearization*, and it works beautifully—as long as you stay close to that operating point.

Consider the inverted pendulum. Its full dynamics are:

$$ml^2\ddot{\theta} + mgl\sin(\theta) = \tau \tag{148}$$

Near the upright equilibrium ($\theta = \pi$, $\dot{\theta} = 0$), we can write $\theta = \pi + \delta$ with δ small. Then $\sin(\theta) = \sin(\pi + \delta) = -\sin(\delta) \approx -\delta$, and the linearized dynamics become $ml^2\ddot{\delta} - mgl\delta = \tau$—a linear system. LQR can stabilize this easily.

But the phase portrait tells the full story. The pendulum has two equilibria: the **hanging position** ($\theta = 0$) which is stable (a center in the conservative case, a stable node with damping), and the **upright position** ($\theta = \pi$) which is unstable (a saddle point). Between them, **separatrices** divide the phase plane into regions with qualitatively different behavior.

To visualize this, plot $\dot{\theta}$ versus θ. Near $\theta = \pi$, the trajectories form closed orbits (for the uncontrolled, undamped case) that correspond to the pendulum oscillating near the top. The separatrices are the trajectories that pass through the saddle point with exactly zero velocity—they separate swinging motions from full rotations. Linear LQR designs a controller valid inside a small neighborhood of $(\theta = \pi, \dot{\theta} = 0)$, but this neighborhood is bounded by the separatrices.

Inside the basin of attraction around $\theta = \pi$, LQR works. Outside it, the controller demands impossible torques, or simply cannot swing the pendulum up—the linearized model predicts behavior that has nothing to do with reality.

The key insight: *linear control is a special case of nonlinear control, not the other way around.* Every real system is nonlinear; linear models are convenient local approximations. When you need global behavior—swing-up, large maneuvers, operation across a wide range—you need nonlinear tools.

How far "from the equilibrium" can you go before linear control breaks down? There is no universal answer; it depends on the system. For the pendulum, linearization is reasonable within about ± 15 of the upright position. For a wheeled robot driving straight, the linear model can be valid over a large speed range because the nonlinearities (tire slip, aerodynamic drag) grow slowly. The phase portrait and simulation are your best friends for answering this question in practice.

11.2 Feedback Linearization

11.2.1 The Idea: Cancel the Nonlinearity

The first tool in our nonlinear toolkit is conceptually the simplest: if you know the nonlinearity, just cancel it.

Suppose your system has the **input-affine** form:

$$\ddot{x} = f(x, \dot{x}) + g(x)\,u \tag{149}$$

where f captures the nonlinear drift dynamics and g is the input coupling. Many mechanical systems fit this form: the nonlinear terms (gravity, Coriolis, centrifugal forces) enter through f, and the actuator

input enters multiplicatively through g. The idea is breathtakingly simple: choose the control input u to *cancel* the nonlinearity and replace it with whatever linear dynamics you want.

Set:

$$u = \frac{v - f(x, \dot{x})}{g(x)} \tag{150}$$

where v is a new "virtual input" that we are free to design. Substituting:

$$\ddot{x} = f(x, \dot{x}) + g(x) \cdot \frac{v - f(x, \dot{x})}{g(x)} = v \tag{151}$$

The nonlinearity is gone. The system is now a pure **double integrator**: $\ddot{x} = v$. You can apply any linear controller to design v—PD, LQR, pole placement, whatever you like.

This is *exact* linearization, not an approximation. We have not dropped higher-order terms or assumed small deviations. The cancellation is algebraically perfect (assuming the model is perfect). Two conditions must hold:

1. The functions $f(x, \dot{x})$ and $g(x)$ must be **known** accurately.
2. The input coupling $g(x) \neq 0$ everywhere in the operating region (otherwise we divide by zero).

Comparison with Jacobian linearization. Jacobian linearization approximates f and g by constants at one point and throws away higher-order terms; the result is only valid locally. Feedback linearization uses the *full* nonlinear f and g in the control law, so the cancellation holds globally (wherever $g \neq 0$). The price is that you must know f and g everywhere, not just at one point.

MIMO extension (input-output linearization). For multi-input systems $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) + G(\mathbf{x})\mathbf{u}$, the idea is the same but the algebra involves Lie derivatives. You differentiate each output until the input appears, stack the results into a matrix equation, and invert. The key concept is *relative degree*: the number of times you must differentiate an output before the input explicitly appears. If the total relative degree equals the system order, the system is *fully* feedback linearizable. If not, there are internal dynamics that remain nonlinear—you must verify that these "zero dynamics" are stable (this is the nonlinear analogue of minimum-phase). For competition robots, SISO feedback linearization covers most cases.

11.2.2 Example: Inverted Pendulum Swing-Up

Return to the inverted pendulum:

$$ml^2\ddot{\theta} + mgl\sin(\theta) = \tau \tag{152}$$

Rewrite as $\ddot{\theta} = -\frac{g}{l}\sin(\theta) + \frac{1}{ml^2}\tau$. Here $f(\theta) = -\frac{g}{l}\sin(\theta)$ and $g = \frac{1}{ml^2}$.

Choose the feedback linearizing control:

$$\tau = mgl\sin(\theta) + ml^2 v \tag{153}$$

The first term cancels the gravity nonlinearity; the second injects the virtual input. The result:

$$\ddot{\theta} = -\frac{g}{l}\sin(\theta) + \frac{1}{ml^2}\big(mgl\sin(\theta) + ml^2 v\big) = v \tag{154}$$

Pure double integrator. Now design v with PD control to drive $\theta \to \pi$ (upright):

$$v = -K_p(\theta - \pi) - K_d\dot{\theta} \tag{155}$$

The closed-loop becomes $\ddot{\theta} + K_d\dot{\theta} + K_p(\theta - \pi) = 0$, a stable second-order linear system with natural frequency $\omega_n = \sqrt{K_p}$ and damping ratio $\zeta = K_d/(2\sqrt{K_p})$. Choose K_p and K_d to place the poles wherever you want—the same design freedom as any linear second-order system.

Here is a minimal Python implementation:

```
import numpy as np

def feedback_linearization_controller(theta, theta_dot,
                                      m, l, g,
                                      Kp, Kd, theta_ref=np.pi):
    """Feedback linearizing controller for inverted pendulum."""
    # Cancel nonlinearity + inject virtual input
    v = -Kp * (theta - theta_ref) - Kd * theta_dot
    tau = m * g * l * np.sin(theta) + m * l**2 * v
    return tau
```

The catch: this requires knowing m, l, and g exactly. If your model says $m = 0.5$ kg but the real mass is 0.55 kg, the $\sin(\theta)$ term is not perfectly cancelled. A 10% mass error leaves a 10% residual nonlinearity that the PD outer loop must handle—and it was not designed for nonlinear disturbances.

To see this explicitly: if the true mass is $\hat{m} = m + \Delta m$, the actual closed-loop dynamics become $\ddot{\theta} = v + \frac{\Delta m}{m}\frac{g}{l}\sin(\theta)$—the residual term grows with the modeling error. For well-characterized systems (carefully measured pendulum in a lab), $\Delta m/m$ is small and the PD loop can absorb it. For a competition robot with unknown payload? Dangerous.

11.2.3 Limitations

Feedback linearization is elegant, but it has real weaknesses:

- **Model sensitivity.** Imperfect cancellation leaves residual nonlinear dynamics that the linear outer loop was not designed to handle.
- **Singularities.** When $g(x) \to 0$, the required control effort $u \to \infty$. For the pendulum, g is constant, so this is not an issue. But for robotic arms near kinematic singularities, it is a showstopper.
- **No inherent robustness.** Unmodeled dynamics (friction, flexibility, backlash) reappear as disturbances that the controller does not anticipate.
- **Full state measurement.** The controller needs to evaluate $f(x, \dot{x})$, which means you need to measure (or accurately estimate) the full state. If your state estimator is noisy, the noise feeds directly into the cancellation term and can excite high-frequency dynamics.

Feedback linearization is best for systems where you have a good model and want clean, predictable behavior. For unknown or poorly characterized systems, we need something fundamentally more robust.

11.3 Sliding Mode Control

11.3.1 The Idea: Robustness Through Discontinuous Switching

Sliding mode control (SMC) takes a fundamentally different philosophy: instead of cancelling the nonlinearity, *overpower it.* Do not try to model every detail of the disturbance—just know an upper bound on how bad it can be, and hit it harder than that. Design a controller that forces the system onto a chosen surface in state space, then keeps it there regardless of disturbances or model errors.

The setup: define a **sliding surface** in state space—a manifold where the system has desirable dynamics. Then design a controller that (1) drives the state onto the surface, and (2) keeps it there. Disturbances and model errors cannot kick the state off the surface because the controller continuously corrects.

For a second-order system tracking a reference $x_d(t)$, with error $e = x - x_d$, a natural choice of sliding surface is:

$$s = \dot{e} + \lambda e \tag{156}$$

where $\lambda > 0$ is a design parameter. This is a line in the $(e, \dot{e})$ phase plane.

Why this surface? On the surface ($s = 0$), the error dynamics become:

$$\dot{e} + \lambda e = 0 \quad \Longrightarrow \quad e(t) = e(0)\, e^{-\lambda t} \tag{157}$$

The error decays exponentially to zero with time constant $1/\lambda$. These dynamics are **independent** of the plant model, disturbances, or parameter variations. Once you are on the surface, convergence is guaranteed by a simple first-order ODE.

The control design has two parts:

1. **Reaching phase:** Drive the state to the sliding surface in finite time.
2. **Sliding phase:** Keep the state on the surface; the reduced-order dynamics handle the rest.

For the reaching phase, we need the **reaching condition**:

$$s\,\dot{s} < -\eta\,|s|, \quad \eta > 0 \tag{158}$$

This guarantees that s decreases in magnitude regardless of its sign, reaching $s = 0$ in finite time $t_r \leq |s(0)|/\eta$.

11.3.2 Example: Velocity Control with Unknown Friction

Consider a motor driving an inertial load:

$$J\,\dot{\omega} = \tau - d(t) \tag{159}$$

where J is the moment of inertia, ω is the angular velocity, τ is the control torque, and $d(t)$ is an unknown time-varying friction/disturbance term bounded by $|d(t)| \le D_{\max}$.

We want to track a reference velocity ω_{ref}. Define the sliding variable:

$$s = \omega - \omega_{\text{ref}} \tag{160}$$

Differentiate: $\dot{s} = \dot{\omega} - \dot{\omega}_{\text{ref}} = \frac{1}{J}(\tau - d(t)) - \dot{\omega}_{\text{ref}}$.

Choose the control law:

$$\tau = J\,\dot{\omega}_{\text{ref}} + K\,\text{sign}(s), \quad K > D_{\max} \tag{161}$$

Substituting:

$$\dot{s} = \frac{1}{J}\big(J\,\dot{\omega}_{\text{ref}} + K\,\text{sign}(s) - d(t)\big) - \dot{\omega}_{\text{ref}} = \frac{K\,\text{sign}(s) - d(t)}{J} \tag{162}$$

Check the reaching condition:

$$s\,\dot{s} = \frac{s\big(K\,\text{sign}(s) - d(t)\big)}{J} = \frac{K|s| - s\,d(t)}{J} \ge \frac{(K - D_{\max})|s|}{J} > 0 \tag{163}$$

Wait—we need $s\,\dot{s} < 0$ for convergence. Let us fix the sign: $\tau = J\,\dot{\omega}_{\text{ref}} - K\,\text{sign}(s)$. Then:

$$s\,\dot{s} = \frac{-K|s| - s\,d(t)}{J} \le \frac{-(K - D_{\max})|s|}{J} < 0 \tag{164}$$

Since $K > D_{\max}$, the reaching condition holds. The state converges to $s = 0$ in finite time, and once there, $\omega = \omega_{\text{ref}}$ exactly—regardless of the unknown friction $d(t)$. This is the power of sliding mode: robustness is *structural*, not dependent on accurate modeling.

Phase plane interpretation. In the $(s, \dot{s})$ phase plane, the sliding surface is the origin line $s = 0$. The reaching condition forces all trajectories to point toward this line, regardless of the disturbance. Once a trajectory hits $s = 0$, it stays there forever—sliding along the surface. The trajectories "above" the surface ($s > 0$) are pushed down; those "below" ($s < 0$) are pushed up. The result is a funnel that captures all trajectories in finite time.

Design parameter λ. For higher-order systems (e.g., position tracking where both position error e and velocity error $\dot{e}$ matter), the sliding surface $s = \dot{e} + \lambda e$ introduces one design choice: λ. A large λ means fast convergence on the surface but requires the controller to be aggressive during the reaching phase. A small λ gives gentle convergence but slow error decay. A good starting point: set λ so that $1/\lambda$ equals the desired settling time on the surface.

11.3.3 The Chattering Problem

There is a price for this robustness. The $\text{sign}(s)$ function switches infinitely fast as s crosses zero. In theory, this is fine. In practice, it produces **chattering**—high-frequency oscillation in the control signal. This is not just annoying; it destroys actuators. Motor coils overheat, gears wear out, and mechanical structures vibrate.

Fix 1: Boundary layer. Replace the discontinuous $\text{sign}(s)$ with a continuous approximation:

$$\text{sat}\Big(\frac{s}{\Phi}\Big) = \begin{cases} s/\Phi & \text{if } |s| \le \Phi \\ \text{sign}(s) & \text{if } |s| > \Phi \end{cases} \tag{165}$$

where $\Phi > 0$ is the boundary layer width. Inside the layer $|s| < \Phi$, the control is proportional (smooth); outside, it is the original bang-bang. The tradeoff: larger Φ gives smoother control but allows a steady-state error of order Φ. Smaller Φ gives better tracking but more chattering. In practice, you tune Φ to match the actuator bandwidth—there is no point switching faster than the motor can respond.

Fix 2: Super-twisting algorithm. A second-order sliding mode controller that achieves $s = 0$ with a *continuous* control signal:

$$u = -k_1|s|^{1/2}\,\text{sign}(s) + w, \qquad \dot{w} = -k_2\,\text{sign}(s) \tag{166}$$

The integral term w builds up the control effort smoothly, eliminating the discontinuity from the applied torque while retaining finite-time convergence to the sliding surface. Sufficient conditions for convergence are $k_1 > 0$ and k_2 sufficiently large relative to the disturbance bound (the exact inequalities depend on the system; see Levant 1993 for the original proof). The super-twisting algorithm is the go-to choice when actuator health matters—which is always, in competition robotics.

Practical recommendation. For most competition applications, start with the boundary layer approach (it is trivial to implement—just replace `sign(s)` with `s/max(abs(s), phi)` in code). If you find that tracking precision is insufficient, try the super-twisting algorithm. The extra integrator state is easy to add, and the improvement in control smoothness is dramatic.

11.4 Gain Scheduling

Sometimes the theoretically elegant approach is not the practical one. If your system is nonlinear but changes "slowly" with the operating point, you do not necessarily need the full machinery of feedback linearization or sliding mode. Gain scheduling is the engineer's pragmatic workaround: design multiple linear controllers, each tuned for a different operating point, and switch or interpolate between them based on the current state. It is the most widely used nonlinear control technique in industry—and often the least recognized as such.

Example: motor speed controller. A DC motor behaves differently at low speed versus high speed:

- **Low speed:** Stiction (static friction) dominates. You need high gains to break free and maintain precise tracking.
- **High speed:** Back-EMF provides natural damping. High gains cause overshoot and oscillation. Low gains suffice.

The simplest implementation is a lookup table:

```
// Gain scheduling lookup table (C)
typedef struct { float speed; float Kp; float Ki; } GainEntry;

static const GainEntry table[] = {
    {   0.0f, 8.0f, 4.0f },   // low speed: high gains
    { 500.0f, 5.0f, 2.5f },   // mid speed
    {1000.0f, 2.0f, 1.0f },   // high speed: low gains
};

// Interpolate Kp, Ki based on current speed
void get_gains(float speed, float *Kp, float *Ki) {
    int i = 0;
    while (i < 2 && speed > table[i+1].speed) i++;
    float alpha = (speed - table[i].speed)
                / (table[i+1].speed - table[i].speed);
    if (alpha < 0.0f) alpha = 0.0f;
    if (alpha > 1.0f) alpha = 1.0f;
    *Kp = table[i].Kp + alpha * (table[i+1].Kp - table[i].Kp);
    *Ki = table[i].Ki + alpha * (table[i+1].Ki - table[i].Ki);
}
```

Advantages:

- Simple to implement, easy to tune experimentally.
- No new math—each operating point uses familiar PID or LQR.
- Works well when the nonlinearity is "slow" (parameters change gradually).
- Easy to verify: test each operating point independently.

Disadvantages:

- No formal stability guarantee during transitions between operating points. If the system moves faster than the gain schedule anticipates, transient instability is possible.
- Requires extensive testing at many operating points. Miss one region and you have an untuned controller lurking in your state space.
- Does not handle fast, coupled nonlinearities well.

- The interpolation between gain sets can produce unexpected behavior if the gain surfaces are not smooth (e.g., a convex combination of two stable gain sets is not guaranteed to be stable).

Gain scheduling is the right tool when your system is "mildly nonlinear"—the dynamics change with operating point, but smoothly and predictably. For violent nonlinearities (like swing-up), use the other methods in this chapter.

A practical note on implementation. In competition robots, gain scheduling often appears informally: teams tune PID gains at low speed, find they oscillate at high speed, reduce the gains for high speed, and glue the two together with an `if` statement. This *is* gain scheduling—just without the fancy name. Making it systematic (a proper lookup table with interpolation, tested at intermediate points) costs little effort and prevents the "weird behavior at 60% speed" problems that plague ad-hoc approaches.

11.5 Lyapunov-Based Design

11.5.1 Energy Shaping for Swing-Up

The most beautiful approach to nonlinear control is to think in terms of **energy**. Instead of cancelling nonlinearities or overpowering disturbances, reshape the system's energy landscape so that the desired state is the natural energy minimum. This is Lyapunov-based design: find a scalar "energy-like" function, and design the controller to make it always decrease. If you can do that, stability follows automatically—no linearization, no switching, no gain tables.

For the inverted pendulum, this approach is particularly natural because the system already has a well-defined energy function. The inverted pendulum has a total energy (kinetic plus potential):

$$E = \frac{1}{2}ml^2\dot{\theta}^2 - mgl\cos(\theta) \tag{167}$$

At the hanging position ($\theta = 0$, $\dot{\theta} = 0$): $E_{\text{hang}} = -mgl$.

At the upright position ($\theta = \pi$, $\dot{\theta} = 0$): $E_{\text{up}} = mgl$.

The goal is to pump energy into the system until $E = E_{\text{up}} = mgl$, then hold it there. Define the energy error:

$$\tilde{E} = E - E_{\text{up}} = \frac{1}{2}ml^2\dot{\theta}^2 - mgl\cos(\theta) - mgl \tag{168}$$

Lyapunov function: Choose $V = \frac{1}{2}\tilde{E}^2 \geq 0$. This is zero only when $E = E_{\text{up}}$, which is exactly what we want.

Differentiate:

$$\dot{V} = \tilde{E}\,\dot{E} \tag{169}$$

The time derivative of energy for the pendulum with torque input τ is:

$$\dot{E} = ml^2\dot{\theta}\ddot{\theta} + mgl\sin(\theta)\dot{\theta} = \dot{\theta}\big(ml^2\ddot{\theta} + mgl\sin(\theta)\big) = \dot{\theta}\,\tau \tag{170}$$

So $\dot{V} = \tilde{E}\,\dot{\theta}\,\tau$. We need $\dot{V} \leq 0$ (Lyapunov stability requires the "energy of the energy error" to decrease). Choose:

$$\tau = k\,\tilde{E}\,\dot{\theta}\cos(\theta), \quad k > 0 \tag{171}$$

Then:

$$\dot{V} = \tilde{E}\,\dot{\theta}\,k\,\tilde{E}\,\dot{\theta}\cos(\theta) = k\,\tilde{E}^2\,\dot{\theta}^2\cos(\theta) \tag{172}$$

Hmm—this is not always negative. The $\cos(\theta)$ factor changes sign. The standard fix is to use the unsigned version of energy pumping:

$$\tau = k\,\tilde{E}\,\dot{\theta} \tag{173}$$

Then $\dot{V} = k\,\tilde{E}^2\,\dot{\theta}^2 \geq 0$—wait, that is non-negative, not non-positive. We need to be more careful. Let us revisit: we want $\tilde{E} \to 0$, so define:

$$\tau = -k\,\tilde{E}\,\dot{\theta}, \quad k > 0 \tag{174}$$

Now $\dot{V} = \tilde{E}\,\dot{\theta}\,(-k\,\tilde{E}\,\dot{\theta}) = -k\,\tilde{E}^2\,\dot{\theta}^2 \leq 0$.

This is *negative semidefinite*—it equals zero when $\dot{\theta} = 0$ or $\tilde{E} = 0$. By LaSalle's invariance principle, the system converges to the largest invariant set where $\dot{V} = 0$. The set $\{\dot{\theta} = 0\}$ includes the upright equilibrium ($\theta = \pi$) and the hanging equilibrium ($\theta = 0$), but the hanging equilibrium has $\tilde{E} \neq 0$, so the

controller will continue to inject energy there—it is not an invariant set of the closed-loop system. The only invariant set where both $\dot{V} = 0$ and the dynamics are consistent is $\tilde{E} = 0$.

In practice, the controller swings the pendulum with increasing amplitude until $E \approx E_{\text{up}}$, at which point the pendulum is near the upright position—ready for a linear controller to take over. The gain k controls how aggressively energy is pumped: larger k means faster swing-up but higher peak torques. With torque limits, choose k small enough that the controller does not saturate, or the Lyapunov argument breaks down.

Key insight: The energy-shaping controller does not try to follow a specific trajectory. It just ensures the energy converges to the right level. The pendulum finds its own way to the top, swinging back and forth with gradually increasing amplitude. This is robust and elegant.

Two-phase control. Energy shaping alone cannot stabilize the upright equilibrium—when $\tilde{E} = 0$ and $\dot{\theta} = 0$ at $\theta = \pi$, the control output is zero, but the equilibrium is unstable. Any perturbation sends the pendulum away. The solution: use energy shaping to get *near* the upright position, then switch to a linear controller (LQR) that is designed for the linearized upright dynamics. This two-phase strategy is how real pendulum swing-up is done, and it generalizes to many other problems.

Why Lyapunov? The Lyapunov approach is more general than the pendulum example suggests. The recipe is: (1) identify a positive-definite function $V(\mathbf{x})$ that is zero at the desired equilibrium; (2) choose the control input to make $\dot{V} \leq 0$; (3) invoke LaSalle's invariance principle to conclude convergence. This works for any system where you can find a suitable V—the hard part is finding one. For mechanical systems, physical energy is a natural starting point. For other systems, constructing Lyapunov functions is an art (and sometimes aided by sum-of-squares programming or other computational tools).

11.6 When to Use What: A Decision Guide

With multiple nonlinear control tools available, choosing the right one matters. Here is a practical guide:

Method	Model Required?	Robustness	Complexity	Best For
PID	No	Low	Low	Simple SISO, near linear
LQR	Yes (linear)	Medium	Medium	MIMO, near equilibrium
Feedback lin.	Yes (nonlinear)	Low	Medium	Known nonlinear systems
Sliding mode	Partial (bounds)	High	Medium	Unknown disturbances
Gain scheduling	Yes (multi-linear)	Medium	Low	Wide operating range
MPC	Yes (any)	Medium	High	Constraints, trajectories

Table 1: Decision guide for control method selection. "Partial" means you need bounds on uncertainty, not a full model.

A few rules of thumb:

- If you have a good model and want clean math: **feedback linearization**.
- If you need robustness and can tolerate some implementation complexity: **sliding mode** (with boundary layer or super-twisting).
- If the system is "mostly linear" but operates over a wide range: **gain scheduling**.
- If you need to swing up or transition between equilibria: **Lyapunov / energy shaping**.
- If you have hard constraints (torque limits, angle limits): **MPC**.
- If in doubt, start with PID and add complexity only when it fails.

In competition settings, the most common pattern is gain-scheduled PID for "routine" control (drivetrain, shooter wheels) and a combination of energy shaping plus LQR for the "spectacular" moves (swing-up, balancing, aggressive maneuvers). Sliding mode appears less frequently in student robotics but is invaluable when you face unpredictable loads or environmental disturbances—for example, a mechanism that must apply consistent force despite varying contact conditions.

Case Study: Inverted Pendulum—From Swing-Up to Stabilization

This case study ties together the chapter by showing how nonlinear and linear control work together on a single system.

Phase 1: Energy shaping (Lyapunov-based swing-up).

The pendulum starts at $\theta = 0$ (hanging down), at rest. The energy at this configuration is

$E = -mgl$, which is $2mgl$ below the target $E_{\text{up}} = mgl$. The energy-shaping controller $\tau = -k\,\tilde{E}\,\dot{\theta}$ pumps energy into the system. On each swing, the pendulum goes a little higher. The controller does not prescribe a trajectory—it just ensures the energy approaches $E_{\text{up}} = mgl$. After several swings (typically 3–5, depending on k and torque limits), the pendulum reaches the neighborhood of the upright position.

Phase 2: LQR stabilization (from Chapter 6).

When the angle error $|\theta - \pi| < 15$ (about 0.26 rad), switch to the LQR controller designed for the linearized upright equilibrium. The LQR gains were computed offline assuming the linearized model $\ddot{\delta} = \frac{g}{l}\delta + \frac{1}{ml^2}\tau$, and they stabilize the pendulum precisely at the upright position.

Switching logic:

```
if abs(theta - pi) < 0.26 and abs(theta_dot) < 2.0:
    tau = -K_lqr @ np.array([theta - np.pi, theta_dot])
else:
    E = 0.5 * m * l**2 * theta_dot**2 - m * g * l * np.cos(theta)
    E_tilde = E - m * g * l
    tau = -k_swing * E_tilde * theta_dot
```

The velocity condition $|\dot{\theta}| < 2.0$ rad/s prevents switching when the pendulum is passing through the upright position at high speed (where LQR would immediately lose control).

The full trajectory:

1. $t = 0$: Pendulum hangs at $\theta = 0$, $\dot{\theta} = 0$.
2. $t \approx 0$–2 s: Energy shaping swings the pendulum with increasing amplitude.
3. $t \approx 2$–3 s: Pendulum reaches $|\theta - \pi| < 15$ with low velocity; controller switches to LQR.
4. $t > 3$ s: LQR stabilizes the pendulum at the upright position. Small oscillations decay exponentially.

The lesson: This two-phase approach generalizes far beyond pendulums. Many robots use a "rough" nonlinear controller to reach the neighborhood of the desired operating point, then switch to a precise linear controller for the final stabilization. Quadrotors recovering from large attitude errors, legged robots transitioning between gaits, robotic arms reaching into tight workspaces—the pattern is the same: nonlinear control for the big moves, linear control for the fine adjustments.

Implementation tip: Add hysteresis to the switching condition. If you switch to LQR at $|\theta - \pi| < 15$ and back to energy shaping at $|\theta - \pi| > 15$, noise near the boundary causes rapid switching ("Zeno behavior"). Instead, switch to LQR at 15 but only switch back to energy shaping at 20. This small dead band eliminates the chatter.

Looking ahead: rotation dynamics are inherently nonlinear—composing rotations is *not* a linear operation, and the Euler angle singularity at ± 90 is a nonlinear phenomenon. The tools in this chapter explain *why* quaternions and rotation matrices behave the way they do, and why attitude control requires the nonlinear thinking we have developed here. Feedback linearization shows up in computed-torque control of robotic arms; sliding mode appears in robust attitude controllers; Lyapunov methods underpin almost every stability proof in rotational dynamics. The next chapter builds that foundation.

Summary of key ideas:

- Linear control is local; nonlinear control can be global.
- Feedback linearization: cancel what you know. Elegant but fragile.
- Sliding mode: overpower what you do not know. Robust but chattery (fixable).
- Gain scheduling: the practical engineer's interpolation. Simple but unproven at transitions.
- Lyapunov / energy shaping: think in energy, not trajectories. Beautiful and powerful.
- In practice, combine them: nonlinear for the big moves, linear for the fine adjustments.

12 Attitude and Rotation Representation

Before we can estimate or control orientation (Ch6 uses Euler angles and quaternions extensively), we need a solid foundation in how rotations work mathematically. This chapter builds that foundation from scratch.

Why does this matter for robotics? Every time you read an IMU, command a gimbal, or fuse sensor data in an EKF, you are working with rotations. Get the math wrong—mix up a convention, compose rotations in the wrong order, or hit a singularity—and your robot will do something unexpected. The good news: the core ideas are not hard. They just need to be understood once, carefully.

12.1 Coordinate Frames

All of 3D robotics begins with a choice: *which way do the axes point?*

12.1.1 The Right-Hand Rule

Hold out your right hand. Point your thumb along the x-axis, your index finger along y, and curl your middle finger—it points along z. This is a **right-handed coordinate frame**, and it is the universal standard in robotics and aerospace.

Several conventions exist that all use the right-hand rule but differ in axis assignment. In this book, we adopt:

NWU (North-West-Up): x points north (forward), y points west (left), z points up. This is the convention used throughout our competition robots—forward is $+x$, left is $+y$, up is $+z$. It is a right-handed frame and matches the body-frame convention where x is "forward."

Other common conventions you will encounter:

- **NED (North-East-Down):** x north, y east, z down. Standard in aerospace and many IMU datasheets. Positive z is "down," so altitude is negative z.
- **ENU (East-North-Up):** x east, y north, z up. Common in ROS and geographic mapping.

All three are perfectly valid right-handed frames. The critical thing is to pick one and be consistent. If your IMU outputs NED data and your control algorithm expects NWU, you need to convert ($y_{\text{NWU}} = -y_{\text{NED}}$, $z_{\text{NWU}} = -z_{\text{NED}}$). This is one of the most common sources of bugs in robotics software.

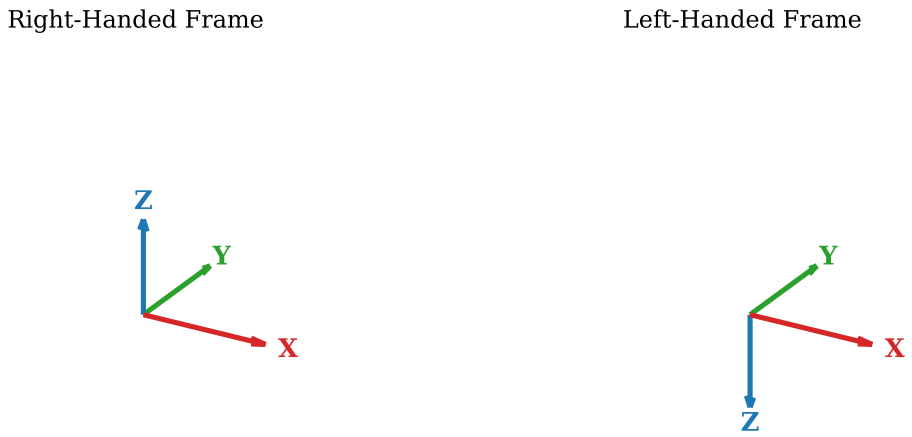

Figure 28: Right-handed vs left-handed coordinate frames. In a right-handed frame, the cross product $\hat{x} \times \hat{y} = \hat{z}$; in a left-handed frame, $\hat{x} \times \hat{y} = -\hat{z}$.

12.1.2 Right-Hand vs Left-Hand Frames

A **left-handed** frame is what you get if you use your left hand instead. Some graphics engines (notably DirectX and Unity's default) use left-handed frames because it makes the z-axis point "into the

screen," which is convenient for rendering. Most robotics software uses right-handed frames. If you mix conventions—say, importing a mesh from a left-handed graphics tool into a right-handed robotics simulator—rotations will appear mirrored. Always check the convention.

12.1.3 Common Frames in Robotics

Two frames appear in virtually every robotics problem:

- **World (inertial) frame:** Fixed to the ground (or to the Earth, if you are working at scales where Earth's rotation matters). This is the "global" reference. We often denote it with a superscript W or subscript world.
- **Body frame:** Fixed to the robot. It moves and rotates with the robot. In our NWU convention, x points forward, y points left, and z points up. Denoted with B or body.

The central question of attitude representation is: *how do we describe the orientation of the body frame relative to the world frame?*

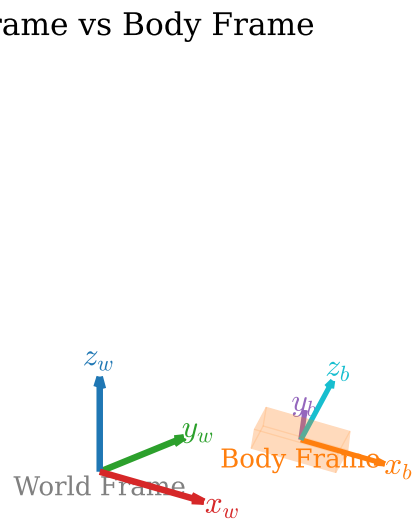

Figure 29: World frame (fixed) and body frame (moves with the robot). The rotation matrix R describes how the body frame is oriented relative to the world frame.

12.2 Rotation Matrices

12.2.1 Definition and Properties

A rotation matrix R is a 3×3 matrix that belongs to the **special orthogonal group** SO(3):

$$\mathrm{SO}(3) = \{R \in \mathbb{R}^{3\times 3} \mid R^T R = I,\ \det(R) = +1\} \tag{175}$$

The condition $R^T R = I$ (orthogonality) means the columns of R are mutually perpendicular unit vectors. The condition $\det(R) = +1$ distinguishes proper rotations from reflections (which have det $= -1$).

Physical meaning: Each column of R tells you where the corresponding original basis vector ends up after the rotation. If $R = [\mathbf{r}_1\ \mathbf{r}_2\ \mathbf{r}_3]$, then $\mathbf{r}_1$ is the image of $\hat{x}$, $\mathbf{r}_2$ is the image of $\hat{y}$, and $\mathbf{r}_3$ is the image of $\hat{z}$. Equivalently, the *rows* of R are the body-frame axes expressed in world coordinates.

The three elementary rotation matrices (counterclockwise when looking from the positive axis toward the origin) are:

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta \\ 0 & \sin\theta & \cos\theta \end{bmatrix} \tag{176}$$

$$R_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{bmatrix} \tag{177}$$

$$R_z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{178}$$

Note the "odd one out": R_y has the $-\sin\theta$ in the top-right and $+\sin\theta$ in the bottom-left, which is the transpose of what you might expect. This is because the y-axis rotation cycles $z \to x$ (not $x \to z$), and consistency with the right-hand rule requires the sign flip.

12.2.2 What Rotation Matrices Actually Do

There are two ways to interpret the same rotation matrix, and confusing them is a guaranteed source of bugs.

Interpretation 1: Active (alibi) rotation. The matrix physically rotates a vector to a new position in the *same* coordinate frame. Think of grabbing a vector and spinning it.

Interpretation 2: Passive (alias) rotation. The matrix re-expresses the same physical vector in a *different* coordinate frame. The vector does not move; we just describe it from a new viewpoint.

Mathematically, the active rotation of $\mathbf{v}$ by angle θ about the z-axis and the passive transformation from frame A to frame B (where B is rotated by $-\theta$ relative to A) use the same matrix. The difference is purely in interpretation.

For robotics, the most important convention is:

$$\mathbf{v}_{\text{world}} = R\,\mathbf{v}_{\text{body}} \quad \Longleftrightarrow \quad \mathbf{v}_{\text{body}} = R^T\mathbf{v}_{\text{world}} \tag{179}$$

Here R is the rotation matrix that transforms vectors from the body frame to the world frame (sometimes written R_B^W or $R_{\text{body}\to\text{world}}$). To go the other way, use $R^T = R^{-1}$—the beauty of orthogonal matrices is that the inverse is just the transpose.

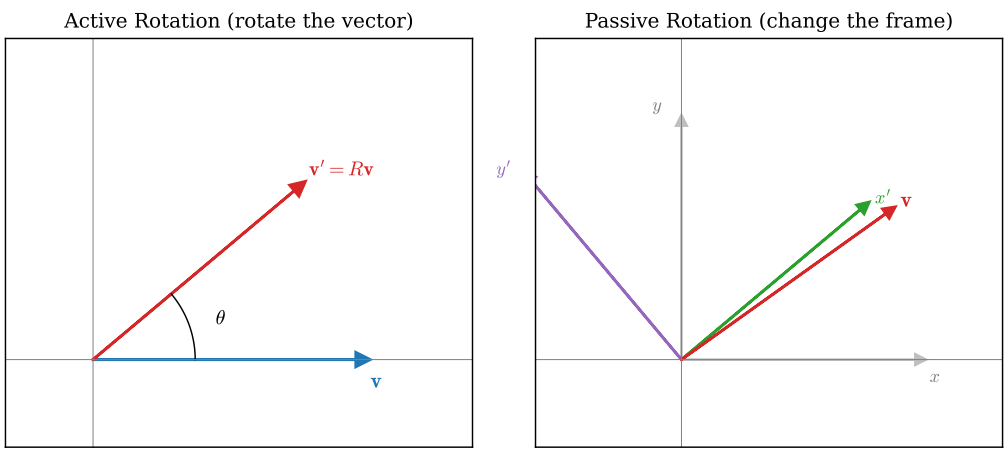

Figure 30: Two interpretations of the same rotation matrix. Active: the vector moves. Passive: the coordinate frame moves. Both use the same R, but the conceptual picture is different.

The #1 source of confusion: Different textbooks and libraries use opposite conventions. Some define R as world-to-body; others as body-to-world. When you import a rotation matrix from a library, *always check which convention it uses.* If you get it backwards, every transformed vector will be wrong, and the error will look like your robot is mirrored or spinning the wrong way.

12.2.3 Composing Rotations: Left vs Right Multiplication

Suppose you want to apply rotation R_1 first, then rotation R_2. The combined rotation is:

$$R_{\text{total}} = R_2 \cdot R_1 \tag{180}$$

This is **right-to-left** order: the rotation applied first is on the right. This follows from the associativity of matrix multiplication: $R_2(R_1\mathbf{v}) = (R_2R_1)\mathbf{v}$.

Now, here is where things get subtle. There are two ways to build up a sequence of rotations:

Extrinsic rotations (world-fixed axes): Each successive rotation is about an axis of the fixed world frame. The total rotation is built by *pre-multiplying* (multiplying on the left):

$$R_{\text{total}} = R_3 \cdot R_2 \cdot R_1$$

Read left-to-right: R_3 is the last rotation applied, but it acts about a world-fixed axis.

Intrinsic rotations (body-fixed axes): Each successive rotation is about an axis of the current (already-rotated) body frame. The total rotation is built by *post-multiplying* (multiplying on the right):

$$R_{\text{total}} = R_1 \cdot R_2 \cdot R_3$$

Here R_1 rotates about the original body axis, R_2 about the once-rotated body axis, and so on.

A remarkable fact: an intrinsic ZYX rotation (yaw about body z, then pitch about the new body y, then roll about the newest body x) produces the *same* matrix as an extrinsic XYZ rotation (roll about world x, then pitch about world y, then yaw about world z). The order reverses. This is purely a consequence of the associativity of matrix multiplication.

For the standard aerospace convention (intrinsic ZYX: yaw ψ, pitch θ, roll ϕ):

$$R = R_z(\psi) \cdot R_y(\theta) \cdot R_x(\phi) \tag{181}$$

Common pitfall: Getting the multiplication order wrong compiles fine and runs fine—it just gives a subtly wrong orientation. There is no runtime error to alert you. Always write down the convention explicitly in your code comments.

12.3 Euler Angles

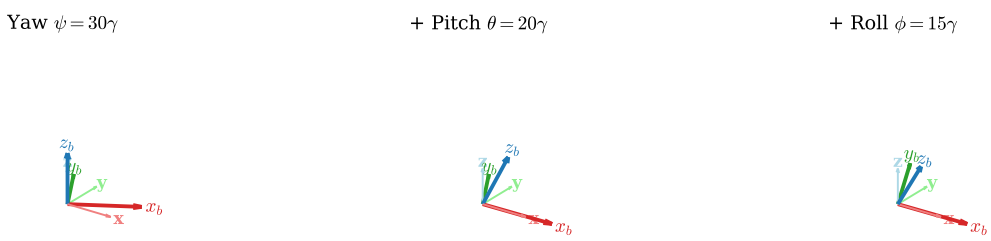

Figure 31: Building up a ZYX rotation step by step. Left: yaw only ($\psi = 30$). Center: yaw + pitch ($\theta = 20$). Right: yaw + pitch + roll ($\phi = 15$). Faint axes show the world frame; bold axes show the body frame.

12.3.1 Definition and Conventions

Euler angles describe an arbitrary rotation as a sequence of three elementary rotations about coordinate axes. There are 12 possible sequences (6 "proper" Euler angles like ZXZ, ZYZ, etc., and 6 Tait–Bryan angles like ZYX, XYZ, etc.). The most common in robotics and aerospace is the **ZYX** (yaw-pitch-roll) convention:

1. Yaw (ψ): rotate about the z-axis.
2. Pitch (θ): rotate about the (new) y-axis.
3. Roll (ϕ): rotate about the (newest) x-axis.

The resulting rotation matrix is (intrinsic ZYX):

$$R = R_z(\psi)\, R_y(\theta)\, R_x(\phi) \tag{182}$$

Expanding this product fully using the elementary matrices:

$$R = \begin{bmatrix} c_\psi c_\theta & c_\psi s_\theta s_\phi - s_\psi c_\phi & c_\psi s_\theta c_\phi + s_\psi s_\phi \\ s_\psi c_\theta & s_\psi s_\theta s_\phi + c_\psi c_\phi & s_\psi s_\theta c_\phi - c_\psi s_\phi \\ -s_\theta & c_\theta s_\phi & c_\theta c_\phi \end{bmatrix} \tag{183}$$

where $c_\psi = \cos\psi$, $s_\psi = \sin\psi$, etc.

Extracting Euler angles from a rotation matrix. Given a rotation matrix $R = [r_{ij}]$, the ZYX Euler angles can be recovered as:

$$\theta = -\arcsin(r_{31}) \tag{184}$$

$$\phi = \operatorname{atan2}(r_{32},\, r_{33}) \tag{185}$$

$$\psi = \operatorname{atan2}(r_{21},\, r_{11}) \tag{186}$$

The use of atan2 (which takes two arguments and returns the angle in the correct quadrant) is essential. Using arctan alone would give ambiguous results.

12.3.2 Gimbal Lock

Gimbal lock is the Achilles' heel of Euler angles, and it is the single most important reason to learn quaternions.

What is it? When the pitch angle reaches $\theta = \pm 90$, the yaw and roll axes become aligned—they rotate about the same physical axis. The system loses one degree of freedom. You can still represent the current orientation, but you cannot distinguish certain rotations near the singularity, and small changes in orientation cause wild jumps in the Euler angles.

Mathematical demonstration. Set $\theta = 90$ (so $\cos\theta = 0$ and $\sin\theta = 1$) in the rotation matrix:

$$R\big|_{\theta=90} = \begin{bmatrix} 0 & \cos\psi\sin\phi - \sin\psi\cos\phi & \cos\psi\cos\phi + \sin\psi\sin\phi \\ 0 & \sin\psi\sin\phi + \cos\psi\cos\phi & \sin\psi\cos\phi - \cos\psi\sin\phi \\ -1 & 0 & 0 \end{bmatrix} \tag{187}$$

Using the angle subtraction identities, this simplifies to:

$$R\big|_{\theta=90} = \begin{bmatrix} 0 & -\sin(\psi-\phi) & \cos(\psi-\phi) \\ 0 & \cos(\psi-\phi) & \sin(\psi-\phi) \\ -1 & 0 & 0 \end{bmatrix} \tag{188}$$

Notice: only the *difference* $\psi - \phi$ appears. We have three parameters (ψ, θ, ϕ) but only two are independent at this configuration—one degree of freedom is lost. There are infinitely many (ψ, ϕ) pairs that give the same rotation, as long as their difference is the same.

Physical analogy. Imagine a three-axis gimbal (like on a camera stabilizer). The inner ring controls roll, the middle ring controls pitch, and the outer ring controls yaw. When the middle ring tilts to 90, the inner and outer rings become coplanar. Rotating the inner ring now produces the same motion as rotating the outer ring—you have lost the ability to independently control one axis.

Why this matters in practice. If you are running an EKF that tracks Euler angles as its state vector, the covariance matrix becomes singular near $\theta = \pm 90$—the filter diverges. Even before reaching

exactly 90, the kinematic equations that relate angular velocity $\boldsymbol{\omega}$ to Euler angle rates contain $\cos\theta$ in the denominator:

$$\begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} 1 & \sin\phi\tan\theta & \cos\phi\tan\theta \\ 0 & \cos\phi & -\sin\phi \\ 0 & \sin\phi/\cos\theta & \cos\phi/\cos\theta \end{bmatrix} \boldsymbol{\omega} \tag{189}$$

The $\tan\theta$ terms blow up as $\theta \to \pm 90$. This is not a numerical precision issue—it is a fundamental topological limitation of representing SO(3) with only three parameters.

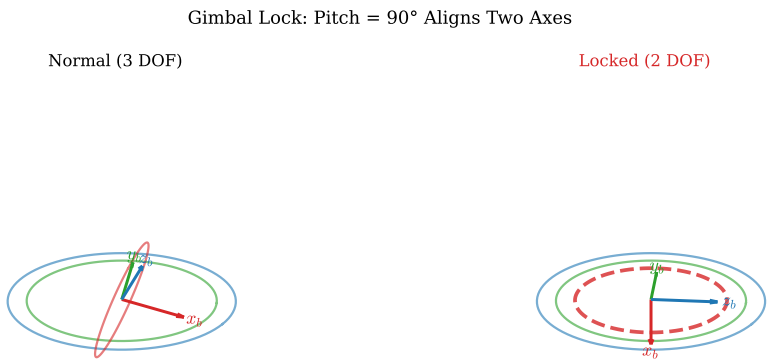

Figure 32: Gimbal lock. Left: normal operation—the three gimbal rings provide three independent degrees of freedom. Right: when pitch reaches 90, the inner and outer rings align, collapsing yaw and roll into the same axis (dashed ring). One DOF is lost.

This motivates the search for a singularity-free representation: quaternions.

12.4 Quaternions

12.4.1 Axis-Angle Rotation: The Gateway to Quaternions

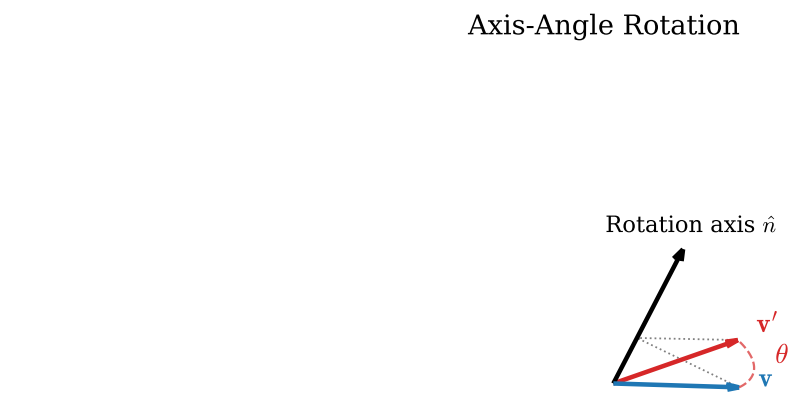

Figure 33: Axis-angle rotation: vector $\mathbf{v}$ is rotated by angle θ about axis $\hat{n}$ to produce $\mathbf{v}'$. The dashed lines show the projection onto the rotation axis.

Before jumping to quaternions, we need one more concept. **Euler's rotation theorem** states that any rotation of a rigid body in 3D can be described by a single axis $\hat{n}$ (a unit vector) and a single angle θ. No matter how complicated the rotation, there is always one axis that stays fixed, and the body has simply rotated around it.

This leads to **Rodrigues' rotation formula**, which rotates a vector $\mathbf{v}$ by angle θ about axis $\hat{n}$:

$$\mathbf{v}' = \mathbf{v}\cos\theta + (\hat{n} \times \mathbf{v})\sin\theta + \hat{n}(\hat{n} \cdot \mathbf{v})(1 - \cos\theta) \tag{190}$$

Derivation. Decompose $\mathbf{v}$ into a component parallel to $\hat{n}$ and a component perpendicular to $\hat{n}$:

$$\mathbf{v}_\parallel = \hat{n}(\hat{n} \cdot \mathbf{v}), \qquad \mathbf{v}_\perp = \mathbf{v} - \mathbf{v}_\parallel$$

The parallel component is unchanged by rotation about $\hat{n}$: $\mathbf{v}'_\parallel = \mathbf{v}_\parallel$. The perpendicular component rotates in the plane spanned by $\mathbf{v}_\perp$ and $\hat{n} \times \mathbf{v}$ (which is perpendicular to both $\hat{n}$ and $\mathbf{v}_\perp$, and has the same magnitude as $\mathbf{v}_\perp$):

$$\mathbf{v}'_\perp = \mathbf{v}_\perp \cos\theta + (\hat{n} \times \mathbf{v})\sin\theta$$

Adding the two components: $\mathbf{v}' = \mathbf{v}_\parallel + \mathbf{v}'_\perp = \hat{n}(\hat{n} \cdot \mathbf{v}) + (\mathbf{v} - \hat{n}(\hat{n} \cdot \mathbf{v}))\cos\theta + (\hat{n} \times \mathbf{v})\sin\theta$. Rearranging gives the formula above.

The corresponding rotation matrix form is:

$$R = I\cos\theta + (1 - \cos\theta)\hat{n}\hat{n}^T + \sin\theta\,[\hat{n}]_\times \tag{191}$$

where $[\hat{n}]_\times$ is the **skew-symmetric matrix** of $\hat{n} = (n_1, n_2, n_3)$:

$$[\hat{n}]_\times = \begin{bmatrix} 0 & -n_3 & n_2 \\ n_3 & 0 & -n_1 \\ -n_2 & n_1 & 0 \end{bmatrix} \tag{192}$$

This matrix has the property that $[\hat{n}]_\times \mathbf{v} = \hat{n} \times \mathbf{v}$ for any vector $\mathbf{v}$.

12.4.2 From Axis-Angle to Quaternions

The axis-angle representation is intuitive, but it does not compose nicely: combining two axis-angle rotations requires going through rotation matrices or trigonometric identities. Quaternions solve this problem.

A **unit quaternion** encodes the same axis-angle rotation $(\hat{n}, \theta)$ as a four-component object:

$$q = \begin{bmatrix} \cos(\theta/2) \\ \sin(\theta/2)\,\hat{n} \end{bmatrix} = \begin{bmatrix} q_w \\ q_x \\ q_y \\ q_z \end{bmatrix} \tag{193}$$

where q_w is the scalar (real) part and (q_x, q_y, q_z) is the vector (imaginary) part.

Why $\theta/2$ and not θ? This is the key insight. Quaternions live in a space where the algebraic structure (the Hamilton product, defined below) naturally composes rotations—but this only works with the half-angle encoding. The deep reason is topological: SO(3) is not simply connected, and the unit quaternion sphere S^3 is its **double cover**. This means that q and $-q$ represent the *same* rotation (negating all four components gives the same axis-angle pair with θ shifted by 2π). This double-cover property is the price we pay for a singularity-free representation.

The set of unit quaternions satisfies $|q| = \sqrt{q_w^2 + q_x^2 + q_y^2 + q_z^2} = 1$, forming the 3-sphere S^3 in four-dimensional space.

12.4.3 Quaternion Operations

Hamilton product (quaternion multiplication). Given two quaternions $p = (p_w, \mathbf{p}_v)$ and $q = (q_w, \mathbf{q}_v)$ where $\mathbf{p}_v$ and $\mathbf{q}_v$ are the vector parts:

$$p \otimes q = \begin{bmatrix} p_w q_w - \mathbf{p}_v \cdot \mathbf{q}_v \\ p_w \mathbf{q}_v + q_w \mathbf{p}_v + \mathbf{p}_v \times \mathbf{q}_v \end{bmatrix} \tag{194}$$

Written out component-wise:

$$p \otimes q = \begin{bmatrix} p_w q_w - p_x q_x - p_y q_y - p_z q_z \\ p_w q_x + p_x q_w + p_y q_z - p_z q_y \\ p_w q_y - p_x q_z + p_y q_w + p_z q_x \\ p_w q_z + p_x q_y - p_y q_x + p_z q_w \end{bmatrix} \tag{195}$$

This product is **associative** $(p \otimes (q \otimes r) = (p \otimes q) \otimes r)$ but **not commutative** $(p \otimes q \neq q \otimes p$ in general). The non-commutativity reflects the physical fact that rotations in 3D do not commute.

Rotating a vector. To rotate a vector $\mathbf{v} \in \mathbb{R}^3$ by the rotation encoded in quaternion q, embed the vector as a "pure quaternion" $\bar{v} = (0, \mathbf{v})$ and compute:

$$\bar{v}' = q \otimes \bar{v} \otimes q^* \tag{196}$$

where q^* is the conjugate. The result $\bar{v}'$ is again a pure quaternion whose vector part is the rotated vector $\mathbf{v}'$. Why the double product? A single multiplication $q \otimes \bar{v}$ would not, in general, produce a pure quaternion (it could have a nonzero scalar part). The "sandwich" product $q \otimes \bar{v} \otimes q^*$ guarantees the scalar part vanishes, giving a proper 3D vector.

Composing rotations. Just like rotation matrices, quaternion rotations compose by multiplication, in the same right-to-left order:

$$q_{\text{total}} = q_2 \otimes q_1 \tag{197}$$

This means: "first apply q_1, then apply q_2." To verify: $q_{\text{total}} \otimes \bar{v} \otimes q_{\text{total}}^* = q_2 \otimes (q_1 \otimes \bar{v} \otimes q_1^*) \otimes q_2^*$, which is indeed q_1 first, then q_2.

Inverse and conjugate. For a unit quaternion $q = (q_w, q_x, q_y, q_z)$:

$$q^{-1} = q^* = (q_w, -q_x, -q_y, -q_z) \tag{198}$$

For unit quaternions, the inverse equals the conjugate (just like $R^{-1} = R^T$ for rotation matrices). This represents the reverse rotation.

Quaternion to rotation matrix. The rotation matrix corresponding to unit quaternion $q = (q_w, q_x, q_y, q_z)$ is:

$$R = \begin{bmatrix} 1 - 2(q_y^2 + q_z^2) & 2(q_x q_y - q_z q_w) & 2(q_x q_z + q_y q_w) \\ 2(q_x q_y + q_z q_w) & 1 - 2(q_x^2 + q_z^2) & 2(q_y q_z - q_x q_w) \\ 2(q_x q_z - q_y q_w) & 2(q_y q_z + q_x q_w) & 1 - 2(q_x^2 + q_y^2) \end{bmatrix} \tag{199}$$

You can verify that $R^T R = I$ and $\det(R) = 1$ using $q_w^2 + q_x^2 + q_y^2 + q_z^2 = 1$.

Quaternion to Euler angles (ZYX convention). Given $q = (q_w, q_x, q_y, q_z)$:

$$\phi = \operatorname{atan2}\big(2(q_w q_x + q_y q_z),\ 1 - 2(q_x^2 + q_y^2)\big) \tag{200}$$

$$\theta = -\arcsin\big(2(q_x q_z - q_w q_y)\big) \tag{201}$$

$$\psi = \operatorname{atan2}\big(2(q_w q_z + q_x q_y),\ 1 - 2(q_y^2 + q_z^2)\big) \tag{202}$$

These formulas follow directly from comparing the quaternion-to-matrix formula with the Euler angle matrix and applying the same extraction procedure.

12.4.4 Why Quaternions Win

Having seen both Euler angles and rotation matrices, here is why quaternions are the preferred representation for state estimation and control:

No gimbal lock. Quaternions parameterize SO(3) smoothly everywhere. There is no configuration where a degree of freedom is lost. The kinematic equation for quaternion propagation is:

$$\dot{q} = \frac{1}{2} q \otimes \begin{bmatrix} 0 \\ \boldsymbol{\omega} \end{bmatrix} \tag{203}$$

where $\boldsymbol{\omega}$ is the angular velocity in the body frame. No tangent terms, no divisions by cosine, no singularities.

Compact representation. A quaternion uses 4 numbers, compared to 9 for a rotation matrix. It has only 1 constraint (unit norm) versus 6 constraints (orthogonality + unit determinant) for a rotation matrix. This makes quaternions easier to normalize and numerically more robust.

Efficient composition. The Hamilton product requires 16 multiplications and 12 additions. A 3×3 matrix product requires 27 multiplications and 18 additions. On a microcontroller, this difference adds up.

Smooth interpolation. Spherical Linear Interpolation (SLERP) provides the shortest, constant-speed path between two orientations on the quaternion sphere:

$$\mathrm{SLERP}(q_0, q_1, t) = q_0 \frac{\sin((1-t)\Omega)}{\sin \Omega} + q_1 \frac{\sin(t\Omega)}{\sin \Omega} \tag{204}$$

where $\Omega = \arccos(q_0 \cdot q_1)$ and $t \in [0, 1]$. This is invaluable for trajectory planning and animation. There is no clean equivalent for Euler angles.

12.5 Practical Summary

The table below compares the four representations we have covered:

Representation	Params	Singularity?	Composition	Best for
Rotation matrix	9 (6 constraints)	No	Matrix multiply	General computation
Euler angles	3	Yes (gimbal lock)	Must convert to matrix	Human-readable display
Quaternion	4 (1 constraint)	No	Hamilton product	State estimation, interpolation
Axis-angle	4 (axis + angle)	At $\theta = 0$	Must convert	Intuitive understanding

In practice, you will often use all four in the same project: quaternions as the internal state of your EKF, rotation matrices for coordinate transformations in your kinematics code, Euler angles for displaying the current attitude to the operator ("yaw: 45°, pitch: 10°, roll: 2°"), and axis-angle for specifying desired rotations in your trajectory planner ("rotate 30° about the vertical axis").

With this foundation, you are now equipped to understand the quaternion-based EKF in the Modern Control chapter. The key insight is: *use quaternions for computation, convert to Euler angles only for display.*

13 Outlook: Data-Driven and Learning-Based Control

All the methods covered so far—PID, state-space, LQR, Kalman filtering, MPC—assume a known *model* of the plant. In practice, however, obtaining an accurate model is often the hardest part. A new generation of techniques attempts to bypass or augment the modeling step by learning control policies directly from data. This chapter surveys these frontiers and discusses their relationship with classical control theory.

13.1 Data-Driven Control

13.1.1 Motivation: From "Model First, Control Second" to "Control Directly from Data"

The traditional pipeline is: collect data → system identification → build model → design controller. Each step introduces errors, and those errors compound. A natural question arises: *can we skip the modeling step and design controllers directly from data?*

13.1.2 Willems' Fundamental Lemma

The theoretical foundation is **Willems' Fundamental Lemma** (Willems et al., 2005; see Markovsky & Dörfler, 2021 for a modern treatment): for a linear time-invariant system, if an input–output trajectory of length T satisfies a *persistent excitation* condition of sufficient order, then *all* possible trajectories of the system can be expressed as linear combinations of the columns of the data Hankel matrix:

$$\begin{bmatrix} U_- \\ X_- \\ U_+ \\ X_+ \end{bmatrix} g = \begin{bmatrix} u_{\text{ini}} \\ x_{\text{ini}} \\ u \\ x \end{bmatrix},$$

where U_-, X_- are Hankel matrices constructed from historical data and g is the coefficient vector to be determined.

Intuition: A single sufficiently rich trajectory "encodes" the complete dynamics of the system—equivalent to knowing the (A, B) matrices.

13.1.3 Data-Driven LQR

Using Willems' Lemma, the LQR problem

$$\min_K \sum_{k=0}^{\infty} (x_k^\top Q\, x_k + u_k^\top R\, u_k)$$

can be solved directly via data-consistency constraints constructed from the Hankel matrix, *without identifying* A or B.

Limitations:

- The data must satisfy the persistent excitation condition—random noise is not enough; the input signal must be sufficiently rich.
- Sensitive to measurement noise.
- Currently limited to linear systems (nonlinear extensions are an active research area).

13.1.4 Data-Enabled Predictive Control (DeePC)

DeePC (Coulson et al., 2019) combines Willems' Lemma with MPC: the data Hankel matrix replaces the prediction model, and the predictive control problem is solved directly in data space. This preserves MPC's constraint-handling capability while eliminating the need for explicit system identification.

13.1.5 Koopman Operator Theory

The methods above apply to linear systems. For nonlinear systems, there is an elegant idea: **lift** the nonlinear dynamics into a higher-dimensional space where they become linear, then apply the entire classical linear toolbox.

The **Koopman operator** is an infinite-dimensional linear operator that acts on *observable functions* of the state rather than the state itself. Given a nonlinear system $x_{k+1} = f(x_k)$, the Koopman operator

$\mathcal{K}$ satisfies $\mathcal{K}g = g \circ f$ for any observable function g. The dynamics of $g(x)$ are *linear* even though the dynamics of x are not.

In practice, we approximate the Koopman operator by choosing a finite dictionary of observable functions $\psi(x) = [\psi_1(x), \ldots, \psi_N(x)]^\top$ and fitting a matrix K such that $\psi(x_{k+1}) \approx K\psi(x_k)$. The most common algorithm for this is **Extended Dynamic Mode Decomposition (EDMD)** (Williams et al., 2015), which solves a least-squares problem on trajectory data.

Why this matters: Once you have a linear model in the lifted space, you can apply LQR, Kalman filtering, and convex MPC *to genuinely nonlinear systems*—no Jacobian linearization, no operating-point restriction.

Recent hardware and algorithmic results (2024–2025) have moved Koopman methods from theoretical curiosities to deployed systems. Invertible Koopman Networks use invertible neural networks to learn both the lifting map and its inverse, solving the long-standing problem of reconstructing the true state from the lifted coordinates. Koopman-based MPC combined with Control Barrier Functions has been shown to formulate safety-critical nonlinear control as a simple QP in the lifted space. On the hardware side, SE(3) Koopman-MPC has been demonstrated on quadrotor UAVs, and real-time tire-dynamics modeling for autonomous vehicles shows the approach scales to fast, safety-critical loops.

Connection to what you already know: Think of the Koopman approach as a principled, data-driven generalization of the Jacobian linearization you learned in the state-space chapter. Jacobian linearization is accurate only near one operating point; Koopman lifting is accurate everywhere the data covers.

13.2 Adaptive Control

All the controllers we have studied so far use *fixed* parameters. What if the plant changes during operation—a drone picks up a payload, a robot arm lifts an unknown object, or a motor's friction changes with temperature? **Adaptive control** adjusts controller parameters online as conditions change.

13.2.1 Model Reference Adaptive Control (MRAC)

The idea is simple: define a **reference model** $\dot{x}_m = A_m x_m + B_m r$ that represents your desired closed-loop behavior, and design an adaptation law that updates the controller gains so that the plant's output tracks the reference model's output.

The adaptation law is typically derived from a **Lyapunov stability argument**—the same tool used in the Modern Control chapter. A candidate Lyapunov function $V(e, \tilde{\theta})$ is constructed from the tracking error $e = x - x_m$ and the parameter error $\tilde{\theta} = \theta - \theta^*$, and the gain update law is chosen to make $\dot{V} \leq 0$.

13.2.2 L1 Adaptive Control

Classical MRAC has a fundamental tension: fast adaptation improves tracking but can cause high-frequency oscillations. **L1 adaptive control** resolves this by inserting a *low-pass filter* between the adaptive estimate and the control signal. This decouples the adaptation rate from the control bandwidth, providing *guaranteed transient performance bounds*—something classical MRAC cannot offer.

The architecture has three components, all of which you have seen before:

1. A **state predictor** (essentially a Luenberger observer),
2. A fast **adaptation law** that estimates uncertainties,
3. A **low-pass filter** that smooths the adaptive signal before it reaches the plant.

13.2.3 When to Use What

Scenario	Recommended approach
Uncertainty bounded and known in advance	Robust control (H_∞, μ-synthesis)
Parameters drift slowly, structure known	MRAC
Fast adaptation needed, bandwidth must be bounded	L1 adaptive control
No model at all, abundant simulation data	Reinforcement learning

13.3 Reinforcement Learning and Control

Adaptive control adjusts fixed-structure controllers online; reinforcement learning (RL) goes further by learning the *structure* of the control policy itself. It generalizes optimal control to nonlinear systems

with arbitrary cost functions and constraints.

13.3.1 Core Concepts

RL rests on three pillars. A **policy** $\pi(s) \to a$ is a mapping from states to actions—in control terms, it *is* the controller. In the linear-quadratic case, $\pi(x) = -Kx$ reduces to the LQR gain we derived earlier. The **value function** $V^\pi(s) = \mathbb{E}\big[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s\big]$ measures the expected cumulative reward from a given state, playing exactly the role of the cost-to-go in dynamic programming. Finally, the **policy gradient** $\nabla_\theta J(\theta)$ provides the direction in which to update the policy parameters to maximize expected reward—gradient ascent on the controller's performance.

13.3.2 Two Paradigms

These building blocks support two broad implementation strategies. **Model-free RL** (PPO, SAC, etc.) learns policies through trial and error without a dynamics model. It is highly general, but *sample efficiency* is extremely low—typically requiring millions of interactions, feasible only in simulation, with sim-to-real transfer needed for deployment. **Model-based RL** (MBPO, Dreamer, etc.) first learns a dynamics model from data, then uses it for planning or policy optimization. It is more sample-efficient and closer in spirit to classical control—essentially system identification followed by model-based optimal control, but with a learned model.

13.3.3 Sim-to-Real Transfer

RL policies are usually trained in simulation, but a gap exists between simulation and the real system. Three strategies are commonly used to bridge this gap. **Domain randomization** randomizes physical parameters (mass, friction, latency, etc.) during training, forcing the policy to learn behaviors robust to parameter variations. **System identification and high-fidelity simulation** carefully calibrate the simulation parameters to narrow the gap at its source. **Fine-tuning** pre-trains in simulation, then adjusts on the real system with a small amount of data—getting most of the way in simulation and polishing the last mile on hardware.

13.3.4 Success Stories and Current Status

RL has achieved remarkable results in research. Quadrupedal robots—MIT Mini Cheetah (Margolis et al., 2022), ANYmal (Miki et al., 2022)—have learned to walk and run on rough terrain via RL alone. OpenAI demonstrated Rubik's cube solving with a dexterous robotic hand, and Google has deployed RL-based grasping in warehouse settings. ETH Zürich's Swift system beat human champions in drone racing (Kaufmann et al., 2023), closing the control loop from raw camera images to motor commands at superhuman speed.

Despite these successes, RL remains rare in engineering competitions and industrial settings. Training requires high-fidelity simulation environments that are expensive to develop. Learned policies are "black boxes," making debugging extremely difficult when something goes wrong on the field. Competition timelines are usually too short for the full RL development cycle. And safety guarantees remain insufficient—you cannot formally prove that a neural-network policy will never command a dangerous action.

13.4 Trajectory Optimization and Optimal Control

The goal of optimal control is to find $u(t)$ that minimizes a cost functional:

$$J = \int_0^T L\big(x(t), u(t)\big)\,\mathrm{d}t + \Phi\big(x(T)\big),$$

subject to the dynamics $\dot{x} = f(x, u)$.

13.4.1 Two Solution Strategies

There are two fundamentally different ways to attack this problem. **Indirect methods**, rooted in Pontryagin's Maximum Principle, derive adjoint (costate) equations to obtain necessary conditions for optimality, then solve the resulting two-point boundary value problem. They are mathematically elegant

but become unwieldy when the system has complex inequality constraints. **Direct methods** (collocation, multiple shooting) take a more engineering-oriented approach: discretize the trajectory into a finite set of decision variables and convert the entire problem into a large-scale nonlinear program (NLP) that off-the-shelf solvers can handle. Modern trajectory optimization tools universally adopt direct methods.

13.4.2 iLQR and DDP

Iterative LQR (iLQR) and **Differential Dynamic Programming (DDP)** are the workhorse algorithms of modern robotic trajectory optimization: linearize the nonlinear dynamics around a nominal trajectory, solve the resulting LQR sub-problem to obtain a correction, update the trajectory, and repeat.

Connection to MPC: Model predictive control is essentially trajectory optimization solved repeatedly in a receding-horizon fashion at each time step. When the dynamics are nonlinear, MPC internally uses iLQR or NLP solvers.

Common tools: CasADi, Drake, ALTRO, Crocoddyl, MATLAB `fmincon`.

13.5 Neural Networks Meet Control

Trajectory optimization gives us optimal solutions *given* a model. But what if parts of the model are too complex to derive from physics? Neural networks can fill in the gaps.

13.5.1 Neural Networks as Controller Components

Neural networks need not replace the entire controller. A more practical approach is to embed them within specific modules of a classical control framework. The most common pattern is **learning the dynamics model**: fit $\dot{x} = f_\theta(x, u)$ with a neural network, then plug the learned model into MPC or iLQR as a drop-in replacement for the physics-based model. When a first-principles model already exists but is not accurate enough, a neural network can **learn the residual**: $\dot{x} = f_{\text{phys}}(x, u) + f_\theta(x, u)$, letting the network capture only what the physics model misses—friction, cable elasticity, aerodynamic effects. A third pattern is **learning the cost function** from human demonstrations (inverse reinforcement learning), which is useful when the task objective is hard to specify mathematically but easy to demonstrate.

13.5.2 Neural ODEs and Physics-Informed Neural Networks

Neural ODEs (Chen et al., 2018) parameterize the state derivative directly as a neural network: $\dot{x} = f_\theta(x, t, u)$, and solve forward using standard ODE solvers (e.g., Runge-Kutta). This gives a continuous-time, state-space model learned from data whose structure is *guaranteed* to satisfy the ODE formulation—exactly the $\dot{x} = f(x, u)$ framework from our state-space chapter, but with f learned rather than derived from physics.

Physics-Informed Neural Networks (PINNs) go one step further: they embed known physics (differential equations, conservation laws, symmetries) as soft constraints in the loss function. The training objective becomes:

$$\mathcal{L} = \underbrace{\mathcal{L}_{\text{data}}}_{\text{fit measured data}} + \lambda \underbrace{\mathcal{L}_{\text{physics}}}_{\text{ODE/PDE residual}\to 0}.$$

This produces models that generalize beyond training data in physically consistent ways—they cannot violate energy conservation or predict negative masses.

Practical impact: Neural ODEs and PINNs can serve as the internal model for MPC, replacing both first-principles derivation *and* traditional system identification. Recent work (2024–2025) has demonstrated PINN-based MPC for chemical processes, oil well control, and Van der Pol oscillators, with inference faster than numerical ODE solvers since it is a single forward pass.

13.5.3 Diffusion Policies for Robot Learning

A breakthrough in robot imitation learning came from an unexpected direction: **diffusion models**, the same generative models behind image generators like Stable Diffusion.

Diffusion Policy (Chi et al., 2023) treats action generation as a denoising process: starting from pure Gaussian noise in action space, the model iteratively denoises conditioned on visual observations until a clean action trajectory emerges. The key advantage over standard behavioral cloning is that

diffusion models can represent **multimodal** action distributions—when there are multiple valid ways to perform a task (go around the obstacle left or right?), the model preserves all modes rather than averaging them into a single (often invalid) action.

Results: Across 12 manipulation benchmarks, Diffusion Policy outperformed prior state-of-the-art by an average of 46.9% success rate. Real-world demonstrations include bimanual assembly and cloth folding. One-Step Diffusion Policy (2025) distills the iterative process into a single forward pass, enabling 50+ Hz real-time control.

Connection to classical control: The denoising process is analogous to iterative trajectory refinement in MPC/iLQR: you start with a rough plan (noise) and refine it step by step, except here the "model" is learned from demonstrations rather than derived from physics.

13.5.4 Foundation Models for Robotics (VLAs)

The latest wave in robot learning leverages **Vision-Language-Action (VLA)** models: large neural networks pre-trained on internet-scale image-text data, then fine-tuned to output robot actions. Given a camera image and a natural-language instruction ("pick up the red cup"), the VLA outputs joint commands directly.

Key milestones:

- **RT-2** (Brohan et al., 2023): First demonstration that a VLM can be fine-tuned for robot control. Showed emergent reasoning—e.g., inferring which object to grasp from abstract instructions never seen during training.
- **Octo** (Ghosh et al., 2024): Open-source generalist policy trained on 1M+ episodes across 22 different robot embodiments.
- π_0 (Black et al., 2024): 3B-parameter VLA using flow matching (a continuous-time generalization of diffusion) for 50 Hz real-time control. Demonstrated dexterous bimanual tasks including laundry folding and box assembly. Weights released open-source.
- $\pi_{0.5}$ (Physical Intelligence, 2025): Demonstrated generalization to entirely new environments unseen during training.

The practical hierarchy: VLAs serve as high-level task planners; classical controllers (PID, impedance control) still execute the low-level joint commands. This mirrors the cascade control architecture from Section 4: VLA = outer loop (task-level semantics), classical controller = inner loop (joint-level dynamics).

13.5.5 Safety Constraints and Verifiability

Combining learning methods with safety guarantees is a hot research direction:

- **Control Barrier Functions (CBF)**: add a safety constraint layer to learned policies, ensuring the system state never leaves a safe set. Mathematically, a CBF $h(x)$ defines a safe set $\{x : h(x) \geq 0\}$ and imposes the constraint $\dot{h}(x) + \alpha h(x) \geq 0$ on the control input—a single linear inequality that can be appended to any QP-based controller.
- **Lyapunov neural networks**: train a neural network to simultaneously learn a controller and a Lyapunov function, thereby providing a stability proof.
- **Reachability analysis**: compute the reachable set of the system under a learned policy and verify whether it satisfies safety requirements.

13.6 Event-Triggered Control

Every controller in this book runs at a *fixed* sampling rate T_s. But does it really need to? If the system is near equilibrium and nothing is changing, computing a new control output every millisecond wastes CPU cycles, bus bandwidth, and battery.

Event-triggered control (ETC) replaces the fixed clock with a **triggering condition**: a new control computation fires only when the state deviation since the last update exceeds a threshold:

$$\|x(t) - x(t_k)\| \geq \sigma \|x(t)\| + \varepsilon,$$

where t_k is the last trigger time, $\sigma > 0$ is a relative threshold, and $\varepsilon > 0$ prevents Zeno behavior (infinitely frequent triggers). Between triggers, the control input is held constant—exactly like a ZOH.

Self-triggered control is a proactive variant: instead of continuously monitoring the state, the controller *predicts* when the next trigger will be needed and schedules it in advance, allowing the processor to sleep in between.

Why this matters for robotics. On a CAN bus shared among 8+ motor controllers, reducing unnecessary messages directly improves worst-case latency for high-priority loops. On battery-powered systems (drones, mobile robots), fewer computations mean longer runtime. And in multi-agent systems—UAV swarms, multi-robot warehouses—event-triggered communication drastically reduces network congestion.

Connection to classical control: ETC is a generalization of sampled-data control theory. Stability analysis uses the same Lyapunov tools from the Modern Control chapter, with an additional step to prove that inter-event intervals are bounded away from zero (ruling out Zeno behavior). The proof typically relies on input-to-state stability (ISS) arguments.

13.7 Method Comparison at a Glance

Method	Model needed?	Handles nonlinearity?	Real-time?	Maturity
PID	No (empirical tuning)	Local	Yes	Industry standard
LQR	Yes (linear)	No	Yes	Mature
MPC (linear)	Yes (linear)	No	Yes	Mature
NMPC	Yes (nonlinear)	Yes	Depends	Early industrial
MRAC / L1 adaptive	Partial (structure)	Limited	Yes	Mature (aerospace)
Data-driven LQR	No (data only)	No	Yes	Research frontier
DeePC	No (data only)	Limited	Depends	Research frontier
Koopman + MPC	No (data-lifted)	Yes (via lifting)	Yes	Research validated
Model-based RL	Learned	Yes	Offline training	Research validated
Model-free RL	No	Yes	Offline training	Research validated
iLQR / DDP	Yes (nonlinear)	Yes	Near real-time	Research / early industrial
Neural ODE / PINN	Hybrid	Yes	Near real-time	Research frontier
Diffusion Policy	No (demos)	Yes	Near real-time	Research validated
VLA (π_0 etc.)	No (demos + VLM)	Yes	Yes (50 Hz)	Research frontier
Event-triggered	Same as base ctrl.	Same as base ctrl.	Reduces load	Mature (theory)

13.8 Advice for the Reader

The big picture. The boundary between control theory and machine learning is dissolving. Data-driven methods bring adaptability; learning-based methods bring the ability to handle complexity. But the foundations covered in this document—state-space modeling, stability analysis, observer design, optimal control—remain the *bedrock* on which all of this is built.

A practical roadmap:

1. **Master the classics first.** PID, LQR, MPC, Kalman filtering—these tools are sufficient for the vast majority of robotics applications, and they are reliable, debuggable, and theoretically grounded.
2. **Introduce data-driven techniques when classical methods hit a wall.** System hard to model? Try data-driven LQR/MPC. Environment varies widely? Consider adaptive control.
3. **Use RL for "things classical methods cannot do."** Extreme agility, high-dimensional dexterous manipulation, complex contact interactions—these are the scenarios where RL's advantages truly shine.
4. **Always keep a safety layer.** No matter how "intelligent" the controller, add control barrier functions or hardware safety limits to ensure the system never takes dangerous actions.

A deep understanding of the classical foundations will make you a better user of advanced methods, not someone replaced by them.

13.9 Further Reading

Clickable links to key papers and resources mentioned in this chapter.

Data-Driven Control

- Markovsky & Dörfler (2021) — Behavioral systems theory and Willems' Fundamental Lemma: a tutorial
- Coulson et al. (2019) — Data-Enabled Predictive Control (DeePC)
- Williams et al. (2015) — Extended Dynamic Mode Decomposition (EDMD) for Koopman approximation

Reinforcement Learning

- Schulman et al. (2017) — Proximal Policy Optimization (PPO)
- Haarnoja et al. (2018) — Soft Actor-Critic (SAC)
- Janner et al. (2019) — Model-Based Policy Optimization (MBPO)
- Hafner et al. (2020) — Dream to Control: Dreamer
- Margolis et al. (2022) — Rapid locomotion via RL (MIT Mini Cheetah)
- Miki et al. (2022) — Learning robust perceptive locomotion (ANYmal)
- Kaufmann et al. (2023) — Champion-level drone racing via deep RL (Swift)

Trajectory Optimization

- Howell et al. (2020) — ALTRO: fast solver for constrained trajectory optimization
- Crocoddyl — Efficient multi-contact optimal control (open-source)
- CasADi — Symbolic framework for nonlinear optimization and optimal control

Neural Networks and Learning for Control

- Chen et al. (2018) — Neural Ordinary Differential Equations
- Chi et al. (2023) — Diffusion Policy: visuomotor policy learning via action diffusion
- Brohan et al. (2023) — RT-2: Vision-Language-Action Models
- Ghosh et al. (2024) — Octo: An Open-Source Generalist Robot Policy
- Black et al. (2024) — π_0: A Vision-Language-Action Flow Model for General Robot Control

14 Appendix: Plug-and-Play C++ Modules

This appendix provides production-ready, header-only C++ implementations of every major algorithm discussed in this note. Each module is self-contained — just drop the header into your project and use it. All modules are designed for embedded systems (no dynamic allocation, no exceptions, no STL containers).

Design philosophy:

- **Header-only**: No `.cpp` files needed. Include and go.
- **No heap allocation**: Everything lives on the stack or is statically sized via templates.
- **Minimal dependencies**: Only `<cmath>` and `<cstring>`. No STL.
- **ISR-safe**: All `update()` methods are safe to call from timer interrupts.
- **Template-based**: Matrix dimensions are compile-time constants for zero-overhead abstraction.

14.1 Low-Pass Filter

First-order IIR low-pass filter. One line to construct, one line to use.

```
// lpf.hpp
#pragma once
#include <cmath>

class LowPassFilter {
public:
    LowPassFilter() : alpha_(1.0f), y_prev_(0.0f), initialized_(false) {}

    /// Configure with cutoff frequency (Hz) and sampling period (s)
    void configure(float cutoff_hz, float dt) {
        float rc = 1.0f / (2.0f * M_PI * cutoff_hz);
        alpha_ = dt / (rc + dt);
    }

    /// Configure directly with smoothing factor (0 < alpha <= 1)
    void set_alpha(float alpha) { alpha_ = alpha; }

    /// Process one sample. Call this at your fixed sample rate.
    float update(float x) {
        if (!initialized_) {
            y_prev_ = x;
            initialized_ = true;
            return x;
        }
        y_prev_ = alpha_ * x + (1.0f - alpha_) * y_prev_;
        return y_prev_;
    }

    /// Reset filter state
    void reset() { initialized_ = false; y_prev_ = 0.0f; }

    /// Get current filtered value without feeding new input
    float value() const { return y_prev_; }

private:
    float alpha_;
    float y_prev_;
    bool  initialized_;
};
```

Usage:

```
LowPassFilter gyro_filter;
gyro_filter.configure(30.0f, 0.001f);  // 30 Hz cutoff, 1 kHz loop

// In your control loop ISR:
float gyro_clean = gyro_filter.update(gyro_raw);
```

14.2 PID Controller

Full-featured PID with anti-windup (back-calculation), derivative-on-measurement, filtered derivative, and output clamping.

```
// pid.hpp
#pragma once
#include <cmath>

struct PIDConfig {
    float kp = 0.0f;
    float ki = 0.0f;
    float kd = 0.0f;
    float dt = 0.001f;         // Sampling period (s)
    float out_min = -1e6f;     // Output lower limit
    float out_max =  1e6f;     // Output upper limit
    float integral_max = 1e6f; // Integral term clamp
    float d_filter_N = 10.0f;  // Derivative filter coefficient (higher = less filtering)
};

class PID {
public:
    PID() = default;
    explicit PID(const PIDConfig& cfg) : cfg_(cfg) {}

    void configure(const PIDConfig& cfg) { cfg_ = cfg; }

    /// Compute PID output. Call once per control cycle.
    /// setpoint: desired value, measurement: actual value
    float update(float setpoint, float measurement) {
        float error = setpoint - measurement;

        // --- Proportional ---
        float p_term = cfg_.kp * error;

        // --- Integral with back-calculation anti-windup ---
        integral_ += cfg_.ki * error * cfg_.dt + anti_windup_;
        integral_ = clamp(integral_, -cfg_.integral_max, cfg_.integral_max);

        // --- Derivative on measurement (no derivative kick) ---
        //     with first-order low-pass filter
        float d_raw = -(measurement - meas_prev_) / cfg_.dt;
        float alpha_d = cfg_.dt * cfg_.d_filter_N
                      / (1.0f + cfg_.dt * cfg_.d_filter_N);
        d_filtered_ = alpha_d * d_raw + (1.0f - alpha_d) * d_filtered_;
        float d_term = cfg_.kd * d_filtered_;

        meas_prev_ = measurement;

        // --- Total output ---
        float output_raw = p_term + integral_ + d_term;
```

```
        float output = clamp(output_raw, cfg_.out_min, cfg_.out_max);

        // --- Back-calculation anti-windup ---
        if (cfg_.ki != 0.0f) {
            anti_windup_ = (output - output_raw) * (1.0f / cfg_.ki)
                         * cfg_.dt * 0.5f;
        } else {
            anti_windup_ = 0.0f;
        }

        return output;
    }

    void reset() {
        integral_ = 0.0f;
        d_filtered_ = 0.0f;
        meas_prev_ = 0.0f;
        anti_windup_ = 0.0f;
    }

    // Access internal state for debugging
    float get_integral() const { return integral_; }

private:
    PIDConfig cfg_;
    float integral_    = 0.0f;
    float d_filtered_  = 0.0f;
    float meas_prev_   = 0.0f;
    float anti_windup_ = 0.0f;

    static float clamp(float v, float lo, float hi) {
        return v < lo ? lo : (v > hi ? hi : v);
    }
};
```

Usage:

```
    PIDConfig cfg;
    cfg.kp = 6.0f;  cfg.ki = 0.3f;  cfg.kd = 3.0f;
    cfg.dt = 0.001f;
    cfg.out_min = -30000.0f;  cfg.out_max = 30000.0f;  // GM6020 range
    cfg.integral_max = 10000.0f;

    PID yaw_pid(cfg);

    // In ISR:
    float torque_cmd = yaw_pid.update(target_angle, current_angle);
```

14.3 Setpoint Ramp (LPF Soft-Start)

A dedicated setpoint smoother — wraps LPF to give ramp-limited setpoint transitions.

```
// ramp.hpp
#pragma once
#include "lpf.hpp"

class SetpointRamp {
public:
    /// ramp_time: approximate time to reach 95% of a step change (in seconds)
```

```
    void configure(float ramp_time, float dt) {
        float cutoff = 3.0f / (2.0f * M_PI * ramp_time);  // 3*tau ~ 95%
        filter_.configure(cutoff, dt);
    }

    float update(float raw_setpoint) {
        return filter_.update(raw_setpoint);
    }

    void reset() { filter_.reset(); }

private:
    LowPassFilter filter_;
};
```

14.4 Complementary Filter (IMU Tilt)

Fuses accelerometer (tilt from gravity) and gyroscope (angular rate).

```
// complementary_filter.hpp
#pragma once
#include <cmath>

class ComplementaryFilter {
public:
    /// alpha: gyro trust factor (0.95-0.99 typical)
    void configure(float alpha, float dt) {
        alpha_ = alpha;
        dt_ = dt;
    }

    /// accel_angle: tilt from accelerometer (rad)
    /// gyro_rate:   angular rate from gyro (rad/s)
    /// returns:     fused angle estimate (rad)
    float update(float accel_angle, float gyro_rate) {
        angle_ = alpha_ * (angle_ + gyro_rate * dt_)
               + (1.0f - alpha_) * accel_angle;
        return angle_;
    }

    void reset(float initial_angle = 0.0f) { angle_ = initial_angle; }
    float angle() const { return angle_; }

private:
    float alpha_ = 0.98f;
    float dt_    = 0.001f;
    float angle_ = 0.0f;
};
```

14.5 Lightweight Matrix Library

Compile-time sized matrix for embedded Kalman/LQR. No heap, no STL.

```
// mat.hpp
#pragma once
#include <cstring>
#include <cmath>

template<int ROWS, int COLS>
```

```
struct Mat {
    float data[ROWS][COLS] = {};

    float& operator()(int r, int c)       { return data[r][c]; }
    float  operator()(int r, int c) const { return data[r][c]; }

    static Mat zeros() { Mat m; return m; }

    static Mat identity() {
        static_assert(ROWS == COLS, "Identity requires square matrix");
        Mat m;
        for (int i = 0; i < ROWS; ++i) m.data[i][i] = 1.0f;
        return m;
    }

    // Matrix addition
    Mat operator+(const Mat& rhs) const {
        Mat r;
        for (int i = 0; i < ROWS; ++i)
            for (int j = 0; j < COLS; ++j)
                r.data[i][j] = data[i][j] + rhs.data[i][j];
        return r;
    }

    // Matrix subtraction
    Mat operator-(const Mat& rhs) const {
        Mat r;
        for (int i = 0; i < ROWS; ++i)
            for (int j = 0; j < COLS; ++j)
                r.data[i][j] = data[i][j] - rhs.data[i][j];
        return r;
    }

    // Matrix multiplication
    template<int COLS2>
    Mat<ROWS, COLS2> operator*(const Mat<COLS, COLS2>& rhs) const {
        Mat<ROWS, COLS2> r;
        for (int i = 0; i < ROWS; ++i)
            for (int j = 0; j < COLS2; ++j)
                for (int k = 0; k < COLS; ++k)
                    r.data[i][j] += data[i][k] * rhs.data[k][j];
        return r;
    }

    // Scalar multiplication
    Mat operator*(float s) const {
        Mat r;
        for (int i = 0; i < ROWS; ++i)
            for (int j = 0; j < COLS; ++j)
                r.data[i][j] = data[i][j] * s;
        return r;
    }

    // Transpose
    Mat<COLS, ROWS> T() const {
        Mat<COLS, ROWS> r;
        for (int i = 0; i < ROWS; ++i)
            for (int j = 0; j < COLS; ++j)
```

```
                r.data[j][i] = data[i][j];
        return r;
    }

    // 2x2 or 3x3 inverse (small matrix only, used by Kalman)
    Mat inverse() const;
};

// 1x1 inverse
template<> inline Mat<1,1> Mat<1,1>::inverse() const {
    Mat<1,1> r;
    r.data[0][0] = 1.0f / data[0][0];
    return r;
}

// 2x2 inverse
template<> inline Mat<2,2> Mat<2,2>::inverse() const {
    Mat<2,2> r;
    float det = data[0][0]*data[1][1] - data[0][1]*data[1][0];
    float inv_det = 1.0f / det;
    r.data[0][0] =  data[1][1] * inv_det;
    r.data[0][1] = -data[0][1] * inv_det;
    r.data[1][0] = -data[1][0] * inv_det;
    r.data[1][1] =  data[0][0] * inv_det;
    return r;
}

// 3x3 inverse
template<> inline Mat<3,3> Mat<3,3>::inverse() const {
    Mat<3,3> r;
    float det = data[0][0]*(data[1][1]*data[2][2]-data[1][2]*data[2][1])
              - data[0][1]*(data[1][0]*data[2][2]-data[1][2]*data[2][0])
              + data[0][2]*(data[1][0]*data[2][1]-data[1][1]*data[2][0]);
    float inv = 1.0f / det;
    r(0,0)=(data[1][1]*data[2][2]-data[1][2]*data[2][1])*inv;
    r(0,1)=(data[0][2]*data[2][1]-data[0][1]*data[2][2])*inv;
    r(0,2)=(data[0][1]*data[1][2]-data[0][2]*data[1][1])*inv;
    r(1,0)=(data[1][2]*data[2][0]-data[1][0]*data[2][2])*inv;
    r(1,1)=(data[0][0]*data[2][2]-data[0][2]*data[2][0])*inv;
    r(1,2)=(data[0][2]*data[1][0]-data[0][0]*data[1][2])*inv;
    r(2,0)=(data[1][0]*data[2][1]-data[1][1]*data[2][0])*inv;
    r(2,1)=(data[0][1]*data[2][0]-data[0][0]*data[2][1])*inv;
    r(2,2)=(data[0][0]*data[1][1]-data[0][1]*data[1][0])*inv;
    return r;
}
```

14.6 Kalman Filter

Generic discrete-time Kalman filter. Template parameters set the state and measurement dimensions at compile time.

```
// kalman.hpp
#pragma once
#include "mat.hpp"

template<int N_STATE, int N_MEAS, int N_INPUT = 1>
class KalmanFilter {
public:
    using StateVec  = Mat<N_STATE, 1>;
```

```
    using MeasVec   = Mat<N_MEAS, 1>;
    using InputVec  = Mat<N_INPUT, 1>;
    using StateMat  = Mat<N_STATE, N_STATE>;
    using InputMat  = Mat<N_STATE, N_INPUT>;
    using MeasMat   = Mat<N_MEAS, N_STATE>;
    using GainMat   = Mat<N_STATE, N_MEAS>;
    using MeasCov   = Mat<N_MEAS, N_MEAS>;

    StateMat A;   // State transition
    InputMat B;   // Input matrix
    MeasMat  C;   // Measurement matrix (called H in some texts)
    StateMat Q;   // Process noise covariance
    MeasCov  R;   // Measurement noise covariance

    StateVec x;   // State estimate
    StateMat P;   // Estimate covariance

    /// Predict step: propagate state and covariance forward
    void predict(const InputVec& u) {
        x = A * x + B * u;
        P = A * P * A.T() + Q;
    }

    /// Predict step (no input)
    void predict() {
        x = A * x;
        P = A * P * A.T() + Q;
    }

    /// Update step: incorporate measurement
    void update(const MeasVec& z) {
        // Innovation
        MeasVec y = z - C * x;
        // Innovation covariance
        MeasCov S = C * P * C.T() + R;
        // Kalman gain
        GainMat K = P * C.T() * S.inverse();
        // State update
        x = x + K * y;
        // Covariance update (Joseph form for numerical stability)
        StateMat I_KC = StateMat::identity() - K * C;
        P = I_KC * P * I_KC.T() + K * R * K.T();
    }

    /// Combined predict + update
    void step(const InputVec& u, const MeasVec& z) {
        predict(u);
        update(z);
    }
};
```

Usage — IMU tilt estimation (2-state Kalman from the case study):

```
    KalmanFilter<2, 1, 1> kf;
    float dt = 0.001f;

    // State: [angle, gyro_bias]
    kf.A(0,0) = 1.0f;  kf.A(0,1) = -dt;
    kf.A(1,0) = 0.0f;  kf.A(1,1) = 1.0f;
```

```
    kf.B(0,0) = dt;
    kf.B(1,0) = 0.0f;

    kf.C(0,0) = 1.0f;  kf.C(0,1) = 0.0f;

    // Tune these
    kf.Q(0,0) = 0.001f;  kf.Q(1,1) = 0.003f;
    kf.R(0,0) = 0.03f;

    kf.P = Mat<2,2>::identity() * 1.0f;

    // In ISR:
    Mat<1,1> u;  u(0,0) = gyro_rate;
    Mat<1,1> z;  z(0,0) = accel_angle;
    kf.step(u, z);
    float estimated_angle = kf.x(0, 0);
    float estimated_bias  = kf.x(1, 0);
```

14.7 LQR Gain (Offline Computation Helper)

LQR gain computation requires solving the Riccati equation, which is typically done *offline* in MATLAB/Python. This module applies the precomputed gain K at runtime.

```
// lqr.hpp
#pragma once
#include "mat.hpp"

template<int N_STATE, int N_INPUT>
class LQRController {
public:
    using StateVec = Mat<N_STATE, 1>;
    using InputVec = Mat<N_INPUT, 1>;
    using GainMat  = Mat<N_INPUT, N_STATE>;

    GainMat K;  // Set this from MATLAB: K = lqr(A, B, Q, R)

    /// Compute optimal input: u = -K * (x - x_ref)
    InputVec compute(const StateVec& x, const StateVec& x_ref) const {
        StateVec error = x - x_ref;
        return (K * error) * (-1.0f);
    }

    /// Compute optimal input for regulation (x_ref = 0)
    InputVec compute(const StateVec& x) const {
        return (K * x) * (-1.0f);
    }
};
```

Usage — balance bot:

```
    // Gain computed offline: K = lqr(A, B, diag([100,1,10,1]), 1)
    LQRController<4, 1> lqr;
    lqr.K(0,0) = -31.62f;  // theta gain
    lqr.K(0,1) = -5.27f;   // theta_dot gain
    lqr.K(0,2) = -3.16f;   // x gain
    lqr.K(0,3) = -4.12f;   // x_dot gain

    // In ISR (state from Kalman filter):
```

```
    Mat<4,1> state;
    state(0,0) = kf.x(0,0);  // theta
    state(1,0) = kf.x(1,0);  // theta_dot
    state(2,0) = encoder_pos;
    state(3,0) = encoder_vel;

    auto u = lqr.compute(state);
    set_motor_torque(u(0,0));
```

14.8 Cascaded PID

Two-loop cascaded controller (outer position + inner velocity) as described in the gimbal case study.

```
// cascaded_pid.hpp
#pragma once
#include "pid.hpp"

class CascadedPID {
public:
    PID outer;   // Position loop (slower)
    PID inner;   // Velocity loop (faster)

    struct Config {
        PIDConfig outer_cfg;
        PIDConfig inner_cfg;
        float ff_gain = 0.0f;  // Velocity feedforward gain
    };

    void configure(const Config& cfg) {
        outer.configure(cfg.outer_cfg);
        inner.configure(cfg.inner_cfg);
        ff_gain_ = cfg.ff_gain;
    }

    /// Full cascaded update
    /// pos_setpoint: desired position
    /// pos_measured: actual position
    /// vel_measured: actual velocity
    /// vel_feedforward: reference velocity for feedforward (optional)
    float update(float pos_setpoint, float pos_measured,
                 float vel_measured, float vel_feedforward = 0.0f) {
        // Outer loop: position -> velocity command
        float vel_cmd = outer.update(pos_setpoint, pos_measured)
                      + ff_gain_ * vel_feedforward;

        // Inner loop: velocity -> torque/current command
        float output = inner.update(vel_cmd, vel_measured);
        return output;
    }

    void reset() { outer.reset(); inner.reset(); }

private:
    float ff_gain_ = 0.0f;
};
```

Usage — gimbal yaw:

```
    CascadedPID::Config cfg;
```

```
    // Outer loop (200 Hz)
    cfg.outer_cfg = {.kp=8.0f, .ki=0.0f, .kd=0.0f, .dt=0.005f,
                     .out_min=-300.0f, .out_max=300.0f};
    // Inner loop (1 kHz)
    cfg.inner_cfg = {.kp=20.0f, .ki=1.0f, .kd=0.0f, .dt=0.001f,
                     .out_min=-30000.0f, .out_max=30000.0f,
                     .integral_max=10000.0f};
    cfg.ff_gain = 1.0f;

    CascadedPID gimbal;
    gimbal.configure(cfg);

    // Inner loop ISR (1 kHz) - only update inner
    float torque = gimbal.inner.update(vel_cmd_cached, gyro_yaw);

    // Outer loop ISR (200 Hz) - update both
    float torque = gimbal.update(target_angle, encoder_yaw,
                                 gyro_yaw, ref_velocity);
```

14.9 Advanced PID (Integral Separation + PI-D + 2-DOF)

Extends the base PID with all improvements from Section 4.5.

```
// pid_advanced.hpp
#pragma once
#include <cmath>

struct AdvancedPIDConfig {
    float kp = 0.0f, ki = 0.0f, kd = 0.0f;
    float dt = 0.001f;
    float out_min = -1e6f, out_max = 1e6f;
    float integral_max = 1e6f;
    float d_filter_N = 10.0f;     // Derivative LPF coefficient
    float setpoint_weight_b = 1.0f; // P setpoint weight (0-1)
    float setpoint_weight_c = 0.0f; // D setpoint weight (0=PI-D, 1=PID)
    float integral_sep_eps = 0.0f;  // 0 = disabled, >0 = threshold
};

class AdvancedPID {
public:
    AdvancedPID() = default;
    explicit AdvancedPID(const AdvancedPIDConfig& c) : cfg_(c) {}
    void configure(const AdvancedPIDConfig& c) { cfg_ = c; }

    float update(float setpoint, float measurement) {
        float error = setpoint - measurement;

        // --- P with setpoint weighting (2-DOF) ---
        float p_term = cfg_.kp * (cfg_.setpoint_weight_b * setpoint
                                  - measurement);

        // --- I with integral separation ---
        if (cfg_.integral_sep_eps <= 0.0f
            || fabsf(error) < cfg_.integral_sep_eps) {
            integral_ += cfg_.ki * error * cfg_.dt;
        }
        // Anti-windup: back-calculation
        integral_ += anti_windup_;
        integral_ = clamp(integral_, -cfg_.integral_max,
```

```
                        cfg_.integral_max);

        // --- D with setpoint weighting + filtered derivative ---
        float d_input = cfg_.setpoint_weight_c * setpoint - measurement;
        float d_raw = -(d_input - d_prev_) / cfg_.dt;
        float alpha = cfg_.dt * cfg_.d_filter_N
                    / (1.0f + cfg_.dt * cfg_.d_filter_N);
        d_filtered_ = alpha * d_raw + (1.0f - alpha) * d_filtered_;
        float d_term = cfg_.kd * d_filtered_;
        d_prev_ = d_input;

        // --- Output ---
        float raw = p_term + integral_ + d_term;
        float out = clamp(raw, cfg_.out_min, cfg_.out_max);

        // Back-calculation anti-windup
        anti_windup_ = (cfg_.ki != 0.0f)
            ? (out - raw) / cfg_.ki * cfg_.dt * 0.5f
            : 0.0f;
        return out;
    }

    void reset() {
        integral_ = 0; d_filtered_ = 0; d_prev_ = 0; anti_windup_ = 0;
    }

private:
    AdvancedPIDConfig cfg_;
    float integral_ = 0, d_filtered_ = 0, d_prev_ = 0, anti_windup_ = 0;
    static float clamp(float v, float lo, float hi) {
        return v < lo ? lo : (v > hi ? hi : v);
    }
};
```

Usage examples:

```
// Standard PID (same as pid.hpp)
AdvancedPIDConfig cfg;
cfg.kp=6; cfg.ki=0.3; cfg.kd=3; cfg.dt=0.001f;

// PI-D (derivative on measurement, no kick)
cfg.setpoint_weight_c = 0.0f;  // D on measurement only

// I-PD (P and D both on measurement)
cfg.setpoint_weight_b = 0.0f;  // P on measurement only
cfg.setpoint_weight_c = 0.0f;  // D on measurement only

// 2-DOF: soften setpoint response, keep disturbance rejection
cfg.setpoint_weight_b = 0.7f;  // reduce P overshoot on setpoint
cfg.setpoint_weight_c = 0.0f;  // no derivative kick

// Integral separation: only integrate when close to target
cfg.integral_sep_eps = 5.0f;   // degrees threshold
```

14.10 EKF for 6-DOF IMU (Pose Estimation)

A ready-to-use EKF that estimates roll, pitch, yaw and gyro biases from a 6-axis IMU.

```
// ekf_imu.hpp | 6-DOF IMU Pose Estimation via EKF
```

```
// State: [roll, pitch, yaw, bias_gx, bias_gy, bias_gz]
// Measurement: accelerometer -> roll, pitch
// Part of: A Practical Guide to Control Theory
// License: MIT
#pragma once
#include "mat.hpp"
#include <cmath>

class EKF_IMU {
public:
    static constexpr int NX = 6;  // states
    static constexpr int NZ = 2;  // measurements (roll, pitch from accel)

    using State  = Mat<NX, 1>;
    using Cov    = Mat<NX, NX>;
    using MeasV  = Mat<NZ, 1>;
    using MeasC  = Mat<NZ, NZ>;
    using KGain  = Mat<NX, NZ>;
    using Hmtx   = Mat<NZ, NX>;

    State x;   // [roll, pitch, yaw, bx, by, bz]
    Cov   P;

    Cov   Q;   // Process noise covariance
    MeasC R;   // Measurement noise covariance

    float dt = 0.001f;  // sampling period

    void init() {
        x = State::zeros();
        P = Cov::identity() * 0.1f;
        // Defaults (tune for your IMU)
        Q = Cov::zeros();
        Q(0,0)=0.001f; Q(1,1)=0.001f; Q(2,2)=0.001f;
        Q(3,3)=0.0001f; Q(4,4)=0.0001f; Q(5,5)=0.0001f;
        R(0,0) = 0.15f; R(1,1) = 0.15f;
    }

    /// Main update. Call at IMU sample rate.
    /// gx,gy,gz: gyro readings (rad/s)
    /// ax,ay,az: accel readings (m/s^2)
    void update(float gx, float gy, float gz,
                float ax, float ay, float az) {
        // === PREDICT ===
        float phi = x(0,0), theta = x(1,0);
        float wx = gx - x(3,0), wy = gy - x(4,0), wz = gz - x(5,0);

        float sp = sinf(phi), cp = cosf(phi);
        float tt = tanf(theta), ct = cosf(theta);

        // Euler kinematics
        float phi_dot   = wx + sp*tt*wy + cp*tt*wz;
        float theta_dot = cp*wy - sp*wz;
        float psi_dot   = (sp/ct)*wy + (cp/ct)*wz;

        // Predict state
        State xp = x;
        xp(0,0) += phi_dot * dt;
```

```
        xp(1,0) += theta_dot * dt;
        xp(2,0) += psi_dot * dt;

        // Jacobian F (6x6)
        Cov F = Cov::identity();
        F(0,0) += (cp*tt*wy - sp*tt*wz)*dt;
        F(0,1)  = (sp/(ct*ct)*wy + cp/(ct*ct)*wz)*dt;
        F(0,3) = -dt;  F(0,4) = -sp*tt*dt;  F(0,5) = -cp*tt*dt;
        F(1,0)  = (-sp*wy - cp*wz)*dt;
        F(1,4) = -cp*dt;  F(1,5) = sp*dt;
        F(2,0)  = (cp/ct*wy - sp/ct*wz)*dt;
        float st = sinf(theta);
        F(2,1)  = (sp*st/(ct*ct)*wy + cp*st/(ct*ct)*wz)*dt;
        F(2,4) = -sp/ct*dt;  F(2,5) = -cp/ct*dt;

        Cov Pp = F * P * F.T() + Q;

        // === UPDATE (accel -> roll, pitch) ===
        float z_roll  = atan2f(ay, az);
        float z_pitch = atan2f(-ax, sqrtf(ay*ay + az*az));

        MeasV z;  z(0,0) = z_roll;  z(1,0) = z_pitch;
        MeasV h;  h(0,0) = xp(0,0); h(1,0) = xp(1,0);

        MeasV innov;
        innov(0,0) = wrap_pi(z(0,0) - h(0,0));
        innov(1,0) = wrap_pi(z(1,0) - h(1,0));

        // H = [1 0 0 0 0 0; 0 1 0 0 0 0]
        Hmtx H = Hmtx::zeros();
        H(0,0) = 1.0f; H(1,1) = 1.0f;

        MeasC S = H * Pp * H.T() + R;
        KGain Kk = Pp * H.T() * S.inverse();

        x = xp + Kk * innov;
        P = (Cov::identity() - Kk * H) * Pp;
    }

    float roll()  const { return x(0,0); }
    float pitch() const { return x(1,0); }
    float yaw()   const { return x(2,0); }
    float bias_x() const { return x(3,0); }
    float bias_y() const { return x(4,0); }
    float bias_z() const { return x(5,0); }

private:
    static float wrap_pi(float a) {
        while (a >  3.14159265f) a -= 6.28318530f;
        while (a < -3.14159265f) a += 6.28318530f;
        return a;
    }
};
```

Usage:

```
    EKF_IMU ekf;
    ekf.dt = 0.002f;  // 500 Hz IMU
    ekf.init();
```

```
    // Tune for your IMU (MPU6050 example):
    ekf.Q(0,0) = 0.001f;  ekf.Q(1,1) = 0.001f;  ekf.Q(2,2) = 0.001f;
    ekf.Q(3,3) = 1e-5f;   ekf.Q(4,4) = 1e-5f;   ekf.Q(5,5) = 1e-5f;
    ekf.R(0,0) = 0.1f;    ekf.R(1,1) = 0.1f;

    // In IMU interrupt (500 Hz):
    ekf.update(gyro_x, gyro_y, gyro_z, accel_x, accel_y, accel_z);

    float roll_deg  = ekf.roll()  * 180.0f / M_PI;
    float pitch_deg = ekf.pitch() * 180.0f / M_PI;
    float yaw_deg   = ekf.yaw()   * 180.0f / M_PI;
```

14.11 TinyMPC Solver

A minimal embedded MPC solver using the Riccati-based approach from TinyMPC (see Section 6 theory). Solves box-constrained LQR problems in real time on microcontrollers.

```
// tiny_mpc.hpp | Riccati-based MPC for embedded systems
// Based on: TinyMPC (Nguyen et al., 2024)
// Precomputes Riccati recursion offline, solves ADMM online.
#pragma once
#include "mat.hpp"

template<int NX, int NU, int N_HORIZON>
class TinyMPC {
public:
    using StateVec = Mat<NX, 1>;
    using InputVec = Mat<NU, 1>;
    using StateMat = Mat<NX, NX>;
    using InputMat = Mat<NX, NU>;
    using GainMat  = Mat<NU, NX>;

    // System model (discrete)
    StateMat Ad;
    InputMat Bd;

    // Cost matrices
    StateMat Q;          // State cost
    Mat<NU,NU> R;        // Input cost
    StateMat Q_terminal; // Terminal cost (set to DARE solution)

    // Constraints
    InputVec u_min, u_max;
    StateVec x_min, x_max;

    // ADMM parameters
    float rho = 1.0f;        // Penalty parameter
    int   max_iter = 10;      // ADMM iterations (5-20 typical)

    // Precomputed Riccati gains (call precompute() once offline)
    GainMat  K_gains[N_HORIZON];    // Feedback gains
    StateMat P_mats[N_HORIZON + 1]; // Cost-to-go matrices

    /// Precompute Riccati recursion (call once, or when model changes)
    void precompute() {
        P_mats[N_HORIZON] = Q_terminal;
        for (int k = N_HORIZON - 1; k >= 0; --k) {
            // P = Q + A'*P_next*A - A'*P_next*B*(R+B'*P_next*B)^-1
```

```
        //     * B'*P_next*A
        auto P_next = P_mats[k + 1];
        auto BtP = Bd.T() * P_next;
        auto S = R + BtP * Bd;  // NU x NU
        auto S_inv = S.inverse();
        K_gains[k] = S_inv * BtP * Ad;  // NU x NX
        auto AtP = Ad.T() * P_next;
        P_mats[k] = Q + AtP * Ad - AtP * Bd * K_gains[k];
    }
}

/// Solve MPC for current state. Returns optimal first input.
InputVec solve(const StateVec& x0,
               const StateVec x_ref[N_HORIZON]) {
    // Initialize with unconstrained LQR rollout
    StateVec x_traj[N_HORIZON + 1];
    InputVec u_traj[N_HORIZON];
    x_traj[0] = x0;

    for (int k = 0; k < N_HORIZON; ++k) {
        StateVec dx = x_traj[k] - x_ref[k];
        u_traj[k] = (K_gains[k] * dx) * (-1.0f);
        // Clamp (projection step)
        for (int i = 0; i < NU; ++i) {
            float v = u_traj[k](i, 0);
            v = v < u_min(i,0) ? u_min(i,0) :
                (v > u_max(i,0) ? u_max(i,0) : v);
            u_traj[k](i, 0) = v;
        }
        x_traj[k + 1] = Ad * x_traj[k] + Bd * u_traj[k];
    }

    // ADMM refinement for constraint satisfaction
    InputVec z[N_HORIZON];   // Slack variables
    InputVec lambda[N_HORIZON]; // Dual variables
    for (int k = 0; k < N_HORIZON; ++k) {
        z[k] = u_traj[k];
        lambda[k] = InputVec::zeros();
    }

    for (int iter = 0; iter < max_iter; ++iter) {
        // Forward pass: solve unconstrained with penalty
        x_traj[0] = x0;
        for (int k = 0; k < N_HORIZON; ++k) {
            StateVec dx = x_traj[k] - x_ref[k];
            // Modified Riccati includes ADMM penalty
            InputVec u_unc = (K_gains[k] * dx) * (-1.0f);
            // ADMM correction
            u_traj[k] = u_unc + (z[k] - lambda[k]) * (rho * 0.5f);
            x_traj[k+1] = Ad * x_traj[k] + Bd * u_traj[k];
        }
        // z-update: projection onto constraints
        for (int k = 0; k < N_HORIZON; ++k) {
            for (int i = 0; i < NU; ++i) {
                float v = u_traj[k](i,0) + lambda[k](i,0);
                v = v < u_min(i,0) ? u_min(i,0) :
                    (v > u_max(i,0) ? u_max(i,0) : v);
                z[k](i, 0) = v;
```

```
                }
            }
            // Dual update
            for (int k = 0; k < N_HORIZON; ++k) {
                lambda[k] = lambda[k] + u_traj[k] - z[k];
            }
        }
        return u_traj[0];
    }
};
```

Usage — chassis trajectory tracking:

```
// 2D position control: state=[x,y,vx,vy], input=[ax,ay]
TinyMPC<4, 2, 10> mpc;  // 4 states, 2 inputs, horizon=10
float dt = 0.01f;

// Discrete double-integrator model
mpc.Ad = Mat<4,4>::identity();
mpc.Ad(0,2) = dt; mpc.Ad(1,3) = dt;
mpc.Bd(2,0) = dt; mpc.Bd(3,1) = dt;

// Costs
mpc.Q = Mat<4,4>::identity() * 10.0f;
mpc.R = Mat<2,2>::identity() * 1.0f;
mpc.Q_terminal = Mat<4,4>::identity() * 100.0f;

// Constraints: max acceleration 5 m/s^2
mpc.u_min(0,0) = -5; mpc.u_min(1,0) = -5;
mpc.u_max(0,0) =  5; mpc.u_max(1,0) =  5;

mpc.precompute();  // Once at startup

// In control loop:
Mat<4,1> x_ref[10];  // Fill from path planner
auto u = mpc.solve(current_state, x_ref);
set_accel(u(0,0), u(1,0));
```

14.12 Cubic Bézier Trajectory

Generates smooth 2D paths through four control points. Useful for swerve drive paths and end-effector trajectories.

```
// bezier.hpp -- Cubic Bezier curve for 2D trajectory generation
#pragma once

struct Vec2 { float x, y; };

class CubicBezier {
public:
    Vec2 p0, p1, p2, p3;  // Control points

    /// Evaluate position at parameter t in [0, 1]
    Vec2 position(float t) const {
        float u = 1.0f - t;
        float u2 = u * u, t2 = t * t;
        float u3 = u2 * u, t3 = t2 * t;
        return {
            u3*p0.x + 3*u2*t*p1.x + 3*u*t2*p2.x + t3*p3.x,
```

```
            u3*p0.y + 3*u2*t*p1.y + 3*u*t2*p2.y + t3*p3.y
        };
    }

    /// Evaluate first derivative (tangent) at parameter t
    Vec2 tangent(float t) const {
        float u = 1.0f - t;
        return {
            3*(u*u*(p1.x-p0.x) + 2*u*t*(p2.x-p1.x) + t*t*(p3.x-p2.x)),
            3*(u*u*(p1.y-p0.y) + 2*u*t*(p2.y-p1.y) + t*t*(p3.y-p2.y))
        };
    }

    /// Evaluate second derivative (curvature direction)
    Vec2 acceleration(float t) const {
        float u = 1.0f - t;
        return {
            6*(u*(p2.x - 2*p1.x + p0.x) + t*(p3.x - 2*p2.x + p1.x)),
            6*(u*(p2.y - 2*p1.y + p0.y) + t*(p3.y - 2*p2.y + p1.y))
        };
    }

    /// Approximate arc length by summing N linear segments
    float arc_length(int N = 100) const {
        float len = 0;
        Vec2 prev = p0;
        for (int i = 1; i <= N; ++i) {
            Vec2 cur = position(static_cast<float>(i) / N);
            float dx = cur.x - prev.x, dy = cur.y - prev.y;
            len += sqrtf(dx*dx + dy*dy);
            prev = cur;
        }
        return len;
    }
};
```

Usage — swerve drive path:

```
CubicBezier path;
path.p0 = {0, 0};        // Start
path.p1 = {1.0f, 0};     // Exit direction: forward
path.p2 = {2.0f, 1.5f};  // Entry direction at end
path.p3 = {3.0f, 1.5f};  // End point

// Track with 50 samples
for (int i = 0; i <= 50; ++i) {
    float t = static_cast<float>(i) / 50;
    Vec2 pos = path.position(t);
    Vec2 vel = path.tangent(t);  // For feedforward heading
    // Send (pos.x, pos.y) to chassis controller
}
```

14.13 Module Dependency Map

Module	File	Depends on
Low-pass filter	`lpf.hpp`	(none)
PID controller	`pid.hpp`	(none)
Advanced PID	`pid_advanced.hpp`	(none)
Setpoint ramp	`ramp.hpp`	`lpf.hpp`
Complementary filter	`complementary_filter.hpp`	(none)
Matrix library	`mat.hpp`	(none)
Kalman filter	`kalman.hpp`	`mat.hpp`
EKF IMU (6-DOF)	`ekf_imu.hpp`	`mat.hpp`
LQR controller	`lqr.hpp`	`mat.hpp`
TinyMPC solver	`tiny_mpc.hpp`	`mat.hpp`
Cascaded PID	`cascaded_pid.hpp`	`pid.hpp`
Cubic Bézier	`bezier.hpp`	(none)

All modules are MIT licensed. Copy into your project's `include/control/` directory, `#include`, and go.

Closing Thoughts

If you have made it this far, congratulations—you now have a solid theoretical foundation for designing control systems on competition robots. Here is a cheat sheet for what to reach for in different situations:

Situation	**Tool**
Noisy sensor data	Low-pass filter (Section 2)
What does my motor actually do?	Motor modeling (Section 3)
Single motor speed/position control	PID (Section 4)
Gimbal tracking	PID + feedforward (Section 4)
Smooth point-to-point moves	Trajectory generation (Section 6)
Getting PID from paper to MCU	Tustin discretization (Section 5)
Balance bot / MIMO system	LQR (Section 6)
Trajectory tracking with constraints	MPC (Section 6)
State estimation from noisy sensors	Kalman filter (Section 6)
IMU + encoder fusion	Extended Kalman filter (Section 6)
Robot can flip/tumble past $\pm 90°$	Quaternion EKF (Section 6)

Remember: theory is a tool, not an end goal. The best controller is the one that **works on the field**, not the one that looks most elegant on paper. Start simple (PID), understand your system, add complexity only when needed, and always, always test on real hardware.

Good luck at your next competition. Go build something awesome.